

Maschinelles Lernen

Tobias Laser

Sommersemester 2026

Inhaltsverzeichnis

Vorwort	19
1 Einführung in maschinelles Lernen	22
1.1 Ziel dieses Kapitels	22
1.2 Was soll ein KI-System leisten?	22
1.3 Daten, Merkmale und Zielwerte	23
1.4 Lernen als Funktionsanpassung	24
1.5 Parameter eines Modells	25
1.6 Trainingsdaten, Testdaten und Generalisierung	26
1.7 Klassifikation und Regression	26
1.7.1 Klassifikation	26
1.7.2 Regression	27
1.8 Arten des Lernens	28
1.8.1 Überwachtes Lernen	28
1.8.2 Halbüberwachtes Lernen	29
1.8.3 Unüberwachtes Lernen	29
1.8.4 Bestärkendes Lernen	29
1.9 Probabilistische Antworten als optimale Inferenz	30
1.9.1 Warum ist Unsicherheit nützlich?	30
1.10 Ein erstes Klassifikationsbeispiel	31
1.11 Merkmalsextraktion und Vorverarbeitung	32
1.12 Modell, Verlustfunktion und Optimierung	32
1.12.1 Modell	33
1.12.2 Verlustfunktion	33
1.12.3 Optimierung	33
1.13 Überanpassung und Unteranpassung	33
1.13.1 Unteranpassung	34
1.13.2 Überanpassung	34
1.14 Parametrische und nichtparametrische Modelle	34
1.15 Warum Wahrscheinlichkeitstheorie?	35
1.16 Mini-Beispiel: Von Daten zu Wahrscheinlichkeiten	36
1.17 Typischer ML-Workflow	36
1.18 Ausblick auf die nächsten Kapitel	37
1.19 Zusammenfassung	37
1.20 Typische Prüfungsfragen	38

2	Grundlagen der Wahrscheinlichkeit	39
2.1	Ziel dieses Kapitels	39
2.2	Zufallsexperimente und Ergebnisse	39
2.3	Zufallsvariablen	40
2.4	Diskrete Wahrscheinlichkeitsverteilungen	41
2.5	Wahrscheinlichkeit als relative Häufigkeit	42
2.6	Das Schlüssel-Frucht-Beispiel	43
2.7	Gemeinsame Wahrscheinlichkeiten	43
2.7.1	Gemeinsame Wahrscheinlichkeit als Tabelle	44
2.8	Randwahrscheinlichkeiten und Summenregel	44
2.9	Bedingte Wahrscheinlichkeiten	45
2.9.1	Definition der bedingten Wahrscheinlichkeit	46
2.10	Produktregel	46
2.11	Summenregel und Produktregel zusammen	47
2.12	Bayes-Regel	48
2.12.1	Herleitung der Bayes-Regel	48
2.13	Prior, Likelihood, Evidence und Posterior	49
2.14	Bayes-Regel im Schlüssel-Frucht-Beispiel	50
2.15	Was bedeutet Bayes-Regel intuitiv?	51
2.16	Bayes-Regel für Klassifikation	52
2.17	Unabhängigkeit	53
2.18	Bedingte Unabhängigkeit	54
2.19	Kettenregel für Wahrscheinlichkeiten	54
2.20	Normalisierung	55
2.21	Häufige Fehler	56
2.22	Mini-Aufgaben	56
2.22.1	Aufgabe 1: Gemeinsame Wahrscheinlichkeit	56
2.22.2	Aufgabe 2: Randwahrscheinlichkeit	56
2.22.3	Aufgabe 3: Bedingte Wahrscheinlichkeit	57
2.23	Zusammenfassung	58
2.24	Typische Prüfungsfragen	58
3	Kontinuierliche Zufallsvariablen	60
3.1	Ziel dieses Kapitels	60
3.2	Von diskreten zu kontinuierlichen Zufallsvariablen	61
3.3	Wahrscheinlichkeitsdichten	62
3.4	Wahrscheinlichkeiten als Integrale	62
3.5	Die Gleichverteilung	63
3.6	Erwartungswert	64
3.7	Mittelwert	64
3.8	Varianz	66
3.8.1	Alternative Formel für die Varianz	66
3.9	Standardabweichung	67

3.10	Kovarianz	68
3.11	Korrelation	68
3.12	Gemeinsame Dichten	69
3.13	Marginalisierung bei Dichten	70
3.14	Bedingte Dichten	70
3.15	Bayes-Regel für Dichten	71
3.16	Histogramme und Dichteschätzung	71
3.17	Sampling-Approximation von Erwartungswerten	73
3.18	Beispiel: Numerische Integration mit Sampling	73
3.19	Analytische Lösung des ersten Integrals	74
3.20	Symmetrische Dichten	75
3.20.1	Intuition	76
3.21	Wichtige Verteilungen	76
3.21.1	Gleichverteilung	76
3.21.2	Normalverteilung	76
3.22	Kontinuierliche Zufallsvektoren	77
3.23	Zusammenhang zu maschinellem Lernen	77
3.24	Häufige Fehler	78
3.25	Mini-Aufgaben	78
3.25.1	Aufgabe 1: Wahrscheinlichkeit bei Gleichverteilung	78
3.25.2	Aufgabe 2: Mittelwert einer Gleichverteilung	79
3.25.3	Aufgabe 3: Varianz aus Momenten	79
3.26	Zusammenfassung	80
3.27	Typische Prüfungsfragen	81
4	Schätzer	82
4.1	Ziel dieses Kapitels	82
4.2	Parameter, Schätzer und Schätzwert	83
4.3	Daten als Zufallsvariablen	84
4.4	Beispiel: Parameter einer Gleichverteilung schätzen	85
4.5	Heuristischer Schätzer mit Minimum und Maximum	85
4.6	Methode der Momente	86
4.6.1	Empirischer Mittelwert	86
4.6.2	Empirische Varianz	86
4.7	Welcher Schätzer ist besser?	88
4.8	Bias eines Schätzers	88
4.9	Varianz eines Schätzers	89
4.10	Mittlerer quadratischer Fehler	89
4.10.1	Bias-Varianz-Zerlegung	89
4.10.2	Herleitung	90
4.11	Bias-Varianz-Trade-off	91
4.12	Konsistenz	91
4.13	Das Stichprobenmittel	92

4.14	Bias des Stichprobenmittels	92
4.15	Varianz des Stichprobenmittels	93
4.16	Die naive Stichprobenvarianz	94
4.17	Bias der naiven Stichprobenvarianz	95
4.17.1	Intuition für die Verzerrung	95
4.18	Unverzerrte Stichprobenvarianz	96
4.18.1	Nachweis der Erwartungstreue	96
4.19	Vergleich der Varianzschätzer	97
4.20	Übersicht: Mittelwert- und Varianzschätzer	98
4.21	Maximum-Likelihood-Schätzung	98
4.22	Likelihood bei i.i.d. Daten	99
4.23	Log-Likelihood	99
4.24	Beispiel: MLE für Bernoulli-Verteilung	100
4.25	MLE für die Normalverteilung	102
4.26	MLE für die Gleichverteilung	103
4.27	Maximum-a-posteriori-Schätzung	104
4.28	Log-Posterior bei MAP	105
4.29	MLE als Spezialfall von MAP	105
4.30	MAP und Regularisierung	106
4.31	Bayesian Estimator	106
4.32	Vergleich: MLE, MAP und Bayesian	107
4.33	Effizienz eines Schätzers	107
4.34	Zusammenhang zu maschinellem Lernen	108
4.35	Häufige Fehler	108
4.36	Mini-Aufgaben	109
4.36.1	Aufgabe 1: Bias eines einfachen Schätzers	109
4.36.2	Aufgabe 2: MSE aus Bias und Varianz	109
4.36.3	Aufgabe 3: Stichprobenmittel	109
4.36.4	Aufgabe 4: Unverzerrte Stichprobenvarianz	110
4.36.5	Aufgabe 5: MLE für Bernoulli	110
4.37	Zusammenfassung	111
4.38	Typische Prüfungsfragen	112
5	Maximum-Likelihood, MAP und Bayessche Schätzung	114
5.1	Ziel dieses Kapitels	114
5.2	Grundsituation der Parameterschätzung	115
5.3	Likelihood	116
5.4	Maximum-Likelihood-Schätzung	116
5.5	Log-Likelihood	117
5.6	Negative Log-Likelihood	118
5.7	Standardrezept für MLE-Herleitungen	118
5.8	Beispiel: Bernoulli-Verteilung	119
5.8.1	Likelihood	120

5.8.2	Log-Likelihood	120
5.8.3	Ableitung	120
5.9	Bernoulli-MLE als Klassifikationsverlust	121
5.10	Beispiel: Normalverteilung mit bekanntem σ^2	122
5.10.1	Likelihood	122
5.10.2	Log-Likelihood	122
5.10.3	Ableitung	122
5.11	Normalverteilung mit unbekanntem μ und σ^2	124
5.12	Regression als Maximum-Likelihood	124
5.13	Prior	125
5.14	Posterior	126
5.15	Maximum-a-posteriori-Schätzung	127
5.16	MLE als Spezialfall von MAP	127
5.17	MAP und Regularisierung	128
5.18	Laplace-Prior und L_1 -Regularisierung	129
5.19	Bayessche Schätzung	129
5.20	Punktschätzer aus dem Posterior	130
5.20.1	Posterior-Mittelwert	130
5.20.2	Posterior-Modus	130
5.21	Prädiktive Verteilung	131
5.22	Beispiel: Beta-Bernoulli-Modell	131
5.22.1	Likelihood	132
5.22.2	Posterior	132
5.23	Konjugierte Priors	132
5.24	Vergleich von MLE, MAP und Bayesscher Schätzung	133
5.25	Interpretation bei wenigen und vielen Daten	133
5.25.1	Wenige Daten	134
5.25.2	Viele Daten	134
5.26	Zusammenhang zu Overfitting	134
5.27	Häufige Fehler	134
5.28	Mini-Aufgaben	135
5.28.1	Aufgabe 1: Bernoulli-MLE	135
5.28.2	Aufgabe 2: Negative Log-Likelihood	136
5.28.3	Aufgabe 3: MAP mit konstantem Prior	136
5.28.4	Aufgabe 4: Beta-Bernoulli-Posterior	137
5.29	Zusammenfassung	137
5.30	Typische Prüfungsfragen	138
6	Lineare Regression	140
6.1	Ziel dieses Kapitels	140
6.2	Regression als überwachtes Lernen	141
6.3	Das einfachste lineare Modell	142
6.4	Lineare Regression mit mehreren Eingabedimensionen	142

6.5	Vektorform mit Bias	143
6.6	Was bedeutet linear?	144
6.7	Basisfunktionen	145
6.7.1	Typische Basisfunktionen	145
6.8	Trainingsdaten in Matrixform	146
6.9	Fehlerfunktion der linearen Regression	147
6.10	Least Squares	148
6.11	Gradient der Fehlerfunktion	149
6.12	Normalengleichung	150
6.13	Geometrische Interpretation	150
6.14	Pseudoinverse	151
6.15	Wann ist die Lösung eindeutig?	151
6.16	Training Error und Test Error	152
6.17	Polynomiale Regression und Overfitting	152
6.18	Probabilistische Interpretation	153
6.19	Likelihood der linearen Regression	153
6.20	Log-Likelihood der linearen Regression	154
6.21	Schätzung der Rauschvarianz	154
6.22	Regularisierung	155
6.23	Ridge Regression	155
6.24	Ridge-Lösung	156
6.25	MAP-Interpretation von Ridge Regression	156
6.26	Gradientenabstieg für lineare Regression	157
6.27	Stochastischer Gradientenabstieg	158
6.28	Mehrdimensionale Zielwerte	158
6.29	Praktische Aspekte	159
6.29.1	Skalierung der Eingaben	159
6.29.2	Multikollinearität	159
6.30	Häufige Fehler	160
6.31	Mini-Aufgaben	160
6.31.1	Aufgabe 1: Einfache Vorhersage	160
6.31.2	Aufgabe 2: Designmatrix	161
6.31.3	Aufgabe 3: Quadratischer Fehler	161
6.31.4	Aufgabe 4: Ridge-Lösung	162
6.32	Zusammenfassung	162
6.33	Typische Prüfungsfragen	163
7	Logistische Regression	165
7.1	Ziel dieses Kapitels	165
7.2	Von Regression zu Klassifikation	166
7.3	Warum nicht einfach lineare Regression?	166
7.4	Die Grundidee der logistischen Regression	167
7.5	Die Sigmoid-Funktion	168

7.6	Interpretation der Sigmoid-Ausgabe	169
7.7	Entscheidungsregel	169
7.8	Bias-Term	170
7.9	Odds und Logit	170
7.10	Bernoulli-Modell für die Zielwerte	172
7.11	Likelihood der logistischen Regression	173
7.12	Log-Likelihood	173
7.13	Negative Log-Likelihood und Kreuzentropie	174
7.14	Intuition der Kreuzentropie	174
7.15	Ableitung der Sigmoid-Funktion	175
7.16	Gradient für einen Datenpunkt	177
7.17	Gradient für den gesamten Datensatz	178
7.18	Matrixform des Gradienten	179
7.19	Warum gibt es keine geschlossene Lösung?	180
7.20	Gradientenabstieg	181
7.21	Stochastischer Gradientenabstieg	181
7.22	Hesse-Matrix und Konvexität	181
7.23	Newton-Verfahren und IRLS	182
7.24	Entscheidungsgrenze	183
7.25	Nichtlineare Entscheidungsgrenzen mit Basisfunktionen	184
7.26	Regularisierte logistische Regression	185
7.27	MAP-Interpretation der Regularisierung	185
7.28	Numerische Stabilität	186
7.29	Evaluation eines binären Klassifikators	186
7.30	Accuracy	187
7.31	Konfusionsmatrix	187
7.32	Precision, Recall und F1-Score	188
7.33	Wahl der Entscheidungsschwelle	188
7.34	Mehrklassen-Klassifikation	189
7.35	Mehrklassen-Kreuzentropie	190
7.36	Softmax als probabilistisches Modell	190
7.37	Zusammenhang zu neuronalen Netzen	191
7.38	Praktisches Beispiel: Binäre Zielvariable erzeugen	191
7.39	Feature-Skalierung	191
7.40	Häufige Fehler	192
7.41	Mini-Aufgaben	192
7.41.1	Aufgabe 1: Sigmoid berechnen	192
7.41.2	Aufgabe 2: Wahrscheinlichkeit und Gegenwahrscheinlichkeit	193
7.41.3	Aufgabe 3: Binäre Kreuzentropie	193
7.41.4	Aufgabe 4: Gradient für einen Datenpunkt	194
7.41.5	Aufgabe 5: Entscheidungsgrenze	194
7.42	Zusammenfassung	195

7.43	Typische Prüfungsfragen	196
8	Perzeptrons	198
8.1	Ziel dieses Kapitels	198
8.2	Motivation	199
8.3	Das Perzeptron-Modell	199
8.4	Bias-Term	200
8.5	Interpretation als künstliches Neuron	201
8.6	Entscheidungsgrenze	202
8.7	Geometrische Interpretation des Gewichtsvektors	203
8.8	Lineare Separierbarkeit	204
8.9	Korrekt und falsch klassifizierte Punkte	205
8.10	Perzeptron-Kriterium	205
8.11	Gradient des Perzeptron-Kriteriums	206
8.12	Perzeptron-Lernregel	207
8.13	Intuition der Lernregel	207
8.13.1	Fall $t_n = +1$	207
8.13.2	Fall $t_n = -1$	208
8.14	Algorithmus des Perzeptrons	208
8.15	Perzeptron-Konvergenz	209
8.16	Beispiel für ein Update	210
8.17	Vergleich mit logistischer Regression	211
8.18	Harte und weiche Entscheidung	211
8.19	Perzeptron-Verlust und Kreuzentropie	212
8.20	Mehrere Ausgabekonventionen	212
8.20.1	Kodierung mit $\{-1, +1\}$	212
8.20.2	Kodierung mit $\{0,1\}$	213
8.21	Limitierung: XOR-Problem	213
8.22	Warum mehrschichtige Netze helfen	214
8.23	Perzeptron als Baustein neuronaler Netze	214
8.24	Aktivierungsfunktionen	215
8.24.1	Sigmoid	215
8.24.2	Tanh	215
8.24.3	ReLU	216
8.25	Warum lineare Schichten allein nicht reichen	216
8.26	Perzeptron und Feature-Transformation	217
8.27	Online-Lernen	217
8.28	Lernrate	218
8.29	Pocket-Algorithmus	218
8.30	Vergleich: Perzeptron, logistische Regression und lineare Regression	219
8.31	Mehrklassen-Perzeptron	219
8.32	Perzeptron mit Margin	220
8.33	Zusammenhang zu Support Vector Machines	220

8.34	Zusammenhang zu Backpropagation	221
8.35	Praktische Aspekte	221
8.35.1	Feature-Skalierung	221
8.35.2	Datenreihenfolge	221
8.35.3	Nicht separierbare Daten	221
8.36	Häufige Fehler	222
8.37	Mini-Aufgaben	222
8.37.1	Aufgabe 1: Perzeptron-Vorhersage	222
8.37.2	Aufgabe 2: Falsch klassifiziert oder korrekt?	223
8.37.3	Aufgabe 3: Perzeptron-Update	223
8.37.4	Aufgabe 4: Entscheidungsgrenze	224
8.37.5	Aufgabe 5: XOR	224
8.38	Zusammenfassung	225
8.39	Typische Prüfungsfragen	226
9	Mehrschichtige Perzeptrons und Backpropagation	227
9.1	Ziel dieses Kapitels	227
9.2	Vom Perzeptron zum neuronalen Netz	228
9.3	Schichten eines neuronalen Netzes	228
9.4	Dimensionen der Parameter	229
9.5	Forward Pass	230
9.6	Warum Nichtlinearitäten notwendig sind	230
9.7	Wichtige Aktivierungsfunktionen	231
9.7.1	Sigmoid	231
9.7.2	Tanh	232
9.7.3	ReLU	232
9.8	Ausgabeschichten	232
9.8.1	Regression	232
9.8.2	Binäre Klassifikation	233
9.8.3	Mehrklassen-Klassifikation	233
9.9	Verlustfunktion für einen Datensatz	233
9.10	Das Problem: Viele Parameter	234
9.11	Kettenregel	234
9.12	Backpropagation-Idee	235
9.13	Ein einfaches Zwei-Schichten-Netz	235
9.14	Gradienten der Ausgabeschicht	236
9.15	Fehlerterm der Ausgabeschicht	236
9.15.1	Quadratischer Fehler	236
9.15.2	Softmax mit Kreuzentropie	237
9.15.3	Sigmoid mit binärer Kreuzentropie	237
9.16	Fehlerterm der versteckten Schicht	237
9.17	Backpropagation-Algorithmus	238
9.18	Gradientenabstieg für MLPs	238

9.19	Mini-Batch-Training	239
9.20	Matrixform für mehrere Datenpunkte	239
9.21	Initialisierung der Gewichte	240
9.21.1	Xavier-Initialisierung	240
9.21.2	He-Initialisierung	240
9.22	Verschwindende und explodierende Gradienten	240
9.23	Tote ReLU-Neuronen	241
9.24	Regularisierung neuronaler Netze	241
9.24.1	L_2 -Regularisierung	242
9.24.2	Early Stopping	242
9.24.3	Dropout	242
9.25	Universal Approximation	242
9.26	Vergleich mit linearen Modellen	242
9.27	Häufige Fehler	243
9.28	Mini-Aufgaben	243
9.28.1	Aufgabe 1: Dimensionen einer Schicht	243
9.28.2	Aufgabe 2: ReLU berechnen	244
9.28.3	Aufgabe 3: Ausgabefehler bei Softmax	244
9.28.4	Aufgabe 4: Gewichtgradient einer Schicht	245
9.28.5	Aufgabe 5: Warum Nichtlinearität?	245
9.29	Zusammenfassung	246
9.30	Typische Prüfungsfragen	247
10	Generalisierung und Validierung	248
10.1	Ziel dieses Kapitels	248
10.2	Trainingsfehler und Generalisierung	249
10.3	Daten erzeugende Verteilung	249
10.4	Erwarteter Fehler	250
10.5	Trainingsfehler als Schätzer	250
10.6	Trainings-, Validierungs- und Testdaten	251
10.6.1	Trainingsfehler	251
10.6.2	Validierungsfehler	251
10.6.3	Testfehler	251
10.7	Überanpassung	251
10.8	Unteranpassung	252
10.9	Modellkomplexität	252
10.10	Beispiel: Polynomiale Regression	253
10.10.1	Kleines M	253
10.10.2	Mittleres M	253
10.10.3	Großes M	254
10.11	Hyperparameter	254
10.12	Validierung	255
10.13	Typische Datenaufteilung	255

10.14	Kreuzvalidierung	256
10.15	Leave-One-Out-Kreuzvalidierung	256
10.16	Stratifizierte Aufteilung	257
10.17	Data Leakage	257
10.18	Vorverarbeitung und Splits	258
10.19	Bias-Varianz-Trade-off	258
10.19.1	Hoher Bias	259
10.19.2	Hohe Varianz	259
10.20	Bias-Varianz-Zerlegung für Regression	259
10.21	Regularisierung	260
10.21.1	L_2 -Regularisierung	260
10.21.2	L_1 -Regularisierung	260
10.22	Regularisierung als MAP	261
10.23	Early Stopping	261
10.24	Lernkurven	262
10.24.1	Fehler über Trainingszeit	262
10.24.2	Fehler über Datenmenge	262
10.25	Diagnose: Underfitting oder Overfitting	263
10.25.1	Fall 1: Beide Fehler hoch	263
10.25.2	Fall 2: Trainingsfehler niedrig, Validierungsfehler hoch	263
10.25.3	Fall 3: Beide Fehler niedrig	264
10.26	Datenaugmentation	264
10.27	Ensembles	264
10.28	Modellselektion	265
10.29	Grid Search und Random Search	265
10.29.1	Grid Search	265
10.29.2	Random Search	266
10.30	Nested Cross-Validation	266
10.31	Verteilungsshift	266
10.32	Metriken für Regression	267
10.32.1	Mean Squared Error	267
10.32.2	Root Mean Squared Error	267
10.32.3	Mean Absolute Error	267
10.33	Metriken für Klassifikation	267
10.33.1	Accuracy	267
10.33.2	Precision	267
10.33.3	Recall	267
10.33.4	F1-Score	268
10.34	Unsicherheit der Testschätzung	268
10.35	Häufige Fehler	268
10.36	Mini-Aufgaben	269
10.36.1	Aufgabe 1: Trainings- und Testfehler interpretieren	269

10.36.2 Aufgabe 2: Underfitting erkennen	269
10.36.3 Aufgabe 3: Kreuzvalidierung	269
10.36.4 Aufgabe 4: Accuracy	270
10.36.5 Aufgabe 5: Precision und Recall	270
10.37 Zusammenfassung	271
10.38 Typische Prüfungsfragen	271
11 PCA und Dimensionsreduktion	273
11.1 Ziel dieses Kapitels	273
11.2 Motivation für Dimensionsreduktion	274
11.3 Warum hohe Dimensionen problematisch sind	275
11.3.1 Rechenaufwand	275
11.3.2 Speicherbedarf	275
11.3.3 Rauschen und irrelevante Merkmale	275
11.3.4 Visualisierung	275
11.4 Der Fluch der Dimensionalität	276
11.5 Lineare Dimensionsreduktion	276
11.6 Zentrierung der Daten	277
11.7 Datenmatrix	278
11.8 Kovarianzmatrix	278
11.9 Symmetrie und positive Semidefinitheit	279
11.10 Varianz in einer Richtung	280
11.11 Erste Hauptkomponente	281
11.12 Herleitung mit Lagrange-Multiplikatoren	281
11.13 Weitere Hauptkomponenten	282
11.14 Projektion auf Hauptkomponenten	283
11.15 Rekonstruktion aus der reduzierten Darstellung	284
11.16 Rekonstruktionsfehler	285
11.17 Erklärte Varianz	285
11.18 Wahl der Anzahl der Komponenten	286
11.18.1 Varianzschwelle	286
11.18.2 Scree Plot	286
11.19 PCA-Algorithmus	287
11.20 PCA und Data Leakage	287
11.21 Standardisierung vor PCA	288
11.22 PCA und Dekorrelation	288
11.23 Whitening	289
11.24 PCA über Singular Value Decomposition	289
11.25 PCA für Visualisierung	290
11.26 PCA für Rauschreduktion	291
11.27 PCA und Kompression	291
11.28 PCA und Klassifikation	292
11.29 Probabilistische Interpretation	292

11.30	Lineare und nichtlineare Dimensionsreduktion	293
11.31	PCA im Vergleich zu Autoencodern	293
11.32	Praktische Aspekte	294
11.32.1	Vorverarbeitung	294
11.32.2	Ausreißer	294
11.32.3	Interpretierbarkeit	294
11.33	Häufige Fehler	294
11.34	Mini-Aufgaben	295
11.34.1	Aufgabe 1: Zentrierung	295
11.34.2	Aufgabe 2: Erklärte Varianz	295
11.34.3	Aufgabe 3: Projektion	296
11.34.4	Aufgabe 4: Rekonstruktion	296
11.34.5	Aufgabe 5: Dimensionen	297
11.35	Zusammenfassung	298
11.36	Typische Prüfungsfragen	299
12	Clustering	301
12.1	Ziel dieses Kapitels	301
12.2	Was ist Clustering?	302
12.3	Clustering versus Klassifikation	302
12.4	Ähnlichkeit und Distanz	303
12.5	Feature-Skalierung	304
12.6	k -means: Grundidee	304
12.7	Cluster-Zuordnungen	305
12.8	Zielfunktion von k -means	306
12.9	Der k -means-Algorithmus	306
12.10	Herleitung des Aktualisierungsschritts	307
12.11	Warum k -means konvergiert	308
12.12	Lokale Minima und Initialisierung	309
12.13	k -means++	309
12.14	Leere Cluster	310
12.15	Wahl der Clusterzahl K	310
12.16	Elbow-Methode	310
12.17	Silhouette-Koeffizient	311
12.18	Grenzen von k -means	312
12.18.1	Sphärische Cluster	312
12.18.2	Empfindlichkeit gegenüber Ausreißern	312
12.18.3	Feste Clusterzahl	312
12.18.4	Harte Zuordnungen	312
12.18.5	Skalierung	312
12.19	Harte und weiche Clusterzuordnung	313
12.20	Hierarchisches Clustering	313
12.21	Agglomeratives Clustering	314

12.22	Linkage-Kriterien	314
12.22.1	Single Linkage	314
12.22.2	Complete Linkage	314
12.22.3	Average Linkage	314
12.22.4	Ward-Linkage	314
12.23	Dendrogramm	315
12.24	DBSCAN	315
12.25	Kernpunkte, Randpunkte und Rauschen	315
12.26	Eigenschaften von DBSCAN	316
12.27	Vergleich wichtiger Clustering-Methoden	316
12.28	Clustering und probabilistische Modelle	317
12.29	Evaluation von Clustering	317
12.29.1	Interne Maße	317
12.29.2	Externe Maße	318
12.30	Interpretation von Clustern	318
12.31	Clustering-Workflow	318
12.32	Mini-Aufgaben	319
12.32.1	Aufgabe 1: Nächstes Zentrum	319
12.32.2	Aufgabe 2: Zentrum aktualisieren	320
12.32.3	Aufgabe 3: k -means-Zielfunktion	320
12.32.4	Aufgabe 4: Silhouette-Wert	321
12.32.5	Aufgabe 5: Hartes oder weiches Clustering	321
12.33	Häufige Fehler	321
12.34	Zusammenfassung	322
12.35	Typische Prüfungsfragen	323
13	Gaussian Mixture Models	325
13.1	Ziel dieses Kapitels	325
13.2	Motivation	326
13.3	Eine einzelne multivariate Gauß-Verteilung	326
13.4	MLE für eine einzelne Gauß-Verteilung	327
13.5	Gaussian Mixture Model	328
13.6	Generative Interpretation	329
13.7	One-Hot-Darstellung der latenten Variablen	330
13.8	Likelihood eines GMMs	330
13.9	Warum direkte MLE schwierig ist	331
13.10	Posterior über Komponenten	332
13.11	Effektive Clustergröße	333
13.12	Expectation-Maximization: Grundidee	333
13.13	E-Schritt für GMMs	334
13.14	M-Schritt für GMMs	334
13.15	Interpretation der M-Schritt-Formeln	335
13.16	EM-Algorithmus für GMMs	335

13.17	Konvergenz von EM	336
13.18	Free Energy und EM-Idee	337
13.19	Initialisierung	337
13.20	Vergleich mit k -means	338
13.21	k -means als Grenzfall eines GMMs	338
13.22	Kovarianztypen	339
13.22.1	Volle Kovarianz	339
13.22.2	Diagonale Kovarianz	339
13.22.3	Sphärische Kovarianz	339
13.22.4	Geteilte Kovarianz	339
13.23	Anzahl der Parameter	339
13.24	Wahl der Komponentenzahl K	340
13.25	Clustering mit GMMs	341
13.26	Anomalieerkennung mit GMMs	341
13.27	Singularitäten und degenerierte Lösungen	342
13.28	Numerische Stabilität	342
13.29	GMMs in hoher Dimension	343
13.30	GMMs und PCA	344
13.31	Vor- und Nachteile von GMMs	344
13.31.1	Vorteile	344
13.31.2	Nachteile	344
13.32	Mini-Aufgaben	345
13.32.1	Aufgabe 1: Mischgewichte prüfen	345
13.32.2	Aufgabe 2: GMM-Dichte	345
13.32.3	Aufgabe 3: Responsibility berechnen	346
13.32.4	Aufgabe 4: Effektive Clustergröße	346
13.32.5	Aufgabe 5: Mischgewicht aktualisieren	346
13.32.6	Aufgabe 6: GMM oder k -means?	347
13.33	Häufige Fehler	347
13.34	Zusammenfassung	348
13.35	Typische Prüfungsfragen	349
14	Expectation-Maximization	351
14.1	Ziel dieses Kapitels	351
14.2	Motivation	352
14.3	Beobachtete und vollständige Daten	352
14.4	Beobachtete Daten-Likelihood	353
14.5	Vollständige Daten-Likelihood	354
14.6	Grundidee von EM	355
14.7	Die Q -Funktion	355
14.8	Der EM-Algorithmus	356
14.9	Warum EM die Log-Likelihood nicht verringert	356
14.10	Untere Schranke mit einer Hilfsverteilung	357

14.11	Zerlegung der unteren Schranke	357
14.12	Zusammenhang mit KL-Divergenz	358
14.13	E-Schritt als Optimierung von q	358
14.14	M-Schritt als Optimierung von θ	359
14.15	Monotonie von EM	360
14.16	EM für Gaussian Mixture Models	360
14.17	Vollständige Daten-Likelihood für GMMs	360
14.18	Vollständige Daten-Log-Likelihood für GMMs	361
14.19	E-Schritt für GMMs	362
14.20	Q-Funktion für GMMs	362
14.21	M-Schritt: effektive Clustergröße	363
14.22	M-Schritt: Update für Mittelwerte	363
14.23	M-Schritt: Update für Kovarianzen	364
14.24	M-Schritt: Update für Mischgewichte	364
14.25	Zusammenfassung der GMM-EM-Updates	366
14.26	EM und k -means	366
14.27	Generalized EM	367
14.28	Hard EM	367
14.29	EM bei fehlenden Daten	368
14.30	Initialisierung	368
14.31	Konvergenzkriterien	369
14.32	Numerische Stabilität	369
14.33	Singularitäten bei GMMs	370
14.34	Vorteile von EM	370
14.35	Nachteile von EM	371
14.36	Mini-Aufgaben	371
14.36.1	Aufgabe 1: Latente Variable erkennen	371
14.36.2	Aufgabe 2: Responsibility berechnen	371
14.36.3	Aufgabe 3: Effektive Clustergröße	372
14.36.4	Aufgabe 4: Mischgewicht	372
14.36.5	Aufgabe 5: Monotonie	373
14.37	Häufige Fehler	373
14.38	Zusammenfassung	373
14.39	Typische Prüfungsfragen	375
15	Nichtparametrische Methoden	377
15.1	Ziel dieses Kapitels	377
15.2	Parametrische und nichtparametrische Modelle	378
15.3	Warum nichtparametrische Methoden?	378
15.4	Dichteschätzung	379
15.5	Histogramm-Dichteschätzung	380
15.6	Bin-Breite und Bias-Varianz-Trade-off	380
15.6.1	Große Bin-Breite	381

15.6.2 Kleine Bin-Breite	381
15.7 Kernel Density Estimation	381
15.8 Kernel	382
15.9 Bandbreite	382
15.9.1 Kleine Bandbreite	382
15.9.2 Große Bandbreite	383
15.10 Mehrdimensionale KDE	383
15.11 KDE und Gaussian Mixture Models	383
15.12 k -nearest neighbors	384
15.13 k -NN für Klassifikation	384
15.14 k -NN für Regression	385
15.15 Wahl von k	386
15.15.1 Kleines k	386
15.15.2 Großes k	386
15.16 Entscheidungsgrenzen bei k -NN	386
15.17 Distanzmaße	386
15.17.1 Euklidische Distanz	387
15.17.2 Manhattan-Distanz	387
15.17.3 Minkowski-Distanz	387
15.18 Feature-Skalierung bei k -NN	387
15.19 Fluch der Dimensionalität	388
15.20 Speicher- und Rechenaufwand	388
15.21 Parzen-Fenster	389
15.22 k -NN-Dichteschätzung	389
15.23 Nichtparametrische Regression als lokale Mittelung	389
15.24 Vergleich parametrisch und nichtparametrisch	390
15.25 Vorteile nichtparametrischer Methoden	390
15.26 Nachteile nichtparametrischer Methoden	391
15.27 Modellwahl und Validierung	391
15.28 Mini-Aufgaben	392
15.28.1 Aufgabe 1: Histogramm-Dichte	392
15.28.2 Aufgabe 2: k -NN-Klassifikation	392
15.28.3 Aufgabe 3: k -NN-Regression	393
15.28.4 Aufgabe 4: Bias und Varianz bei k	393
15.28.5 Aufgabe 5: KDE als GMM	393
15.29 Häufige Fehler	394
15.30 Zusammenfassung	394
15.31 Typische Prüfungsfragen	396

Vorwort

Dieses Skript ist als Lernskript zur Lehrveranstaltung *Maschinelles Lernen* gedacht. Es orientiert sich an der Themenliste der Vorlesung und des Proseminars und erklärt die Themen von Grund auf.

Das Ziel ist nicht eine kurze Zusammenfassung, sondern ein verständlicher Aufbau der wichtigsten Begriffe, Formeln, Herleitungen, Beispiele und Prüfungszusammenhänge.

Sprache und Begriffe

Das Skript ist vollständig auf Deutsch geschrieben. Wichtige englische Fachbegriffe werden beim ersten Auftreten in eckigen Klammern ergänzt, zum Beispiel *überwachtes Lernen* [*supervised learning*], *Wahrscheinlichkeitsdichte* [*probability density function*] oder *Maximum-Likelihood-Schätzung* [*maximum likelihood estimation*].

Aufbau der Kapitel

Jedes Kapitel ist nach einem ähnlichen Schema aufgebaut:

- Definitionen wichtiger Begriffe,
- intuitive Erklärung der Grundideen,
- zentrale Formeln,
- Herleitungen,
- Beispiele,
- prüfungsrelevante Hinweise.

Boxen

In diesem Skript werden verschiedene Boxen verwendet:

- `definitionbox`: wichtige Begriffe,
- `conceptbox`: zentrale Idee,
- `formulabox`: zentrale Formeln,
- `derivationbox`: Herleitungen,

- `examplebox`: Beispiele,
- `examtipbox`: Hinweise für die Prüfung.

Hinweis

Das Skript ist als Arbeitsdokument gedacht. Es ersetzt nicht die Vorlesung, sondern soll helfen, die Themen systematisch zu verstehen und zu wiederholen.

“‘latex

1 Einführung in maschinelles Lernen

1.1 Ziel dieses Kapitels

Dieses Kapitel führt die Grundbegriffe des maschinellen Lernens (machine learning) ein. Es geht noch nicht darum, einen bestimmten Algorithmus vollständig zu beherrschen. Stattdessen soll klar werden, was maschinelles Lernen überhaupt leisten soll, welche Arten von Lernproblemen es gibt und warum Wahrscheinlichkeitstheorie (probability theory) im maschinellen Lernen eine zentrale Rolle spielt.

Idee

Maschinelles Lernen (machine learning) beschäftigt sich mit Systemen, die aus Daten Muster lernen und dieses gelernte Wissen auf neue, noch nicht gesehene Daten anwenden sollen.

Das zentrale Ziel ist also nicht bloß, vorhandene Daten auswendig zu lernen, sondern Generalisierung (generalization).

Nach diesem Kapitel solltest du folgende Fragen beantworten können:

- Was ist der Unterschied zwischen künstlicher Intelligenz (artificial intelligence) und maschinellem Lernen (machine learning)?
- Was ist ein Datensatz (data set) und was ist ein Modell (model)?
- Was bedeutet Lernen in einem mathematischen Sinn?
- Was ist der Unterschied zwischen überwachtem Lernen (supervised learning), halbüberwachtem Lernen (semi-supervised learning), unüberwachtem Lernen (unsupervised learning) und bestärkendem Lernen (reinforcement learning)?
- Was ist der Unterschied zwischen Klassifikation (classification) und Regression (regression)?
- Warum liefern viele moderne Modelle keine festen Antworten, sondern Wahrscheinlichkeiten?
- Was bedeutet Training (training), Testen (testing) und Generalisierung (generalization)?

1.2 Was soll ein KI-System leisten?

Ein System der künstlichen Intelligenz (artificial intelligence) soll Aufgaben lösen, die typischerweise als intelligent wahrgenommen werden. Beispiele sind:

- ein Bild einer Ziffer erkennen,
- eine E-Mail als Spam oder Nicht-Spam klassifizieren,
- den zukünftigen Verkaufspreis einer Wohnung vorhersagen,
- ähnliche Patientinnen und Patienten gruppieren,
- einem Roboter beibringen, sich in einer Umgebung sinnvoll zu bewegen.

Maschinelles Lernen (machine learning) ist ein Teilgebiet davon. Die Grundidee ist: Wir schreiben nicht für jede mögliche Situation eine feste Regel. Stattdessen geben wir dem System Daten und lassen es aus diesen Daten eine Regel, Funktion oder Verteilung lernen.

Definition

Ein Modell (model) ist im maschinellen Lernen eine mathematische Vorschrift, die Eingaben in Ausgaben umwandelt oder die Struktur der Daten beschreibt.

Typisch schreibt man:

$$y = f(x).$$

Dabei ist x die Eingabe (input), f das gelernte Modell und y die Ausgabe (output).

Die Eingabe x kann sehr unterschiedlich aussehen:

- ein einzelner Messwert, zum Beispiel Temperatur,
- ein Vektor von Messwerten, zum Beispiel Blutdruck, Alter und Puls,
- ein Bild, das als sehr großer Vektor von Pixelwerten dargestellt wird,
- ein Text, der zuerst in eine numerische Darstellung umgewandelt wird.

In den meisten ML-Modellen werden Eingaben als Vektoren geschrieben:

$$\vec{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_D \end{pmatrix} \in \mathbb{R}^D.$$

Hier ist D die Dimension (dimension) der Eingabe. Bei einem Bild mit vielen Pixeln kann D sehr groß sein.

1.3 Daten, Merkmale und Zielwerte

Ein Datensatz (data set) besteht aus Beobachtungen. Eine einzelne Beobachtung wird oft als Datenpunkt (data point) bezeichnet.

Definition

Ein Datenpunkt (data point) ist eine einzelne Beobachtung. Er besteht meistens aus einem Eingabevektor $\vec{x}$ und, falls vorhanden, einem Zielwert t .

Ein überwachter Trainingsdatensatz hat typischerweise die Form

$$\mathcal{D} = \{(\vec{x}_1, t_1), (\vec{x}_2, t_2), \dots, (\vec{x}_N, t_N)\}.$$

Dabei ist N die Anzahl der Datenpunkte.

Die Komponenten eines Eingabevektors nennt man Merkmale (features). Ein Beispiel:

$$\vec{x} = \begin{pmatrix} \text{Größe} \\ \text{Gewicht} \\ \text{Alter} \end{pmatrix}.$$

In einem echten ML-System würden diese Einträge natürlich als Zahlen dargestellt werden, also zum Beispiel:

$$\vec{x} = \begin{pmatrix} 178 \\ 72 \\ 23 \end{pmatrix}.$$

Hinweis

In mathematischen Herleitungen wird oft einfach x geschrieben, obwohl eigentlich ein Vektor $\vec{x}$ gemeint ist. Das ist eine übliche Abkürzung. In diesem Skript wird möglichst klar unterschieden, aber in späteren Kapiteln kann x je nach Kontext auch für einen Vektor stehen.

1.4 Lernen als Funktionsanpassung

Eine einfache Sichtweise auf maschinelles Lernen ist:

Wir suchen eine Funktion, die zu den Daten passt und für neue Daten gute Vorhersagen macht.

Angenommen, wir haben Eingaben $\vec{x}_n$ und Zielwerte t_n . Dann suchen wir ein Modell f , sodass

$$f(\vec{x}_n) \approx t_n.$$

Das Zeichen $\approx$ ist wichtig: In echten Daten gibt es fast immer Rauschen (noise), Messfehler oder unvollständige Information. Deshalb erwarten wir meistens nicht, dass das Modell alle Daten perfekt trifft.

Beispiel

Angenommen, wir wollen die Miete einer Wohnung vorhersagen. Die Eingabe könnte sein:

$$\vec{x} = \begin{pmatrix} \text{Wohnfläche} \\ \text{Anzahl Zimmer} \\ \text{Entfernung zum Zentrum} \end{pmatrix}.$$

Der Zielwert t wäre dann die tatsächliche Miete.

Ein Modell f soll aus diesen Eingaben eine vorhergesagte Miete berechnen:

$$\hat{t} = f(\vec{x}).$$

Der Wert $\hat{t}$ ist die Vorhersage (prediction).

1.5 Parameter eines Modells

Viele Modelle enthalten Parameter (parameters). Diese Parameter bestimmen, wie genau das Modell rechnet.

Ein sehr einfaches Modell für eine eindimensionale Eingabe ist eine Gerade:

$$y(x, w_0, w_1) = w_0 + w_1 x.$$

Dabei sind w_0 und w_1 Parameter. Man kann sie zusammen als Vektor schreiben:

$$\vec{w} = \begin{pmatrix} w_0 \\ w_1 \end{pmatrix}.$$

Dann schreibt man das Modell oft als

$$y(x, \vec{w}).$$

Definition

Lernen (learning) bedeutet in vielen ML-Modellen, geeignete Parameter $\vec{w}$ aus den Daten zu bestimmen.

Informell:

$$\text{Daten} \longrightarrow \text{Lernalgorithmus} \longrightarrow \text{gelernte Parameter}.$$

In späteren Kapiteln wird diese Idee präziser. Bei der linearen Regression (linear regression) werden die Parameter zum Beispiel so gewählt, dass ein Fehlermaß (error function) möglichst klein wird. Bei probabilistischen Modellen werden die Parameter oft über Maximum-Likelihood-Schätzung (maximum likelihood estimation) oder Maximum-a-posteriori-Schätzung (maximum a-posteriori estimation) bestimmt.

1.6 Trainingsdaten, Testdaten und Generalisierung

Ein Modell soll nicht nur die Daten erklären, die es schon gesehen hat. Es soll auch bei neuen Daten funktionieren. Deshalb unterscheidet man zwischen Trainingsdaten (training data) und Testdaten (test data).

Definition

Trainingsdaten (training data) sind Daten, mit denen das Modell gelernt wird.

Testdaten (test data) sind Daten, die beim Lernen nicht verwendet werden und danach zur Bewertung des Modells dienen.

Generalisierung (generalization) bedeutet, dass ein Modell auch auf neuen, bisher unbekannten Daten gute Vorhersagen macht.

Die zentrale Herausforderung ist:

gute Leistung auf Trainingsdaten $\nRightarrow$ gute Leistung auf neuen Daten.

Ein Modell kann die Trainingsdaten sehr gut treffen und trotzdem schlecht generalisieren. Dieses Problem nennt man Überanpassung (overfitting).

Idee

Ein Modell soll die Struktur in den Daten lernen, nicht bloß die Trainingsdaten auswendig.

Wenn ein Modell auch zufälliges Rauschen (noise) lernt, spricht man von Überanpassung (overfitting).

1.7 Klassifikation und Regression

Zwei grundlegende Problemtypen im überwachten Lernen sind Klassifikation (classification) und Regression (regression).

1.7.1 Klassifikation

Bei einer Klassifikation (classification) soll das Modell eine Eingabe einer von mehreren Klassen (classes) zuordnen.

Definition

Bei einer Klassifikation (classification) ist der Zielwert diskret. Das heißt, er stammt aus einer endlichen oder abzählbaren Menge von Klassen:

$$t \in \{1, 2, \dots, C\}.$$

Dabei ist C die Anzahl der Klassen.

Beispiele:

- Bild einer Ziffer $\rightarrow$ Klasse $0, 1, \dots, 9$,

- E-Mail $\rightarrow$ Spam oder Nicht-Spam,
- medizinischer Befund $\rightarrow$ krank oder gesund,
- Foto $\rightarrow$ Katze, Hund, Auto, Person.

Für eine Ziffernerkennung könnte das Modell eine Eingabe $\vec{x}$ bekommen, die ein Bild beschreibt. Die Ausgabe ist dann zum Beispiel:

$$y(\vec{x}) = 7.$$

Das bedeutet: Das Modell entscheidet, dass das Bild zur Klasse 7 gehört.

Hinweis

In modernen probabilistischen Klassifikationsmodellen gibt das Modell oft nicht direkt eine Klasse aus, sondern Wahrscheinlichkeiten für alle Klassen:

$$p(c = 0 \mid \vec{x}) = 0,01, \quad p(c = 1 \mid \vec{x}) = 0,03, \quad p(c = 2 \mid \vec{x}) = 0,91, \quad \dots$$

Die endgültige Klasse kann dann als die wahrscheinlichste Klasse gewählt werden.

1.7.2 Regression

Bei einer Regression (regression) soll das Modell einen kontinuierlichen Wert vorhersagen.

Definition

Bei einer Regression (regression) ist der Zielwert kontinuierlich, also typischerweise eine reelle Zahl:

$$t \in \mathbb{R}.$$

Beispiele:

- Wohnungsdaten $\rightarrow$ Mietpreis,
- Wetterdaten $\rightarrow$ Temperatur morgen,
- medizinische Daten $\rightarrow$ erwarteter Laborwert,
- technische Prozessdaten $\rightarrow$ Produktionsausbeute.

Ein Regressionsmodell gibt also nicht eine Klasse wie Katze oder Hund aus, sondern zum Beispiel:

$$\hat{t} = 23,7.$$

Prüfungshinweis

Prüfungsfragen unterscheiden oft zwischen Klassifikation (classification) und Regression (regression).

- Diskrete Ausgabe? $\Rightarrow$ Klassifikation (classification).
- Kontinuierliche Ausgabe? $\Rightarrow$ Regression (regression).

Wichtig ist die Art des Zielwerts, nicht die Art der Eingabe. Ein Bild kann zum Beispiel für Klassifikation verwendet werden, aber auch für Regression, wenn man daraus einen kontinuierlichen Wert vorhersagen will.

1.8 Arten des Lernens

Maschinelles Lernen kann danach eingeteilt werden, welche Information beim Lernen verfügbar ist.

1.8.1 Überwachtes Lernen

Definition

Beim überwachten Lernen (supervised learning) bestehen die Trainingsdaten aus Eingaben und bekannten Zielwerten:

$$\mathcal{D} = \{(\vec{x}_1, t_1), \dots, (\vec{x}_N, t_N)\}.$$

Das Modell lernt, Eingaben $\vec{x}$ auf Zielwerte t abzubilden.

Man kann sich das wie Lernen mit einer Lehrperson vorstellen: Für jeden Trainingsfall ist bekannt, was die richtige Antwort gewesen wäre.

Typische Aufgaben:

- Klassifikation (classification),
- Regression (regression).

Beispiel

Bei der Ziffernerkennung besteht ein Trainingsbeispiel aus einem Bild und dem zugehörigen Label:

(Bild einer 7, 7).

Das Modell soll aus vielen solchen Beispielen lernen, neue Bilder korrekt zu klassifizieren.

1.8.2 Halbüberwachtes Lernen

Definition

Beim halbüberwachten Lernen (semi-supervised learning) gibt es einige Datenpunkte mit Labels und viele Datenpunkte ohne Labels.

Die Daten haben also ungefähr die Form:

$$\mathcal{D} = \underbrace{\{(\vec{x}_1, t_1), \dots, (\vec{x}_M, t_M)\}}_{\text{gelabelte Daten}} \cup \underbrace{\{\vec{x}_{M+1}, \dots, \vec{x}_N\}}_{\text{ungelabelte Daten}}.$$

Das ist praktisch wichtig, weil Labels oft teuer sind. Zum Beispiel kann man viele medizinische Bilder speichern, aber ihre korrekte Diagnose muss von Expertinnen oder Experten erstellt werden.

1.8.3 Unüberwachtes Lernen

Definition

Beim unüberwachten Lernen (unsupervised learning) gibt es keine Zielwerte. Die Daten bestehen nur aus Eingaben:

$$\mathcal{D} = \{\vec{x}_1, \dots, \vec{x}_N\}.$$

Das Ziel ist, Struktur in den Daten zu finden.

Typische Aufgaben sind:

- Cluster finden (clustering),
- Wahrscheinlichkeitsdichten schätzen (density estimation),
- Daten in niedrigere Dimensionen projizieren (dimensionality reduction),
- Muster oder latente Faktoren finden (latent factors).

Beispiel

Angenommen, wir haben Messwerte von vielen Patientinnen und Patienten, aber keine festen Diagnosen. Ein Clustering-Algorithmus könnte versuchen, Gruppen ähnlicher Patientinnen und Patienten zu finden. Diese Gruppen müssen nicht vorher bekannt sein.

1.8.4 Bestärkendes Lernen

Definition

Beim bestärkenden Lernen (reinforcement learning) lernt ein Agent (agent) durch Interaktion mit einer Umgebung (environment). Der Agent führt Aktionen (actions) aus und erhält Belohnungen (rewards). Ziel ist es, langfristig eine möglichst hohe Belohnung zu erreichen.

Die Grundelemente sind:

- Zustand (state),
- Aktion (action),
- Belohnung (reward),
- Strategie (policy),
- Umgebung (environment).

Anders als beim überwachten Lernen bekommt der Agent nicht für jede Situation die perfekte Antwort gezeigt. Er muss durch Versuch und Irrtum (trial and error) lernen.

Beispiel

Ein Spielagent sieht eine Spielsituation, wählt einen Zug und bekommt später eine Belohnung, zum Beispiel +1 für einen Sieg und −1 für eine Niederlage.
Das Schwierige ist: Ein Zug kann erst viele Schritte später wichtig werden. Man muss also lernen, welche früheren Aktionen zu späterem Erfolg beigetragen haben.

Hinweis

Bestärkendes Lernen (reinforcement learning) ist wichtig, aber in diesem Skript liegt der Schwerpunkt zunächst auf probabilistischen Grundlagen, Regression, Klassifikation, neuronalen Netzen, Dimensionsreduktion, Clustering, Gaussian Mixture Models und dem EM-Algorithmus.

1.9 Probabilistische Antworten als optimale Inferenz

Viele ML-Probleme haben keine absolut sichere Antwort. Ein Bild kann undeutlich sein, Messwerte können rauschen, Daten können unvollständig sein. Deshalb ist es oft sinnvoller, nicht nur eine feste Antwort zu geben, sondern Unsicherheit mitzuschätzen.

Idee

Ein probabilistisches Modell (probabilistic model) gibt nicht nur eine Vorhersage aus, sondern beschreibt Unsicherheit mit Wahrscheinlichkeiten.

Bei einer Klassifikation könnte ein Modell zum Beispiel ausgeben:

$$p(c = 0 \mid \vec{x}) = 0,01, \quad p(c = 1 \mid \vec{x}) = 0,03, \quad p(c = 2 \mid \vec{x}) = 0,91, \quad \dots$$

Dann wäre Klasse 2 die wahrscheinlichste Klasse. Aber das Modell sagt zusätzlich: Es ist sich zu etwa 91% sicher.

1.9.1 Warum ist Unsicherheit nützlich?

Unsicherheit ist nützlich, weil sie bessere Entscheidungen erlaubt.

- Bei hoher Sicherheit kann ein System automatisch entscheiden.
- Bei niedriger Sicherheit kann ein Mensch nachprüfen.
- Bei medizinischen oder sicherheitskritischen Aufgaben ist Unsicherheit besonders wichtig.
- Wahrscheinlichkeiten erlauben es, mehrere Informationsquellen mathematisch sauber zu kombinieren.

Beispiel

Ein medizinisches Modell klassifiziert ein Bild als unauffällig oder auffällig.

Fall A:

$$p(\text{auffällig} \mid \vec{x}) = 0,99.$$

Fall B:

$$p(\text{auffällig} \mid \vec{x}) = 0,51.$$

In beiden Fällen wäre die wahrscheinlichste Klasse auffällig. Trotzdem sind die Situationen sehr verschieden: In Fall A ist das Modell sehr sicher, in Fall B fast unsicher.

1.10 Ein erstes Klassifikationsbeispiel

Wir betrachten ein einfaches Klassifikationsproblem. Ein System bekommt ein Bild einer handgeschriebenen Ziffer. Die möglichen Klassen sind:

$$c \in \{0,1,2,3,4,5,6,7,8,9\}.$$

Das Bild wird als Eingabevektor $\vec{x}$ dargestellt. Wenn das Bild zum Beispiel 28×28 Pixel hat, dann enthält es

$$28 \cdot 28 = 784$$

Pixelwerte. Das Bild kann also als Vektor in $\mathbb{R}^{784}$ dargestellt werden:

$$\vec{x} \in \mathbb{R}^{784}.$$

Das Modell soll aus $\vec{x}$ die richtige Ziffer bestimmen.

Definition

Bei der Ziffernklassifikation ist

$$\vec{x} = \text{Bild als Vektor}, \quad c = \text{Ziffernklasse}.$$

Ein probabilistischer Klassifikator modelliert dann Wahrscheinlichkeiten der Form

$$p(c \mid \vec{x}).$$

Die endgültige Entscheidung könnte durch die wahrscheinlichste Klasse getroffen werden:

$$\hat{c} = \arg \max_{c \in \{0, \dots, 9\}} p(c \mid \vec{x}).$$

Das bedeutet: Wir wählen die Klasse c , für die die bedingte Wahrscheinlichkeit $p(c \mid \vec{x})$ maximal ist.

Hinweis

Das Symbol $\arg \max$ liefert nicht den maximalen Wert selbst, sondern die Stelle, an der das Maximum erreicht wird.

Wenn $p(c = 7 \mid \vec{x})$ am größten ist, dann ist

$$\arg \max_c p(c \mid \vec{x}) = 7.$$

1.11 Merkmalsextraktion und Vorverarbeitung

Rohdaten sind oft nicht direkt ideal für ein ML-Modell. Deshalb werden Daten häufig vorverarbeitet (preprocessed).

Definition

Vorverarbeitung (preprocessing) bezeichnet Transformationen der Rohdaten, bevor sie in das Modell eingegeben werden.

Merkmalsextraktion (feature extraction) bedeutet, aus Rohdaten geeignete Merkmale (features) zu berechnen.

Beispiele:

- Bilder werden skaliert oder zentriert.
- Messwerte werden normalisiert.
- Texte werden in numerische Vektoren umgewandelt.
- Sehr hochdimensionale Daten werden auf wenige Dimensionen reduziert.

Prüfungshinweis

Wichtig: Die gleiche Vorverarbeitung muss auf Trainingsdaten und Testdaten angewendet werden. Wenn Trainingsdaten anders verarbeitet werden als Testdaten, misst man nicht mehr die echte Modellleistung.

1.12 Modell, Verlustfunktion und Optimierung

Viele ML-Verfahren lassen sich in drei Bausteine zerlegen:

1. Modell (model),
2. Verlustfunktion (loss function),
3. Optimierungsverfahren (optimization method).

1.12.1 Modell

Das Modell beschreibt, welche Funktionen überhaupt möglich sind. Zum Beispiel:

$$y(x, \vec{w}) = w_0 + w_1 x$$

für eine Gerade oder ein neuronales Netz (neural network) für eine komplexere Abbildung.

1.12.2 Verlustfunktion

Die Verlustfunktion (loss function) misst, wie schlecht das Modell auf den Trainingsdaten ist. Eine typische Idee ist:

großer Fehler $\Rightarrow$ großer Verlust.

Für Regression wird oft ein quadratischer Fehler verwendet:

$$E(\vec{w}) = \frac{1}{2} \sum_{n=1}^N (y(\vec{x}_n, \vec{w}) - t_n)^2.$$

Der Faktor $\frac{1}{2}$ ist mathematisch praktisch, weil er sich beim Ableiten mit dem Faktor 2 aus dem Quadrat kürzt.

1.12.3 Optimierung

Beim Training sucht man Parameter, die den Verlust möglichst klein machen:

$$\vec{w}^* = \arg \min_{\vec{w}} E(\vec{w}).$$

Definition

Training (training) bedeutet oft:

Finde Parameter, die eine Verlustfunktion minimieren.

Mathematisch schreibt man:

$$\vec{w}^* = \arg \min_{\vec{w}} E(\vec{w}).$$

Später werden wir sehen, dass dieses Optimierungsproblem bei manchen Modellen direkt gelöst werden kann, während man bei anderen Modellen iterative Verfahren wie Gradientenabstieg (gradient descent) verwendet.

1.13 Überanpassung und Unteranpassung

Ein Modell kann auf verschiedene Arten schlecht sein.

1.13.1 Unteranpassung

Unteranpassung (underfitting) bedeutet, dass das Modell zu einfach ist, um die Struktur der Daten zu erfassen.

Beispiel

Wenn die Daten ungefähr einer gekrümmten Kurve folgen, aber man nur eine Gerade anpasst, kann die Gerade die Struktur nicht gut treffen. Das Modell ist dann zu unflexibel.

1.13.2 Überanpassung

Überanpassung (overfitting) bedeutet, dass das Modell zu stark an die Trainingsdaten angepasst ist und dabei auch Rauschen (noise) lernt.

Beispiel

Ein sehr komplexes Polynom kann durch jeden Trainingspunkt laufen. Das sieht auf den Trainingsdaten perfekt aus, kann aber zwischen den Datenpunkten stark oszillieren und auf neuen Daten schlecht sein.

Idee

Gutes maschinelles Lernen bedeutet, die richtige Balance zu finden:

Modell zu einfach $\Rightarrow$ Unteranpassung (underfitting).

Modell zu komplex $\Rightarrow$ Überanpassung (overfitting).

1.14 Parametrische und nichtparametrische Modelle

In diesem Kurs tauchen später auch parametrische und nichtparametrische Methoden auf.

Definition

Ein parametrisches Modell (parametric model) hat eine feste Anzahl von Parametern. Die Modellform wird vorher festgelegt, und das Lernen besteht darin, diese Parameter aus den Daten zu bestimmen.

Beispiel:

$$y(x, w_0, w_1) = w_0 + w_1 x.$$

Dieses Modell hat zwei Parameter, egal ob wir 10, 100 oder 10^6 Datenpunkte haben.

Definition

Ein nichtparametrisches Modell (nonparametric model) hat nicht unbedingt eine feste Parameterzahl. Die effektive Komplexität kann mit der Datenmenge wachsen.

Beispiele für nichtparametrische Methoden sind Histogramme (histograms), Kernel-Dichteschätzung (kernel density estimation) und k -nächste-Nachbarn (k -nearest neighbours).

Hinweis

Nichtparametrisch bedeutet nicht, dass es gar keine Parameter gibt. Es bedeutet eher, dass die Modellkomplexität nicht durch eine feste, kleine Parameterzahl vorgegeben ist.

1.15 Warum Wahrscheinlichkeitstheorie?

Wahrscheinlichkeitstheorie (probability theory) ist im maschinellen Lernen nicht nur ein Zusatz, sondern eine Grundlage. Sie beantwortet Fragen wie:

- Wie beschreibt man Unsicherheit mathematisch?
- Wie kombiniert man Vorwissen und neue Daten?
- Wie lernt man Parameter aus Daten?
- Wie entscheidet man zwischen mehreren möglichen Klassen?
- Wie modelliert man Rauschen (noise)?

Viele zentrale Verfahren des Kurses werden probabilistisch motiviert:

- Bayes-Regel (Bayes' rule),
- Maximum-Likelihood-Schätzung (maximum likelihood estimation),
- Maximum-a-posteriori-Schätzung (maximum a-posteriori estimation),
- logistische Regression (logistic regression),
- Gaussian Mixture Models,
- Expectation-Maximization-Algorithmus (expectation maximization algorithm).

Idee

Die probabilistische Sichtweise lautet:

Lernen $\approx$ Schließen aus unsicheren Daten.

Englisch nennt man das inference under uncertainty. Ein Modell soll nicht nur eine Antwort geben, sondern begründen, wie plausibel verschiedene Antworten unter den Daten sind.

1.16 Mini-Beispiel: Von Daten zu Wahrscheinlichkeiten

Angenommen, wir haben ein einfaches Experiment mit Früchten. Es gibt zwei Schüsseln:

$$s \in \{A, B\}.$$

Aus einer Schüssel wird zufällig eine Frucht gezogen. Die Frucht kann eine Orange oder eine weiße Beere sein:

$$y \in \{O, W\}.$$

Eine typische Frage wäre:

Wenn ich eine Orange beobachtet habe, wie wahrscheinlich ist es, dass sie aus Schüssel B kam?

Mathematisch ist das:

$$p(s = B \mid y = O).$$

Das ist eine bedingte Wahrscheinlichkeit (conditional probability). In späteren Kapiteln lernen wir, wie man solche Wahrscheinlichkeiten mit der Bayes-Regel (Bayes' rule) berechnet.

Hinweis

Dieses Beispiel ist wichtig, weil es die typische ML-Situation vereinfacht zeigt:

- Es gibt beobachtete Daten.
- Es gibt unbekannte Ursachen oder Klassen.
- Wir wollen aus Beobachtungen auf plausible Ursachen schließen.

Genau diese Idee steckt hinter vielen Klassifikationsmodellen.

1.17 Typischer ML-Workflow

Ein praktischer ML-Workflow sieht oft so aus:

1. **Problem definieren:** Was soll vorhergesagt oder gelernt werden?
2. **Daten sammeln:** Welche Eingaben und Zielwerte stehen zur Verfügung?
3. **Daten vorverarbeiten:** Fehlende Werte, Skalierung, Kodierung, Merkmalsextraktion.
4. **Modellklasse wählen:** Zum Beispiel lineare Regression, logistische Regression, neuronales Netz, PCA oder k -means.
5. **Verlustfunktion oder probabilistisches Ziel festlegen:** Zum Beispiel quadratischer Fehler, negative Log-Likelihood oder ELBO.
6. **Training durchführen:** Parameter aus den Trainingsdaten bestimmen.

7. **Validieren und testen:** Modell auf neuen Daten bewerten.

8. **Interpretieren:** Verstehen, was das Modell gut kann, wo es unsicher ist und wo es versagt.

Prüfungshinweis

Für Prüfungsfragen ist oft wichtig, dass du die Begriffe sauber trennst:

- Modell (model): Welche Funktionsklasse oder Verteilung wird verwendet?
- Parameter (parameters): Welche Größen werden gelernt?
- Training (training): Wie werden die Parameter angepasst?
- Verlustfunktion (loss function): Was wird minimiert?
- Generalisierung (generalization): Wie gut funktioniert das Modell auf neuen Daten?

1.18 Ausblick auf die nächsten Kapitel

Dieses Kapitel hat die Grundideen eingeführt. Die nächsten Kapitel machen diese Ideen mathematisch präzise.

- In Kapitel 2 behandeln wir diskrete Wahrscheinlichkeiten, bedingte Wahrscheinlichkeiten, Summenregel, Produktregel und Bayes-Regel.
- In Kapitel 3 betrachten wir kontinuierliche Zufallsvariablen, Wahrscheinlichkeitsdichten, Erwartungswerte, Varianz, Kovarianz und Korrelation.
- Danach lernen wir Schätzer (estimators), Maximum-Likelihood (maximum likelihood), Maximum-a-posteriori (maximum a-posteriori) und konkrete Modelle wie lineare Regression und logistische Regression.
- Später folgen neuronale Netze, Backpropagation, PCA, Clustering, Gaussian Mixture Models und der EM-Algorithmus.

1.19 Zusammenfassung

- Maschinelles Lernen (machine learning) bedeutet, Muster aus Daten zu lernen.
- Ein Modell (model) bildet Eingaben auf Ausgaben ab oder beschreibt die Struktur der Daten.
- Beim überwachten Lernen (supervised learning) gibt es Eingaben und Zielwerte.
- Beim unüberwachten Lernen (unsupervised learning) gibt es keine Zielwerte.
- Beim bestärkenden Lernen (reinforcement learning) lernt ein Agent durch Belohnungen.
- Klassifikation (classification) bedeutet Vorhersage diskreter Klassen.

- Regression (regression) bedeutet Vorhersage kontinuierlicher Werte.
- Generalisierung (generalization) ist das zentrale Ziel: Das Modell soll auf neuen Daten gut sein.
- Probabilistische Modelle (probabilistic models) beschreiben Unsicherheit explizit.
- Wahrscheinlichkeitstheorie (probability theory) ist deshalb eine zentrale Grundlage für maschinelles Lernen.

1.20 Typische Prüfungsfragen

Prüfungshinweis

Mögliche Fragen zu diesem Kapitel:

1. Erkläre den Unterschied zwischen Klassifikation (classification) und Regression (regression).
2. Was ist der Unterschied zwischen überwachtem Lernen (supervised learning) und unüberwachtem Lernen (unsupervised learning)?
3. Was bedeutet Generalisierung (generalization)?
4. Warum reicht es nicht, nur den Trainingsfehler zu minimieren?
5. Was ist Überanpassung (overfitting)?
6. Warum sind Wahrscheinlichkeiten im maschinellen Lernen nützlich?
7. Was ist der Unterschied zwischen Modell (model), Parameter (parameters) und Lernalgorithmus (learning algorithm)?

““

2 Grundlagen der Wahrscheinlichkeit

2.1 Ziel dieses Kapitels

Dieses Kapitel führt die wichtigsten Grundlagen der diskreten Wahrscheinlichkeitstheorie (probability theory) ein. Diese Grundlagen sind notwendig, um später probabilistische Modelle, Maximum-Likelihood-Schätzung (maximum likelihood estimation), Maximum-a-posteriori-Schätzung (maximum a-posteriori estimation), logistische Regression (logistic regression), Gaussian Mixture Models und den EM-Algorithmus zu verstehen.

Im vorherigen Kapitel wurde motiviert, dass maschinelles Lernen häufig mit Unsicherheit umgehen muss. In diesem Kapitel wird diese Unsicherheit mathematisch beschrieben.

Idee

Wahrscheinlichkeitstheorie (probability theory) ist die mathematische Sprache für Unsicherheit.

Im maschinellen Lernen verwenden wir Wahrscheinlichkeiten, um auszudrücken, wie plausibel bestimmte Klassen, Ursachen, Parameter oder zukünftige Beobachtungen unter den gegebenen Daten sind.

Nach diesem Kapitel solltest du folgende Fragen beantworten können:

- Was ist eine Zufallsvariable (random variable)?
- Was ist eine Wahrscheinlichkeitsverteilung (probability distribution)?
- Was ist der Unterschied zwischen einer gemeinsamen Wahrscheinlichkeit (joint probability), einer Randwahrscheinlichkeit (marginal probability) und einer bedingten Wahrscheinlichkeit (conditional probability)?
- Was sagt die Summenregel (sum rule)?
- Was sagt die Produktregel (product rule)?
- Wie wird die Bayes-Regel (Bayes' rule) hergeleitet?
- Was ist der Unterschied zwischen Prior, Likelihood und Posterior?
- Warum ist Bayes' Regel für maschinelles Lernen so wichtig?

2.2 Zufallsexperimente und Ergebnisse

Ein Zufallsexperiment (random experiment) ist ein Vorgang, dessen Ergebnis vor der Durchführung nicht sicher bekannt ist. Trotzdem kennt man meistens die Menge der möglichen Ergebnisse.

Beispiel

Beispiele für Zufallsexperimente:

- Ein Würfel wird geworfen.
- Eine Münze wird geworfen.
- Aus einer Schüssel wird zufällig eine Frucht gezogen.
- Ein zufälliges Bild aus einem Datensatz wird betrachtet.
- Ein Messwert wird unter Rauschen (noise) aufgenommen.

Die Menge aller möglichen Ergebnisse nennt man Ergebnisraum (sample space). Oft wird er mit Ω bezeichnet.

Definition

Der Ergebnisraum (sample space) Ω ist die Menge aller möglichen Ergebnisse eines Zufallsexperiments.

Ein einzelnes mögliches Ergebnis nennt man Elementarereignis (elementary event).

Beim Würfeln ist der Ergebnisraum zum Beispiel

$$\Omega = \{1,2,3,4,5,6\}.$$

Beim Ziehen einer Frucht könnte der Ergebnisraum sein

$$\Omega = \{\text{Orange, wei\ss e Beere}\}.$$

Ein Ereignis (event) ist eine Teilmenge des Ergebnisraums. Zum Beispiel ist beim Würfeln das Ereignis, eine gerade Zahl zu würfeln,

$$A = \{2,4,6\}.$$

Hinweis

In der Wahrscheinlichkeitstheorie unterscheidet man zwischen einem konkreten Ergebnis und einem Ereignis.

Ein konkretes Ergebnis wäre zum Beispiel $X = 4$. Ein Ereignis kann mehrere Ergebnisse enthalten, zum Beispiel $X \in \{2,4,6\}$.

2.3 Zufallsvariablen

In der Mathematik arbeitet man selten direkt mit den rohen Ergebnissen eines Experiments. Stattdessen verwendet man Zufallsvariablen (random variables).

Definition

Eine Zufallsvariable (random variable) ist eine Variable, deren Wert vom Ausgang eines Zufallsexperiments abhängt.

Eine Zufallsvariable wird meistens mit einem Großbuchstaben geschrieben, zum Beispiel X . Ein konkreter Wert der Zufallsvariable wird meistens mit einem Kleinbuchstaben geschrieben, zum Beispiel x .

Beispiel:

X = Augenzahl eines Würfels.

Dann kann X einen der Werte

$$x \in \{1, 2, 3, 4, 5, 6\}$$

annehmen.

Die Menge aller möglichen Werte von X wird oft mit Ω_x bezeichnet:

$$\Omega_x = \{1, 2, 3, 4, 5, 6\}.$$

Hinweis

Großbuchstaben wie X , Y , S bezeichnen Zufallsvariablen. Kleinbuchstaben wie x , y , s bezeichnen konkrete Werte dieser Zufallsvariablen.

Streng wäre also:

$$p(X = x).$$

In vielen ML-Texten schreibt man aber kürzer:

$$p(x).$$

Das bedeutet dann: die Wahrscheinlichkeit, dass die Zufallsvariable X den Wert x annimmt.

2.4 Diskrete Wahrscheinlichkeitsverteilungen

In diesem Kapitel betrachten wir zunächst diskrete Zufallsvariablen (discrete random variables). Eine diskrete Zufallsvariable kann nur endlich viele oder abzählbar viele Werte annehmen.

Definition

Eine diskrete Wahrscheinlichkeitsverteilung (discrete probability distribution) ordnet jedem möglichen Wert $x \in \Omega_x$ eine Wahrscheinlichkeit $p(X = x)$ zu.

Für eine diskrete Wahrscheinlichkeitsverteilung müssen zwei Bedingungen gelten:

$$p(X = x) \geq 0 \quad \text{für alle } x \in \Omega_x,$$

und

$$\sum_{x \in \Omega_x} p(X = x) = 1.$$

Die erste Bedingung sagt: Wahrscheinlichkeiten können nicht negativ sein.

Die zweite Bedingung sagt: Irgendein möglicher Wert muss eintreten. Die Gesamtwahrscheinlichkeit über alle Möglichkeiten ist daher 1.

Beispiel

Bei einem fairen Würfel gilt:

$$p(X = 1) = p(X = 2) = \dots = p(X = 6) = \frac{1}{6}.$$

Damit ist

$$\sum_{x=1}^6 p(X = x) = 6 \cdot \frac{1}{6} = 1.$$

2.5 Wahrscheinlichkeit als relative Häufigkeit

In Datenanwendungen entsteht Wahrscheinlichkeit oft aus beobachteten Häufigkeiten. Wenn ein Ereignis A in N Experimenten insgesamt $n(A)$ -mal beobachtet wird, dann kann man die Wahrscheinlichkeit durch die relative Häufigkeit schätzen:

$$p(A) \approx \frac{n(A)}{N}.$$

Idee

In maschinellem Lernen sehen wir Wahrscheinlichkeiten oft aus zwei Perspektiven:

- als theoretische Größen eines Modells,
- als Größen, die aus Daten geschätzt werden.

Der Übergang von Daten zu Wahrscheinlichkeiten ist eine zentrale Idee statistischen Lernens.

Beispiel

Wenn in einem Datensatz mit $N = 1000$ Bildern genau $n(c = 7) = 93$ Bilder die Ziffer 7 zeigen, dann kann man schätzen:

$$p(c = 7) \approx \frac{93}{1000} = 0,093.$$

2.6 Das Schlüssel-Frucht-Beispiel

Wir betrachten ein einfaches Beispiel, das die wichtigsten Regeln der Wahrscheinlichkeit motiviert.

Es gibt zwei Schlüssel:

$$s \in \{A, B\}.$$

Die Zufallsvariable S beschreibt, aus welcher Schlüssel gezogen wurde. Außerdem gibt es zwei Fruchttypen:

$$y \in \{W, O\}.$$

Dabei steht W für weiße Beere und O für Orange. Die Zufallsvariable Y beschreibt, welche Frucht gezogen wurde.

Der Ablauf des Experiments ist:

1. Zuerst wird zufällig eine Schlüssel gewählt.
2. Danach wird zufällig eine Frucht aus dieser Schlüssel gezogen.
3. Dann wird die Frucht beobachtet.

Hinweis

Dieses Beispiel ist für maschinelles Lernen wichtig, weil es eine einfache Version eines typischen Inferenzproblems zeigt:

Ursache $\longrightarrow$ Beobachtung.

Die Schlüssel S ist die unbekannte Ursache. Die Frucht Y ist die beobachtete Information. Oft interessiert uns aber die umgekehrte Richtung:

Beobachtung $\longrightarrow$ wahrscheinliche Ursache.

2.7 Gemeinsame Wahrscheinlichkeiten

Eine gemeinsame Wahrscheinlichkeit (joint probability) beschreibt die Wahrscheinlichkeit, dass mehrere Aussagen gleichzeitig eintreten.

Definition

Die gemeinsame Wahrscheinlichkeit (joint probability)

$$p(S = s, Y = y)$$

ist die Wahrscheinlichkeit, dass gleichzeitig $S = s$ und $Y = y$ gilt.

Im Schlüssel-Frucht-Beispiel bedeutet

$$p(S = A, Y = W)$$

die Wahrscheinlichkeit, dass aus Schüssel A gezogen wurde und die gezogene Frucht eine weiße Beere ist.

Oft schreibt man kürzer:

$$p(s, y)$$

statt

$$p(S = s, Y = y).$$

Hinweis

Die Schreibweise $p(s, y)$ ist eine Abkürzung. Sie bedeutet nicht, dass s und y selbst Zufallsvariablen sind. Eigentlich sind s und y konkrete Werte der Zufallsvariablen S und Y .

2.7.1 Gemeinsame Wahrscheinlichkeit als Tabelle

Für zwei diskrete Zufallsvariablen kann man die gemeinsame Verteilung als Tabelle darstellen.

	$Y = W$	$Y = O$	$p(S = s)$
$S = A$	$p(A, W)$	$p(A, O)$	$p(A)$
$S = B$	$p(B, W)$	$p(B, O)$	$p(B)$
$p(Y = y)$	$p(W)$	$p(O)$	1

Die inneren Einträge sind gemeinsame Wahrscheinlichkeiten. Die Randwerte sind Randwahrscheinlichkeiten (marginal probabilities).

2.8 Randwahrscheinlichkeiten und Summenregel

Eine Randwahrscheinlichkeit (marginal probability) beschreibt die Wahrscheinlichkeit einer Variable, ohne die andere Variable festzulegen.

Zum Beispiel fragt

$$p(S = A)$$

nur: Wie wahrscheinlich ist es, dass Schüssel A gewählt wurde?

Dabei ist egal, welche Frucht gezogen wurde. Die Frucht kann entweder W oder O sein. Daher gilt:

$$p(S = A) = p(S = A, Y = W) + p(S = A, Y = O).$$

Das ist ein Spezialfall der Summenregel (sum rule).

Definition

Für zwei diskrete Zufallsvariablen X und Y lautet die Summenregel (sum rule):

$$p(X = x) = \sum_{y \in \Omega_y} p(X = x, Y = y).$$

In Kurzschreibweise:

$$p(x) = \sum_y p(x, y).$$

Analog gilt:

$$p(Y = y) = \sum_{x \in \Omega_x} p(X = x, Y = y).$$

In Kurzschreibweise:

$$p(y) = \sum_x p(x, y).$$

Idee

Die Summenregel (sum rule) bedeutet: Wenn eine Variable nicht interessiert, summiert man über alle ihre möglichen Werte.

Dieses Heraussummieren nennt man Marginalisierung (marginalization).

Beispiel

Im Schlüssel-Frucht-Beispiel gilt:

$$p(Y = O) = p(S = A, Y = O) + p(S = B, Y = O).$$

Die Wahrscheinlichkeit für eine Orange ist also die Summe der Wahrscheinlichkeiten aller Wege, die zu einer Orange führen.

2.9 Bedingte Wahrscheinlichkeiten

Eine bedingte Wahrscheinlichkeit (conditional probability) beschreibt, wie wahrscheinlich ein Ereignis ist, wenn eine bestimmte Information bereits bekannt ist.

Definition

Die bedingte Wahrscheinlichkeit (conditional probability)

$$p(Y = y \mid X = x)$$

ist die Wahrscheinlichkeit, dass $Y = y$ gilt, unter der Bedingung, dass $X = x$ bereits bekannt ist.

Man liest

$$p(Y = y \mid X = x)$$

als:

Wahrscheinlichkeit von $Y = y$ gegeben $X = x$.

Im Schlüssel-Frucht-Beispiel bedeutet

$$p(Y = O \mid S = B)$$

die Wahrscheinlichkeit, eine Orange zu ziehen, wenn bereits bekannt ist, dass aus Schlüssel B gezogen wurde.

2.9.1 Definition der bedingten Wahrscheinlichkeit

Die bedingte Wahrscheinlichkeit wird über gemeinsame und marginale Wahrscheinlichkeiten definiert:

$$p(Y = y \mid X = x) = \frac{p(X = x, Y = y)}{p(X = x)}$$

für $p(X = x) > 0$.

In Kurzschreibweise:

$$p(y \mid x) = \frac{p(x, y)}{p(x)}.$$

Hinweis

Die Bedingung $p(X = x) > 0$ ist notwendig, weil man nicht durch 0 teilen darf. Außerdem kann man nicht sinnvoll auf ein Ereignis konditionieren, das unmöglich ist.

2.10 Produktregel

Aus der Definition der bedingten Wahrscheinlichkeit folgt direkt die Produktregel (product rule).

Aus

$$p(y \mid x) = \frac{p(x, y)}{p(x)}$$

folgt durch Multiplikation mit $p(x)$:

$$p(x, y) = p(y \mid x)p(x).$$

Da die gemeinsame Wahrscheinlichkeit symmetrisch ist,

$$p(x,y) = p(y,x),$$

gilt auch

$$p(x,y) = p(x | y)p(y).$$

Definition

Für zwei diskrete Zufallsvariablen X und Y lautet die Produktregel (product rule):

$$p(x,y) = p(y | x)p(x) = p(x | y)p(y).$$

Idee

Die Produktregel (product rule) bedeutet:

gemeinsame Wahrscheinlichkeit = bedingte Wahrscheinlichkeit · Randwahrscheinlichkeit.

Sie beschreibt also, wie man eine Wahrscheinlichkeit für zwei gemeinsam eintretende Ereignisse zerlegen kann.

Beispiel

Im Schlüssel-Frucht-Beispiel gilt:

$$p(S = A, Y = W) = p(Y = W | S = A)p(S = A).$$

Man liest das als:

Die Wahrscheinlichkeit, Schlüssel A zu wählen und eine weiße Beere zu ziehen, ist die Wahrscheinlichkeit für Schlüssel A , multipliziert mit der Wahrscheinlichkeit für eine weiße Beere unter der Bedingung Schlüssel A .

2.11 Summenregel und Produktregel zusammen

Die Summenregel und die Produktregel sind zwei der wichtigsten Regeln im gesamten maschinellen Lernen.

$$p(x) = \sum_y p(x,y)$$

und

$$p(x,y) = p(y | x)p(x).$$

Setzt man die Produktregel in die Summenregel ein, erhält man:

$$p(x) = \sum_y p(x | y)p(y).$$

Diese Gleichung ist sehr wichtig. Sie sagt: Die Wahrscheinlichkeit von x kann berechnet werden, indem man über alle möglichen Ursachen y summiert.

Idee

Die Gleichung

$$p(x) = \sum_y p(x | y)p(y)$$

heißt oft totale Wahrscheinlichkeit (law of total probability). Sie ist eine Kombination aus Summenregel und Produktregel.

2.12 Bayes-Regel

Die Bayes-Regel (Bayes' rule) erlaubt es, bedingte Wahrscheinlichkeiten umzudrehen.

Oft kennen wir

$$p(y | x),$$

brauchen aber

$$p(x | y).$$

Oder umgekehrt.

2.12.1 Herleitung der Bayes-Regel

Wir starten mit der Produktregel:

$$p(x,y) = p(y | x)p(x).$$

Gleichzeitig gilt:

$$p(x,y) = p(x | y)p(y).$$

Da beide rechten Seiten dieselbe gemeinsame Wahrscheinlichkeit $p(x,y)$ beschreiben, setzen wir sie gleich:

$$p(y | x)p(x) = p(x | y)p(y).$$

Nun teilen wir durch $p(x)$, sofern $p(x) > 0$:

$$p(y | x) = \frac{p(x | y)p(y)}{p(x)}.$$

Damit erhalten wir die Bayes-Regel.

Definition

Die Bayes-Regel (Bayes' rule) lautet:

$$p(y | x) = \frac{p(x | y)p(y)}{p(x)}.$$

Mit der Summenregel kann der Nenner geschrieben werden als:

$$p(x) = \sum_{y'} p(x | y')p(y').$$

Daher gilt auch:

$$p(y | x) = \frac{p(x | y)p(y)}{\sum_{y'} p(x | y')p(y')}.$$

Hinweis

Im Nenner verwendet man oft y' statt y , weil y im Zähler bereits der konkrete Wert ist, für den man die Wahrscheinlichkeit berechnen möchte. Die Variable y' ist nur eine Summationsvariable.

2.13 Prior, Likelihood, Evidence und Posterior

Die Bayes-Regel hat im maschinellen Lernen eine spezielle Interpretation:

$$p(y | x) = \frac{p(x | y)p(y)}{p(x)}.$$

Die einzelnen Teile haben Namen:

$$\underbrace{p(y | x)}_{\text{Posterior}} = \frac{\underbrace{p(x | y)}_{\text{Likelihood}} \underbrace{p(y)}_{\text{Prior}}}{\underbrace{p(x)}_{\text{Evidence}}}.$$

Definition

Der Prior $p(y)$ ist die Wahrscheinlichkeit von y , bevor die Beobachtung x berücksichtigt wird.

Die Likelihood $p(x | y)$ beschreibt, wie plausibel die Beobachtung x wäre, wenn y gelten würde.

Die Evidence $p(x)$ ist die Gesamtwahrscheinlichkeit der Beobachtung x .

Der Posterior $p(y | x)$ ist die aktualisierte Wahrscheinlichkeit von y , nachdem x beobachtet wurde.

Idee

Bayes' Regel beschreibt Lernen als Aktualisierung von Wahrscheinlichkeiten:

$$\text{Posterior} \propto \text{Likelihood} \cdot \text{Prior}.$$

Das Zeichen $\propto$ bedeutet proportional zu. Der Nenner $p(x)$ sorgt dafür, dass die Posterior-Wahrscheinlichkeiten wieder zu 1 summieren.

2.14 Bayes-Regel im Schlüssel-Frucht-Beispiel

Wir wollen wissen:

Wenn eine Orange beobachtet wurde, wie wahrscheinlich ist es, dass aus Schlüssel B gezogen wurde?

Mathematisch suchen wir:

$$p(S = B \mid Y = O).$$

Angenommen, die Schlüssel werden so gewählt:

$$p(S = A) = \frac{2}{3}, \quad p(S = B) = \frac{1}{3}.$$

Außerdem gelten die bedingten Wahrscheinlichkeiten:

$$p(Y = O \mid S = A) = \frac{1}{3}, \quad p(Y = O \mid S = B) = \frac{3}{4}.$$

Dann liefert Bayes' Regel:

$$p(S = B \mid Y = O) = \frac{p(Y = O \mid S = B)p(S = B)}{p(Y = O \mid S = A)p(S = A) + p(Y = O \mid S = B)p(S = B)}.$$

Einsetzen ergibt:

$$p(S = B \mid Y = O) = \frac{\frac{3}{4} \cdot \frac{1}{3}}{\frac{1}{3} \cdot \frac{2}{3} + \frac{3}{4} \cdot \frac{1}{3}}.$$

Der Zähler ist:

$$\frac{3}{4} \cdot \frac{1}{3} = \frac{3}{12} = \frac{1}{4}.$$

Der Nenner ist:

$$\frac{1}{3} \cdot \frac{2}{3} + \frac{3}{4} \cdot \frac{1}{3} = \frac{2}{9} + \frac{1}{4}.$$

Auf gemeinsamen Nenner gebracht:

$$\frac{2}{9} + \frac{1}{4} = \frac{8}{36} + \frac{9}{36} = \frac{17}{36}.$$

Also:

$$p(S = B \mid Y = O) = \frac{\frac{1}{4}}{\frac{17}{36}} = \frac{1}{4} \cdot \frac{36}{17} = \frac{9}{17} \approx 0,529.$$

Damit gilt:

$$p(S = B \mid Y = O) \approx 53\%.$$

Beispiel

Vor dem Beobachten der Frucht war die Wahrscheinlichkeit für Schlüssel B

$$p(S = B) = \frac{1}{3} \approx 33\%.$$

Nach dem Beobachten einer Orange ist sie

$$p(S = B \mid Y = O) = \frac{9}{17} \approx 53\%.$$

Die Beobachtung $Y = O$ hat unsere Einschätzung also verändert.

2.15 Was bedeutet Bayes-Regel intuitiv?

Bayes' Regel verbindet drei Ideen:

1. Wie plausibel war die Ursache vor der Beobachtung?
2. Wie gut erklärt diese Ursache die Beobachtung?
3. Wie plausibel ist die Ursache nach der Beobachtung?

Im Schlüssel-Frucht-Beispiel:

- Der Prior $p(S = B)$ sagt, wie wahrscheinlich Schlüssel B vor der Beobachtung war.
- Die Likelihood $p(Y = O \mid S = B)$ sagt, wie gut Schlüssel B die Orange erklärt.
- Der Posterior $p(S = B \mid Y = O)$ sagt, wie wahrscheinlich Schlüssel B nach der Beobachtung ist.

Idee

Bayes' Regel ist im maschinellen Lernen so wichtig, weil viele Probleme genau diese Struktur haben:

unbekannte Klasse oder Ursache $\longrightarrow$ beobachtete Daten.

Wir beobachten aber die Daten und wollen auf die Klasse oder Ursache schließen:

beobachtete Daten $\longrightarrow$ wahrscheinliche Klasse oder Ursache.

2.16 Bayes-Regel für Klassifikation

In einem Klassifikationsproblem ist c eine Klasse und $\vec{x}$ eine beobachtete Eingabe. Gesucht ist oft:

$$p(c \mid \vec{x}).$$

Das ist die Wahrscheinlichkeit der Klasse c , nachdem die Eingabe $\vec{x}$ beobachtet wurde. Mit Bayes' Regel gilt:

$$p(c \mid \vec{x}) = \frac{p(\vec{x} \mid c)p(c)}{p(\vec{x})}.$$

Der Nenner ist:

$$p(\vec{x}) = \sum_{c'} p(\vec{x} \mid c')p(c').$$

Also:

$$p(c \mid \vec{x}) = \frac{p(\vec{x} \mid c)p(c)}{\sum_{c'} p(\vec{x} \mid c')p(c')}.$$

Definition

In der Klassifikation bedeutet:

$$p(c)$$

Prior-Wahrscheinlichkeit der Klasse c ,

$$p(\vec{x} \mid c)$$

Likelihood der Eingabe $\vec{x}$ unter Klasse c ,

$$p(c \mid \vec{x})$$

Posterior-Wahrscheinlichkeit der Klasse c nach Beobachtung von $\vec{x}$.

Eine typische Entscheidungsregel ist:

$$\hat{c} = \arg \max_c p(c \mid \vec{x}).$$

Das bedeutet: Wir wählen die Klasse mit der höchsten Posterior-Wahrscheinlichkeit.

Prüfungshinweis

Für Klassifikationsaufgaben ist wichtig:

$$p(c \mid \vec{x})$$

ist meistens die Größe, die wir für die Entscheidung brauchen.
Bayes' Regel erlaubt es, diese Größe aus

$$p(\vec{x} \mid c) \quad \text{und} \quad p(c)$$

zu berechnen.

2.17 Unabhängigkeit

Zwei Zufallsvariablen sind unabhängig (independent), wenn die Information über die eine Variable nichts an der Wahrscheinlichkeit der anderen Variable ändert.

Definition

Zwei diskrete Zufallsvariablen X und Y heißen unabhängig (independent), wenn für alle $x \in \Omega_x$ und $y \in \Omega_y$ gilt:

$$p(x,y) = p(x)p(y).$$

Aus der Produktregel

$$p(x,y) = p(y \mid x)p(x)$$

und der Unabhängigkeit

$$p(x,y) = p(x)p(y)$$

folgt für $p(x) > 0$:

$$p(y \mid x) = p(y).$$

Das bedeutet: Wenn X und Y unabhängig sind, dann verändert das Wissen über $X = x$ die Wahrscheinlichkeit von $Y = y$ nicht.

Idee

Unabhängigkeit bedeutet:

$$p(y \mid x) = p(y).$$

Abhängigkeit bedeutet:

$$p(y \mid x) \neq p(y).$$

Beispiel

Wenn in beiden Schüsseln der gleiche Anteil an Orangen liegt, dann sagt die beobachtete Frucht wenig oder nichts darüber aus, aus welcher Schüssel gezogen wurde.
Dann wären Schüssel und Fruchttyp unabhängig oder näherungsweise unabhängig.

2.18 Bedingte Unabhängigkeit

In maschinellern Lernen ist bedingte Unabhängigkeit (conditional independence) oft wichtiger als normale Unabhängigkeit.

Definition

Zwei Zufallsvariablen X und Y heißen bedingt unabhängig gegeben Z , wenn gilt:

$$p(x, y | z) = p(x | z)p(y | z).$$

Man schreibt manchmal:

$$X \perp Y | Z.$$

Das bedeutet: Wenn Z bekannt ist, liefert X keine zusätzliche Information über Y .

Hinweis

Bedingte Unabhängigkeit ist eine der wichtigsten Ideen hinter probabilistischen graphischen Modellen, Naive Bayes, Hidden Markov Models und vielen generativen Modellen. In diesem Kapitel reicht es, die Grundidee zu kennen.

2.19 Kettenregel für Wahrscheinlichkeiten

Die Produktregel kann auf mehr als zwei Variablen erweitert werden. Für drei Variablen gilt:

$$p(x, y, z) = p(x | y, z)p(y, z).$$

Dann zerlegen wir $p(y, z)$ wieder mit der Produktregel:

$$p(y, z) = p(y | z)p(z).$$

Damit erhalten wir:

$$p(x, y, z) = p(x | y, z)p(y | z)p(z).$$

Definition

Die Kettenregel (chain rule) der Wahrscheinlichkeit lautet für mehrere Zufallsvariablen:

$$p(x_1, x_2, \dots, x_D) = p(x_1 | x_2, \dots, x_D)p(x_2 | x_3, \dots, x_D) \cdots p(x_{D-1} | x_D)p(x_D).$$

Eine alternative Reihenfolge ist genauso möglich:

$$p(x_1, x_2, \dots, x_D) = p(x_D \mid x_1, \dots, x_{D-1}) \cdots p(x_2 \mid x_1) p(x_1).$$

Hinweis

Die Kettenregel ist keine zusätzliche Annahme. Sie folgt aus wiederholter Anwendung der Produktregel.

2.20 Normalisierung

Eine Wahrscheinlichkeitsverteilung muss immer auf 1 summieren.

Für eine diskrete Zufallsvariable Y gilt:

$$\sum_y p(y) = 1.$$

Auch für eine bedingte Verteilung gilt:

$$\sum_y p(y \mid x) = 1$$

für jedes feste x .

Idee

Der Nenner in Bayes' Regel ist eine Normalisierungskonstante. Er sorgt dafür, dass die Posterior-Verteilung über alle möglichen Werte von y wieder zu 1 summiert.

Wir können das direkt sehen:

$$p(y \mid x) = \frac{p(x \mid y)p(y)}{\sum_{y'} p(x \mid y')p(y')}.$$

Wenn wir über alle y summieren, erhalten wir:

$$\sum_y p(y \mid x) = \sum_y \frac{p(x \mid y)p(y)}{\sum_{y'} p(x \mid y')p(y')}.$$

Der Nenner hängt nicht von y ab, also kann man ihn aus der Summe ziehen:

$$\sum_y p(y \mid x) = \frac{\sum_y p(x \mid y)p(y)}{\sum_{y'} p(x \mid y')p(y')}.$$

Zähler und Nenner sind dieselbe Summe, nur mit anderen Summationsnamen:

$$\sum_y p(y \mid x) = 1.$$

2.21 Häufige Fehler

Prüfungshinweis

Typische Fehler bei Wahrscheinlichkeiten:

1. $p(x | y)$ und $p(y | x)$ verwechseln.
2. Den Nenner in Bayes' Regel vergessen.
3. Bei der Summenregel nicht über alle möglichen Werte zu summieren.
4. Gemeinsame Wahrscheinlichkeit $p(x,y)$ mit bedingter Wahrscheinlichkeit $p(x | y)$ verwechseln.
5. Unabhängigkeit anzunehmen, obwohl sie nicht gegeben ist.
6. Die Kurzschreibweise $p(x)$ zu wörtlich zu nehmen und zu vergessen, dass eigentlich $p(X = x)$ gemeint ist.

2.22 Mini-Aufgaben

2.22.1 Aufgabe 1: Gemeinsame Wahrscheinlichkeit

Angenommen,

$$p(S = A) = 0,6$$

und

$$p(Y = O | S = A) = 0,2.$$

Berechne

$$p(S = A, Y = O).$$

Beispiel

Mit der Produktregel gilt:

$$p(S = A, Y = O) = p(Y = O | S = A)p(S = A).$$

Also:

$$p(S = A, Y = O) = 0,2 \cdot 0,6 = 0,12.$$

2.22.2 Aufgabe 2: Randwahrscheinlichkeit

Angenommen,

$$p(S = A, Y = O) = 0,12$$

und

$$p(S = B, Y = O) = 0,30.$$

Berechne

$$p(Y = O).$$

Beispiel

Mit der Summenregel gilt:

$$p(Y = O) = p(S = A, Y = O) + p(S = B, Y = O).$$

Also:

$$p(Y = O) = 0,12 + 0,30 = 0,42.$$

2.22.3 Aufgabe 3: Bedingte Wahrscheinlichkeit

Angenommen,

$$p(S = A, Y = O) = 0,12$$

und

$$p(S = A) = 0,6.$$

Berechne

$$p(Y = O \mid S = A).$$

Beispiel

Mit der Definition der bedingten Wahrscheinlichkeit gilt:

$$p(Y = O \mid S = A) = \frac{p(S = A, Y = O)}{p(S = A)}.$$

Also:

$$p(Y = O \mid S = A) = \frac{0,12}{0,6} = 0,2.$$

2.23 Zusammenfassung

- Eine Zufallsvariable (random variable) beschreibt eine Größe, deren Wert vom Zufall abhängt.
- Eine diskrete Wahrscheinlichkeitsverteilung ordnet jedem möglichen Wert eine Wahrscheinlichkeit zu.
- Wahrscheinlichkeiten sind nichtnegativ und summieren sich zu 1.
- Eine gemeinsame Wahrscheinlichkeit (joint probability) beschreibt das gleichzeitige Auftreten mehrerer Ereignisse.
- Eine Randwahrscheinlichkeit (marginal probability) erhält man durch Summieren über nicht interessierende Variablen.
- Die Summenregel (sum rule) lautet:

$$p(x) = \sum_y p(x,y).$$

- Eine bedingte Wahrscheinlichkeit (conditional probability) beschreibt Wahrscheinlichkeit unter gegebener Information:

$$p(y \mid x) = \frac{p(x,y)}{p(x)}.$$

- Die Produktregel (product rule) lautet:

$$p(x,y) = p(y \mid x)p(x) = p(x \mid y)p(y).$$

- Die Bayes-Regel (Bayes' rule) lautet:

$$p(y \mid x) = \frac{p(x \mid y)p(y)}{p(x)}.$$

- Prior, Likelihood, Evidence und Posterior sind zentrale Begriffe probabilistischen Lernens.
- Unabhängigkeit bedeutet:

$$p(x,y) = p(x)p(y).$$

2.24 Typische Prüfungsfragen

Prüfungshinweis

Mögliche Prüfungsfragen zu diesem Kapitel:

1. Definiere eine diskrete Zufallsvariable.
2. Was ist der Unterschied zwischen $p(x,y)$, $p(x)$ und $p(y \mid x)$?
3. Erkläre die Summenregel (sum rule).

4. Erkläre die Produktregel (product rule).
5. Leite die Bayes-Regel (Bayes' rule) aus der Produktregel her.
6. Was bedeuten Prior, Likelihood und Posterior?
7. Warum steht im Nenner der Bayes-Regel eine Summe?
8. Was bedeutet Unabhängigkeit zweier Zufallsvariablen?
9. Warum ist Bayes' Regel für Klassifikation nützlich?
10. Berechne in einem Schlüssel-Frucht-Beispiel eine Posterior-Wahrscheinlichkeit.

3 Kontinuierliche Zufallsvariablen

3.1 Ziel dieses Kapitels

Im vorherigen Kapitel wurden diskrete Zufallsvariablen (discrete random variables) betrachtet. Eine diskrete Zufallsvariable kann nur endlich viele oder abzählbar viele Werte annehmen. Viele Größen in Anwendungen sind aber nicht sinnvoll diskret, sondern kontinuierlich (continuous).

Beispiele sind:

- Körpergröße,
- Temperatur,
- Zeit,
- Geschwindigkeit,
- Position auf einer Linie,
- Messwerte mit Rauschen (noise),
- Pixelintensitäten, wenn sie als reelle Werte modelliert werden.

In diesem Kapitel lernen wir, wie man solche Größen mit Wahrscheinlichkeitsdichten (probability density functions) beschreibt.

Idee

Bei diskreten Zufallsvariablen ordnet man einzelnen Werten Wahrscheinlichkeiten zu. Bei kontinuierlichen Zufallsvariablen ordnet man einzelnen Werten keine direkten Wahrscheinlichkeiten zu. Stattdessen verwendet man eine Wahrscheinlichkeitsdichte (probability density function), die über Intervalle integriert wird.

Nach diesem Kapitel solltest du folgende Fragen beantworten können:

- Was ist eine kontinuierliche Zufallsvariable (continuous random variable)?
- Warum ist $p(X = x)$ bei kontinuierlichen Zufallsvariablen meistens 0?
- Was ist eine Wahrscheinlichkeitsdichte (probability density function)?
- Wie berechnet man Wahrscheinlichkeiten über Intervalle?
- Welche Bedingungen muss eine Wahrscheinlichkeitsdichte erfüllen?
- Wie funktionieren Summenregel, Produktregel und Bayes-Regel für Dichten?

- Was ist ein Histogramm (histogram) und wie hängt es mit Dichteschätzung zusammen?
- Was sind Erwartungswert (expectation), Mittelwert (mean), Varianz (variance), Kovarianz (covariance) und Korrelation (correlation)?
- Was bedeutet Sampling-Approximation (sampling approximation)?

3.2 Von diskreten zu kontinuierlichen Zufallsvariablen

Eine diskrete Zufallsvariable X kann nur einzelne Werte annehmen. Zum Beispiel beim Würfeln:

$$X \in \{1,2,3,4,5,6\}.$$

Dann ist eine Wahrscheinlichkeit wie

$$p(X = 3)$$

sinnvoll und kann zum Beispiel den Wert

$$p(X = 3) = \frac{1}{6}$$

haben.

Bei einer kontinuierlichen Zufallsvariable ist die Situation anders. Angenommen, X beschreibt eine Position auf einer Linie:

$$X \in \mathbb{R}.$$

Dann gibt es unendlich viele mögliche Werte. Die Wahrscheinlichkeit, genau einen einzelnen Wert zu treffen, ist in typischen kontinuierlichen Modellen gleich 0:

$$p(X = x) = 0.$$

Das bedeutet aber nicht, dass nichts passieren kann. Sinnvoll sind Wahrscheinlichkeiten für Intervalle:

$$p(X \in [a,b]).$$

Beispiel

Wenn X die Körpergröße einer zufällig ausgewählten Person in Zentimetern beschreibt, ist die Wahrscheinlichkeit, exakt

$$X = 178,0000000000 \dots$$

zu erhalten, praktisch 0.

Sinnvoll ist dagegen eine Frage wie:

$$p(175 \leq X \leq 180).$$

Hinweis

Bei kontinuierlichen Zufallsvariablen sind einzelne Punkte normalerweise nicht wichtig. Wichtig sind Bereiche, Intervalle oder allgemeiner Mengen.

3.3 Wahrscheinlichkeitsdichten

Für kontinuierliche Zufallsvariablen verwendet man Wahrscheinlichkeitsdichten (probability density functions). Eine Wahrscheinlichkeitsdichte beschreibt nicht direkt die Wahrscheinlichkeit eines einzelnen Wertes, sondern wie stark Wahrscheinlichkeit in einem Bereich konzentriert ist.

Definition

Eine Wahrscheinlichkeitsdichte (probability density function) einer kontinuierlichen Zufallsvariable X ist eine Funktion

$$p : \Omega_x \rightarrow \mathbb{R},$$

wobei Ω_x der Wertebereich von X ist.

Für eine eindimensionale kontinuierliche Zufallsvariable schreibt man meistens $p(x)$.

Eine Dichte $p(x)$ muss zwei wichtige Eigenschaften erfüllen:

$$p(x) \geq 0 \quad \text{für alle } x \in \Omega_x,$$

und

$$\int_{x \in \Omega_x} p(x) dx = 1.$$

Falls $\Omega_x = \mathbb{R}$, schreibt man:

$$\int_{-\infty}^{\infty} p(x) dx = 1.$$

Idee

Eine Wahrscheinlichkeitsdichte muss nicht kleiner oder gleich 1 sein. Wichtig ist nicht die Höhe der Dichte an einem Punkt, sondern die Fläche unter der Dichte. Diese gesamte Fläche muss 1 ergeben.

3.4 Wahrscheinlichkeiten als Integrale

Bei kontinuierlichen Zufallsvariablen erhält man Wahrscheinlichkeiten durch Integration der Dichte.

Definition

Für eine kontinuierliche Zufallsvariable X mit Dichte $p(x)$ gilt:

$$p(X \in [a,b]) = \int_a^b p(x) dx.$$

Die Wahrscheinlichkeit, dass X im Intervall $[a,b]$ liegt, ist also die Fläche unter der Dichte zwischen a und b .

Beispiel

Wenn $p(x)$ die Dichte einer Messgröße ist, dann bedeutet

$$\int_2^5 p(x) dx$$

die Wahrscheinlichkeit, dass der Messwert zwischen 2 und 5 liegt.

Hinweis

Für kontinuierliche Zufallsvariablen ist es meistens egal, ob man

$$[a,b], \quad (a,b), \quad [a,b), \quad (a,b]$$

verwendet. Die Randpunkte haben Wahrscheinlichkeit 0. Daher gilt typischerweise:

$$p(X \in [a,b]) = p(X \in (a,b)).$$

3.5 Die Gleichverteilung

Ein einfaches Beispiel für eine kontinuierliche Verteilung ist die Gleichverteilung (uniform distribution).

Definition

Eine kontinuierliche Zufallsvariable X ist gleichverteilt auf dem Intervall $[\alpha, \beta]$, wenn ihre Dichte gegeben ist durch

$$p(x \mid \alpha, \beta) = \begin{cases} \frac{1}{\beta - \alpha}, & x \in [\alpha, \beta], \\ 0, & \text{sonst.} \end{cases}$$

Dabei gilt $\alpha < \beta$.

Die Dichte ist auf dem Intervall konstant. Außerhalb des Intervalls ist sie 0.

Die Normalisierung stimmt, denn:

$$\int_{-\infty}^{\infty} p(x \mid \alpha, \beta) dx = \int_{\alpha}^{\beta} \frac{1}{\beta - \alpha} dx.$$

Da $\frac{1}{\beta - \alpha}$ konstant ist, gilt:

$$\int_{\alpha}^{\beta} \frac{1}{\beta - \alpha} dx = \frac{1}{\beta - \alpha} \int_{\alpha}^{\beta} 1 dx = \frac{1}{\beta - \alpha} (\beta - \alpha) = 1.$$

Beispiel

Für eine Gleichverteilung auf $[0,1]$ gilt:

$$p(x) = \begin{cases} 1, & x \in [0,1], \\ 0, & \text{sonst.} \end{cases}$$

Dann ist zum Beispiel:

$$p(X \in [0,2,0,7]) = \int_{0,2}^{0,7} 1 dx = 0,5.$$

3.6 Erwartungswert

Der Erwartungswert (expectation) ist eine der wichtigsten Größen in der Wahrscheinlichkeitstheorie. Er beschreibt den durchschnittlich erwarteten Wert einer Zufallsvariable oder einer Funktion einer Zufallsvariable.

Definition

Sei X eine kontinuierliche Zufallsvariable mit Dichte $p(x)$. Für eine Funktion $f(X)$ ist der Erwartungswert (expectation) definiert als

$$\langle f(X) \rangle_p = \int_{x \in \Omega_x} p(x) f(x) dx.$$

Alternativ schreibt man oft:

$$\mathbb{E}_p[f(X)] = \int_{x \in \Omega_x} p(x) f(x) dx.$$

Wenn klar ist, welche Verteilung gemeint ist, schreibt man auch einfach:

$$\mathbb{E}[f(X)].$$

Hinweis

Die Notationen

$$\langle f(X) \rangle_p \quad \text{und} \quad \mathbb{E}_p[f(X)]$$

bedeuten dasselbe: Erwartungswert von $f(X)$ bezüglich der Verteilung p .

3.7 Mittelwert

Der Mittelwert (mean) ist der Erwartungswert der Identitätsfunktion. Das heißt, man setzt

$$f(X) = X.$$

Definition

Der Mittelwert (mean) einer kontinuierlichen Zufallsvariable X mit Dichte $p(x)$ ist

$$\mu = \mathbb{E}[X] = \int_{x \in \Omega_x} p(x)x \, dx.$$

Für $\Omega_x = \mathbb{R}$ schreibt man:

$$\mu = \int_{-\infty}^{\infty} p(x)x \, dx.$$

Beispiel

Für die Gleichverteilung auf $[\alpha, \beta]$ gilt:

$$p(x \mid \alpha, \beta) = \frac{1}{\beta - \alpha} \quad \text{für } x \in [\alpha, \beta].$$

Der Mittelwert ist:

$$\mathbb{E}[X] = \int_{\alpha}^{\beta} \frac{1}{\beta - \alpha} x \, dx.$$

Also:

$$\mathbb{E}[X] = \frac{1}{\beta - \alpha} \int_{\alpha}^{\beta} x \, dx.$$

Mit

$$\int_{\alpha}^{\beta} x \, dx = \left[\frac{x^2}{2} \right]_{\alpha}^{\beta} = \frac{\beta^2 - \alpha^2}{2}$$

folgt:

$$\mathbb{E}[X] = \frac{1}{\beta - \alpha} \frac{\beta^2 - \alpha^2}{2}.$$

Da

$$\beta^2 - \alpha^2 = (\beta - \alpha)(\beta + \alpha)$$

gilt:

$$\mathbb{E}[X] = \frac{\alpha + \beta}{2}.$$

Idee

Der Mittelwert der Gleichverteilung auf $[\alpha, \beta]$ liegt genau in der Mitte des Intervalls:

$$\mathbb{E}[X] = \frac{\alpha + \beta}{2}.$$

3.8 Varianz

Der Mittelwert beschreibt die Lage einer Verteilung. Er sagt aber nicht, wie stark die Werte um diesen Mittelwert streuen. Dafür verwendet man die Varianz (variance).

Definition

Die Varianz (variance) einer kontinuierlichen Zufallsvariable X mit Mittelwert $\mu = \mathbb{E}[X]$ ist

$$\text{Var}(X) = \mathbb{E}[(X - \mu)^2].$$

Als Integral:

$$\text{Var}(X) = \int_{x \in \Omega_x} p(x)(x - \mu)^2 dx.$$

Die Varianz ist immer nichtnegativ:

$$\text{Var}(X) \geq 0.$$

Das liegt daran, dass

$$(x - \mu)^2 \geq 0$$

für alle x gilt.

Idee

Die Varianz misst die mittlere quadratische Abweichung vom Mittelwert.

Große Varianz bedeutet: Die Werte streuen stark.

Kleine Varianz bedeutet: Die Werte liegen nah am Mittelwert.

3.8.1 Alternative Formel für die Varianz

Die Varianz kann auch anders geschrieben werden:

$$\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2.$$

Herleitung:

$$\text{Var}(X) = \mathbb{E}[(X - \mu)^2].$$

Ausmultiplizieren ergibt:

$$(X - \mu)^2 = X^2 - 2\mu X + \mu^2.$$

Also:

$$\text{Var}(X) = \mathbb{E}[X^2 - 2\mu X + \mu^2].$$

Wegen Linearität des Erwartungswerts gilt:

$$\text{Var}(X) = \mathbb{E}[X^2] - 2\mu\mathbb{E}[X] + \mathbb{E}[\mu^2].$$

Da μ konstant ist, gilt:

$$\mathbb{E}[\mu^2] = \mu^2.$$

Außerdem ist

$$\mathbb{E}[X] = \mu.$$

Damit:

$$\text{Var}(X) = \mathbb{E}[X^2] - 2\mu^2 + \mu^2 = \mathbb{E}[X^2] - \mu^2.$$

Also:

$$\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2.$$

Prüfungshinweis

Die Formel

$$\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$$

ist oft praktisch für Rechnungen. Sie ist aber nicht die Definition, sondern eine äquivalente Umformung.

3.9 Standardabweichung

Die Varianz hat quadrierte Einheiten. Wenn X zum Beispiel in Metern gemessen wird, dann hat $\text{Var}(X)$ die Einheit Quadratmeter. Deshalb verwendet man oft die Standardabweichung (standard deviation).

Definition

Die Standardabweichung (standard deviation) ist die Wurzel der Varianz:

$$\sigma = \sqrt{\text{Var}(X)}.$$

Wenn man schreibt

$$\sigma^2 = \text{Var}(X),$$

dann ist σ die Standardabweichung und σ^2 die Varianz.

3.10 Kovarianz

Bisher wurde die Streuung einer einzelnen Zufallsvariable betrachtet. Oft interessiert man sich aber für den Zusammenhang zwischen zwei Zufallsvariablen. Dafür verwendet man die Kovarianz (covariance).

Definition

Seien X und Y zwei Zufallsvariablen mit Mittelwerten

$$\mu_X = \mathbb{E}[X], \quad \mu_Y = \mathbb{E}[Y].$$

Die Kovarianz (covariance) ist definiert als

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)].$$

Für kontinuierliche Zufallsvariablen mit gemeinsamer Dichte $p(x, y)$ gilt:

$$\text{Cov}(X, Y) = \int \int p(x, y)(x - \mu_X)(y - \mu_Y) dx dy.$$

Idee

Die Kovarianz beschreibt, ob zwei Variablen gemeinsam steigen oder fallen.

- $\text{Cov}(X, Y) > 0$: Große Werte von X treten eher mit großen Werten von Y auf.
- $\text{Cov}(X, Y) < 0$: Große Werte von X treten eher mit kleinen Werten von Y auf.
- $\text{Cov}(X, Y) \approx 0$: Kein klarer linearer Zusammenhang.

Eine alternative Formel ist:

$$\text{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y].$$

3.11 Korrelation

Die Kovarianz hängt von den Einheiten der Variablen ab. Um einen einheitenlosen Zusammenhang zu erhalten, verwendet man die Korrelation (correlation).

Definition

Die Korrelation (correlation) von X und Y ist

$$\rho_{X,Y} = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y},$$

wobei σ_X und σ_Y die Standardabweichungen von X und Y sind.

Es gilt:

$$-1 \leq \rho_{X,Y} \leq 1.$$

- $\rho_{X,Y} = 1$: perfekter positiver linearer Zusammenhang.
- $\rho_{X,Y} = -1$: perfekter negativer linearer Zusammenhang.
- $\rho_{X,Y} = 0$: kein linearer Zusammenhang.

Hinweis

Korrelation 0 bedeutet nicht automatisch Unabhängigkeit. Sie bedeutet nur, dass kein linearer Zusammenhang gemessen wird. Nichtlineare Zusammenhänge können trotzdem existieren.

3.12 Gemeinsame Dichten

Bei zwei kontinuierlichen Zufallsvariablen X und Y verwendet man eine gemeinsame Dichte (joint density)

$$p(x,y).$$

Definition

Eine gemeinsame Dichte (joint density) $p(x,y)$ beschreibt die gemeinsame Verteilung zweier kontinuierlicher Zufallsvariablen X und Y .

Sie muss erfüllen:

$$p(x,y) \geq 0$$

und

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} p(x,y) dx dy = 1.$$

Die Wahrscheinlichkeit, dass X in $[a,b]$ und Y in $[c,d]$ liegt, ist:

$$p(X \in [a,b], Y \in [c,d]) = \int_a^b \int_c^d p(x,y) dy dx.$$

Hinweis

Die Reihenfolge der Integrale ist hier nicht wesentlich, solange die Integrationsbereiche korrekt sind. Je nach Konvention schreibt man auch

$$\int_c^d \int_a^b p(x,y) dx dy.$$

3.13 Marginalisierung bei Dichten

Bei diskreten Zufallsvariablen erhält man Randwahrscheinlichkeiten durch Summieren. Bei kontinuierlichen Zufallsvariablen ersetzt man Summen durch Integrale.

Definition

Aus einer gemeinsamen Dichte $p(x,y)$ erhält man die Randdichte (marginal density) von X durch Integration über y :

$$p(x) = \int_{y \in \Omega_y} p(x,y) dy.$$

Analog:

$$p(y) = \int_{x \in \Omega_x} p(x,y) dx.$$

Idee

Diskreter Fall:

$$p(x) = \sum_y p(x,y).$$

Kontinuierlicher Fall:

$$p(x) = \int p(x,y) dy.$$

Die Idee ist dieselbe: Man entfernt eine nicht interessierende Variable durch Summieren oder Integrieren.

3.14 Bedingte Dichten

Auch für kontinuierliche Zufallsvariablen gibt es bedingte Dichten (conditional densities).

Definition

Die bedingte Dichte (conditional density) von Y gegeben $X = x$ ist

$$p(y | x) = \frac{p(x,y)}{p(x)}$$

für $p(x) > 0$.

Daraus folgt die Produktregel für Dichten:

$$p(x, y) = p(y | x)p(x) = p(x | y)p(y).$$

Hinweis

Die Schreibweise sieht genauso aus wie im diskreten Fall. Der Unterschied ist: Bei kontinuierlichen Variablen sind $p(x)$, $p(y)$ und $p(x, y)$ Dichten, keine Punktwahrscheinlichkeiten.

3.15 Bayes-Regel für Dichten

Die Bayes-Regel gilt auch für Wahrscheinlichkeitsdichten. Für kontinuierliches x und diskretes y erhält man:

$$p(y | x) = \frac{p(x | y)p(y)}{\sum_{y'} p(x | y')p(y')}.$$

Wenn auch y kontinuierlich ist, wird die Summe im Nenner durch ein Integral ersetzt:

$$p(y | x) = \frac{p(x | y)p(y)}{\int p(x | y')p(y') dy'}.$$

Definition

Bayes-Regel (Bayes' rule) für kontinuierliche Zufallsvariablen:

$$p(y | x) = \frac{p(x | y)p(y)}{p(x)},$$

wobei

$$p(x) = \int p(x | y')p(y') dy'.$$

Prüfungshinweis

In kontinuierlichen Fällen gilt dieselbe Logik wie im diskreten Fall.

Der wichtigste Unterschied ist:

$$\sum \quad \text{wird ersetzt durch} \quad \int.$$

3.16 Histogramme und Dichteschätzung

In der Praxis kennt man die echte Dichte $p(x)$ oft nicht. Man hat nur Datenpunkte

$$x_1, x_2, \dots, x_N.$$

Eine einfache Methode zur Schätzung einer Dichte ist das Histogramm (histogram).

Definition

Ein Histogramm (histogram) teilt den Wertebereich in Intervalle, sogenannte Bins (bins), und zählt, wie viele Datenpunkte in jedem Bin liegen.

Sei I_i ein Bin. Die Breite des Bins sei

$$|I_i|.$$

Die Anzahl der Datenpunkte im Bin I_i sei

$$n_i.$$

Dann ist die relative Häufigkeit im Bin:

$$\frac{n_i}{N}.$$

Diese relative Häufigkeit approximiert die Wahrscheinlichkeit, dass ein Datenpunkt in I_i liegt:

$$p(X \in I_i) \approx \frac{n_i}{N}.$$

Andererseits gilt für die Dichte:

$$p(X \in I_i) = \int_{I_i} p(x) dx.$$

Wenn das Intervall I_i klein ist und $p(x)$ darin ungefähr konstant ist, dann gilt:

$$\int_{I_i} p(x) dx \approx p(\bar{x}_i)|I_i|,$$

wobei $\bar{x}_i$ der Mittelpunkt des Bins ist. Damit:

$$\frac{n_i}{N} \approx p(\bar{x}_i)|I_i|.$$

Also:

$$p(\bar{x}_i) \approx \frac{n_i}{N|I_i|}.$$

Idee

Die Histogramm-Schätzung einer Dichte lautet:

$$p(x) \approx \frac{n_i}{N|I_i|} \quad \text{für } x \in I_i.$$

Dabei ist n_i die Anzahl der Datenpunkte im Bin I_i , N die Gesamtzahl der Datenpunkte und $|I_i|$ die Binbreite.

Hinweis

Wenn die Bins sehr breit sind, wird die Schätzung grob. Wenn die Bins sehr schmal sind, wird die Schätzung stark verrauscht. Die Wahl der Binbreite ist daher wichtig.

3.17 Sampling-Approximation von Erwartungswerten

Erwartungswerte sind Integrale. Viele Integrale sind analytisch schwer oder gar nicht lösbar. Wenn man aber Stichproben (samples)

$$x_1, x_2, \dots, x_N$$

aus der Verteilung $p(x)$ ziehen kann, kann man Erwartungswerte näherungsweise berechnen.

Definition

Seien $x_1, \dots, x_N$ Stichproben aus einer Verteilung $p(x)$. Dann kann man den Erwartungswert von $f(X)$ approximieren durch

$$\mathbb{E}_p[f(X)] = \int p(x)f(x) dx \approx \frac{1}{N} \sum_{n=1}^N f(x_n).$$

Das nennt man Sampling-Approximation (sampling approximation) oder Monte-Carlo-Approximation (Monte Carlo approximation).

Idee

Die Sampling-Approximation ersetzt ein Integral durch einen Durchschnitt über Stichproben:

$$\int p(x)f(x) dx \approx \frac{1}{N} \sum_{n=1}^N f(x_n).$$

Je größer N ist, desto besser wird die Approximation typischerweise. Die Genauigkeit hängt aber auch von der Varianz der Funktion $f(X)$ ab.

3.18 Beispiel: Numerische Integration mit Sampling

Wir betrachten das Integral

$$\int_{-\infty}^{\infty} e^{-x^2} x^2 dx.$$

Dieses Integral sieht nicht direkt wie ein Erwartungswert aus. Wir können es aber so umformen, dass eine Normalverteilung erscheint.

Die Normalverteilung mit Mittelwert 0 und Varianz $\frac{1}{2}$ hat die Dichte

$$\mathcal{N}\left(x \mid 0, \frac{1}{2}\right) = \frac{1}{\sqrt{\pi}} e^{-x^2}.$$

Daraus folgt:

$$e^{-x^2} = \sqrt{\pi} \mathcal{N}\left(x \mid 0, \frac{1}{2}\right).$$

Damit:

$$\int_{-\infty}^{\infty} e^{-x^2} x^2 dx = \sqrt{\pi} \int_{-\infty}^{\infty} \mathcal{N}\left(x \mid 0, \frac{1}{2}\right) x^2 dx.$$

Das Integral auf der rechten Seite ist ein Erwartungswert:

$$\int_{-\infty}^{\infty} \mathcal{N}\left(x \mid 0, \frac{1}{2}\right) x^2 dx = \mathbb{E}[X^2]$$

für

$$X \sim \mathcal{N}\left(0, \frac{1}{2}\right).$$

Mit Stichproben

$$x_n \sim \mathcal{N}\left(0, \frac{1}{2}\right)$$

gilt daher:

$$\int_{-\infty}^{\infty} e^{-x^2} x^2 dx \approx \frac{\sqrt{\pi}}{N} \sum_{n=1}^N x_n^2.$$

Beispiel

Das Integral

$$\int_{-\infty}^{\infty} e^{-x^2} \cos(x) dx$$

kann analog geschrieben werden als

$$\sqrt{\pi} \int_{-\infty}^{\infty} \mathcal{N}\left(x \mid 0, \frac{1}{2}\right) \cos(x) dx.$$

Mit Stichproben $x_n \sim \mathcal{N}(0, \frac{1}{2})$ erhält man:

$$\int_{-\infty}^{\infty} e^{-x^2} \cos(x) dx \approx \frac{\sqrt{\pi}}{N} \sum_{n=1}^N \cos(x_n).$$

3.19 Analytische Lösung des ersten Integrals

Das Integral

$$\int_{-\infty}^{\infty} e^{-x^2} x^2 dx$$

kann mit der Varianz der Normalverteilung interpretiert werden.

Für

$$X \sim \mathcal{N}\left(0, \frac{1}{2}\right)$$

gilt:

$$\mathbb{E}[X] = 0$$

und

$$\text{Var}(X) = \frac{1}{2}.$$

Da der Mittelwert 0 ist, gilt:

$$\text{Var}(X) = \mathbb{E}[X^2].$$

Also:

$$\mathbb{E}[X^2] = \frac{1}{2}.$$

Daher:

$$\int_{-\infty}^{\infty} e^{-x^2} x^2 dx = \sqrt{\pi} \mathbb{E}[X^2] = \sqrt{\pi} \frac{1}{2}.$$

Somit:

$$\int_{-\infty}^{\infty} e^{-x^2} x^2 dx = \frac{\sqrt{\pi}}{2}.$$

Prüfungshinweis

Die wichtige Idee ist nicht nur das Ergebnis, sondern die Umformung:

$$e^{-x^2} = \sqrt{\pi} \mathcal{N}\left(x \mid 0, \frac{1}{2}\right).$$

Dadurch wird ein Integral zu einem Erwartungswert bezüglich einer bekannten Verteilung.

3.20 Symmetrische Dichten

Eine Dichte ist symmetrisch um μ , wenn links und rechts von μ dieselbe Dichteform liegt.

Definition

Eine Dichte $p(x)$ ist symmetrisch um μ , wenn für alle $x \in \mathbb{R}$ gilt:

$$p(\mu - x) = p(\mu + x).$$

Für eine solche Dichte ist der Erwartungswert gleich dem Symmetriezentrum:

$$\mathbb{E}[X] = \mu.$$

3.20.1 Intuition

Wenn die Dichte um μ symmetrisch ist, dann gibt es zu jeder Abweichung nach rechts eine gleich wahrscheinliche Abweichung nach links. Die Beiträge heben sich im Durchschnitt auf. Deshalb liegt der Mittelwert genau bei μ .

Beispiel

Die Normalverteilung

$$\mathcal{N}(x \mid \mu, \sigma^2)$$

ist symmetrisch um μ . Daher gilt:

$$\mathbb{E}[X] = \mu.$$

3.21 Wichtige Verteilungen

In diesem Abschnitt sammeln wir einige wichtige kontinuierliche Verteilungen. Sie werden später immer wieder verwendet.

3.21.1 Gleichverteilung

Die Gleichverteilung (uniform distribution) auf $[\alpha, \beta]$ ist:

$$p(x \mid \alpha, \beta) = \begin{cases} \frac{1}{\beta - \alpha}, & x \in [\alpha, \beta], \\ 0, & \text{sonst.} \end{cases}$$

Der Mittelwert ist:

$$\mathbb{E}[X] = \frac{\alpha + \beta}{2}.$$

Die Varianz ist:

$$\text{Var}(X) = \frac{(\beta - \alpha)^2}{12}.$$

3.21.2 Normalverteilung

Die Normalverteilung (Gaussian distribution) ist eine der wichtigsten Verteilungen im maschinellen Lernen. In einer Dimension lautet ihre Dichte:

$$\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Dabei ist μ der Mittelwert und σ^2 die Varianz.

$$\mathbb{E}[X] = \mu, \quad \text{Var}(X) = \sigma^2.$$

Idee

Die Normalverteilung ist wichtig, weil sie in vielen Modellen als Rauschmodell (noise model) verwendet wird. Außerdem führt sie bei Maximum-Likelihood-Schätzung oft auf quadratische Fehlerfunktionen.

3.22 Kontinuierliche Zufallsvektoren

Bisher haben wir eindimensionale kontinuierliche Zufallsvariablen betrachtet. In maschinellern Lernen sind Eingaben aber meistens Vektoren:

$$\vec{x} = \begin{pmatrix} x_1 \\ \vdots \\ x_D \end{pmatrix} \in \mathbb{R}^D.$$

Dann spricht man von einem kontinuierlichen Zufallsvektor (continuous random vector).

Definition

Ein kontinuierlicher Zufallsvektor $\vec{X}$ mit Dichte $p(\vec{x})$ erfüllt:

$$p(\vec{x}) \geq 0 \quad \text{für alle } \vec{x} \in \Omega_{\vec{x}},$$

und

$$\int_{\vec{x} \in \Omega_{\vec{x}}} p(\vec{x}) d\vec{x} = 1.$$

Für $\vec{x} \in \mathbb{R}^D$ bedeutet $d\vec{x}$:

$$d\vec{x} = dx_1 dx_2 \cdots dx_D.$$

Hinweis

In Machine Learning schreibt man häufig $p(\vec{x})$ für die Dichte eines Datenvektors. Das ist keine Wahrscheinlichkeit eines einzelnen Vektors, sondern eine Dichte. Wahrscheinlichkeiten erhält man durch Integration über Bereiche im Datenraum.

3.23 Zusammenhang zu maschinellern Lernen

Kontinuierliche Zufallsvariablen treten im maschinellern Lernen an vielen Stellen auf:

- Eingabevektoren $\vec{x}$ werden oft als kontinuierlich modelliert.
- Regressionsziele t sind meistens kontinuierlich.

- Rauschen (noise) wird häufig mit Normalverteilungen modelliert.
- Parameter können in Bayesian Learning als Zufallsvariablen mit Dichten beschrieben werden.
- Gaussian Mixture Models verwenden Mischungen kontinuierlicher Dichten.
- PCA und probabilistische PCA arbeiten mit kontinuierlichen Vektoren.

Idee

Viele ML-Modelle können als Aussagen über Dichten verstanden werden:

$$p(\vec{x}), \quad p(t \mid \vec{x}), \quad p(c \mid \vec{x}), \quad p(\vec{x} \mid c).$$

Je nachdem, welche dieser Größen modelliert wird, entstehen unterschiedliche Arten von Lernverfahren.

3.24 Häufige Fehler

Prüfungshinweis

Typische Fehler bei kontinuierlichen Zufallsvariablen:

1. Eine Dichte $p(x)$ mit einer Wahrscheinlichkeit $p(X = x)$ verwechseln.
2. Vergessen, dass Wahrscheinlichkeiten durch Integrale berechnet werden.
3. Annehmen, dass eine Dichte nie größer als 1 sein darf.
4. Die Normalisierungsbedingung $\int p(x) dx = 1$ vergessen.
5. Bei Randdichten Summen statt Integrale verwenden.
6. Bei Sampling-Approximation vergessen, dass die Stichproben aus der passenden Verteilung kommen müssen.
7. Korrelation 0 mit Unabhängigkeit verwechseln.

3.25 Mini-Aufgaben

3.25.1 Aufgabe 1: Wahrscheinlichkeit bei Gleichverteilung

Sei

$$X \sim \text{Uniform}(0,4).$$

Berechne

$$p(X \in [1,3]).$$

Beispiel

Die Dichte ist

$$p(x) = \frac{1}{4} \quad \text{für } x \in [0,4].$$

Also:

$$p(X \in [1,3]) = \int_1^3 \frac{1}{4} dx = \frac{1}{4}(3 - 1) = \frac{1}{2}.$$

3.25.2 Aufgabe 2: Mittelwert einer Gleichverteilung

Sei

$$X \sim \text{Uniform}(2,8).$$

Berechne

$$\mathbb{E}[X].$$

Beispiel

Für eine Gleichverteilung auf $[\alpha, \beta]$ gilt:

$$\mathbb{E}[X] = \frac{\alpha + \beta}{2}.$$

Mit $\alpha = 2$ und $\beta = 8$:

$$\mathbb{E}[X] = \frac{2 + 8}{2} = 5.$$

3.25.3 Aufgabe 3: Varianz aus Momenten

Angenommen,

$$\mathbb{E}[X] = 3$$

und

$$\mathbb{E}[X^2] = 13.$$

Berechne

$$\text{Var}(X).$$

Beispiel

Mit

$$\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$$

folgt:

$$\text{Var}(X) = 13 - 3^2 = 13 - 9 = 4.$$

3.26 Zusammenfassung

- Kontinuierliche Zufallsvariablen (continuous random variables) nehmen Werte in kontinuierlichen Räumen wie $\mathbb{R}$ an.
- Einzelne Werte haben typischerweise Wahrscheinlichkeit 0.
- Wahrscheinlichkeiten über Intervalle berechnet man durch Integrale.
- Eine Wahrscheinlichkeitsdichte (probability density function) erfüllt:

$$p(x) \geq 0$$

und

$$\int p(x) dx = 1.$$

- Für Intervalle gilt:

$$p(X \in [a, b]) = \int_a^b p(x) dx.$$

- Der Erwartungswert (expectation) ist:

$$\mathbb{E}_p[f(X)] = \int p(x)f(x) dx.$$

- Der Mittelwert (mean) ist:

$$\mathbb{E}[X] = \int p(x)x dx.$$

- Die Varianz (variance) ist:

$$\text{Var}(X) = \mathbb{E}[(X - \mu)^2].$$

- Die Kovarianz (covariance) misst den gemeinsamen linearen Zusammenhang zweier Variablen.
- Die Korrelation (correlation) ist die normierte Kovarianz.
- Summenregeln werden bei kontinuierlichen Variablen zu Integralen:

$$p(x) = \int p(x, y) dy.$$

- Sampling-Approximation ersetzt Integrale durch Mittelwerte über Stichproben:

$$\int p(x)f(x) dx \approx \frac{1}{N} \sum_{n=1}^N f(x_n).$$

3.27 Typische Prüfungsfragen

Prüfungshinweis

Mögliche Prüfungsfragen zu diesem Kapitel:

1. Warum ist $p(X = x) = 0$ für typische kontinuierliche Zufallsvariablen?
2. Was ist eine Wahrscheinlichkeitsdichte (probability density function)?
3. Welche zwei Bedingungen muss eine Dichte erfüllen?
4. Wie berechnet man $p(X \in [a,b])$?
5. Was ist der Unterschied zwischen Wahrscheinlichkeit und Dichte?
6. Definiere Erwartungswert, Mittelwert und Varianz.
7. Zeige, dass $\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$.
8. Was ist ein Histogramm und wie schätzt es eine Dichte?
9. Erkläre die Sampling-Approximation eines Erwartungswerts.
10. Wie werden Summenregel und Bayes-Regel im kontinuierlichen Fall geschrieben?
11. Warum ist die Normalverteilung für ML wichtig?
12. Was ist der Unterschied zwischen Kovarianz und Korrelation?

4 Schätzer

4.1 Ziel dieses Kapitels

In den vorherigen Kapiteln haben wir Wahrscheinlichkeiten, Wahrscheinlichkeitsdichten, Erwartungswerte, Varianz und Kovarianz kennengelernt. Jetzt betrachten wir die umgekehrte Situation:

Wir kennen die Verteilung nicht vollständig, sondern haben nur Daten.

Aus diesen Daten möchten wir unbekannte Größen einer Verteilung bestimmen. Solche unbekannten Größen nennt man Parameter (parameters). Eine Vorschrift, die aus Daten einen Parameter schätzt, nennt man Schätzer (estimator).

Idee

Ein Schätzer (estimator) ist eine Funktion der Daten, mit der wir eine unbekannte Größe einer Verteilung approximieren.

Typische unbekannte Größen sind:

- ein Mittelwert μ ,
- eine Varianz σ^2 ,
- ein Parameter einer Gleichverteilung,
- ein Parameter einer Bernoulli-Verteilung,
- die Gewichte eines Regressionsmodells.

Nach diesem Kapitel solltest du folgende Fragen beantworten können:

- Was ist ein statistischer Schätzer (statistical estimator)?
- Was ist der Unterschied zwischen Parameter, Schätzer und Schätzwert?
- Was bedeutet i.i.d. (independent and identically distributed)?
- Was ist Bias eines Schätzers?
- Was ist die Varianz eines Schätzers?
- Was ist der mittlere quadratische Fehler (mean squared error)?
- Wie hängen Bias, Varianz und Fehler zusammen?
- Warum ist das Stichprobenmittel ein erwartungstreuer Schätzer?

- Warum ist die naive Stichprobenvarianz verzerrt?
- Warum teilt man bei der unverzerrten Stichprobenvarianz durch $N - 1$?
- Was ist Maximum-Likelihood-Schätzung (maximum likelihood estimation)?
- Was ist Maximum-a-posteriori-Schätzung (maximum a-posteriori estimation)?
- Was ist ein Bayesian Estimator?

4.2 Parameter, Schätzer und Schätzwert

Wir beginnen mit einer wichtigen sprachlichen Unterscheidung.

Definition

Ein Parameter (parameter) ist eine feste, aber unbekannte Größe eines Modells oder einer Verteilung.

Ein Schätzer (estimator) ist eine Funktion der Daten, die diesen Parameter schätzen soll.

Ein Schätzwert (estimate) ist der konkrete Zahlenwert, den der Schätzer für einen konkreten Datensatz liefert.

Angenommen, die Zufallsvariable X hat einen unbekannten Mittelwert

$$\mu_X = \mathbb{E}[X].$$

Wir beobachten Daten

$$x_1, x_2, \dots, x_N.$$

Ein möglicher Schätzer für μ_X ist das Stichprobenmittel (sample mean):

$$\hat{\mu}_N = \frac{1}{N} \sum_{n=1}^N X_n.$$

Wichtig ist: Der Schätzer $\hat{\mu}_N$ ist selbst eine Zufallsvariable, weil er von den Zufallsvariablen $X_1, \dots, X_N$ abhängt. Wenn wir konkrete Daten $x_1, \dots, x_N$ einsetzen, erhalten wir einen konkreten Schätzwert:

$$\hat{\mu}_N(x_1, \dots, x_N) = \frac{1}{N} \sum_{n=1}^N x_n.$$

Hinweis

In der Praxis schreibt man oft einfach $\hat{\mu}_N$, obwohl eigentlich eine Funktion der Daten gemeint ist.

Strenger wäre:

$$\hat{\mu}_N = \hat{\mu}_N(X_1, \dots, X_N).$$

Diese Unterscheidung ist wichtig: Der wahre Parameter μ_X ist eine feste Zahl. Der Schätzer $\hat{\mu}_N$ ist eine Zufallsvariable.

4.3 Daten als Zufallsvariablen

In der Statistik und im maschinellen Lernen nehmen wir häufig an, dass Datenpunkte zufällige Realisierungen einer Verteilung sind.

Wir schreiben:

$$X_1, X_2, \dots, X_N.$$

Das sind Zufallsvariablen. Eine konkrete Beobachtung schreiben wir als

$$x_1, x_2, \dots, x_N.$$

Definition

Die Daten $X_1, \dots, X_N$ heißen unabhängig und identisch verteilt (independent and identically distributed), kurz i.i.d., wenn gilt:

- Alle X_n haben dieselbe Verteilung.
- Die Zufallsvariablen $X_1, \dots, X_N$ sind unabhängig.

Identisch verteilt bedeutet:

$$X_n \sim p(x) \quad \text{für alle } n \in \{1, \dots, N\}.$$

Unabhängig bedeutet:

$$p(x_1, \dots, x_N) = \prod_{n=1}^N p(x_n).$$

Idee

Die i.i.d.-Annahme ist eine Standardannahme in vielen Herleitungen.

Sie bedeutet: Alle Datenpunkte stammen aus derselben Quelle und beeinflussen sich nicht gegenseitig.

Beispiel

Wenn wir $N = 100$ zufällig ausgewählte Personen messen und X_n die Körpergröße der n -ten Person ist, dann modellieren wir oft:

$$X_1, \dots, X_N \stackrel{\text{i.i.d.}}{\sim} p(x).$$

Das bedeutet: Alle Körpergrößen werden als unabhängige Ziehungen aus derselben Ver-

teilung betrachtet.

4.4 Beispiel: Parameter einer Gleichverteilung schätzen

Ein einfaches Beispiel ist die Gleichverteilung (uniform distribution).

Angenommen, die Daten stammen aus einer Gleichverteilung auf einem unbekannten Intervall:

$$X \sim \text{Uniform}(a,b).$$

Die Dichte ist:

$$p(x \mid a,b) = \begin{cases} \frac{1}{b-a}, & x \in [a,b], \\ 0, & \text{sonst.} \end{cases}$$

Die Parameter sind a und b . Wir beobachten Daten:

$$x_1, x_2, \dots, x_N.$$

Die Frage lautet:

Wie können wir aus den Daten a und b schätzen?

4.5 Heuristischer Schätzer mit Minimum und Maximum

Eine naheliegende Idee ist: Das kleinste beobachtete x_n sollte ungefähr bei a liegen, und das größte beobachtete x_n ungefähr bei b .

Wir definieren:

$$\hat{a}_{\text{mima}} = \min_{n \in \{1, \dots, N\}} X_n,$$

und

$$\hat{b}_{\text{mima}} = \max_{n \in \{1, \dots, N\}} X_n.$$

Der Name mima steht hier für Minimum-Maximum.

Definition

Der Minimum-Maximum-Schätzer (minimum-maximum estimator) für die Gleichverteilung ist:

$$\hat{a}_{\text{mima}} = \min_n X_n, \quad \hat{b}_{\text{mima}} = \max_n X_n.$$

Idee

Die Idee ist sehr intuitiv: Wenn die Daten aus dem Intervall $[a,b]$ kommen, dann muss a links vom kleinsten Datenpunkt und b rechts vom größten Datenpunkt liegen. Daher verwendet man den kleinsten und größten Datenpunkt als Schätzung der Intervallgrenzen.

Dieser Schätzer ist einfach, aber er ist nicht perfekt. Bei endlichem N ist der kleinste Datenpunkt typischerweise größer als a , und der größte Datenpunkt typischerweise kleiner als b . Das Intervall wird also tendenziell zu klein geschätzt.

4.6 Methode der Momente

Eine zweite Idee ist die Methode der Momente (method of moments). Sie nutzt bekannte Formeln für Erwartungswert und Varianz einer Verteilung.

Für die Gleichverteilung auf $[a,b]$ gilt:

$$\mathbb{E}[X] = \frac{a+b}{2},$$

und

$$\text{Var}(X) = \frac{(b-a)^2}{12}.$$

Aus Daten können wir Mittelwert und Varianz schätzen. Dann lösen wir diese Gleichungen nach a und b auf.

4.6.1 Empirischer Mittelwert

Der empirische Mittelwert (empirical mean) ist:

$$\hat{\mu}_N = \frac{1}{N} \sum_{n=1}^N X_n.$$

4.6.2 Empirische Varianz

Eine einfache empirische Varianz ist:

$$\hat{\sigma}_N^2 = \frac{1}{N} \sum_{n=1}^N (X_n - \hat{\mu}_N)^2.$$

Für die Gleichverteilung gilt:

$$\mu = \frac{a+b}{2},$$

und

$$\sigma^2 = \frac{(b-a)^2}{12}.$$

Daraus folgt:

$$a + b = 2\mu,$$

und

$$b - a = \sqrt{12\sigma^2}.$$

Wir setzen

$$d = b - a.$$

Dann gilt:

$$d = \sqrt{12\sigma^2}.$$

Außerdem:

$$a = \mu - \frac{d}{2}, \quad b = \mu + \frac{d}{2}.$$

Einsetzen von $d = \sqrt{12\sigma^2}$ ergibt:

$$a = \mu - \sqrt{3\sigma^2},$$

und

$$b = \mu + \sqrt{3\sigma^2}.$$

Für die Schätzer verwenden wir $\hat{\mu}_N$ und $\hat{\sigma}_N^2$:

$$\hat{a}_{\text{MoM}} = \hat{\mu}_N - \sqrt{3\hat{\sigma}_N^2},$$

und

$$\hat{b}_{\text{MoM}} = \hat{\mu}_N + \sqrt{3\hat{\sigma}_N^2}.$$

Definition

Die Methode der Momente (method of moments) schätzt Parameter, indem theoretische Momente einer Verteilung mit empirischen Momenten aus den Daten gleichgesetzt werden. Für die Gleichverteilung ergibt sich:

$$\hat{a}_{\text{MoM}} = \hat{\mu}_N - \sqrt{3\hat{\sigma}_N^2},$$

$$\hat{b}_{\text{MoM}} = \hat{\mu}_N + \sqrt{3\hat{\sigma}_N^2}.$$

Hinweis

Momente (moments) sind Erwartungswerte von Potenzen einer Zufallsvariable. Das erste Moment hängt mit dem Mittelwert zusammen. Das zweite zentrale Moment ist die Varianz.

4.7 Welcher Schätzer ist besser?

Für die Gleichverteilung haben wir nun mindestens zwei Möglichkeiten:

$$(\hat{a}_{\text{mima}}, \hat{b}_{\text{mima}})$$

und

$$(\hat{a}_{\text{MoM}}, \hat{b}_{\text{MoM}}).$$

Die Frage ist:

Welcher Schätzer ist besser?

Um das mathematisch zu beantworten, brauchen wir Kriterien. Die wichtigsten Kriterien sind:

- Bias,
- Varianz,
- mittlerer quadratischer Fehler (mean squared error),
- Konsistenz (consistency).

4.8 Bias eines Schätzers

Ein Schätzer kann systematisch zu groß oder zu klein sein. Diese systematische Abweichung nennt man Bias.

Definition

Sei $\hat{\theta}_N$ ein Schätzer für einen wahren Parameter θ . Der Bias des Schätzers ist

$$\text{Bias}[\hat{\theta}_N] = \mathbb{E}[\hat{\theta}_N] - \theta.$$

Falls

$$\text{Bias}[\hat{\theta}_N] = 0,$$

dann heißt der Schätzer erwartungstreu oder unverzerrt (unbiased).

Definition

Ein Schätzer $\hat{\theta}_N$ heißt erwartungstreu (unbiased), wenn gilt:

$$\mathbb{E}[\hat{\theta}_N] = \theta.$$

Idee

Bias misst systematischen Fehler.

Ein positiver Bias bedeutet: Der Schätzer ist im Durchschnitt zu groß.

Ein negativer Bias bedeutet: Der Schätzer ist im Durchschnitt zu klein.

4.9 Varianz eines Schätzers

Neben systematischem Fehler gibt es zufällige Schwankung. Ein Schätzer kann bei verschiedenen Datensätzen stark unterschiedliche Werte liefern. Diese Schwankung misst die Varianz des Schätzers.

Definition

Die Varianz eines Schätzers $\hat{\theta}_N$ ist

$$\text{Var}[\hat{\theta}_N] = \mathbb{E} \left[\left(\hat{\theta}_N - \mathbb{E}[\hat{\theta}_N] \right)^2 \right].$$

Idee

Die Varianz eines Schätzers misst, wie stark der Schätzer von Datensatz zu Datensatz schwankt.

Kleine Varianz bedeutet: Der Schätzer ist stabil.

Große Varianz bedeutet: Der Schätzer reagiert stark auf die zufällige Auswahl der Daten.

4.10 Mittlerer quadratischer Fehler

Bias und Varianz beschreiben unterschiedliche Fehlerquellen. Der mittlere quadratische Fehler (mean squared error, MSE) kombiniert beide.

Definition

Der mittlere quadratische Fehler (mean squared error) eines Schätzers $\hat{\theta}_N$ für einen Parameter θ ist

$$\text{MSE}[\hat{\theta}_N] = \mathbb{E} \left[\left(\hat{\theta}_N - \theta \right)^2 \right].$$

4.10.1 Bias-Varianz-Zerlegung

Der MSE kann zerlegt werden in Bias und Varianz:

$$\text{MSE}[\hat{\theta}_N] = \text{Bias}[\hat{\theta}_N]^2 + \text{Var}[\hat{\theta}_N].$$

4.10.2 Herleitung

Wir starten mit der Definition:

$$\text{MSE}[\hat{\theta}_N] = \mathbb{E} \left[\left(\hat{\theta}_N - \theta \right)^2 \right].$$

Wir addieren und subtrahieren $\mathbb{E}[\hat{\theta}_N]$:

$$\hat{\theta}_N - \theta = \hat{\theta}_N - \mathbb{E}[\hat{\theta}_N] + \mathbb{E}[\hat{\theta}_N] - \theta.$$

Setze

$$A = \hat{\theta}_N - \mathbb{E}[\hat{\theta}_N],$$

und

$$B = \mathbb{E}[\hat{\theta}_N] - \theta.$$

Dann gilt:

$$\hat{\theta}_N - \theta = A + B.$$

Also:

$$\text{MSE}[\hat{\theta}_N] = \mathbb{E}[(A + B)^2].$$

Ausmultiplizieren ergibt:

$$\mathbb{E}[(A + B)^2] = \mathbb{E}[A^2] + 2\mathbb{E}[AB] + \mathbb{E}[B^2].$$

Da B konstant ist, gilt:

$$\mathbb{E}[AB] = B\mathbb{E}[A].$$

Außerdem:

$$\mathbb{E}[A] = \mathbb{E} \left[\hat{\theta}_N - \mathbb{E}[\hat{\theta}_N] \right] = \mathbb{E}[\hat{\theta}_N] - \mathbb{E}[\hat{\theta}_N] = 0.$$

Daher verschwindet der Kreuzterm:

$$2\mathbb{E}[AB] = 0.$$

Weiter gilt:

$$\mathbb{E}[A^2] = \text{Var}[\hat{\theta}_N],$$

und

$$\mathbb{E}[B^2] = B^2 = \left(\mathbb{E}[\hat{\theta}_N] - \theta\right)^2 = \text{Bias}[\hat{\theta}_N]^2.$$

Damit:

$$\text{MSE}[\hat{\theta}_N] = \text{Var}[\hat{\theta}_N] + \text{Bias}[\hat{\theta}_N]^2.$$

Idee

Die Bias-Varianz-Zerlegung lautet:

$$\text{MSE} = \text{Bias}^2 + \text{Varianz}.$$

Ein guter Schätzer sollte idealerweise kleinen Bias und kleine Varianz haben. Oft stehen diese beiden Ziele aber in Konkurrenz.

4.11 Bias-Varianz-Trade-off

Ein Schätzer mit sehr kleinem Bias kann hohe Varianz haben. Ein Schätzer mit kleiner Varianz kann dafür verzerrt sein. Dieses Spannungsverhältnis nennt man Bias-Varianz-Trade-off (bias-variance trade-off).

Idee

Beim Bias-Varianz-Trade-off versucht man, den gesamten erwarteten Fehler zu minimieren, nicht nur Bias oder Varianz einzeln.

Ein leicht verzerrter Schätzer kann besser sein als ein unverzerrter Schätzer, wenn seine Varianz deutlich kleiner ist.

Prüfungshinweis

Wichtig: Unverzerrt bedeutet nicht automatisch gut.

Ein Schätzer kann Bias 0 haben, aber sehr stark schwanken. Dann kann sein MSE trotzdem groß sein.

4.12 Konsistenz

Ein Schätzer sollte besser werden, wenn mehr Daten verfügbar sind. Diese Eigenschaft nennt man Konsistenz (consistency).

Definition

Ein Schätzer $\hat{\theta}_N$ heißt konsistent (consistent), wenn er für $N \rightarrow \infty$ gegen den wahren Parameter θ konvergiert.

Eine einfache ausreichende Intuition ist:

$$\lim_{N \rightarrow \infty} \text{Bias}[\hat{\theta}_N] = 0,$$

und

$$\lim_{N \rightarrow \infty} \text{Var}[\hat{\theta}_N] = 0.$$

Idee

Konsistenz bedeutet: Mit unendlich vielen Daten verschwindet der Schätzfehler. Für maschinelles Lernen ist das wichtig, weil wir erwarten, dass mehr Daten zu besseren Schätzungen führen.

4.13 Das Stichprobenmittel

Das Stichprobenmittel (sample mean) ist einer der wichtigsten Schätzer.

Seien

$$X_1, \dots, X_N$$

i.i.d. Zufallsvariablen mit

$$\mathbb{E}[X_n] = \mu_X$$

und

$$\text{Var}(X_n) = \sigma_X^2.$$

Der Schätzer für den Mittelwert ist:

$$\hat{\mu}_N = \frac{1}{N} \sum_{n=1}^N X_n.$$

Definition

Das Stichprobenmittel (sample mean, empirical mean) ist

$$\hat{\mu}_N = \frac{1}{N} \sum_{n=1}^N X_n.$$

Es schätzt den wahren Mittelwert

$$\mu_X = \mathbb{E}[X].$$

4.14 Bias des Stichprobenmittels

Wir berechnen:

$$\text{Bias}[\hat{\mu}_N] = \mathbb{E}[\hat{\mu}_N] - \mu_X.$$

Zuerst:

$$\mathbb{E}[\hat{\mu}_N] = \mathbb{E}\left[\frac{1}{N} \sum_{n=1}^N X_n\right].$$

Wegen Linearität des Erwartungswerts:

$$\mathbb{E}[\hat{\mu}_N] = \frac{1}{N} \sum_{n=1}^N \mathbb{E}[X_n].$$

Da alle X_n identisch verteilt sind:

$$\mathbb{E}[X_n] = \mu_X.$$

Also:

$$\mathbb{E}[\hat{\mu}_N] = \frac{1}{N} \sum_{n=1}^N \mu_X = \frac{1}{N} N \mu_X = \mu_X.$$

Damit:

$$\text{Bias}[\hat{\mu}_N] = \mu_X - \mu_X = 0.$$

Idee

Das Stichprobenmittel ist ein erwartungstreuer Schätzer (unbiased estimator) für den wahren Mittelwert:

$$\mathbb{E}[\hat{\mu}_N] = \mu_X.$$

4.15 Varianz des Stichprobenmittels

Nun berechnen wir die Varianz von $\hat{\mu}_N$:

$$\text{Var}[\hat{\mu}_N] = \text{Var}\left[\frac{1}{N} \sum_{n=1}^N X_n\right].$$

Konstanten ziehen quadratisch aus der Varianz:

$$\text{Var}[\hat{\mu}_N] = \frac{1}{N^2} \text{Var}\left[\sum_{n=1}^N X_n\right].$$

Da die X_n unabhängig sind, addieren sich die Varianzen:

$$\text{Var}\left[\sum_{n=1}^N X_n\right] = \sum_{n=1}^N \text{Var}[X_n].$$

Da alle X_n identisch verteilt sind:

$$\text{Var}[X_n] = \sigma_X^2.$$

Also:

$$\sum_{n=1}^N \text{Var}[X_n] = N\sigma_X^2.$$

Damit:

$$\text{Var}[\hat{\mu}_N] = \frac{1}{N^2} N\sigma_X^2 = \frac{\sigma_X^2}{N}.$$

Definition

Für das Stichprobenmittel gilt:

$$\text{Bias}[\hat{\mu}_N] = 0,$$

und

$$\text{Var}[\hat{\mu}_N] = \frac{\sigma_X^2}{N}.$$

Idee

Je größer N ist, desto kleiner ist die Varianz des Stichprobenmittels.

$$\text{Var}[\hat{\mu}_N] = \frac{\sigma_X^2}{N} \longrightarrow 0 \quad \text{für} \quad N \rightarrow \infty.$$

Das Stichprobenmittel wird also mit mehr Daten stabiler.

Da der Bias 0 ist, ist der MSE des Stichprobenmittels gleich seiner Varianz:

$$\text{MSE}[\hat{\mu}_N] = \text{Var}[\hat{\mu}_N] = \frac{\sigma_X^2}{N}.$$

4.16 Die naive Stichprobenvarianz

Wir möchten nun die wahre Varianz

$$\sigma_X^2 = \text{Var}(X)$$

schätzen.

Eine naheliegende Idee ist:

$$\hat{\sigma}_N^2 = \frac{1}{N} \sum_{n=1}^N (X_n - \hat{\mu}_N)^2.$$

Definition

Die naive Stichprobenvarianz ist

$$\hat{\sigma}_N^2 = \frac{1}{N} \sum_{n=1}^N (X_n - \hat{\mu}_N)^2.$$

Sie verwendet das Stichprobenmittel $\hat{\mu}_N$ anstelle des unbekannten wahren Mittelwerts μ_X .

Diese Formel wirkt natürlich, ist aber verzerrt. Sie unterschätzt die wahre Varianz im Mittel.

4.17 Bias der naiven Stichprobenvarianz

Für die naive Stichprobenvarianz gilt:

$$\mathbb{E}[\hat{\sigma}_N^2] = \frac{N-1}{N} \sigma_X^2.$$

Damit ist der Bias:

$$\text{Bias}[\hat{\sigma}_N^2] = \mathbb{E}[\hat{\sigma}_N^2] - \sigma_X^2.$$

Einsetzen ergibt:

$$\text{Bias}[\hat{\sigma}_N^2] = \frac{N-1}{N} \sigma_X^2 - \sigma_X^2.$$

Wir schreiben

$$\sigma_X^2 = \frac{N}{N} \sigma_X^2.$$

Dann:

$$\text{Bias}[\hat{\sigma}_N^2] = \frac{N-1}{N} \sigma_X^2 - \frac{N}{N} \sigma_X^2 = -\frac{1}{N} \sigma_X^2.$$

Idee

Die naive Stichprobenvarianz ist verzerrt:

$$\text{Bias}[\hat{\sigma}_N^2] = -\frac{1}{N} \sigma_X^2.$$

Sie unterschätzt die wahre Varianz im Durchschnitt.

4.17.1 Intuition für die Verzerrung

Der Grund ist: Wir kennen den wahren Mittelwert μ_X nicht. Stattdessen verwenden wir das aus denselben Daten berechnete Stichprobenmittel $\hat{\mu}_N$.

Das Stichprobenmittel liegt näher an den Daten als der wahre Mittelwert. Dadurch werden die Abstände

$$X_n - \hat{\mu}_N$$

im Durchschnitt zu klein. Also wird auch die Varianz zu klein geschätzt.

4.18 Unverzerrte Stichprobenvarianz

Um den Bias zu korrigieren, multiplizieren wir die naive Stichprobenvarianz mit

$$\frac{N}{N-1}.$$

Wir definieren:

$$\hat{s}_N^2 = \frac{N}{N-1} \hat{\sigma}_N^2.$$

Einsetzen der naiven Stichprobenvarianz ergibt:

$$\hat{s}_N^2 = \frac{N}{N-1} \frac{1}{N} \sum_{n=1}^N (X_n - \hat{\mu}_N)^2.$$

Damit:

$$\hat{s}_N^2 = \frac{1}{N-1} \sum_{n=1}^N (X_n - \hat{\mu}_N)^2.$$

Definition

Die unverzernte Stichprobenvarianz ist

$$\hat{s}_N^2 = \frac{1}{N-1} \sum_{n=1}^N (X_n - \hat{\mu}_N)^2.$$

Sie ist ein erwartungstreuer Schätzer für σ_X^2 .

4.18.1 Nachweis der Erwartungstreue

Wir wissen:

$$\mathbb{E}[\hat{\sigma}_N^2] = \frac{N-1}{N} \sigma_X^2.$$

Da

$$\hat{s}_N^2 = \frac{N}{N-1} \hat{\sigma}_N^2$$

gilt:

$$\mathbb{E}[\hat{s}_N^2] = \mathbb{E}\left[\frac{N}{N-1} \hat{\sigma}_N^2\right].$$

Der Faktor ist konstant:

$$\mathbb{E}[\hat{s}_N^2] = \frac{N}{N-1} \mathbb{E}[\hat{\sigma}_N^2].$$

Einsetzen:

$$\mathbb{E}[\hat{s}_N^2] = \frac{N}{N-1} \frac{N-1}{N} \sigma_X^2.$$

Damit:

$$\mathbb{E}[\hat{s}_N^2] = \sigma_X^2.$$

Also:

$$\text{Bias}[\hat{s}_N^2] = 0.$$

Prüfungshinweis

Der Grund für $N - 1$ im Nenner ist die Bias-Korrektur.

$$\frac{1}{N} \sum_{n=1}^N (X_n - \hat{\mu}_N)^2$$

ist verzerrt.

$$\frac{1}{N-1} \sum_{n=1}^N (X_n - \hat{\mu}_N)^2$$

ist unverzerrt.

4.19 Vergleich der Varianzschätzer

Wir haben zwei Schätzer für σ_X^2 :

$$\hat{\sigma}_N^2 = \frac{1}{N} \sum_{n=1}^N (X_n - \hat{\mu}_N)^2,$$

und

$$\hat{s}_N^2 = \frac{1}{N-1} \sum_{n=1}^N (X_n - \hat{\mu}_N)^2.$$

Zwischen ihnen gilt:

$$\hat{s}_N^2 = \frac{N}{N-1} \hat{\sigma}_N^2.$$

Daher gilt für die Varianzen:

$$\text{Var}[\hat{s}_N^2] = \text{Var} \left[\frac{N}{N-1} \hat{\sigma}_N^2 \right].$$

Konstanten gehen quadratisch in die Varianz ein:

$$\text{Var}[\hat{s}_N^2] = \left(\frac{N}{N-1} \right)^2 \text{Var}[\hat{\sigma}_N^2].$$

Da

$$\frac{N}{N-1} > 1$$

für $N > 1$, gilt:

$$\text{Var}[\hat{s}_N^2] > \text{Var}[\hat{\sigma}_N^2].$$

Idee

Die unverzernte Stichprobenvarianz hat Bias 0, aber etwas größere Varianz als die naive Stichprobenvarianz.

Das ist ein Beispiel für den Bias-Varianz-Trade-off.

4.20 Übersicht: Mittelwert- und Varianzschätzer

Zielgröße	Schätzer	Bias	Varianz
μ_X	$\hat{\mu}_N = \frac{1}{N} \sum_{n=1}^N X_n$	0	$\frac{1}{N} \sigma_X^2$
σ_X^2	$\hat{\sigma}_N^2 = \frac{1}{N} \sum_{n=1}^N (X_n - \hat{\mu}_N)^2$	$-\frac{1}{N} \sigma_X^2$	$\text{Var}[\hat{\sigma}_N^2]$
σ_X^2	$\hat{s}_N^2 = \frac{1}{N-1} \sum_{n=1}^N (X_n - \hat{\mu}_N)^2$	0	$\left(\frac{N}{N-1} \right)^2 \text{Var}[\hat{\sigma}_N^2]$

Hinweis

Die Tabelle zeigt: Der unverzernte Varianzschätzer reduziert den Bias auf 0, zahlt dafür aber mit etwas höherer Varianz. Für große N wird der Unterschied zwischen N und $N-1$ klein.

4.21 Maximum-Likelihood-Schätzung

Die Maximum-Likelihood-Schätzung (maximum likelihood estimation, MLE) ist eine der wichtigsten Methoden im maschinellen Lernen.

Die Idee ist:

Wähle die Parameter so, dass die beobachteten Daten unter dem Modell möglichst wahrscheinlich sind.

Definition

Sei $p(\vec{X} \mid \theta)$ die Wahrscheinlichkeit oder Dichte der Daten unter dem Parameter θ . Die Likelihood ist

$$L(\theta; \vec{X}) = p(\vec{X} \mid \theta).$$

Der Maximum-Likelihood-Schätzer ist

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} p(\vec{X} \mid \theta).$$

Hier bezeichnet

$$\vec{X} = (X_1, \dots, X_N)^\top$$

den gesamten Datensatz als Zufallsvektor.

Für konkrete Daten

$$\vec{x} = (x_1, \dots, x_N)^\top$$

betrachtet man

$$L(\theta; \vec{x}) = p(\vec{x} \mid \theta)$$

als Funktion von θ .

Hinweis

Die Likelihood $L(\theta; \vec{x})$ ist keine Wahrscheinlichkeitsverteilung über θ .

Sie ist eine Funktion, die sagt, wie plausibel die beobachteten Daten $\vec{x}$ für verschiedene Parameterwerte θ sind.

4.22 Likelihood bei i.i.d. Daten

Wenn die Daten i.i.d. sind, gilt:

$$p(\vec{x} \mid \theta) = p(x_1, \dots, x_N \mid \theta) = \prod_{n=1}^N p(x_n \mid \theta).$$

Damit ist der Maximum-Likelihood-Schätzer:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \prod_{n=1}^N p(x_n \mid \theta).$$

Idee

Bei i.i.d. Daten faktorisiert die Likelihood als Produkt der Einzelwahrscheinlichkeiten oder Einzeldichten:

$$L(\theta; \vec{x}) = \prod_{n=1}^N p(x_n \mid \theta).$$

4.23 Log-Likelihood

Produkte vieler Wahrscheinlichkeiten sind mathematisch unpraktisch. Deshalb nimmt man meistens den Logarithmus. Da der Logarithmus streng monoton wächst, ändert sich die Stelle des

Maximums nicht.

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} L(\theta; \vec{x})$$

ist äquivalent zu

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \log L(\theta; \vec{x}).$$

Für i.i.d. Daten gilt:

$$\log L(\theta; \vec{x}) = \log \prod_{n=1}^N p(x_n | \theta).$$

Der Logarithmus macht aus dem Produkt eine Summe:

$$\log L(\theta; \vec{x}) = \sum_{n=1}^N \log p(x_n | \theta).$$

Definition

Die Log-Likelihood ist

$$\ell(\theta) = \log L(\theta; \vec{x}) = \sum_{n=1}^N \log p(x_n | \theta).$$

Prüfungshinweis

Standardtechnik für Maximum-Likelihood-Herleitungen:

1. Likelihood $L(\theta; \vec{x})$ aufschreiben.
2. Log-Likelihood $\ell(\theta)$ bilden.
3. Nach θ ableiten.
4. Ableitung gleich 0 setzen.
5. Nach θ lösen.
6. Prüfen, ob die Lösung zulässig ist und wirklich ein Maximum liefert.

4.24 Beispiel: MLE für Bernoulli-Verteilung

Die Bernoulli-Verteilung (Bernoulli distribution) modelliert binäre Daten:

$$X \in \{0,1\}.$$

Der Parameter sei

$$\pi \in [0,1].$$

Die Verteilung ist:

$$p(X = x \mid \pi) = \pi^x (1 - \pi)^{1-x}.$$

Wir beobachten i.i.d. Daten:

$$x_1, \dots, x_N \in \{0,1\}.$$

Die Likelihood ist:

$$L(\pi; \vec{x}) = \prod_{n=1}^N \pi^{x_n} (1 - \pi)^{1-x_n}.$$

Die Log-Likelihood ist:

$$\ell(\pi) = \sum_{n=1}^N [x_n \log \pi + (1 - x_n) \log(1 - \pi)].$$

Ableiten nach π :

$$\frac{\partial \ell}{\partial \pi} = \sum_{n=1}^N \left[\frac{x_n}{\pi} - \frac{1 - x_n}{1 - \pi} \right].$$

Setze die Ableitung gleich 0:

$$\sum_{n=1}^N \left[\frac{x_n}{\pi} - \frac{1 - x_n}{1 - \pi} \right] = 0.$$

Das ist:

$$\frac{1}{\pi} \sum_{n=1}^N x_n - \frac{1}{1 - \pi} \sum_{n=1}^N (1 - x_n) = 0.$$

Setze

$$S = \sum_{n=1}^N x_n.$$

Dann ist

$$\sum_{n=1}^N (1 - x_n) = N - S.$$

Also:

$$\frac{S}{\pi} - \frac{N - S}{1 - \pi} = 0.$$

Damit:

$$\frac{S}{\pi} = \frac{N - S}{1 - \pi}.$$

Kreuzmultiplizieren:

$$S(1 - \pi) = \pi(N - S).$$

Ausmultiplizieren:

$$S - S\pi = \pi N - \pi S.$$

Die Terme $-S\pi$ und $-\pi S$ heben sich auf:

$$S = \pi N.$$

Also:

$$\hat{\pi}_{\text{ML}} = \frac{S}{N} = \frac{1}{N} \sum_{n=1}^N x_n.$$

Idee

Der Maximum-Likelihood-Schätzer für den Parameter einer Bernoulli-Verteilung ist der Anteil der Einsen:

$$\hat{\pi}_{\text{ML}} = \frac{1}{N} \sum_{n=1}^N x_n.$$

4.25 MLE für die Normalverteilung

Seien

$$X_1, \dots, X_N \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(\mu, \sigma^2).$$

Die Parameter sind μ und σ^2 . Für die Normalverteilung ergibt Maximum-Likelihood:

$$\hat{\mu}_{\text{ML}} = \frac{1}{N} \sum_{n=1}^N x_n,$$

und

$$\hat{\sigma}_{\text{ML}}^2 = \frac{1}{N} \sum_{n=1}^N (x_n - \hat{\mu}_{\text{ML}})^2.$$

Hinweis

Der ML-Schätzer für die Varianz der Normalverteilung teilt durch N , nicht durch $N - 1$. Er ist daher identisch mit der naiven Stichprobenvarianz und ist bei endlichem N verzerrt.

Idee

Maximum-Likelihood-Schätzer sind oft sehr allgemein herleitbar. Sie sind deshalb für maschinelles Lernen besonders wichtig.

Aber: MLE ist nicht automatisch unverzerrt.

4.26 MLE für die Gleichverteilung

Für die Gleichverteilung

$$X \sim \text{Uniform}(a,b)$$

ist die Dichte:

$$p(x \mid a,b) = \begin{cases} \frac{1}{b-a}, & x \in [a,b], \\ 0, & \text{sonst.} \end{cases}$$

Für i.i.d. Daten ergibt sich die Likelihood:

$$L(a,b;\vec{x}) = \prod_{n=1}^N p(x_n \mid a,b).$$

Damit gilt:

$$L(a,b;\vec{x}) = \begin{cases} \left(\frac{1}{b-a}\right)^N, & \text{wenn } a \leq x_n \leq b \text{ für alle } n, \\ 0, & \text{sonst.} \end{cases}$$

Damit die Likelihood nicht 0 ist, muss das Intervall $[a,b]$ alle Datenpunkte enthalten:

$$a \leq \min_n x_n, \quad b \geq \max_n x_n.$$

Wenn diese Bedingung erfüllt ist, ist

$$L(a,b;\vec{x}) = \left(\frac{1}{b-a}\right)^N.$$

Diese Likelihood wird größer, wenn $b-a$ kleiner wird. Das kleinste Intervall, das alle Daten enthält, ist:

$$\hat{a}_{\text{ML}} = \min_n x_n,$$

und

$$\hat{b}_{\text{ML}} = \max_n x_n.$$

Definition

Für die Gleichverteilung auf $[a,b]$ ist der Maximum-Likelihood-Schätzer:

$$\hat{a}_{\text{ML}} = \min_n x_n, \quad \hat{b}_{\text{ML}} = \max_n x_n.$$

Hinweis

Für die Gleichverteilung fällt der Maximum-Likelihood-Schätzer mit dem Minimum-Maximum-Schätzer zusammen.

Das ist ein gutes Beispiel dafür, dass MLE nicht immer alle gewünschten Eigenschaften hat: Bei endlichem N schätzt dieser Schätzer das Intervall tendenziell zu klein.

4.27 Maximum-a-posteriori-Schätzung

Maximum-Likelihood verwendet nur die Daten. Manchmal haben wir aber Vorwissen über plausible Parameter. Dieses Vorwissen modellieren wir mit einem Prior.

Definition

Bei der Maximum-a-posteriori-Schätzung (maximum a-posteriori estimation, MAP) maximiert man die Posterior-Verteilung der Parameter:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\theta \mid \vec{x}).$$

Mit Bayes' Regel gilt:

$$p(\theta \mid \vec{x}) = \frac{p(\vec{x} \mid \theta)p(\theta)}{p(\vec{x})}.$$

Dabei ist:

- $p(\vec{x} \mid \theta)$ die Likelihood,
- $p(\theta)$ der Prior,
- $p(\theta \mid \vec{x})$ der Posterior,
- $p(\vec{x})$ die Evidence.

Da $p(\vec{x})$ nicht von θ abhängt, ist es für die Maximierung konstant. Daher:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\vec{x} \mid \theta)p(\theta).$$

Oder proportional geschrieben:

$$p(\theta \mid \vec{x}) \propto p(\vec{x} \mid \theta)p(\theta).$$

Idee

MAP kombiniert Daten und Vorwissen:

$$\text{Posterior} \propto \text{Likelihood} \cdot \text{Prior}.$$

MLE maximiert nur die Likelihood. MAP maximiert Likelihood mal Prior.

4.28 Log-Posterior bei MAP

Wie bei MLE verwendet man oft den Logarithmus:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} [\log p(\vec{x} \mid \theta) + \log p(\theta)].$$

Für i.i.d. Daten ist:

$$\log p(\vec{x} \mid \theta) = \sum_{n=1}^N \log p(x_n \mid \theta).$$

Damit:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} \left[\sum_{n=1}^N \log p(x_n \mid \theta) + \log p(\theta) \right].$$

Prüfungshinweis

Standardtechnik für MAP-Herleitungen:

1. Likelihood $p(\vec{x} \mid \theta)$ aufschreiben.
2. Prior $p(\theta)$ aufschreiben.
3. Produkt $p(\vec{x} \mid \theta)p(\theta)$ bilden.
4. Logarithmus nehmen.
5. Nach θ ableiten.
6. Ableitung gleich 0 setzen.
7. Zulässige optimale Lösung wählen.

4.29 MLE als Spezialfall von MAP

Wenn der Prior konstant ist, dann ist MAP äquivalent zu MLE.

Angenommen:

$$p(\theta) = \text{konstant}.$$

Dann gilt:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\vec{x} \mid \theta) p(\theta).$$

Da $p(\theta)$ konstant ist, ändert es die Stelle des Maximums nicht:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\vec{x} \mid \theta) = \hat{\theta}_{\text{ML}}.$$

Idee

MLE ist MAP mit einem konstanten oder uninformativem Prior.

MAP erweitert MLE, indem Vorwissen über plausible Parameterwerte berücksichtigt wird.

4.30 MAP und Regularisierung

MAP ist wichtig, weil es direkt mit Regularisierung (regularization) zusammenhängt.

Wenn wir einen Prior verwenden, der große Parameterwerte unwahrscheinlich macht, dann werden solche Parameterwerte beim Schätzen bestraft.

Ein Beispiel ist ein Gauß-Prior auf einen Parametervektor $\vec{w}$:

$$p(\vec{w}) = \mathcal{N}(\vec{w} \mid \vec{0}, \lambda^{-1} I).$$

Dieser Prior bevorzugt kleine Gewichte. Der Log-Prior enthält dann einen Term der Form:

$$-\lambda \vec{w}^{\top} \vec{w}.$$

Beim Maximieren des Log-Posterior entspricht das einer Bestrafung großer Gewichte. Beim Minimieren eines negativen Log-Posterior wird daraus ein Regularisierungsterm:

$$\lambda \vec{w}^{\top} \vec{w}.$$

Idee

MAP-Schätzung kann als Maximum-Likelihood-Schätzung mit zusätzlichem Prior-Wissen verstanden werden.

In vielen Modellen führt ein Gauß-Prior auf Parameter zu L_2 -Regularisierung. Bei linearer Regression ergibt das Ridge Regression.

4.31 Bayesian Estimator

MLE und MAP liefern jeweils einen einzelnen besten Parameterwert:

$$\hat{\theta}_{\text{ML}}$$

oder

$$\hat{\theta}_{\text{MAP}}.$$

Bayesian Estimation geht einen Schritt weiter: Statt nur einen Punkt zu schätzen, betrachtet man die ganze Posterior-Verteilung.

Definition

Ein Bayesian Estimator verwendet die Posterior-Verteilung

$$p(\theta \mid \vec{x}) = \frac{p(\vec{x} \mid \theta)p(\theta)}{p(\vec{x})}.$$

Die Unsicherheit über θ bleibt also als Verteilung erhalten.

Man kann schreiben:

$$\hat{\Theta}_N \sim p(\theta \mid \vec{x}).$$

Das bedeutet: Die Schätzung ist nicht nur ein Punkt, sondern wird durch eine Verteilung über mögliche Parameterwerte beschrieben.

Idee

Bayesian Estimation modelliert Unsicherheit über Parameter explizit.

Das ist besonders nützlich, wenn nur wenige Daten vorhanden sind oder wenn Unsicherheit in Entscheidungen einfließen soll.

4.32 Vergleich: MLE, MAP und Bayesian

Methode	Was wird berechnet?	Verwendet Prior?
MLE	$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} p(\vec{x} \mid \theta)$	Nein
MAP	$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\theta \mid \vec{x})$	Ja
Bayesian	$p(\theta \mid \vec{x})$	Ja

Hinweis

MLE und MAP geben Punktschätzungen.

Bayesian Estimation gibt eine ganze Verteilung über Parameter.

4.33 Effizienz eines Schätzers

Ein Schätzer ist effizient (efficient), wenn er im Vergleich zu anderen Schätzern einen kleinen erwarteten Fehler hat. Meistens betrachtet man dafür den MSE.

Definition

Ein Schätzer $\hat{\theta}_N^A$ ist effizienter als ein Schätzer $\hat{\theta}_N^B$, wenn gilt:

$$\text{MSE}[\hat{\theta}_N^A] < \text{MSE}[\hat{\theta}_N^B].$$

Hinweis

Effizienz hängt vom gewählten Fehlermaß ab. In diesem Kurs ist der MSE das zentrale Fehlermaß für Schätzer.

4.34 Zusammenhang zu maschinellem Lernen

Schätzer sind eine Grundlage für viele ML-Algorithmen.

- In der linearen Regression schätzen wir Gewichte $\vec{w}$.
- In der logistischen Regression schätzen wir Parameter eines Klassifikators.
- In Gaussian Mixture Models schätzen wir Mittelwerte, Kovarianzen und Mischgewichte.
- Im EM-Algorithmus wechseln sich Posterior-Berechnung und Parameterschätzung ab.
- In neuronalen Netzen schätzen wir sehr viele Gewichte durch Optimierung einer Verlustfunktion.

Idee

Viele Lernalgorithmen können als Parameterschätzung verstanden werden:

Daten $\longrightarrow$ Schätzer $\longrightarrow$ Parameter $\longrightarrow$ Modell.

4.35 Häufige Fehler

Prüfungshinweis

Typische Fehler bei Schätzern:

1. Parameter θ , Schätzer $\hat{\theta}_N$ und Schätzwert $\hat{\theta}_N(\vec{x})$ verwechseln.
2. Vergessen, dass ein Schätzer eine Zufallsvariable ist.
3. Unbiased mit gut gleichsetzen.
4. Bias und Varianz getrennt betrachten, aber den MSE vergessen.
5. Bei der Stichprobenvarianz durch N teilen, obwohl ein unverzerrter Schätzer gefragt ist.
6. MLE für automatisch unverzerrt halten.
7. Likelihood $p(\vec{x} | \theta)$ mit Posterior $p(\theta | \vec{x})$ verwechseln.
8. Bei MAP den Prior vergessen.
9. Bei der Log-Likelihood das Produkt nicht korrekt in eine Summe umwandeln.

4.36 Mini-Aufgaben

4.36.1 Aufgabe 1: Bias eines einfachen Schätzers

Sei $\hat{\theta}_N$ ein Schätzer für θ . Angenommen:

$$\mathbb{E}[\hat{\theta}_N] = \theta + 2.$$

Berechne den Bias.

Beispiel

Nach Definition:

$$\text{Bias}[\hat{\theta}_N] = \mathbb{E}[\hat{\theta}_N] - \theta.$$

Einsetzen ergibt:

$$\text{Bias}[\hat{\theta}_N] = (\theta + 2) - \theta = 2.$$

4.36.2 Aufgabe 2: MSE aus Bias und Varianz

Angenommen:

$$\text{Bias}[\hat{\theta}_N] = -3,$$

und

$$\text{Var}[\hat{\theta}_N] = 5.$$

Berechne den MSE.

Beispiel

Mit der Bias-Varianz-Zerlegung:

$$\text{MSE}[\hat{\theta}_N] = \text{Bias}[\hat{\theta}_N]^2 + \text{Var}[\hat{\theta}_N].$$

Also:

$$\text{MSE}[\hat{\theta}_N] = (-3)^2 + 5 = 9 + 5 = 14.$$

4.36.3 Aufgabe 3: Stichprobenmittel

Gegeben sind die Daten:

$$x_1 = 2, \quad x_2 = 4, \quad x_3 = 6.$$

Berechne das Stichprobenmittel.

Beispiel

Das Stichprobenmittel ist:

$$\hat{\mu}_3 = \frac{1}{3}(2 + 4 + 6) = 4.$$

4.36.4 Aufgabe 4: Unverzerrte Stichprobenvarianz

Für dieselben Daten

$$x_1 = 2, \quad x_2 = 4, \quad x_3 = 6$$

ist

$$\hat{\mu}_3 = 4.$$

Berechne die unverzerrte Stichprobenvarianz.

Beispiel

Die unverzerrte Stichprobenvarianz ist:

$$\hat{s}_3^2 = \frac{1}{3-1} \sum_{n=1}^3 (x_n - \hat{\mu}_3)^2.$$

Einsetzen:

$$\hat{s}_3^2 = \frac{1}{2} [(2-4)^2 + (4-4)^2 + (6-4)^2].$$

Also:

$$\hat{s}_3^2 = \frac{1}{2}(4 + 0 + 4) = 4.$$

4.36.5 Aufgabe 5: MLE für Bernoulli

Gegeben sind binäre Daten:

$$x_1 = 1, \quad x_2 = 0, \quad x_3 = 1, \quad x_4 = 1.$$

Berechne den Maximum-Likelihood-Schätzer für π .

Beispiel

Für Bernoulli-Daten gilt:

$$\hat{\pi}_{\text{ML}} = \frac{1}{N} \sum_{n=1}^N x_n.$$

Hier ist $N = 4$ und

$$\sum_{n=1}^4 x_n = 1 + 0 + 1 + 1 = 3.$$

Also:

$$\hat{\pi}_{\text{ML}} = \frac{3}{4} = 0,75.$$

4.37 Zusammenfassung

- Ein Parameter θ ist eine feste, unbekannte Modellgröße.
- Ein Schätzer $\hat{\theta}_N$ ist eine Funktion der Daten.
- Ein Schätzwert ist der konkrete Wert des Schätzers auf einem konkreten Datensatz.
- i.i.d. bedeutet unabhängig und identisch verteilt.
- Bias misst systematische Abweichung:

$$\text{Bias}[\hat{\theta}_N] = \mathbb{E}[\hat{\theta}_N] - \theta.$$

- Varianz misst die Schwankung des Schätzers:

$$\text{Var}[\hat{\theta}_N] = \mathbb{E} \left[\left(\hat{\theta}_N - \mathbb{E}[\hat{\theta}_N] \right)^2 \right].$$

- Der mittlere quadratische Fehler ist:

$$\text{MSE}[\hat{\theta}_N] = \mathbb{E} \left[(\hat{\theta}_N - \theta)^2 \right].$$

- Es gilt:

$$\text{MSE} = \text{Bias}^2 + \text{Varianz}.$$

- Das Stichprobenmittel ist unverzerrt:

$$\text{Bias}[\hat{\mu}_N] = 0.$$

- Die Varianz des Stichprobenmittels ist:

$$\text{Var}[\hat{\mu}_N] = \frac{\sigma_X^2}{N}.$$

- Die naive Stichprobenvarianz ist verzerrt:

$$\text{Bias}[\hat{\sigma}_N^2] = -\frac{1}{N}\sigma_X^2.$$

- Die unverzerzte Stichprobenvarianz ist:

$$\hat{s}_N^2 = \frac{1}{N-1} \sum_{n=1}^N (X_n - \hat{\mu}_N)^2.$$

- Maximum-Likelihood schätzt Parameter durch Maximieren der Datenwahrscheinlichkeit:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} p(\vec{x} \mid \theta).$$

- MAP maximiert den Posterior:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\theta \mid \vec{x}).$$

- Bayesian Estimation betrachtet die ganze Posterior-Verteilung $p(\theta \mid \vec{x})$.

4.38 Typische Prüfungsfragen

Prüfungshinweis

Mögliche Prüfungsfragen zu diesem Kapitel:

1. Was ist der Unterschied zwischen Parameter, Schätzer und Schätzwert?
2. Warum ist ein Schätzer eine Zufallsvariable?
3. Was bedeutet i.i.d.?
4. Definiere Bias eines Schätzers.
5. Definiere Varianz eines Schätzers.
6. Definiere MSE und leite die Bias-Varianz-Zerlegung her.
7. Zeige, dass das Stichprobenmittel erwartungstreu ist.
8. Berechne die Varianz des Stichprobenmittels.
9. Warum ist die naive Stichprobenvarianz verzerrt?
10. Warum verwendet man bei der unverzerrten Stichprobenvarianz $N - 1$?
11. Was ist der Bias der naiven Stichprobenvarianz?
12. Was ist Maximum-Likelihood-Schätzung?
13. Warum verwendet man die Log-Likelihood?
14. Leite den MLE für eine Bernoulli-Verteilung her.
15. Was ist der Unterschied zwischen MLE und MAP?

16. Welche Rolle spielt der Prior bei MAP?
17. Warum ist Bayesian Estimation mehr als eine Punktschätzung?
18. Erkläre den Bias-Varianz-Trade-off.
19. Warum ist ein unverzerrter Schätzer nicht automatisch der beste Schätzer?
20. Wie hängt Parameterschätzung mit maschinellem Lernen zusammen?

5 Maximum-Likelihood, MAP und Bayessche Schätzung

5.1 Ziel dieses Kapitels

Im vorherigen Kapitel haben wir Schätzer (estimators) allgemein betrachtet. Wir haben gesehen, dass ein Schätzer eine Funktion der Daten ist und dass man Schätzer nach Bias, Varianz, mittlerem quadratischem Fehler und Konsistenz beurteilen kann.

In diesem Kapitel betrachten wir drei zentrale Prinzipien zur Parameterschätzung:

- Maximum-Likelihood-Schätzung (maximum likelihood estimation),
- Maximum-a-posteriori-Schätzung (maximum a-posteriori estimation),
- Bayessche Schätzung (Bayesian estimation).

Idee

Maximum-Likelihood-Schätzung (maximum likelihood estimation) verwendet nur die Daten.

Maximum-a-posteriori-Schätzung (maximum a-posteriori estimation) verwendet Daten und einen Prior.

Bayessche Schätzung (Bayesian estimation) berechnet nicht nur einen besten Parameterwert, sondern eine ganze Posterior-Verteilung über Parameter.

Nach diesem Kapitel solltest du folgende Fragen beantworten können:

- Was ist eine Likelihood?
- Warum ist die Likelihood keine Wahrscheinlichkeitsverteilung über Parameter?
- Wie funktioniert Maximum-Likelihood-Schätzung?
- Warum verwendet man die Log-Likelihood?
- Wie hängt Maximum-Likelihood mit Verlustfunktionen zusammen?
- Warum führt Gaußsches Rauschen zu quadratischem Fehler?
- Was ist ein Prior?
- Was ist ein Posterior?
- Wie wird MAP aus Bayes' Regel hergeleitet?

- Warum kann MAP als regularisierte Maximum-Likelihood-Schätzung verstanden werden?
- Was unterscheidet MLE, MAP und Bayesian Estimation?
- Was ist eine prädiktive Verteilung (predictive distribution)?

5.2 Grundsituation der Parameterschätzung

Wir betrachten ein Modell mit Parameter θ . Der Parameter θ kann eine einzelne Zahl, ein Vektor oder auch eine Menge mehrerer Parameter sein.

Beispiele:

$$\theta = \mu$$

für den Mittelwert einer Normalverteilung,

$$\theta = (\mu, \sigma^2)$$

für Mittelwert und Varianz einer Normalverteilung,
oder

$$\theta = \vec{w}$$

für die Gewichte eines linearen Modells.

Wir beobachten Daten

$$\mathcal{D} = \{x_1, x_2, \dots, x_N\}.$$

Oft nehmen wir an, dass die Daten unabhängig und identisch verteilt sind (independent and identically distributed), kurz i.i.d.:

$$x_1, \dots, x_N \stackrel{\text{i.i.d.}}{\sim} p(x \mid \theta).$$

Das bedeutet:

$$p(\mathcal{D} \mid \theta) = p(x_1, \dots, x_N \mid \theta) = \prod_{n=1}^N p(x_n \mid \theta).$$

Definition

Die Größe

$$p(\mathcal{D} \mid \theta)$$

beschreibt, wie wahrscheinlich oder wie dicht die beobachteten Daten $\mathcal{D}$ unter dem Parameter θ sind.

5.3 Likelihood

Die Likelihood ist dieselbe mathematische Größe wie $p(\mathcal{D} \mid \theta)$, aber mit anderer Interpretation.

Definition

Für feste beobachtete Daten $\mathcal{D}$ ist die Likelihood (likelihood) eine Funktion des Parameters θ :

$$L(\theta) = p(\mathcal{D} \mid \theta).$$

Sie misst, wie plausibel verschiedene Parameterwerte θ die beobachteten Daten machen.

Wichtig ist die Perspektive:

$$p(\mathcal{D} \mid \theta)$$

als Wahrscheinlichkeitsmodell: θ ist fest, $\mathcal{D}$ ist zufällig.

$$L(\theta)$$

als Likelihood: $\mathcal{D}$ ist fest beobachtet, θ wird variiert.

Hinweis

Die Likelihood $L(\theta)$ ist im Allgemeinen keine Wahrscheinlichkeitsverteilung über θ . Das heißt: Es gilt im Allgemeinen nicht

$$\int L(\theta) d\theta = 1.$$

Oder im diskreten Fall:

$$\sum_{\theta} L(\theta) = 1.$$

Die Likelihood ist nur bis auf ihre relative Größe für verschiedene Parameterwerte wichtig.

5.4 Maximum-Likelihood-Schätzung

Die Maximum-Likelihood-Schätzung (maximum likelihood estimation, MLE) wählt den Parameterwert, der die beobachteten Daten unter dem Modell maximal plausibel macht.

Definition

Der Maximum-Likelihood-Schätzer ist definiert als

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} p(\mathcal{D} \mid \theta).$$

Mit der Likelihood-Schreibweise:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} L(\theta).$$

Für i.i.d. Daten gilt:

$$p(\mathcal{D} \mid \theta) = \prod_{n=1}^N p(x_n \mid \theta).$$

Daher:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \prod_{n=1}^N p(x_n \mid \theta).$$

Idee

Maximum-Likelihood bedeutet:

Wähle θ so, dass die beobachteten Daten möglichst wahrscheinlich sind.

5.5 Log-Likelihood

Produkte vieler kleiner Wahrscheinlichkeiten oder Dichten sind numerisch und algebraisch unpraktisch. Deshalb maximiert man meistens den Logarithmus der Likelihood.

Definition

Die Log-Likelihood (log-likelihood) ist

$$\ell(\theta) = \log L(\theta) = \log p(\mathcal{D} \mid \theta).$$

Da der Logarithmus streng monoton wachsend ist, gilt:

$$\arg \max_{\theta} L(\theta) = \arg \max_{\theta} \log L(\theta).$$

Für i.i.d. Daten:

$$\ell(\theta) = \log \prod_{n=1}^N p(x_n \mid \theta).$$

Mit der Logarithmusregel

$$\log \prod_{n=1}^N a_n = \sum_{n=1}^N \log a_n$$

folgt:

$$\ell(\theta) = \sum_{n=1}^N \log p(x_n \mid \theta).$$

Idee

Die Log-Likelihood wandelt ein Produkt in eine Summe um:

$$\prod_{n=1}^N p(x_n | \theta) \longrightarrow \sum_{n=1}^N \log p(x_n | \theta).$$

Summen sind leichter abzuleiten, numerisch stabiler und passen direkt zu Verlustfunktionen.

5.6 Negative Log-Likelihood

In maschinellem Lernen formuliert man viele Probleme als Minimierung einer Verlustfunktion (loss function). Da Maximum-Likelihood ein Maximierungsproblem ist, verwendet man oft die negative Log-Likelihood.

Definition

Die negative Log-Likelihood (negative log-likelihood, NLL) ist

$$\mathcal{L}(\theta) = -\ell(\theta) = -\log p(\mathcal{D} | \theta).$$

Maximum-Likelihood ist äquivalent zur Minimierung der negativen Log-Likelihood:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \log p(\mathcal{D} | \theta)$$

ist äquivalent zu

$$\hat{\theta}_{\text{ML}} = \arg \min_{\theta} [-\log p(\mathcal{D} | \theta)].$$

Für i.i.d. Daten:

$$-\log p(\mathcal{D} | \theta) = -\sum_{n=1}^N \log p(x_n | \theta).$$

Also:

$$\mathcal{L}(\theta) = \sum_{n=1}^N -\log p(x_n | \theta).$$

Idee

Viele Verlustfunktionen im maschinellen Lernen sind negative Log-Likelihoods. Das bedeutet: Man kann viele Trainingsziele probabilistisch interpretieren.

5.7 Standardrezept für MLE-Herleitungen

Viele MLE-Aufgaben folgen demselben Muster.

Prüfungshinweis

Standardvorgehen für Maximum-Likelihood-Schätzung:

1. Modell $p(x \mid \theta)$ aufschreiben.
2. Likelihood für alle Daten bilden:

$$L(\theta) = \prod_{n=1}^N p(x_n \mid \theta).$$

3. Log-Likelihood bilden:

$$\ell(\theta) = \sum_{n=1}^N \log p(x_n \mid \theta).$$

4. Nach θ ableiten.
5. Ableitung gleich 0 setzen.
6. Nach θ auflösen.
7. Prüfen, ob ein Maximum vorliegt und ob die Lösung zulässig ist.

5.8 Beispiel: Bernoulli-Verteilung

Die Bernoulli-Verteilung (Bernoulli distribution) modelliert eine binäre Zufallsvariable:

$$X \in \{0,1\}.$$

Der Parameter ist

$$\pi \in [0,1].$$

Die Verteilung lautet:

$$p(x \mid \pi) = \pi^x (1 - \pi)^{1-x}.$$

Dabei gilt:

$$p(X = 1 \mid \pi) = \pi,$$

und

$$p(X = 0 \mid \pi) = 1 - \pi.$$

Wir beobachten Daten

$$x_1, \dots, x_N \in \{0,1\}.$$

5.8.1 Likelihood

Für i.i.d. Daten ist die Likelihood:

$$L(\pi) = \prod_{n=1}^N \pi^{x_n} (1 - \pi)^{1-x_n}.$$

5.8.2 Log-Likelihood

Die Log-Likelihood ist:

$$\ell(\pi) = \sum_{n=1}^N \log [\pi^{x_n} (1 - \pi)^{1-x_n}].$$

Mit Logarithmusregeln:

$$\ell(\pi) = \sum_{n=1}^N [x_n \log \pi + (1 - x_n) \log(1 - \pi)].$$

5.8.3 Ableitung

Ableiten nach π :

$$\frac{\partial \ell}{\partial \pi} = \sum_{n=1}^N \left[\frac{x_n}{\pi} - \frac{1 - x_n}{1 - \pi} \right].$$

Setze

$$S = \sum_{n=1}^N x_n.$$

Dann gilt:

$$\sum_{n=1}^N (1 - x_n) = N - S.$$

Damit:

$$\frac{\partial \ell}{\partial \pi} = \frac{S}{\pi} - \frac{N - S}{1 - \pi}.$$

Setze die Ableitung gleich 0:

$$\frac{S}{\pi} - \frac{N - S}{1 - \pi} = 0.$$

Also:

$$\frac{S}{\pi} = \frac{N - S}{1 - \pi}.$$

Kreuzmultiplizieren:

$$S(1 - \pi) = \pi(N - S).$$

Ausmultiplizieren:

$$S - S\pi = \pi N - \pi S.$$

Damit:

$$S = \pi N.$$

Also:

$$\hat{\pi}_{\text{ML}} = \frac{S}{N} = \frac{1}{N} \sum_{n=1}^N x_n.$$

Idee

Der Maximum-Likelihood-Schätzer für den Bernoulli-Parameter ist der relative Anteil der Einsen:

$$\hat{\pi}_{\text{ML}} = \frac{1}{N} \sum_{n=1}^N x_n.$$

5.9 Bernoulli-MLE als Klassifikationsverlust

Die negative Log-Likelihood der Bernoulli-Verteilung ist:

$$-\ell(\pi) = - \sum_{n=1}^N [x_n \log \pi + (1 - x_n) \log(1 - \pi)].$$

Für ein Modell, das für jeden Datenpunkt eine Wahrscheinlichkeit

$$y_n = p(t_n = 1 \mid \vec{x}_n)$$

ausgibt, erhält man den Verlust

$$\mathcal{L} = - \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

Definition

Der Ausdruck

$$- [t \log y + (1 - t) \log(1 - y)]$$

heißt binäre Kreuzentropie (binary cross-entropy).

Sie ist die negative Log-Likelihood eines Bernoulli-Modells.

Hinweis

Die binäre Kreuzentropie ist also nicht nur eine willkürliche Verlustfunktion. Sie entsteht direkt aus Maximum-Likelihood für Bernoulli-verteilte Zielwerte.

5.10 Beispiel: Normalverteilung mit bekanntem σ^2

Nun betrachten wir kontinuierliche Daten. Seien

$$X_1, \dots, X_N \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(\mu, \sigma^2),$$

wobei σ^2 bekannt ist und μ geschätzt werden soll.

Die Dichte ist:

$$p(x_n \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_n - \mu)^2}{2\sigma^2}\right).$$

5.10.1 Likelihood

Die Likelihood ist:

$$L(\mu) = \prod_{n=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_n - \mu)^2}{2\sigma^2}\right).$$

5.10.2 Log-Likelihood

Die Log-Likelihood ist:

$$\ell(\mu) = \sum_{n=1}^N \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{(x_n - \mu)^2}{2\sigma^2} \right].$$

Also:

$$\ell(\mu) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2.$$

Der erste Term hängt nicht von μ ab. Daher maximieren wir $\ell(\mu)$, indem wir minimieren:

$$\sum_{n=1}^N (x_n - \mu)^2.$$

Idee

Für Gaußsches Rauschen führt Maximum-Likelihood auf quadratische Fehler.
Das ist der Grund, warum Least Squares und Normalverteilung eng zusammenhängen.

5.10.3 Ableitung

Wir minimieren:

$$E(\mu) = \sum_{n=1}^N (x_n - \mu)^2.$$

Ableiten:

$$\frac{\partial E}{\partial \mu} = \sum_{n=1}^N 2(x_n - \mu)(-1).$$

Also:

$$\frac{\partial E}{\partial \mu} = -2 \sum_{n=1}^N (x_n - \mu).$$

Setze gleich 0:

$$-2 \sum_{n=1}^N (x_n - \mu) = 0.$$

Damit:

$$\sum_{n=1}^N (x_n - \mu) = 0.$$

Also:

$$\sum_{n=1}^N x_n - \sum_{n=1}^N \mu = 0.$$

Da μ konstant ist:

$$\sum_{n=1}^N \mu = N\mu.$$

Somit:

$$\sum_{n=1}^N x_n - N\mu = 0.$$

Daraus folgt:

$$\hat{\mu}_{\text{ML}} = \frac{1}{N} \sum_{n=1}^N x_n.$$

Definition

Für normalverteilte Daten mit bekannter Varianz ist der Maximum-Likelihood-Schätzer des Mittelwerts das Stichprobenmittel:

$$\hat{\mu}_{\text{ML}} = \frac{1}{N} \sum_{n=1}^N x_n.$$

5.11 Normalverteilung mit unbekanntem μ und σ^2

Jetzt seien beide Parameter unbekannt:

$$\theta = (\mu, \sigma^2).$$

Die Daten seien

$$X_1, \dots, X_N \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(\mu, \sigma^2).$$

Die Log-Likelihood ist:

$$\ell(\mu, \sigma^2) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^N (x_n - \mu)^2.$$

Maximum-Likelihood liefert:

$$\hat{\mu}_{\text{ML}} = \frac{1}{N} \sum_{n=1}^N x_n,$$

und

$$\hat{\sigma}_{\text{ML}}^2 = \frac{1}{N} \sum_{n=1}^N (x_n - \hat{\mu}_{\text{ML}})^2.$$

Hinweis

Der MLE für die Varianz der Normalverteilung verwendet den Nenner N , nicht $N - 1$.
Er ist daher bei endlichem N verzerrt:

$$\mathbb{E}[\hat{\sigma}_{\text{ML}}^2] = \frac{N-1}{N} \sigma^2.$$

5.12 Regression als Maximum-Likelihood

Maximum-Likelihood erklärt auch die lineare Regression (linear regression).

Wir betrachten Daten

$$(\vec{x}_1, t_1), \dots, (\vec{x}_N, t_N).$$

Ein lineares Modell ist:

$$y(\vec{x}, \vec{w}) = \vec{w}^\top \vec{x}.$$

Wir nehmen an, dass die Zielwerte mit Gaußschem Rauschen entstehen:

$$t_n = \vec{w}^\top \vec{x}_n + \epsilon_n,$$

wobei

$$\epsilon_n \sim \mathcal{N}(0, \sigma^2).$$

Dann gilt:

$$p(t_n | \vec{x}_n, \vec{w}, \sigma^2) = \mathcal{N}(t_n | \vec{w}^\top \vec{x}_n, \sigma^2).$$

Für i.i.d. Daten ist:

$$p(\vec{t} | X, \vec{w}, \sigma^2) = \prod_{n=1}^N \mathcal{N}(t_n | \vec{w}^\top \vec{x}_n, \sigma^2).$$

Die negative Log-Likelihood ist bis auf Konstanten:

$$\mathcal{L}(\vec{w}) = \frac{1}{2\sigma^2} \sum_{n=1}^N (t_n - \vec{w}^\top \vec{x}_n)^2.$$

Da $\frac{1}{2\sigma^2}$ positiv und konstant ist, ist das äquivalent zu:

$$E(\vec{w}) = \frac{1}{2} \sum_{n=1}^N (t_n - \vec{w}^\top \vec{x}_n)^2.$$

Idee

Lineare Regression mit quadratischem Fehler ist Maximum-Likelihood unter der Annahme von Gaußschem Rauschen.

$$\text{Gaußsches Rauschen} \implies \text{quadratischer Fehler.}$$

5.13 Prior

Maximum-Likelihood verwendet nur die Daten. In vielen Situationen haben wir aber Vorwissen über Parameter. Dieses Vorwissen wird durch einen Prior beschrieben.

Definition

Ein Prior $p(\theta)$ ist eine Wahrscheinlichkeitsverteilung über Parameter vor Betrachtung der aktuellen Daten.

Beispiele:

$$p(\mu) = \mathcal{N}(\mu | 0, \tau^2)$$

bedeutet: Vor den Daten halten wir Werte von μ nahe 0 für plausibler als sehr große Werte.

Für einen Gewichtsvektor $\vec{w}$ kann ein Prior sein:

$$p(\vec{w}) = \mathcal{N}(\vec{w} \mid \vec{0}, \alpha^{-1}I).$$

Dieser Prior bevorzugt kleine Gewichte.

Hinweis

Der Prior ist kein Trick, sondern eine Modellannahme.

Er beschreibt, welche Parameterwerte vor Beobachtung der Daten plausibel sind.

5.14 Posterior

Wenn Daten beobachtet werden, aktualisiert Bayes' Regel die Verteilung über Parameter.

$$p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \theta)p(\theta)}{p(\mathcal{D})}.$$

Definition

Der Posterior $p(\theta \mid \mathcal{D})$ ist die Verteilung über Parameter nach Beobachtung der Daten.

Die einzelnen Bestandteile sind:

$$\underbrace{p(\theta \mid \mathcal{D})}_{\text{Posterior}} = \frac{\underbrace{p(\mathcal{D} \mid \theta)}_{\text{Likelihood}} \underbrace{p(\theta)}_{\text{Prior}}}{\underbrace{p(\mathcal{D})}_{\text{Evidence}}}.$$

Die Evidence ist:

$$p(\mathcal{D}) = \int p(\mathcal{D} \mid \theta)p(\theta) d\theta.$$

Im diskreten Fall:

$$p(\mathcal{D}) = \sum_{\theta} p(\mathcal{D} \mid \theta)p(\theta).$$

Idee

Bayes' Regel für Parameter lautet:

$$\text{Posterior} = \frac{\text{Likelihood} \cdot \text{Prior}}{\text{Evidence}}.$$

Oder proportional:

$$p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta)p(\theta).$$

5.15 Maximum-a-posteriori-Schätzung

Die Maximum-a-posteriori-Schätzung (maximum a-posteriori estimation, MAP) wählt den Parameterwert, der den Posterior maximiert.

Definition

Der MAP-Schätzer ist

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\theta \mid \mathcal{D}).$$

Mit Bayes' Regel:

$$p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \theta)p(\theta)}{p(\mathcal{D})}.$$

Da $p(\mathcal{D})$ nicht von θ abhängt, ist der Nenner für die Maximierung irrelevant:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\mathcal{D} \mid \theta)p(\theta).$$

Mit Logarithmus:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} [\log p(\mathcal{D} \mid \theta) + \log p(\theta)].$$

Idee

MAP maximiert die Summe aus Log-Likelihood und Log-Prior:

$$\log p(\theta \mid \mathcal{D}) = \log p(\mathcal{D} \mid \theta) + \log p(\theta) - \log p(\mathcal{D}).$$

Für die Maximierung nach θ kann $\log p(\mathcal{D})$ ignoriert werden.

5.16 MLE als Spezialfall von MAP

Wenn der Prior konstant ist, dann ist MAP dasselbe wie MLE.

Angenommen:

$$p(\theta) = \text{konstant}.$$

Dann:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\mathcal{D} \mid \theta)p(\theta).$$

Da $p(\theta)$ konstant ist:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\mathcal{D} \mid \theta) = \hat{\theta}_{\text{ML}}.$$

Hinweis

Man sagt oft: MLE entspricht MAP mit einem flachen oder uninformativem Prior. Das ist eine nützliche Intuition, aber mathematisch muss man bei kontinuierlichen Parametern vorsichtig sein, weil ein wirklich konstanter Prior auf ganz $\mathbb{R}$ nicht normalisierbar ist.

5.17 MAP und Regularisierung

MAP ist besonders wichtig, weil es Regularisierung (regularization) probabilistisch erklärt.

Wir betrachten wieder ein lineares Modell:

$$y(\vec{x}, \vec{w}) = \vec{w}^\top \vec{x}.$$

Die Likelihood unter Gaußschem Rauschen führt auf den quadratischen Fehler:

$$\frac{1}{2\sigma^2} \sum_{n=1}^N \left(t_n - \vec{w}^\top \vec{x}_n \right)^2.$$

Nun nehmen wir einen Gauß-Prior auf $\vec{w}$ an:

$$p(\vec{w}) = \mathcal{N}(\vec{w} \mid \vec{0}, \alpha^{-1} I).$$

Die Dichte ist proportional zu:

$$p(\vec{w}) \propto \exp\left(-\frac{\alpha}{2} \vec{w}^\top \vec{w}\right).$$

Der negative Log-Prior ist daher bis auf Konstanten:

$$-\log p(\vec{w}) = \frac{\alpha}{2} \vec{w}^\top \vec{w} + \text{const.}$$

Die negative Log-Posterior ist also bis auf Konstanten:

$$\mathcal{L}_{\text{MAP}}(\vec{w}) = \frac{1}{2\sigma^2} \sum_{n=1}^N \left(t_n - \vec{w}^\top \vec{x}_n \right)^2 + \frac{\alpha}{2} \vec{w}^\top \vec{w}.$$

Multipliziert man mit der positiven Konstanten σ^2 , erhält man ein äquivalentes Optimierungsproblem:

$$E(\vec{w}) = \frac{1}{2} \sum_{n=1}^N \left(t_n - \vec{w}^\top \vec{x}_n \right)^2 + \frac{\lambda}{2} \vec{w}^\top \vec{w},$$

mit

$$\lambda = \alpha\sigma^2.$$

Idee

Ein Gauß-Prior auf Gewichte führt zu L_2 -Regularisierung:

$$\frac{\lambda}{2} \vec{w}^\top \vec{w}.$$

Das ist die probabilistische Interpretation von Ridge Regression.

5.18 Laplace-Prior und L_1 -Regularisierung

Nicht nur Gauß-Priors sind möglich. Ein Laplace-Prior auf Gewichte hat die Form:

$$p(w_i) \propto \exp(-\lambda |w_i|).$$

Für einen Gewichtsvektor mit unabhängigen Komponenten:

$$p(\vec{w}) \propto \exp\left(-\lambda \sum_i |w_i|\right).$$

Dann ist der negative Log-Prior proportional zu:

$$\lambda \sum_i |w_i|.$$

Idee

Ein Laplace-Prior führt zu L_1 -Regularisierung:

$$\lambda \sum_i |w_i|.$$

L_1 -Regularisierung fördert sparse Lösungen, also Lösungen, bei denen viele Gewichte exakt oder nahezu 0 sind.

5.19 Bayessche Schätzung

MLE und MAP liefern jeweils einen einzelnen Parameterwert. Bayessche Schätzung (Bayesian estimation) betrachtet dagegen die ganze Posterior-Verteilung.

Definition

Bei Bayesscher Schätzung wird der Parameter θ selbst als Zufallsvariable behandelt. Nach Beobachtung der Daten beschreibt der Posterior

$$p(\theta \mid \mathcal{D})$$

die Unsicherheit über θ .

Das Ziel ist also nicht nur:

$$\hat{\theta}$$

zu finden, sondern die gesamte Verteilung

$$p(\theta \mid \mathcal{D})$$

zu berechnen oder zu approximieren.

Idee

Bayesian Estimation behält Unsicherheit über Parameter bei.

Das ist besonders wichtig, wenn wenige Daten vorhanden sind oder wenn Entscheidungen die Unsicherheit berücksichtigen sollen.

5.20 Punktschätzer aus dem Posterior

Auch in der Bayesschen Statistik kann man aus dem Posterior einen einzelnen Punktschätzer ableiten.

5.20.1 Posterior-Mittelwert

Der Posterior-Mittelwert ist:

$$\hat{\theta}_{\text{Bayes}} = \mathbb{E}[\theta \mid \mathcal{D}].$$

Im kontinuierlichen Fall:

$$\hat{\theta}_{\text{Bayes}} = \int \theta p(\theta \mid \mathcal{D}) d\theta.$$

5.20.2 Posterior-Modus

Der Posterior-Modus ist der Wert mit größter Posterior-Dichte:

$$\hat{\theta}_{\text{mode}} = \arg \max_{\theta} p(\theta \mid \mathcal{D}).$$

Das ist genau der MAP-Schätzer:

$$\hat{\theta}_{\text{mode}} = \hat{\theta}_{\text{MAP}}.$$

Hinweis

MAP ist ein Punktschätzer aus dem Posterior.

Bayessche Schätzung als Ganzes ist aber mehr als MAP: Sie betrachtet die gesamte Posterior-Verteilung.

5.21 Prädiktive Verteilung

In maschinellem Lernen wollen wir oft nicht nur Parameter schätzen, sondern Vorhersagen für neue Daten machen.

Sei x_* ein neuer Eingabewert und t_* der unbekannte Zielwert. Wenn θ unbekannt ist, sollte man die Unsicherheit über θ in der Vorhersage berücksichtigen.

Definition

Die prädiktive Verteilung (predictive distribution) ist

$$p(t_* | x_*, \mathcal{D}) = \int p(t_* | x_*, \theta) p(\theta | \mathcal{D}) d\theta.$$

Diese Formel mittelt Vorhersagen über alle möglichen Parameterwerte, gewichtet mit deren Posterior-Wahrscheinlichkeit.

Idee

Bayessche Vorhersage integriert über Parameterunsicherheit:

Vorhersage = Mittel über alle plausiblen Parameter.

Das unterscheidet Bayesian Prediction von einer reinen Punktschätzung.

5.22 Beispiel: Beta-Bernoulli-Modell

Ein klassisches Beispiel für Bayessche Schätzung ist das Beta-Bernoulli-Modell.

Wir beobachten binäre Daten:

$$x_n \in \{0,1\}.$$

Die Likelihood ist Bernoulli:

$$p(x_n | \pi) = \pi^{x_n} (1 - \pi)^{1-x_n}.$$

Für den Parameter π verwenden wir einen Beta-Prior:

$$p(\pi) = \text{Beta}(\pi | a, b).$$

Die Beta-Verteilung hat die Form:

$$\text{Beta}(\pi | a, b) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \pi^{a-1} (1-\pi)^{b-1},$$

für

$$0 \leq \pi \leq 1.$$

5.22.1 Likelihood

Für N Datenpunkte:

$$p(\mathcal{D} \mid \pi) = \prod_{n=1}^N \pi^{x_n} (1 - \pi)^{1-x_n}.$$

Setze

$$S = \sum_{n=1}^N x_n.$$

Dann ist

$$\sum_{n=1}^N (1 - x_n) = N - S.$$

Also:

$$p(\mathcal{D} \mid \pi) = \pi^S (1 - \pi)^{N-S}.$$

5.22.2 Posterior

Nach Bayes:

$$p(\pi \mid \mathcal{D}) \propto p(\mathcal{D} \mid \pi) p(\pi).$$

Einsetzen:

$$p(\pi \mid \mathcal{D}) \propto \pi^S (1 - \pi)^{N-S} \pi^{a-1} (1 - \pi)^{b-1}.$$

Zusammenfassen der Exponenten:

$$p(\pi \mid \mathcal{D}) \propto \pi^{a+S-1} (1 - \pi)^{b+N-S-1}.$$

Das ist wieder eine Beta-Verteilung:

$$p(\pi \mid \mathcal{D}) = \text{Beta}(\pi \mid a + S, b + N - S).$$

Idee

Beim Beta-Bernoulli-Modell ist der Posterior wieder eine Beta-Verteilung:

$$\text{Beta}(a, b) \longrightarrow \text{Beta}(a + S, b + N - S).$$

Der Parameter a zählt sozusagen vorherige Erfolge, b vorherige Misserfolge.

5.23 Konjugierte Priors

Das Beta-Bernoulli-Beispiel zeigt eine wichtige Idee.

Definition

Ein Prior heißt konjugierter Prior (conjugate prior) für eine Likelihood, wenn der Posterior zur selben Verteilungsfamilie gehört wie der Prior.

Beim Beta-Bernoulli-Modell:

Prior = Beta,

Likelihood = Bernoulli,

Posterior = Beta.

Hinweis

Konjugierte Priors sind mathematisch praktisch, weil der Posterior analytisch berechnet werden kann.

In komplexeren ML-Modellen ist der Posterior oft nicht analytisch berechenbar. Dann braucht man Approximationen.

5.24 Vergleich von MLE, MAP und Bayesscher Schätzung

Methode	Ziel	Prior	Ergebnis
MLE	$\max_{\theta} p(\mathcal{D} \mid \theta)$	nein	$\hat{\theta}_{\text{ML}}$
MAP	$\max_{\theta} p(\theta \mid \mathcal{D})$	ja	$\hat{\theta}_{\text{MAP}}$
Bayesian	$p(\theta \mid \mathcal{D})$	ja	Posterior-Verteilung

Idee

MLE:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} p(\mathcal{D} \mid \theta).$$

MAP:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\mathcal{D} \mid \theta)p(\theta).$$

Bayesian:

$$p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \theta)p(\theta)}{p(\mathcal{D})}.$$

5.25 Interpretation bei wenigen und vielen Daten

Der Unterschied zwischen MLE, MAP und Bayesian Estimation ist besonders bei kleinen Datensätzen wichtig.

5.25.1 Wenige Daten

Wenn nur wenige Daten vorhanden sind, kann MLE stark schwanken. Ein Prior kann dann helfen, un plausible Parameterwerte zu vermeiden.

wenige Daten $\Rightarrow$ Prior hat starken Einfluss.

5.25.2 Viele Daten

Wenn sehr viele Daten vorhanden sind, dominiert oft die Likelihood. Dann wird der Einfluss des Priors relativ kleiner.

viele Daten $\Rightarrow$ Likelihood dominiert.

Hinweis

Das bedeutet nicht, dass der Prior bei vielen Daten immer egal ist. Aber in vielen einfachen regulären Modellen wird sein Einfluss im Vergleich zur Datenmenge kleiner.

5.26 Zusammenhang zu Overfitting

Maximum-Likelihood kann bei flexiblen Modellen zu Überanpassung (overfitting) führen. Das passiert, wenn Parameter so gewählt werden, dass sie die Trainingsdaten sehr gut erklären, aber auf neuen Daten schlecht generalisieren.

MAP kann dem entgegenwirken, wenn der Prior komplexe oder extreme Parameterwerte bestraft.

Idee

Regularisierung kann probabilistisch als Prior interpretiert werden.

Regularisierung $\Longleftrightarrow$ Prior auf Parameter.

Beispiel:

$$\frac{\lambda}{2} \vec{w}^\top \vec{w}$$

bestraft große Gewichte. Das entspricht einem Gauß-Prior auf $\vec{w}$.

5.27 Häufige Fehler

Prüfungshinweis

Typische Fehler bei MLE, MAP und Bayesian Estimation:

1. Likelihood $p(\mathcal{D} \mid \theta)$ mit Posterior $p(\theta \mid \mathcal{D})$ verwechseln.
2. Denken, dass die Likelihood eine Wahrscheinlichkeitsverteilung über θ sein muss.

3. Bei MLE den Logarithmus falsch verwenden.
4. Vergessen, dass Maximieren der Log-Likelihood äquivalent zum Maximieren der Likelihood ist.
5. Negative Log-Likelihood und Log-Likelihood mit falschem Vorzeichen verwenden.
6. Bei MAP den Prior vergessen.
7. Die Evidence $p(\mathcal{D})$ bei der Maximierung nach θ unnötig mitoptimieren.
8. MAP mit vollständiger Bayesscher Schätzung gleichsetzen.
9. Quadratischen Fehler als rein willkürlichen Verlust betrachten, obwohl er aus Gaußschem Rauschen folgt.
10. L_2 -Regularisierung nicht als Gauß-Prior erkennen.

5.28 Mini-Aufgaben

5.28.1 Aufgabe 1: Bernoulli-MLE

Gegeben sind die Daten

$$x_1 = 1, \quad x_2 = 0, \quad x_3 = 1, \quad x_4 = 1, \quad x_5 = 0.$$

Berechne den Maximum-Likelihood-Schätzer für π .

Beispiel

Für Bernoulli-Daten gilt:

$$\hat{\pi}_{\text{ML}} = \frac{1}{N} \sum_{n=1}^N x_n.$$

Hier ist:

$$N = 5$$

und

$$\sum_{n=1}^5 x_n = 1 + 0 + 1 + 1 + 0 = 3.$$

Also:

$$\hat{\pi}_{\text{ML}} = \frac{3}{5} = 0,6.$$

5.28.2 Aufgabe 2: Negative Log-Likelihood

Für ein Bernoulli-Modell sei

$$t = 1$$

und das Modell sagt

$$y = 0,8.$$

Berechne die negative Log-Likelihood für diesen einzelnen Datenpunkt.

Beispiel

Die negative Log-Likelihood ist:

$$- [t \log y + (1 - t) \log(1 - y)].$$

Mit $t = 1$:

$$- [1 \cdot \log(0,8) + 0 \cdot \log(0,2)] = -\log(0,8).$$

Also:

$$\mathcal{L} = -\log(0,8).$$

5.28.3 Aufgabe 3: MAP mit konstantem Prior

Zeige, dass MAP mit konstantem Prior zu MLE wird.

Beispiel

MAP ist:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\mathcal{D} \mid \theta) p(\theta).$$

Wenn

$$p(\theta) = c$$

konstant ist, dann:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} c p(\mathcal{D} \mid \theta).$$

Da c positiv und konstant ist, ändert es die Stelle des Maximums nicht:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\mathcal{D} \mid \theta) = \hat{\theta}_{\text{ML}}.$$

5.28.4 Aufgabe 4: Beta-Bernoulli-Posterior

Ein Prior sei

$$\pi \sim \text{Beta}(2,3).$$

In den Daten gibt es

$$S = 4$$

Einsen und

$$N - S = 6$$

Nullen. Berechne den Posterior.

Beispiel

Beim Beta-Bernoulli-Modell gilt:

$$\text{Beta}(a,b) \longrightarrow \text{Beta}(a + S, b + N - S).$$

Hier ist:

$$a = 2, \quad b = 3, \quad S = 4, \quad N - S = 6.$$

Also:

$$\pi \mid \mathcal{D} \sim \text{Beta}(2 + 4, 3 + 6).$$

Damit:

$$\pi \mid \mathcal{D} \sim \text{Beta}(6,9).$$

5.29 Zusammenfassung

- Die Likelihood ist:

$$L(\theta) = p(\mathcal{D} \mid \theta).$$

- Bei i.i.d. Daten faktorisiert sie:

$$p(\mathcal{D} \mid \theta) = \prod_{n=1}^N p(x_n \mid \theta).$$

- Die Log-Likelihood ist:

$$\ell(\theta) = \sum_{n=1}^N \log p(x_n \mid \theta).$$

- MLE maximiert die Likelihood:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} p(\mathcal{D} \mid \theta).$$

- Negative Log-Likelihood ist eine Verlustfunktion:

$$\mathcal{L}(\theta) = -\log p(\mathcal{D} \mid \theta).$$

- Bernoulli-MLE ergibt den Anteil der Einsen:

$$\hat{\pi}_{\text{ML}} = \frac{1}{N} \sum_{n=1}^N x_n.$$

- Gaußsche Likelihood führt zu quadratischem Fehler.
- Ein Prior $p(\theta)$ beschreibt Vorwissen über Parameter.
- Der Posterior ist:

$$p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \theta)p(\theta)}{p(\mathcal{D})}.$$

- MAP maximiert den Posterior:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\theta \mid \mathcal{D}).$$

- MAP ist äquivalent zu:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\mathcal{D} \mid \theta)p(\theta).$$

- Ein Gauß-Prior auf Gewichte führt zu L_2 -Regularisierung.
- Ein Laplace-Prior führt zu L_1 -Regularisierung.
- Bayessche Schätzung betrachtet die gesamte Posterior-Verteilung $p(\theta \mid \mathcal{D})$.
- Die prädiktive Verteilung integriert über Parameterunsicherheit:

$$p(t_* \mid x_*, \mathcal{D}) = \int p(t_* \mid x_*, \theta)p(\theta \mid \mathcal{D}) d\theta.$$

5.30 Typische Prüfungsfragen

Prüfungshinweis

Mögliche Prüfungsfragen zu diesem Kapitel:

1. Was ist eine Likelihood?
2. Warum ist die Likelihood keine Wahrscheinlichkeitsverteilung über Parameter?
3. Definiere den Maximum-Likelihood-Schätzer.

4. Warum verwendet man die Log-Likelihood?
5. Was ist die negative Log-Likelihood?
6. Leite den MLE für den Parameter einer Bernoulli-Verteilung her.
7. Warum führt Bernoulli-Likelihood zur binären Kreuzentropie?
8. Leite den MLE für den Mittelwert einer Normalverteilung mit bekannter Varianz her.
9. Warum führt Gaußsches Rauschen zu quadratischem Fehler?
10. Was ist ein Prior?
11. Was ist ein Posterior?
12. Leite MAP aus Bayes' Regel her.
13. Warum kann die Evidence bei MAP ignoriert werden?
14. Wann ist MAP gleich MLE?
15. Wie hängt MAP mit Regularisierung zusammen?
16. Warum führt ein Gauß-Prior zu L_2 -Regularisierung?
17. Warum führt ein Laplace-Prior zu L_1 -Regularisierung?
18. Was ist der Unterschied zwischen MAP und vollständiger Bayesscher Schätzung?
19. Was ist eine prädiktive Verteilung?
20. Erkläre das Beta-Bernoulli-Modell.

6 Lineare Regression

6.1 Ziel dieses Kapitels

In diesem Kapitel betrachten wir lineare Regression (linear regression). Lineare Regression ist eines der wichtigsten Grundmodelle im maschinellen Lernen. Sie ist mathematisch überschaubar, aber viele zentrale Ideen späterer Modelle tauchen hier bereits auf:

- Modellfunktion (model function),
- Parameter oder Gewichte (parameters, weights),
- Designmatrix (design matrix),
- Fehlerfunktion (error function),
- kleinste Quadrate (least squares),
- Maximum-Likelihood-Interpretation (maximum likelihood interpretation),
- Regularisierung (regularization),
- Bias-Term,
- Basisfunktionen (basis functions),
- Generalisierung (generalization).

Idee

Lineare Regression (linear regression) ist ein überwachtes Lernverfahren (supervised learning) für kontinuierliche Zielwerte.

Das Modell sagt einen reellen Zielwert t aus einer Eingabe $\vec{x}$ vorher.

Nach diesem Kapitel solltest du folgende Fragen beantworten können:

- Was ist lineare Regression (linear regression)?
- Was bedeutet linear in den Parametern?
- Was ist ein Bias-Term?
- Was ist eine Designmatrix (design matrix)?
- Wie schreibt man lineare Regression in Matrixform?
- Was ist die Sum-of-Squares-Fehlerfunktion?

- Wie leitet man die Normalengleichung (normal equation) her?
- Wann ist die Lösung eindeutig?
- Wie hängt lineare Regression mit Maximum-Likelihood zusammen?
- Warum führt Gaußsches Rauschen zu quadratischem Fehler?
- Was ist Ridge Regression?
- Wie hängt Ridge Regression mit MAP zusammen?
- Was sind Basisfunktionen?
- Warum kann ein Modell nichtlinear in der Eingabe, aber linear in den Parametern sein?

6.2 Regression als überwachtes Lernen

Bei der Regression (regression) wollen wir einen kontinuierlichen Zielwert vorhersagen. Ein Trainingsdatensatz besteht aus Eingaben und Zielwerten:

$$\mathcal{D} = \{(\vec{x}_1, t_1), (\vec{x}_2, t_2), \dots, (\vec{x}_N, t_N)\}.$$

Dabei ist

$$\vec{x}_n \in \mathbb{R}^D$$

der Eingabevektor des n -ten Datenpunkts und

$$t_n \in \mathbb{R}$$

der zugehörige Zielwert.

Das Ziel ist, ein Modell $y(\vec{x})$ zu lernen, sodass

$$y(\vec{x}_n) \approx t_n$$

für die Trainingsdaten gilt und das Modell auch für neue Daten gut generalisiert.

Definition

Regression (regression) ist ein überwachtes Lernproblem, bei dem der Zielwert kontinuierlich ist:

$$t \in \mathbb{R}.$$

Beispiel

Beispiele für Regression:

- Wohnungsdaten $\vec{x} \rightarrow$ Mietpreis t ,

- Wetterdaten $\vec{x} \rightarrow$ Temperatur t ,
- medizinische Messwerte $\vec{x} \rightarrow$ Laborwert t ,
- Sensordaten $\vec{x} \rightarrow$ erwartete Lebensdauer eines Bauteils t .

6.3 Das einfachste lineare Modell

Wir beginnen mit einer eindimensionalen Eingabe

$$x \in \mathbb{R}.$$

Ein einfaches lineares Modell ist eine Gerade:

$$y(x, w_0, w_1) = w_0 + w_1 x.$$

Dabei sind w_0 und w_1 Parameter.

Definition

Das Modell

$$y(x, w_0, w_1) = w_0 + w_1 x$$

heißt lineares Regressionsmodell für eine eindimensionale Eingabe.

Der Parameter w_0 ist der Bias oder Achsenabschnitt (bias, intercept). Der Parameter w_1 ist die Steigung (slope).

Idee

Der Bias-Term w_0 erlaubt dem Modell, die Gerade nach oben oder unten zu verschieben. Ohne Bias-Term hätte das Modell die Form

$$y(x, w_1) = w_1 x$$

und müsste immer durch den Ursprung gehen.

6.4 Lineare Regression mit mehreren Eingabedimensionen

In maschinellern Lernen ist die Eingabe meistens ein Vektor:

$$\vec{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_D \end{pmatrix} \in \mathbb{R}^D.$$

Ein lineares Modell mit Bias-Term ist:

$$y(\vec{x}, \vec{w}) = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_D x_D.$$

Hier sind

$$w_0, w_1, \dots, w_D$$

die Parameter des Modells.

Definition

Die lineare Regression für $\vec{x} \in \mathbb{R}^D$ hat die Form

$$y(\vec{x}, \vec{w}) = w_0 + \sum_{j=1}^D w_j x_j.$$

Der Parameter w_0 ist wieder der Bias-Term. Die Parameter $w_1, \dots, w_D$ bestimmen, wie stark die einzelnen Merkmale (features) auf die Vorhersage wirken.

Beispiel

Angenommen,

$$\vec{x} = \begin{pmatrix} \text{Wohnfläche} \\ \text{Anzahl Zimmer} \\ \text{Entfernung zum Zentrum} \end{pmatrix}.$$

Dann könnte ein lineares Modell sein:

$$y(\vec{x}, \vec{w}) = w_0 + w_1 \cdot \text{Wohnfläche} + w_2 \cdot \text{Anzahl Zimmer} + w_3 \cdot \text{Entfernung zum Zentrum}.$$

Wenn $w_1 > 0$, dann erhöht größere Wohnfläche die vorhergesagte Miete. Wenn $w_3 < 0$, dann senkt größere Entfernung zum Zentrum die vorhergesagte Miete.

6.5 Vektorform mit Bias

Es ist praktisch, den Bias-Term in den Eingabevektor einzubauen. Dazu erweitern wir den Eingabevektor um eine konstante Komponente:

$$\tilde{\vec{x}} = \begin{pmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_D \end{pmatrix} \in \mathbb{R}^{D+1}.$$

Der Gewichtsvektor ist:

$$\vec{w} = \begin{pmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_D \end{pmatrix} \in \mathbb{R}^{D+1}.$$

Dann kann das Modell kompakt geschrieben werden als:

$$y(\vec{x}, \vec{w}) = \vec{w}^\top \tilde{\vec{x}}.$$

Hinweis

Oft schreibt man wieder einfach $\vec{x}$ statt $\tilde{\vec{x}}$, obwohl die 1 für den Bias bereits eingefügt wurde.

Dann gilt:

$$y(\vec{x}, \vec{w}) = \vec{w}^\top \vec{x}.$$

Dabei muss aus dem Kontext klar sein, ob $\vec{x}$ den Bias-Eintrag enthält.

6.6 Was bedeutet linear?

Der Begriff linear in linearer Regression kann verwirrend sein. Wichtig ist:

Lineare Regression ist linear in den Parametern, nicht notwendigerweise linear in der ursprünglichen Eingabe.

Das einfache Modell

$$y(x, \vec{w}) = w_0 + w_1 x$$

ist linear in x und linear in den Parametern.

Aber auch das Modell

$$y(x, \vec{w}) = w_0 + w_1 x + w_2 x^2$$

ist linear in den Parametern w_0, w_1, w_2 , obwohl es nichtlinear in x ist.

Definition

Ein Modell ist linear in den Parametern, wenn es als Linearkombination von festen Funktionen geschrieben werden kann:

$$y(x, \vec{w}) = \sum_{j=0}^{M-1} w_j \phi_j(x).$$

Die Funktionen $\phi_j(x)$ heißen Basisfunktionen (basis functions).

Beispiel

Für polynomiale Basisfunktionen gilt:

$$\phi_0(x) = 1, \quad \phi_1(x) = x, \quad \phi_2(x) = x^2, \quad \phi_3(x) = x^3.$$

Dann ist

$$y(x, \vec{w}) = w_0 + w_1x + w_2x^2 + w_3x^3.$$

Dieses Modell ist linear in $\vec{w}$, aber nichtlinear in x .

6.7 Basisfunktionen

Basisfunktionen (basis functions) transformieren Eingaben in neue Merkmale. Statt direkt mit x zu arbeiten, arbeitet das Modell mit

$$\phi_0(x), \phi_1(x), \dots, \phi_{M-1}(x).$$

Das Modell lautet:

$$y(x, \vec{w}) = \sum_{j=0}^{M-1} w_j \phi_j(x).$$

In Vektorform:

$$\vec{\phi}(x) = \begin{pmatrix} \phi_0(x) \\ \phi_1(x) \\ \vdots \\ \phi_{M-1}(x) \end{pmatrix}.$$

Dann:

$$y(x, \vec{w}) = \vec{w}^\top \vec{\phi}(x).$$

Definition

Eine Basisfunktion (basis function) ist eine feste Funktion $\phi_j(x)$, die aus der Eingabe ein neues Merkmal berechnet.

Das Modell bleibt linear in den Gewichten:

$$y(x, \vec{w}) = \vec{w}^\top \vec{\phi}(x).$$

6.7.1 Typische Basisfunktionen

Typische Basisfunktionen sind:

- Polynomiale Basisfunktionen:

$$\phi_j(x) = x^j.$$

- Gaußsche Basisfunktionen:

$$\phi_j(x) = \exp\left(-\frac{(x - \mu_j)^2}{2s^2}\right).$$

- Sigmoid-Basisfunktionen:

$$\phi_j(x) = \frac{1}{1 + \exp(-a_j(x - b_j))}.$$

Idee

Basisfunktionen erlauben nichtlineare Kurven, obwohl das Modell linear in den Parametern bleibt.

Dadurch können wir viele Methoden der linearen Algebra weiterverwenden.

6.8 Trainingsdaten in Matrixform

Wir betrachten nun N Trainingsdatenpunkte

$$(x_1, t_1), \dots, (x_N, t_N)$$

und M Basisfunktionen.

Für jeden Datenpunkt x_n bilden wir den Feature-Vektor

$$\vec{\phi}(x_n) = \begin{pmatrix} \phi_0(x_n) \\ \phi_1(x_n) \\ \vdots \\ \phi_{M-1}(x_n) \end{pmatrix}.$$

Die Designmatrix (design matrix) Φ enthält diese Feature-Vektoren als Zeilen:

$$\Phi = \begin{pmatrix} \phi_0(x_1) & \phi_1(x_1) & \dots & \phi_{M-1}(x_1) \\ \phi_0(x_2) & \phi_1(x_2) & \dots & \phi_{M-1}(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0(x_N) & \phi_1(x_N) & \dots & \phi_{M-1}(x_N) \end{pmatrix}.$$

Damit gilt:

$$\Phi \in \mathbb{R}^{N \times M}.$$

Der Zielwertvektor ist:

$$\vec{t} = \begin{pmatrix} t_1 \\ t_2 \\ \vdots \\ t_N \end{pmatrix} \in \mathbb{R}^N.$$

Der Gewichtsvektor ist:

$$\vec{w} = \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_{M-1} \end{pmatrix} \in \mathbb{R}^M.$$

Die Vorhersagen für alle Datenpunkte sind:

$$\vec{y} = \Phi \vec{w}.$$

Definition

Die Designmatrix (design matrix) Φ ist die Matrix aller transformierten Eingaben:

$$\Phi_{nj} = \phi_j(x_n).$$

Jede Zeile entspricht einem Datenpunkt. Jede Spalte entspricht einer Basisfunktion.

Hinweis

Die Dimensionen sind wichtig:

$$\Phi \in \mathbb{R}^{N \times M}, \quad \vec{w} \in \mathbb{R}^M, \quad \vec{t} \in \mathbb{R}^N.$$

Dann gilt:

$$\Phi \vec{w} \in \mathbb{R}^N.$$

6.9 Fehlerfunktion der linearen Regression

Wir möchten, dass die Vorhersagen

$$y(x_n, \vec{w})$$

möglichst nahe bei den Zielwerten t_n liegen. Ein natürlicher Fehler ist die Differenz

$$y(x_n, \vec{w}) - t_n.$$

Da positive und negative Fehler sich nicht gegenseitig aufheben sollen, quadrieren wir sie. Die Sum-of-Squares-Fehlerfunktion ist:

$$E_D(\vec{w}) = \frac{1}{2} \sum_{n=1}^N (y(x_n, \vec{w}) - t_n)^2.$$

In Matrixform:

$$E_D(\vec{w}) = \frac{1}{2} \|\Phi \vec{w} - \vec{t}\|^2.$$

Das bedeutet:

$$E_D(\vec{w}) = \frac{1}{2} (\Phi \vec{w} - \vec{t})^\top (\Phi \vec{w} - \vec{t}).$$

Definition

Die quadratische Fehlerfunktion (sum-of-squares error function) der linearen Regression ist

$$E_D(\vec{w}) = \frac{1}{2} \sum_{n=1}^N \left(\vec{w}^\top \vec{\phi}(x_n) - t_n \right)^2.$$

In Matrixform:

$$E_D(\vec{w}) = \frac{1}{2} \|\Phi \vec{w} - \vec{t}\|^2.$$

Hinweis

Der Faktor $\frac{1}{2}$ ändert die Lage des Minimums nicht. Er ist nur praktisch, weil beim Ableiten des Quadrats ein Faktor 2 entsteht, der sich mit $\frac{1}{2}$ kürzt.

6.10 Least Squares

Das Trainingsproblem der linearen Regression lautet:

$$\vec{w}^* = \arg \min_{\vec{w}} E_D(\vec{w}).$$

Also:

$$\vec{w}^* = \arg \min_{\vec{w}} \frac{1}{2} \|\Phi \vec{w} - \vec{t}\|^2.$$

Definition

Least Squares oder kleinste Quadrate bezeichnet das Minimieren der Summe quadratischer Fehler:

$$\min_{\vec{w}} \sum_{n=1}^N (y(x_n, \vec{w}) - t_n)^2.$$

6.11 Gradient der Fehlerfunktion

Um das Minimum zu finden, leiten wir $E_D(\vec{w})$ nach $\vec{w}$ ab.

Wir starten mit:

$$E_D(\vec{w}) = \frac{1}{2}(\Phi\vec{w} - \vec{t})^\top(\Phi\vec{w} - \vec{t}).$$

Ausmultiplizieren:

$$E_D(\vec{w}) = \frac{1}{2} \left(\vec{w}^\top \Phi^\top \Phi \vec{w} - \vec{w}^\top \Phi^\top \vec{t} - \vec{t}^\top \Phi \vec{w} + \vec{t}^\top \vec{t} \right).$$

Da

$$\vec{w}^\top \Phi^\top \vec{t}$$

ein Skalar ist, gilt:

$$\vec{w}^\top \Phi^\top \vec{t} = \vec{t}^\top \Phi \vec{w}.$$

Also:

$$E_D(\vec{w}) = \frac{1}{2} \left(\vec{w}^\top \Phi^\top \Phi \vec{w} - 2\vec{w}^\top \Phi^\top \vec{t} + \vec{t}^\top \vec{t} \right).$$

Wir verwenden die Ableitungsregeln:

$$\nabla_{\vec{w}} \left(\frac{1}{2} \vec{w}^\top A \vec{w} \right) = A \vec{w}$$

für symmetrisches A , und

$$\nabla_{\vec{w}} \left(\vec{w}^\top \vec{b} \right) = \vec{b}.$$

Hier ist

$$A = \Phi^\top \Phi$$

symmetrisch. Damit:

$$\nabla_{\vec{w}} E_D(\vec{w}) = \Phi^\top \Phi \vec{w} - \Phi^\top \vec{t}.$$

Definition

Der Gradient der quadratischen Fehlerfunktion ist

$$\nabla_{\vec{w}} E_D(\vec{w}) = \Phi^\top (\Phi \vec{w} - \vec{t}).$$

Äquivalent:

$$\nabla_{\vec{w}} E_D(\vec{w}) = \Phi^\top \Phi \vec{w} - \Phi^\top \vec{t}.$$

6.12 Normalengleichung

Am Minimum muss der Gradient verschwinden:

$$\nabla_{\vec{w}} E_D(\vec{w}) = \vec{0}.$$

Also:

$$\Phi^\top \Phi \vec{w} - \Phi^\top \vec{t} = \vec{0}.$$

Damit:

$$\Phi^\top \Phi \vec{w} = \Phi^\top \vec{t}.$$

Diese Gleichung heißt Normalengleichung (normal equation).

Definition

Die Normalengleichung der linearen Regression lautet:

$$\Phi^\top \Phi \vec{w} = \Phi^\top \vec{t}.$$

Wenn $\Phi^\top \Phi$ invertierbar ist, folgt:

$$\vec{w}_{\text{LS}} = (\Phi^\top \Phi)^{-1} \Phi^\top \vec{t}.$$

Idee

Die Least-Squares-Lösung ist:

$$\vec{w}_{\text{LS}} = (\Phi^\top \Phi)^{-1} \Phi^\top \vec{t}.$$

Diese Formel gilt, wenn $\Phi^\top \Phi$ invertierbar ist.

6.13 Geometrische Interpretation

Die Vorhersage

$$\vec{y} = \Phi \vec{w}$$

liegt im Spaltenraum der Matrix Φ . Der Zielvektor $\vec{t}$ liegt im Allgemeinen nicht exakt in diesem Spaltenraum.

Least Squares sucht den Vektor $\vec{y}$ im Spaltenraum von Φ , der $\vec{t}$ am nächsten liegt.

Der Residuenvektor ist:

$$\vec{r} = \vec{t} - \Phi \vec{w}.$$

Am Optimum ist der Residuenvektor orthogonal zum Spaltenraum von Φ :

$$\Phi^\top \vec{r} = \vec{0}.$$

Einsetzen von $\vec{r} = \vec{t} - \Phi \vec{w}$:

$$\Phi^\top (\vec{t} - \Phi \vec{w}) = \vec{0}.$$

Das ist äquivalent zu:

$$\Phi^\top \Phi \vec{w} = \Phi^\top \vec{t}.$$

Hinweis

Die Normalengleichung bedeutet geometrisch: Der Fehlervektor steht senkrecht auf allen Richtungen, die das Modell durch Änderung der Gewichte erreichen kann.

6.14 Pseudoinverse

Wenn $\Phi^\top \Phi$ nicht invertierbar ist, kann man die Moore-Penrose-Pseudoinverse (Moore-Penrose pseudoinverse) verwenden.

Man schreibt:

$$\vec{w}_{\text{LS}} = \Phi^\dagger \vec{t}.$$

Falls $\Phi^\top \Phi$ invertierbar ist, gilt:

$$\Phi^\dagger = (\Phi^\top \Phi)^{-1} \Phi^\top.$$

Definition

Die Pseudoinverse $\Phi^\dagger$ verallgemeinert die Inverse auf nicht-quadratische oder singuläre Matrizen.

Die Least-Squares-Lösung kann geschrieben werden als

$$\vec{w}_{\text{LS}} = \Phi^\dagger \vec{t}.$$

6.15 Wann ist die Lösung eindeutig?

Die Lösung

$$\vec{w}_{\text{LS}} = (\Phi^\top \Phi)^{-1} \Phi^\top \vec{t}$$

ist eindeutig, wenn $\Phi^\top \Phi$ invertierbar ist. Das ist genau dann der Fall, wenn die Spalten von Φ linear unabhängig sind.

Idee

Für eine eindeutige Least-Squares-Lösung braucht man:

$$\text{rank}(\Phi) = M.$$

Das bedeutet: Die M Spalten der Designmatrix müssen linear unabhängig sein.

Wenn $M > N$, also mehr Parameter als Datenpunkte vorhanden sind, kann Φ keinen vollen Spaltenrang haben. Dann ist die Lösung nicht eindeutig. Das ist ein typischer Fall von Unterbestimmtheit (underdetermined system).

6.16 Training Error und Test Error

Der Trainingsfehler (training error) ist der Fehler auf den Daten, die zum Lernen verwendet wurden:

$$E_{\text{train}} = \frac{1}{2} \sum_{n=1}^N (y(\vec{x}_n, \vec{w}) - t_n)^2.$$

Der Testfehler (test error) wird auf neuen Daten gemessen:

$$E_{\text{test}} = \frac{1}{2} \sum_{m=1}^{N_{\text{test}}} (y(\vec{x}_m^{\text{test}}, \vec{w}) - t_m^{\text{test}})^2.$$

Idee

Ein kleiner Trainingsfehler bedeutet nicht automatisch gute Generalisierung.

Ein Modell kann die Trainingsdaten sehr gut treffen und trotzdem auf neuen Daten schlecht sein. Das nennt man Überanpassung (overfitting).

6.17 Polynomiale Regression und Overfitting

Ein klassisches Beispiel für Overfitting ist polynomiale Regression.

Wir verwenden Basisfunktionen:

$$\phi_j(x) = x^j$$

für

$$j = 0, 1, \dots, M-1.$$

Das Modell ist:

$$y(x, \vec{w}) = \sum_{j=0}^{M-1} w_j x^j.$$

Wenn M klein ist, ist das Modell sehr eingeschränkt. Es kann unteranpassen (underfitting).

Wenn M sehr groß ist, kann das Modell sehr flexibel werden. Es kann dann die Trainingspunkte sehr genau treffen, aber zwischen den Punkten stark oszillieren.

Idee

Kleines M :

einfaches Modell $\Rightarrow$ mögliche Unteranpassung.

Großes M :

flexibles Modell $\Rightarrow$ mögliche Überanpassung.

6.18 Probabilistische Interpretation

Lineare Regression kann probabilistisch interpretiert werden. Wir nehmen an, dass die Zielwerte aus einer Modellvorhersage plus Rauschen entstehen:

$$t_n = y(\vec{x}_n, \vec{w}) + \epsilon_n.$$

Das Rauschen sei normalverteilt:

$$\epsilon_n \sim \mathcal{N}(0, \beta^{-1}).$$

Dabei ist β die Präzision (precision), also der Kehrwert der Varianz:

$$\beta = \frac{1}{\sigma^2}.$$

Dann gilt:

$$p(t_n \mid \vec{x}_n, \vec{w}, \beta) = \mathcal{N}(t_n \mid y(\vec{x}_n, \vec{w}), \beta^{-1}).$$

Definition

Das probabilistische Modell der linearen Regression ist:

$$p(t \mid \vec{x}, \vec{w}, \beta) = \mathcal{N}(t \mid \vec{w}^\top \vec{\phi}(\vec{x}), \beta^{-1}).$$

6.19 Likelihood der linearen Regression

Für i.i.d. Daten gilt:

$$p(\vec{t} \mid X, \vec{w}, \beta) = \prod_{n=1}^N \mathcal{N}(t_n \mid \vec{w}^\top \vec{\phi}(\vec{x}_n), \beta^{-1}).$$

Die Normalverteilungsdichte ist:

$$\mathcal{N}(t_n | y_n, \beta^{-1}) = \left(\frac{\beta}{2\pi}\right)^{1/2} \exp\left(-\frac{\beta}{2}(t_n - y_n)^2\right),$$

wobei

$$y_n = \vec{w}^\top \vec{\phi}(\vec{x}_n).$$

Damit:

$$p(\vec{t} | X, \vec{w}, \beta) = \prod_{n=1}^N \left(\frac{\beta}{2\pi}\right)^{1/2} \exp\left(-\frac{\beta}{2}(t_n - y_n)^2\right).$$

6.20 Log-Likelihood der linearen Regression

Die Log-Likelihood ist:

$$\log p(\vec{t} | X, \vec{w}, \beta) = \sum_{n=1}^N \left[\frac{1}{2} \log \beta - \frac{1}{2} \log(2\pi) - \frac{\beta}{2} (t_n - y_n)^2 \right].$$

Zusammenfassen:

$$\log p(\vec{t} | X, \vec{w}, \beta) = \frac{N}{2} \log \beta - \frac{N}{2} \log(2\pi) - \frac{\beta}{2} \sum_{n=1}^N (t_n - y_n)^2.$$

Für festes β hängen die ersten beiden Terme nicht von $\vec{w}$ ab. Also maximiert man die Log-Likelihood nach $\vec{w}$, indem man minimiert:

$$\sum_{n=1}^N (t_n - y_n)^2.$$

Idee

Maximum-Likelihood für lineare Regression mit Gaußschem Rauschen ist äquivalent zu Least Squares.

$$\text{Gauß-Likelihood} \iff \text{quadratischer Fehler.}$$

6.21 Schätzung der Rauschvarianz

Neben $\vec{w}$ kann auch β geschätzt werden. Die Maximum-Likelihood-Lösung für die Varianz ist:

$$\hat{\sigma}_{\text{ML}}^2 = \frac{1}{N} \sum_{n=1}^N (t_n - y(\vec{x}_n, \vec{w}_{\text{ML}}))^2.$$

Da

$$\beta = \frac{1}{\sigma^2},$$

gilt:

$$\hat{\beta}_{\text{ML}} = \frac{1}{\hat{\sigma}_{\text{ML}}^2}.$$

Also:

$$\hat{\beta}_{\text{ML}}^{-1} = \frac{1}{N} \sum_{n=1}^N (t_n - y(\vec{x}_n, \vec{w}_{\text{ML}}))^2.$$

Hinweis

Wie bei der Normalverteilung ist der ML-Schätzer der Varianz bei endlichem N verzerrt. Für reine Modellschätzung wird trotzdem oft die ML-Form verwendet.

6.22 Regularisierung

Bei flexiblen Modellen kann Least Squares zu Overfitting führen. Eine typische Lösung ist Regularisierung (regularization).

Dabei ergänzt man die Fehlerfunktion um einen Term, der große Gewichte bestraft.

$$E(\vec{w}) = E_D(\vec{w}) + \lambda E_W(\vec{w}).$$

Hier ist:

$$E_D(\vec{w})$$

der Datenfehler (data error), und

$$E_W(\vec{w})$$

der Regularisierungsterm (regularization term).

Idee

Regularisierung verhindert, dass die Gewichte zu groß werden.

Große Gewichte können dazu führen, dass das Modell stark schwankt und die Trainingsdaten überanpasst.

6.23 Ridge Regression

Eine häufige Regularisierung ist L_2 -Regularisierung:

$$E_W(\vec{w}) = \frac{1}{2} \vec{w}^\top \vec{w}.$$

Dann ist die gesamte Fehlerfunktion:

$$E(\vec{w}) = \frac{1}{2} \|\Phi \vec{w} - \vec{t}\|^2 + \frac{\lambda}{2} \vec{w}^\top \vec{w}.$$

Das nennt man Ridge Regression.

Definition

Ridge Regression minimiert:

$$E(\vec{w}) = \frac{1}{2} \|\Phi \vec{w} - \vec{t}\|^2 + \frac{\lambda}{2} \vec{w}^\top \vec{w}.$$

Der Parameter $\lambda \geq 0$ steuert die Stärke der Regularisierung.

6.24 Ridge-Lösung

Wir leiten die Lösung her. Die Fehlerfunktion ist:

$$E(\vec{w}) = \frac{1}{2} (\Phi \vec{w} - \vec{t})^\top (\Phi \vec{w} - \vec{t}) + \frac{\lambda}{2} \vec{w}^\top \vec{w}.$$

Der Gradient ist:

$$\nabla_{\vec{w}} E(\vec{w}) = \Phi^\top \Phi \vec{w} - \Phi^\top \vec{t} + \lambda \vec{w}.$$

Setze den Gradienten gleich 0:

$$\Phi^\top \Phi \vec{w} - \Phi^\top \vec{t} + \lambda \vec{w} = \vec{0}.$$

Zusammenfassen:

$$(\Phi^\top \Phi + \lambda I) \vec{w} = \Phi^\top \vec{t}.$$

Wenn $\Phi^\top \Phi + \lambda I$ invertierbar ist, folgt:

$$\vec{w}_{\text{ridge}} = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top \vec{t}.$$

Idee

Die Ridge-Lösung ist:

$$\vec{w}_{\text{ridge}} = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top \vec{t}.$$

Für $\lambda > 0$ ist die Matrix oft besser konditioniert als $\Phi^\top \Phi$.

Hinweis

In manchen Konventionen wird der Bias-Term w_0 nicht regularisiert. Dann ersetzt man λI durch eine Diagonalmatrix, deren erster Eintrag 0 ist.

6.25 MAP-Interpretation von Ridge Regression

Ridge Regression kann probabilistisch als MAP-Schätzung interpretiert werden.

Wir verwenden die Likelihood:

$$p(\vec{t} \mid X, \vec{w}, \beta) = \prod_{n=1}^N \mathcal{N}(t_n \mid \vec{w}^\top \vec{\phi}(\vec{x}_n), \beta^{-1}).$$

Außerdem verwenden wir einen Gauß-Prior auf die Gewichte:

$$p(\vec{w} \mid \alpha) = \mathcal{N}(\vec{w} \mid \vec{0}, \alpha^{-1} I).$$

Der negative Log-Posterior ist bis auf Konstanten:

$$\frac{\beta}{2} \|\Phi \vec{w} - \vec{t}\|^2 + \frac{\alpha}{2} \vec{w}^\top \vec{w}.$$

Multipliziert man mit $\frac{1}{\beta}$, erhält man ein äquivalentes Optimierungsproblem:

$$\frac{1}{2} \|\Phi \vec{w} - \vec{t}\|^2 + \frac{\alpha}{2\beta} \vec{w}^\top \vec{w}.$$

Also entspricht Ridge Regression einem Gauß-Prior mit

$$\lambda = \frac{\alpha}{\beta}.$$

Idee

Ridge Regression ist MAP-Schätzung bei:

Gauß-Likelihood + Gauß-Prior auf $\vec{w}$.

Der Regularisierungsparameter ist:

$$\lambda = \frac{\alpha}{\beta}.$$

6.26 Gradientenabstieg für lineare Regression

Obwohl lineare Regression eine geschlossene Lösung hat, kann man sie auch iterativ mit Gradientenabstieg (gradient descent) trainieren.

Die Fehlerfunktion ist:

$$E_D(\vec{w}) = \frac{1}{2} \|\Phi \vec{w} - \vec{t}\|^2.$$

Der Gradient ist:

$$\nabla_{\vec{w}} E_D(\vec{w}) = \Phi^\top (\Phi \vec{w} - \vec{t}).$$

Gradientenabstieg aktualisiert:

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} - \eta \nabla_{\vec{w}} E_D(\vec{w}^{(\tau)}),$$

also:

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} - \eta \Phi^\top (\Phi \vec{w}^{(\tau)} - \vec{t}).$$

Dabei ist $\eta > 0$ die Lernrate (learning rate).

Hinweis

Für große Datensätze ist eine iterative Optimierung oft praktischer als das direkte Invertieren von $\Phi^\top \Phi$.

Das ist besonders wichtig bei sehr vielen Datenpunkten oder sehr vielen Parametern.

6.27 Stochastischer Gradientenabstieg

Beim stochastischen Gradientenabstieg (stochastic gradient descent, SGD) verwendet man nicht alle Daten auf einmal. Stattdessen aktualisiert man die Gewichte mit einem einzelnen Datenpunkt oder einem Mini-Batch.

Für einen einzelnen Datenpunkt ist der Fehler:

$$E_n(\vec{w}) = \frac{1}{2} (y(\vec{x}_n, \vec{w}) - t_n)^2.$$

Der Gradient ist:

$$\nabla_{\vec{w}} E_n(\vec{w}) = (y(\vec{x}_n, \vec{w}) - t_n) \vec{\phi}(\vec{x}_n).$$

Das Update ist:

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} - \eta \left(y(\vec{x}_n, \vec{w}^{(\tau)}) - t_n \right) \vec{\phi}(\vec{x}_n).$$

Idee

SGD ist wichtig, weil es auf sehr große Datensätze skaliert. Viele moderne ML-Modelle, insbesondere neuronale Netze, werden mit Varianten von SGD trainiert.

6.28 Mehrdimensionale Zielwerte

Bisher war der Zielwert eindimensional:

$$t_n \in \mathbb{R}.$$

Man kann lineare Regression auch für mehrere Zielwerte verwenden:

$$\vec{t}_n \in \mathbb{R}^K.$$

Dann verwendet man eine Gewichtsmatrix W :

$$\vec{y}(\vec{x}) = W^\top \vec{\phi}(\vec{x}).$$

Die Zielmatrix ist:

$$T = \begin{pmatrix} \vec{t}_1^\top \\ \vec{t}_2^\top \\ \vdots \\ \vec{t}_N^\top \end{pmatrix} \in \mathbb{R}^{N \times K}.$$

Die Vorhersagematrix ist:

$$Y = \Phi W.$$

Die Least-Squares-Lösung ist:

$$W_{\text{LS}} = (\Phi^\top \Phi)^{-1} \Phi^\top T.$$

Hinweis

Mehrdimensionale lineare Regression kann als mehrere lineare Regressionsprobleme gleichzeitig verstanden werden. Jede Ausgabedimension hat ihre eigenen Gewichte.

6.29 Praktische Aspekte

6.29.1 Skalierung der Eingaben

Lineare Regression ist empfindlich gegenüber Skalierung. Wenn ein Merkmal sehr große Werte hat und ein anderes sehr kleine, können numerische Probleme entstehen.

Eine typische Standardisierung ist:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}.$$

Dabei sind μ_j und σ_j Mittelwert und Standardabweichung des j -ten Merkmals im Trainingsdatensatz.

Prüfungshinweis

Wichtig: Die Skalierungsparameter μ_j und σ_j werden nur auf den Trainingsdaten berechnet. Danach werden dieselben Werte auf Validierungs- und Testdaten angewendet.

6.29.2 Multikollinearität

Wenn Spalten der Designmatrix stark linear abhängig sind, ist $\Phi^\top \Phi$ schlecht konditioniert. Dann können kleine Änderungen in den Daten große Änderungen in den Gewichten verursachen.

Ridge Regression hilft hier, weil

$$\Phi^\top \Phi + \lambda I$$

stabiler invertierbar ist.

6.30 Häufige Fehler

Prüfungshinweis

Typische Fehler bei linearer Regression:

1. Den Bias-Term w_0 vergessen.
2. Verwechseln, ob $\vec{x}$ den Bias-Eintrag 1 enthält.
3. Denken, dass lineare Regression immer linear in der ursprünglichen Eingabe sein muss.
4. Designmatrix Φ falsch dimensionieren.
5. $\Phi\vec{w}$ und $\vec{w}^\top\Phi$ verwechseln.
6. Die Normalengleichung falsch herum schreiben.
7. Vergessen, dass $\Phi^\top\Phi$ invertierbar sein muss.
8. Overfitting bei zu vielen Basisfunktionen ignorieren.
9. Den Zusammenhang zwischen Gaußischem Rauschen und quadratischem Fehler nicht erkennen.
10. Ridge Regression nur als Trick sehen, nicht als MAP mit Gauß-Prior.
11. Trainingsfehler mit Testfehler verwechseln.

6.31 Mini-Aufgaben

6.31.1 Aufgabe 1: Einfache Vorhersage

Gegeben sei das Modell

$$y(x, w_0, w_1) = w_0 + w_1 x.$$

Mit

$$w_0 = 2, \quad w_1 = 3, \quad x = 4$$

berechne y .

Beispiel

Einsetzen:

$$y = 2 + 3 \cdot 4 = 2 + 12 = 14.$$

6.31.2 Aufgabe 2: Designmatrix

Gegeben seien die Datenpunkte

$$x_1 = 1, \quad x_2 = 2, \quad x_3 = 3.$$

Wir verwenden Basisfunktionen

$$\phi_0(x) = 1, \quad \phi_1(x) = x.$$

Schreibe die Designmatrix Φ .

Beispiel

Die Designmatrix hat eine Zeile pro Datenpunkt:

$$\Phi = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}.$$

6.31.3 Aufgabe 3: Quadratischer Fehler

Gegeben seien Zielwerte

$$\vec{t} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

und Vorhersagen

$$\vec{y} = \begin{pmatrix} 2 \\ 4 \end{pmatrix}.$$

Berechne

$$E_D = \frac{1}{2} \|\vec{y} - \vec{t}\|^2.$$

Beispiel

Zuerst:

$$\vec{y} - \vec{t} = \begin{pmatrix} 2 \\ 4 \end{pmatrix} - \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

Dann:

$$\|\vec{y} - \vec{t}\|^2 = 1^2 + 2^2 = 5.$$

Also:

$$E_D = \frac{1}{2}5 = 2,5.$$

6.31.4 Aufgabe 4: Ridge-Lösung

Gegeben sei

$$\Phi^\top \Phi = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}, \quad \Phi^\top \vec{t} = \begin{pmatrix} 4 \\ 9 \end{pmatrix},$$

und

$$\lambda = 1.$$

Berechne $\vec{w}_{\text{ridge}}$.

Beispiel

Ridge Regression verwendet:

$$\vec{w}_{\text{ridge}} = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top \vec{t}.$$

Zuerst:

$$\Phi^\top \Phi + \lambda I = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} + \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 3 & 0 \\ 0 & 4 \end{pmatrix}.$$

Die Inverse ist:

$$(\Phi^\top \Phi + \lambda I)^{-1} = \begin{pmatrix} \frac{1}{3} & 0 \\ 0 & \frac{1}{4} \end{pmatrix}.$$

Also:

$$\vec{w}_{\text{ridge}} = \begin{pmatrix} \frac{1}{3} & 0 \\ 0 & \frac{1}{4} \end{pmatrix} \begin{pmatrix} 4 \\ 9 \end{pmatrix} = \begin{pmatrix} \frac{4}{3} \\ \frac{9}{4} \end{pmatrix}.$$

6.32 Zusammenfassung

- Lineare Regression (linear regression) ist ein überwachtes Lernverfahren für kontinuierliche Zielwerte.
- Das einfache lineare Modell ist:

$$y(x, w_0, w_1) = w_0 + w_1 x.$$

- Mit mehreren Merkmalen:

$$y(\vec{x}, \vec{w}) = w_0 + \sum_{j=1}^D w_j x_j.$$

- Mit Bias in Vektorform:

$$y(\vec{x}, \vec{w}) = \vec{w}^\top \vec{x}.$$

- Lineare Regression ist linear in den Parametern, nicht notwendigerweise linear in der Eingabe.

- Mit Basisfunktionen:

$$y(x, \vec{w}) = \sum_{j=0}^{M-1} w_j \phi_j(x).$$

- Die Designmatrix ist:

$$\Phi_{nj} = \phi_j(x_n).$$

- Die Vorhersagen für alle Datenpunkte sind:

$$\vec{y} = \Phi \vec{w}.$$

- Die quadratische Fehlerfunktion ist:

$$E_D(\vec{w}) = \frac{1}{2} \|\Phi \vec{w} - \vec{t}\|^2.$$

- Die Normalengleichung lautet:

$$\Phi^\top \Phi \vec{w} = \Phi^\top \vec{t}.$$

- Die Least-Squares-Lösung ist:

$$\vec{w}_{\text{LS}} = (\Phi^\top \Phi)^{-1} \Phi^\top \vec{t}.$$

- Mit Gaußschem Rauschen ist Least Squares äquivalent zu Maximum-Likelihood.

- Ridge Regression minimiert:

$$E(\vec{w}) = \frac{1}{2} \|\Phi \vec{w} - \vec{t}\|^2 + \frac{\lambda}{2} \vec{w}^\top \vec{w}.$$

- Die Ridge-Lösung ist:

$$\vec{w}_{\text{ridge}} = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top \vec{t}.$$

- Ridge Regression entspricht MAP mit Gauß-Likelihood und Gauß-Prior auf die Gewichte.

6.33 Typische Prüfungsfragen

Prüfungshinweis

Mögliche Prüfungsfragen zu diesem Kapitel:

1. Was ist lineare Regression?
2. Was ist der Unterschied zwischen Klassifikation und Regression?

3. Was ist ein Bias-Term?
4. Warum erweitert man den Eingabevektor um eine 1?
5. Was bedeutet linear in den Parametern?
6. Gib ein Beispiel für ein Modell, das nichtlinear in x , aber linear in $\vec{w}$ ist.
7. Was ist eine Basisfunktion?
8. Was ist die Designmatrix?
9. Welche Dimensionen haben Φ , $\vec{w}$ und $\vec{t}$?
10. Schreibe die quadratische Fehlerfunktion in Summenform und Matrixform.
11. Leite den Gradienten von $E_D(\vec{w})$ her.
12. Leite die Normalengleichung her.
13. Wann ist die Least-Squares-Lösung eindeutig?
14. Was ist die Pseudoinverse?
15. Erkläre die geometrische Interpretation der Normalengleichung.
16. Warum kann polynomiale Regression überanpassen?
17. Wie hängt lineare Regression mit Maximum-Likelihood zusammen?
18. Warum führt Gaußsches Rauschen zu quadratischem Fehler?
19. Was ist Ridge Regression?
20. Leite die Ridge-Lösung her.
21. Wie hängt Ridge Regression mit MAP zusammen?
22. Was ist der Unterschied zwischen Trainingsfehler und Testfehler?

7 Logistische Regression

7.1 Ziel dieses Kapitels

In den vorherigen Kapiteln haben wir lineare Regression (linear regression), Maximum-Likelihood-Schätzung (maximum likelihood estimation), Maximum-a-posteriori-Schätzung (maximum a-posteriori estimation) und negative Log-Likelihood kennengelernt. Lineare Regression eignet sich für kontinuierliche Zielwerte. In diesem Kapitel betrachten wir nun Klassifikation (classification), also Probleme mit diskreten Zielwerten.

Die logistische Regression (logistic regression) ist eines der wichtigsten Grundmodelle für binäre Klassifikation (binary classification). Trotz des Namens ist sie ein Klassifikationsmodell und kein Regressionsmodell im üblichen Sinn.

Idee

Logistische Regression (logistic regression) ist ein probabilistisches Klassifikationsmodell. Für eine Eingabe $\vec{x}$ berechnet das Modell eine Wahrscheinlichkeit

$$p(t = 1 \mid \vec{x}).$$

Diese Wahrscheinlichkeit wird dann verwendet, um zwischen zwei Klassen zu entscheiden.

Nach diesem Kapitel solltest du folgende Fragen beantworten können:

- Was ist binäre Klassifikation (binary classification)?
- Warum ist lineare Regression für Klassifikation ungeeignet?
- Was ist die Sigmoid-Funktion (sigmoid function)?
- Was ist der Zusammenhang zwischen Sigmoid, Odds und Logit?
- Wie modelliert logistische Regression $p(t = 1 \mid \vec{x})$?
- Wie entsteht die Bernoulli-Likelihood?
- Warum führt Maximum-Likelihood zu Kreuzentropie (cross-entropy)?
- Wie leitet man den Gradienten der logistischen Regression her?
- Warum gibt es keine geschlossene Lösung wie bei linearer Regression?
- Wie trainiert man logistische Regression mit Gradientenabstieg (gradient descent)?
- Was ist eine Entscheidungsgrenze (decision boundary)?

- Wie erweitert man logistische Regression auf mehrere Klassen?
- Wie hängt Regularisierung mit MAP zusammen?

7.2 Von Regression zu Klassifikation

Bei Regression (regression) ist der Zielwert kontinuierlich:

$$t \in \mathbb{R}.$$

Bei Klassifikation (classification) ist der Zielwert diskret. Bei binärer Klassifikation gibt es genau zwei Klassen:

$$t \in \{0,1\}.$$

Definition

Binäre Klassifikation (binary classification) ist ein überwachtes Lernproblem, bei dem jeder Datenpunkt zu einer von zwei Klassen gehört.

Der Trainingsdatensatz hat die Form

$$\mathcal{D} = \{(\vec{x}_1, t_1), \dots, (\vec{x}_N, t_N)\},$$

wobei

$$\vec{x}_n \in \mathbb{R}^D$$

und

$$t_n \in \{0,1\}.$$

Beispiele für binäre Klassifikation:

- E-Mail: Spam oder Nicht-Spam,
- medizinischer Test: krank oder gesund,
- Bild: Katze oder keine Katze,
- Kreditentscheidung: Ausfall oder kein Ausfall,
- Wohnungspreis-Beispiel: teuer oder nicht teuer.

7.3 Warum nicht einfach lineare Regression?

Man könnte versuchen, ein lineares Regressionsmodell zu verwenden:

$$y(\vec{x}, \vec{w}) = \vec{w}^\top \vec{x}.$$

Dann könnte man sagen:

$$y(\vec{x}, \vec{w}) \geq 0,5 \quad \Rightarrow \quad \hat{t} = 1,$$

und

$$y(\vec{x}, \vec{w}) < 0,5 \quad \Rightarrow \quad \hat{t} = 0.$$

Das ist aber problematisch.

Idee

Lineare Regression ist für Klassifikation ungeeignet, weil die Ausgabe

$$y(\vec{x}, \vec{w}) = \vec{w}^\top \vec{x}$$

beliebige reelle Werte annehmen kann.

Für eine Wahrscheinlichkeit brauchen wir aber Werte im Intervall

$$[0,1].$$

Eine Wahrscheinlichkeit muss erfüllen:

$$0 \leq p(t = 1 \mid \vec{x}) \leq 1.$$

Ein lineares Modell garantiert das nicht. Zum Beispiel könnte gelten:

$$\vec{w}^\top \vec{x} = -3$$

oder

$$\vec{w}^\top \vec{x} = 4,7.$$

Beides kann keine Wahrscheinlichkeit sein.

7.4 Die Grundidee der logistischen Regression

Die Idee der logistischen Regression ist:

1. Berechne zuerst einen linearen Score:

$$a = \vec{w}^\top \vec{x}.$$

2. Wandle diesen Score mit einer Sigmoid-Funktion in eine Wahrscheinlichkeit um:

$$y = \sigma(a).$$

Definition

Das Modell der binären logistischen Regression lautet:

$$y(\vec{x}, \vec{w}) = \sigma(\vec{w}^\top \vec{x}),$$

wobei

$$y(\vec{x}, \vec{w}) = p(t = 1 \mid \vec{x}, \vec{w}).$$

Der Wert

$$a = \vec{w}^\top \vec{x}$$

heißt Aktivierung (activation), Score oder Logit.

Die Ausgabe

$$y = \sigma(a)$$

ist eine Wahrscheinlichkeit.

7.5 Die Sigmoid-Funktion

Die Sigmoid-Funktion (sigmoid function) ist definiert als

$$\sigma(a) = \frac{1}{1 + \exp(-a)}.$$

Definition

Die Sigmoid-Funktion ist

$$\sigma(a) = \frac{1}{1 + \exp(-a)}.$$

Sie bildet reelle Zahlen auf das Intervall (0,1) ab:

$$\sigma : \mathbb{R} \rightarrow (0,1).$$

Wichtige Eigenschaften:

$$\lim_{a \rightarrow \infty} \sigma(a) = 1,$$

$$\lim_{a \rightarrow -\infty} \sigma(a) = 0,$$

und

$$\sigma(0) = \frac{1}{2}.$$

Idee

Die Sigmoid-Funktion macht aus einem beliebigen linearen Score $a \in \mathbb{R}$ eine gültige Wahrscheinlichkeit.

$$a \in \mathbb{R} \longrightarrow \sigma(a) \in (0,1).$$

7.6 Interpretation der Sigmoid-Ausgabe

Bei logistischer Regression setzen wir:

$$p(t = 1 \mid \vec{x}, \vec{w}) = y(\vec{x}, \vec{w}) = \sigma(\vec{w}^\top \vec{x}).$$

Da es nur zwei Klassen gibt, muss gelten:

$$p(t = 0 \mid \vec{x}, \vec{w}) = 1 - p(t = 1 \mid \vec{x}, \vec{w}).$$

Also:

$$p(t = 0 \mid \vec{x}, \vec{w}) = 1 - y(\vec{x}, \vec{w}).$$

Definition

Für binäre logistische Regression gilt:

$$p(t = 1 \mid \vec{x}, \vec{w}) = y,$$

und

$$p(t = 0 \mid \vec{x}, \vec{w}) = 1 - y,$$

wobei

$$y = \sigma(\vec{w}^\top \vec{x}).$$

7.7 Entscheidungsregel

Aus der Wahrscheinlichkeit kann eine Klassenentscheidung gemacht werden. Die Standardregel ist:

$$\hat{t} = \begin{cases} 1, & y \geq 0,5, \\ 0, & y < 0,5. \end{cases}$$

Da

$$\sigma(0) = 0,5$$

und die Sigmoid-Funktion monoton wachsend ist, gilt:

$$y \geq 0,5 \iff \vec{w}^\top \vec{x} \geq 0.$$

Idee

Die Entscheidungsgrenze (decision boundary) der logistischen Regression ist

$$\vec{w}^\top \vec{x} = 0.$$

Auf der einen Seite entscheidet das Modell Klasse 1, auf der anderen Seite Klasse 0.

7.8 Bias-Term

Wie bei linearer Regression verwenden wir meistens einen Bias-Term. Für eine Eingabe

$$\vec{x} = \begin{pmatrix} x_1 \\ \vdots \\ x_D \end{pmatrix}$$

erweitern wir:

$$\tilde{\vec{x}} = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_D \end{pmatrix}.$$

Der Gewichtsvektor ist:

$$\vec{w} = \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_D \end{pmatrix}.$$

Dann gilt:

$$a = \vec{w}^\top \tilde{\vec{x}} = w_0 + \sum_{j=1}^D w_j x_j.$$

Hinweis

Der Bias-Term w_0 verschiebt die Entscheidungsgrenze.

Ohne Bias müsste die Entscheidungsgrenze durch den Ursprung gehen.

Im Folgenden schreiben wir oft einfach $\vec{x}$ statt $\tilde{\vec{x}}$, wenn der Bias-Eintrag bereits enthalten ist.

7.9 Odds und Logit

Die logistische Regression hat eine wichtige Interpretation über Odds und Logit.

Sei

$$y = p(t = 1 \mid \vec{x}).$$

Dann ist

$$1 - y = p(t = 0 \mid \vec{x}).$$

Die Odds sind definiert als:

$$\frac{y}{1 - y}.$$

Definition

Die Odds (odds) sind das Verhältnis der Wahrscheinlichkeit für Klasse 1 zur Wahrscheinlichkeit für Klasse 0:

$$\text{odds} = \frac{p(t = 1 \mid \vec{x})}{p(t = 0 \mid \vec{x})} = \frac{y}{1 - y}.$$

Der Logit ist der Logarithmus der Odds:

$$\text{logit}(y) = \log\left(\frac{y}{1 - y}\right).$$

Bei logistischer Regression gilt:

$$y = \sigma(a) = \frac{1}{1 + \exp(-a)}.$$

Dann ist:

$$1 - y = 1 - \frac{1}{1 + \exp(-a)} = \frac{\exp(-a)}{1 + \exp(-a)}.$$

Also:

$$\frac{y}{1 - y} = \frac{\frac{1}{1 + \exp(-a)}}{\frac{\exp(-a)}{1 + \exp(-a)}} = \frac{1}{\exp(-a)} = \exp(a).$$

Damit:

$$\log\left(\frac{y}{1 - y}\right) = \log(\exp(a)) = a.$$

Da $a = \vec{w}^\top \vec{x}$, folgt:

$$\text{logit}(y) = \vec{w}^\top \vec{x}.$$

Idee

Logistische Regression modelliert den Logit linear:

$$\log \left(\frac{p(t = 1 \mid \vec{x})}{p(t = 0 \mid \vec{x})} \right) = \vec{w}^\top \vec{x}.$$

Die Wahrscheinlichkeit selbst ist nicht linear in $\vec{x}$, aber die Log-Odds sind linear.

7.10 Bernoulli-Modell für die Zielwerte

Da

$$t_n \in \{0,1\}$$

ist, modellieren wir t_n mit einer Bernoulli-Verteilung (Bernoulli distribution).

Für einen Datenpunkt gilt:

$$p(t_n \mid \vec{x}_n, \vec{w}) = y_n^{t_n} (1 - y_n)^{1-t_n},$$

wobei

$$y_n = y(\vec{x}_n, \vec{w}) = \sigma(\vec{w}^\top \vec{x}_n).$$

Definition

Das probabilistische Modell der logistischen Regression ist:

$$p(t \mid \vec{x}, \vec{w}) = y^t (1 - y)^{1-t},$$

mit

$$y = \sigma(\vec{w}^\top \vec{x}).$$

Prüfen wir die beiden Fälle:

Falls

$$t = 1,$$

dann:

$$p(t = 1 \mid \vec{x}, \vec{w}) = y^1 (1 - y)^0 = y.$$

Falls

$$t = 0,$$

dann:

$$p(t = 0 \mid \vec{x}, \vec{w}) = y^0(1 - y)^1 = 1 - y.$$

7.11 Likelihood der logistischen Regression

Für einen Datensatz

$$\mathcal{D} = \{(\vec{x}_1, t_1), \dots, (\vec{x}_N, t_N)\}$$

nehmen wir i.i.d. Daten an. Dann ist die Likelihood:

$$p(\vec{t} \mid X, \vec{w}) = \prod_{n=1}^N p(t_n \mid \vec{x}_n, \vec{w}).$$

Einsetzen des Bernoulli-Modells ergibt:

$$p(\vec{t} \mid X, \vec{w}) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n}.$$

Definition

Die Likelihood der logistischen Regression lautet:

$$L(\vec{w}) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n},$$

wobei

$$y_n = \sigma(\vec{w}^\top \vec{x}_n).$$

7.12 Log-Likelihood

Die Log-Likelihood ist:

$$\ell(\vec{w}) = \log L(\vec{w}).$$

Also:

$$\ell(\vec{w}) = \log \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n}.$$

Der Logarithmus macht aus dem Produkt eine Summe:

$$\ell(\vec{w}) = \sum_{n=1}^N \log [y_n^{t_n} (1 - y_n)^{1-t_n}].$$

Mit Logarithmusregeln:

$$\ell(\vec{w}) = \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

Definition

Die Log-Likelihood der logistischen Regression ist:

$$\ell(\vec{w}) = \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

7.13 Negative Log-Likelihood und Kreuzentropie

In maschinellem Lernen minimieren wir meistens eine Verlustfunktion. Daher betrachten wir die negative Log-Likelihood:

$$E(\vec{w}) = -\ell(\vec{w}).$$

Damit:

$$E(\vec{w}) = - \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

Definition

Die Fehlerfunktion der logistischen Regression ist die binäre Kreuzentropie (binary cross-entropy):

$$E(\vec{w}) = - \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

Für einen einzelnen Datenpunkt ist der Verlust:

$$E_n(\vec{w}) = - [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

Idee

Kreuzentropie (cross-entropy) ist die negative Log-Likelihood eines Bernoulli-Modells. Sie ist also keine willkürlich gewählte Fehlerfunktion, sondern folgt aus Maximum-Likelihood.

7.14 Intuition der Kreuzentropie

Betrachten wir einen einzelnen Datenpunkt.

Falls

$$t = 1,$$

dann ist der Verlust:

$$E = -\log y.$$

Wenn das Modell

$$y \approx 1$$

vorhersagt, dann ist

$$-\log y \approx 0.$$

Wenn das Modell aber

$$y \approx 0$$

vorhersagt, dann wird

$$-\log y$$

sehr groß.

Falls

$$t = 0,$$

dann ist der Verlust:

$$E = -\log(1 - y).$$

Wenn das Modell

$$y \approx 0$$

vorhersagt, ist der Verlust klein. Wenn das Modell

$$y \approx 1$$

vorhersagt, ist der Verlust groß.

Idee

Kreuzentropie bestraft sichere falsche Vorhersagen besonders stark.

Das ist sinnvoll, weil ein Modell nicht nur falsch liegt, sondern auch sehr sicher falsch sein kann.

7.15 Ableitung der Sigmoid-Funktion

Für den Gradienten der logistischen Regression brauchen wir die Ableitung der Sigmoid-Funktion.

Die Sigmoid-Funktion ist:

$$\sigma(a) = \frac{1}{1 + \exp(-a)}.$$

Wir schreiben:

$$\sigma(a) = (1 + \exp(-a))^{-1}.$$

Ableiten:

$$\frac{d\sigma}{da} = -(1 + \exp(-a))^{-2} \cdot (-\exp(-a)).$$

Also:

$$\frac{d\sigma}{da} = \frac{\exp(-a)}{(1 + \exp(-a))^2}.$$

Nun gilt:

$$\sigma(a) = \frac{1}{1 + \exp(-a)},$$

und

$$1 - \sigma(a) = \frac{\exp(-a)}{1 + \exp(-a)}.$$

Daher:

$$\sigma(a)(1 - \sigma(a)) = \frac{1}{1 + \exp(-a)} \cdot \frac{\exp(-a)}{1 + \exp(-a)} = \frac{\exp(-a)}{(1 + \exp(-a))^2}.$$

Somit:

$$\frac{d\sigma}{da} = \sigma(a)(1 - \sigma(a)).$$

Definition

Die Ableitung der Sigmoid-Funktion ist:

$$\sigma'(a) = \sigma(a)(1 - \sigma(a)).$$

Wenn

$$y = \sigma(a),$$

dann gilt:

$$\frac{dy}{da} = y(1 - y).$$

7.16 Gradient für einen Datenpunkt

Für einen einzelnen Datenpunkt ist der Verlust:

$$E_n(\vec{w}) = -[t_n \log y_n + (1 - t_n) \log(1 - y_n)],$$

wobei

$$y_n = \sigma(a_n)$$

und

$$a_n = \vec{w}^\top \vec{x}_n.$$

Wir möchten den Gradienten nach $\vec{w}$ berechnen:

$$\nabla_{\vec{w}} E_n(\vec{w}).$$

Zuerst leiten wir nach y_n ab:

$$\frac{\partial E_n}{\partial y_n} = - \left[\frac{t_n}{y_n} - \frac{1 - t_n}{1 - y_n} \right].$$

Also:

$$\frac{\partial E_n}{\partial y_n} = - \frac{t_n}{y_n} + \frac{1 - t_n}{1 - y_n}.$$

Auf gemeinsamen Nenner gebracht:

$$\frac{\partial E_n}{\partial y_n} = \frac{-t_n(1 - y_n) + (1 - t_n)y_n}{y_n(1 - y_n)}.$$

Der Zähler ist:

$$-t_n + t_n y_n + y_n - t_n y_n = y_n - t_n.$$

Also:

$$\frac{\partial E_n}{\partial y_n} = \frac{y_n - t_n}{y_n(1 - y_n)}.$$

Nun:

$$\frac{\partial y_n}{\partial a_n} = y_n(1 - y_n).$$

Mit der Kettenregel:

$$\frac{\partial E_n}{\partial a_n} = \frac{\partial E_n}{\partial y_n} \frac{\partial y_n}{\partial a_n}.$$

Einsetzen:

$$\frac{\partial E_n}{\partial a_n} = \frac{y_n - t_n}{y_n(1 - y_n)} y_n(1 - y_n).$$

Damit:

$$\frac{\partial E_n}{\partial a_n} = y_n - t_n.$$

Schließlich gilt:

$$a_n = \vec{w}^\top \vec{x}_n.$$

Also:

$$\frac{\partial a_n}{\partial \vec{w}} = \vec{x}_n.$$

Damit:

$$\nabla_{\vec{w}} E_n(\vec{w}) = (y_n - t_n) \vec{x}_n.$$

Definition

Für einen einzelnen Datenpunkt ist der Gradient der Kreuzentropie:

$$\nabla_{\vec{w}} E_n(\vec{w}) = (y_n - t_n) \vec{x}_n.$$

Dabei ist

$$y_n = \sigma(\vec{w}^\top \vec{x}_n).$$

Idee

Die Kombination aus Sigmoid und Kreuzentropie führt zu einem besonders einfachen Gradienten:

$$\text{Gradient} = \text{Vorhersagefehler} \cdot \text{Eingabe}.$$

Also:

$$(y_n - t_n) \vec{x}_n.$$

7.17 Gradient für den gesamten Datensatz

Die gesamte Fehlerfunktion ist:

$$E(\vec{w}) = \sum_{n=1}^N E_n(\vec{w}).$$

Daher ist der Gradient:

$$\nabla_{\vec{w}} E(\vec{w}) = \sum_{n=1}^N \nabla_{\vec{w}} E_n(\vec{w}).$$

Mit dem Einzeldatenpunkt-Gradienten:

$$\nabla_{\vec{w}} E(\vec{w}) = \sum_{n=1}^N (y_n - t_n) \vec{x}_n.$$

Definition

Der Gradient der logistischen Regression ist:

$$\nabla_{\vec{w}} E(\vec{w}) = \sum_{n=1}^N (y_n - t_n) \vec{x}_n.$$

7.18 Matrixform des Gradienten

Wir definieren die Designmatrix:

$$X = \begin{pmatrix} \vec{x}_1^\top \\ \vec{x}_2^\top \\ \vdots \\ \vec{x}_N^\top \end{pmatrix} \in \mathbb{R}^{N \times D}.$$

Der Zielvektor ist:

$$\vec{t} = \begin{pmatrix} t_1 \\ t_2 \\ \vdots \\ t_N \end{pmatrix}.$$

Die Aktivierungen sind:

$$\vec{a} = X \vec{w}.$$

Die Vorhersagen sind:

$$\vec{y} = \sigma(\vec{a}),$$

wobei σ komponentenweise angewendet wird:

$$y_n = \sigma(a_n).$$

Dann lautet der Gradient:

$$\nabla_{\vec{w}} E(\vec{w}) = X^\top (\vec{y} - \vec{t}).$$

Definition

In Matrixform ist der Gradient:

$$\nabla_{\vec{w}} E(\vec{w}) = X^\top (\vec{y} - \vec{t}).$$

Hinweis

Die Dimensionen sind:

$$X \in \mathbb{R}^{N \times D}, \quad \vec{y} - \vec{t} \in \mathbb{R}^N, \quad X^\top (\vec{y} - \vec{t}) \in \mathbb{R}^D.$$

7.19 Warum gibt es keine geschlossene Lösung?

Bei linearer Regression mit quadratischem Fehler konnten wir die Normalengleichung herleiten:

$$\Phi^\top \Phi \vec{w} = \Phi^\top \vec{t}.$$

Daraus folgte eine geschlossene Lösung.

Bei logistischer Regression ist die Situation anders. Die Fehlerfunktion enthält die Sigmoid-Funktion und Logarithmen:

$$E(\vec{w}) = - \sum_{n=1}^N \left[t_n \log \sigma(\vec{w}^\top \vec{x}_n) + (1 - t_n) \log(1 - \sigma(\vec{w}^\top \vec{x}_n)) \right].$$

Der Gradient ist zwar einfach:

$$\nabla_{\vec{w}} E(\vec{w}) = X^\top (\vec{y} - \vec{t}),$$

aber $\vec{y}$ hängt nichtlinear von $\vec{w}$ ab:

$$\vec{y} = \sigma(X\vec{w}).$$

Die Gleichung

$$X^\top (\vec{y} - \vec{t}) = \vec{0}$$

ist daher nichtlinear in $\vec{w}$.

Idee

Logistische Regression hat im Allgemeinen keine geschlossene Lösung wie lineare Regression.

Man verwendet iterative Optimierungsverfahren, zum Beispiel Gradientenabstieg (gradient descent), Newton-Verfahren oder quasi-Newton-Verfahren.

7.20 Gradientenabstieg

Gradientenabstieg (gradient descent) aktualisiert die Gewichte in Richtung des negativen Gradienten:

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} - \eta \nabla_{\vec{w}} E(\vec{w}^{(\tau)}).$$

Dabei ist

$$\eta > 0$$

die Lernrate (learning rate).

Für logistische Regression:

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} - \eta X^\top (\vec{y}^{(\tau)} - \vec{t}).$$

Dabei ist:

$$\vec{y}^{(\tau)} = \sigma(X \vec{w}^{(\tau)}).$$

Definition

Batch-Gradientenabstieg für logistische Regression:

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} - \eta X^\top (\sigma(X \vec{w}^{(\tau)}) - \vec{t}).$$

7.21 Stochastischer Gradientenabstieg

Beim stochastischen Gradientenabstieg (stochastic gradient descent, SGD) wird nicht der ganze Datensatz verwendet. Stattdessen wird ein einzelner Datenpunkt oder ein Mini-Batch verwendet.

Für einen einzelnen Datenpunkt lautet das Update:

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} - \eta (y_n - t_n) \vec{x}_n.$$

Dabei ist:

$$y_n = \sigma((\vec{w}^{(\tau)})^\top \vec{x}_n).$$

Idee

SGD ist wichtig, weil es bei großen Datensätzen effizienter ist als Batch-Gradientenabstieg. Viele moderne Modelle werden mit Mini-Batch-Varianten von SGD trainiert.

7.22 Hesse-Matrix und Konvexität

Die Fehlerfunktion der logistischen Regression ist konvex. Das bedeutet, dass jedes lokale Minimum auch ein globales Minimum ist.

Die Hesse-Matrix (Hessian matrix) ist:

$$H = \nabla_{\vec{w}}^2 E(\vec{w}).$$

Für logistische Regression gilt:

$$H = X^\top R X,$$

wobei R eine Diagonalmatrix ist:

$$R = \text{diag}(y_1(1 - y_1), y_2(1 - y_2), \dots, y_N(1 - y_N)).$$

Da

$$0 < y_n(1 - y_n) \leq \frac{1}{4},$$

ist R positiv semidefinit. Damit ist auch

$$X^\top R X$$

positiv semidefinit.

Definition

Die Hesse-Matrix der logistischen Regression ist:

$$H = X^\top R X,$$

mit

$$R_{nn} = y_n(1 - y_n).$$

Idee

Die Kreuzentropie der logistischen Regression ist konvex in $\vec{w}$.

Das bedeutet: Es gibt keine schlechten lokalen Minima. Trotzdem muss das Minimum numerisch gefunden werden.

7.23 Newton-Verfahren und IRLS

Neben Gradientenabstieg kann man auch das Newton-Verfahren (Newton's method) verwenden. Das Update lautet:

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} - H^{-1} \nabla_{\vec{w}} E(\vec{w}^{(\tau)}).$$

Für logistische Regression:

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} - (X^\top R X)^{-1} X^\top (\vec{y} - \vec{t}).$$

Dieses Verfahren kann auch als iterativ gewichtete kleinste Quadrate (iteratively reweighted least squares, IRLS) interpretiert werden.

Hinweis

Newton-Verfahren kann schneller konvergieren als Gradientenabstieg, ist aber pro Iteration teurer, weil eine Matrix invertiert oder ein lineares Gleichungssystem gelöst werden muss.

7.24 Entscheidungsgrenze

Die Entscheidung des Modells basiert auf:

$$y(\vec{x}, \vec{w}) = \sigma(\vec{w}^\top \vec{x}).$$

Mit Schwelle 0,5:

$$\hat{t} = 1 \iff y(\vec{x}, \vec{w}) \geq 0,5.$$

Da

$$\sigma(a) \geq 0,5 \iff a \geq 0,$$

gilt:

$$\hat{t} = 1 \iff \vec{w}^\top \vec{x} \geq 0.$$

Die Entscheidungsgrenze ist daher:

$$\vec{w}^\top \vec{x} = 0.$$

Definition

Die Entscheidungsgrenze (decision boundary) der logistischen Regression ist eine Hyper-ebene:

$$\vec{w}^\top \vec{x} = 0.$$

Für zwei Eingabedimensionen mit Bias-Term:

$$\vec{x} = \begin{pmatrix} 1 \\ x_1 \\ x_2 \end{pmatrix},$$

ist die Entscheidungsgrenze:

$$w_0 + w_1 x_1 + w_2 x_2 = 0.$$

Falls

$$w_2 \neq 0,$$

kann man nach x_2 auflösen:

$$x_2 = -\frac{w_0}{w_2} - \frac{w_1}{w_2}x_1.$$

Das ist eine Gerade.

Idee

Logistische Regression ist ein linearer Klassifikator.

Die Wahrscheinlichkeiten sind nichtlinear, aber die Entscheidungsgrenze ist linear im Merkmalsraum.

7.25 Nichtlineare Entscheidungsgrenzen mit Basisfunktionen

Wie bei linearer Regression kann man Basisfunktionen verwenden. Statt direkt $\vec{x}$ zu verwenden, nutzt man

$$\vec{\phi}(\vec{x}).$$

Das Modell lautet:

$$y(\vec{x}, \vec{w}) = \sigma(\vec{w}^\top \vec{\phi}(\vec{x})).$$

Die Entscheidungsgrenze ist:

$$\vec{w}^\top \vec{\phi}(\vec{x}) = 0.$$

Im transformierten Merkmalsraum ist die Grenze linear. Im ursprünglichen Eingaberaum kann sie nichtlinear sein.

Beispiel

Für eine eindimensionale Eingabe kann man polynomiale Basisfunktionen verwenden:

$$\vec{\phi}(x) = \begin{pmatrix} 1 \\ x \\ x^2 \end{pmatrix}.$$

Dann ist:

$$y(x, \vec{w}) = \sigma(w_0 + w_1x + w_2x^2).$$

Die Entscheidungsgrenze ist:

$$w_0 + w_1x + w_2x^2 = 0.$$

Diese Gleichung kann bis zu zwei Lösungen haben.

7.26 Regularisierte logistische Regression

Wie bei linearer Regression kann logistische Regression überanpassen. Daher verwendet man Regularisierung.

Mit L_2 -Regularisierung lautet die Fehlerfunktion:

$$E_{\text{reg}}(\vec{w}) = E(\vec{w}) + \frac{\lambda}{2} \vec{w}^\top \vec{w}.$$

Dabei ist:

$$E(\vec{w}) = - \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

Definition

Regularisierte logistische Regression mit L_2 -Regularisierung minimiert:

$$E_{\text{reg}}(\vec{w}) = - \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)] + \frac{\lambda}{2} \vec{w}^\top \vec{w}.$$

Der Gradient ist:

$$\nabla_{\vec{w}} E_{\text{reg}}(\vec{w}) = X^\top (\vec{y} - \vec{t}) + \lambda \vec{w}.$$

Hinweis

Oft wird der Bias-Term w_0 nicht regularisiert. Dann verwendet man statt $\vec{w}^\top \vec{w}$ nur die Summe über die Gewichte ohne Bias.

7.27 MAP-Interpretation der Regularisierung

Regularisierung kann probabilistisch als Prior interpretiert werden. Wir verwenden einen Gauß-Prior auf die Gewichte:

$$p(\vec{w}) = \mathcal{N}(\vec{w} \mid \vec{0}, \alpha^{-1} I).$$

Dieser Prior hat die Form:

$$p(\vec{w}) \propto \exp\left(-\frac{\alpha}{2} \vec{w}^\top \vec{w}\right).$$

Der negative Log-Prior ist bis auf Konstanten:

$$-\log p(\vec{w}) = \frac{\alpha}{2} \vec{w}^\top \vec{w} + \text{const.}$$

Die MAP-Schätzung minimiert:

$$-\log p(\vec{t} | X, \vec{w}) - \log p(\vec{w}).$$

Das ergibt:

$$E_{\text{MAP}}(\vec{w}) = E(\vec{w}) + \frac{\alpha}{2} \vec{w}^\top \vec{w} + \text{const.}$$

Idee

L_2 -regularisierte logistische Regression ist MAP-Schätzung mit Bernoulli-Likelihood und Gauß-Prior auf die Gewichte.

$$\text{Bernoulli-Likelihood} + \text{Gauß-Prior} \implies \text{Kreuzentropie} + L_2\text{-Regularisierung.}$$

7.28 Numerische Stabilität

In Implementierungen muss man auf numerische Stabilität achten. Die Sigmoid-Funktion

$$\sigma(a) = \frac{1}{1 + \exp(-a)}$$

kann für sehr große negative a numerisch problematisch sein, weil

$$\exp(-a)$$

sehr groß wird.

Auch die Kreuzentropie

$$- [t \log y + (1 - t) \log(1 - y)]$$

ist problematisch, wenn y exakt 0 oder 1 wird, weil

$$\log(0)$$

nicht definiert ist.

Hinweis

In praktischen Implementierungen verwendet man oft numerisch stabile Varianten der Sigmoid- und Kreuzentropie-Berechnung.

Außerdem werden Wahrscheinlichkeiten manchmal auf ein kleines Intervall beschränkt, zum Beispiel

$$y \in [\varepsilon, 1 - \varepsilon].$$

7.29 Evaluation eines binären Klassifikators

Nach dem Training muss das Modell bewertet werden. Dazu verwendet man Testdaten:

$$\mathcal{D}_{\text{test}} = \{(\vec{x}_1^{\text{test}}, t_1^{\text{test}}), \dots, (\vec{x}_M^{\text{test}}, t_M^{\text{test}})\}.$$

Für jeden Testpunkt berechnet man:

$$y_m = p(t = 1 \mid \vec{x}_m^{\text{test}}, \vec{w}).$$

Dann wählt man eine Schwelle, meistens:

$$\tau = 0,5.$$

Die vorhergesagte Klasse ist:

$$\hat{t}_m = \begin{cases} 1, & y_m \geq \tau, \\ 0, & y_m < \tau. \end{cases}$$

7.30 Accuracy

Die Accuracy (accuracy) ist der Anteil korrekt klassifizierter Datenpunkte.

$$\text{accuracy} = \frac{\text{Anzahl korrekter Vorhersagen}}{\text{Anzahl aller Vorhersagen}}.$$

Mathematisch:

$$\text{accuracy} = \frac{1}{M} \sum_{m=1}^M \mathbb{K}(\hat{t}_m = t_m).$$

Definition

Die Accuracy ist:

$$\text{accuracy} = \frac{1}{M} \sum_{m=1}^M \mathbb{K}(\hat{t}_m = t_m).$$

Dabei ist $\mathbb{K}(\hat{t}_m = t_m)$ gleich 1, wenn die Vorhersage korrekt ist, und sonst 0.

Hinweis

Accuracy kann irreführend sein, wenn die Klassen stark unausgeglichen sind.

Wenn 95% der Daten Klasse 0 sind, erreicht ein Modell, das immer 0 vorhersagt, bereits 95% Accuracy.

7.31 Konfusionsmatrix

Für binäre Klassifikation unterscheidet man:

- True Positive (TP): $t = 1$ und $\hat{t} = 1$,

- True Negative (TN): $t = 0$ und $\hat{t} = 0$,
- False Positive (FP): $t = 0$, aber $\hat{t} = 1$,
- False Negative (FN): $t = 1$, aber $\hat{t} = 0$.

Die Konfusionsmatrix (confusion matrix) ist:

	$\hat{t} = 1$	$\hat{t} = 0$
$t = 1$	TP	FN
$t = 0$	FP	TN

Definition

Die Konfusionsmatrix zeigt, welche Klassen korrekt oder falsch vorhergesagt wurden. Sie ist besonders hilfreich, wenn Accuracy allein nicht aussagekräftig ist.

7.32 Precision, Recall und F1-Score

Aus der Konfusionsmatrix lassen sich weitere Maße berechnen.

Precision (precision):

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}.$$

Recall (recall):

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

F1-Score:

$$F_1 = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

Idee

Precision fragt:

Wie viele positive Vorhersagen waren wirklich positiv?

Recall fragt:

Wie viele wirklich positive Fälle wurden gefunden?

7.33 Wahl der Entscheidungsschwelle

Die Standardschwelle ist:

$$\tau = 0,5.$$

Manchmal ist eine andere Schwelle sinnvoll. Die allgemeine Entscheidungsregel lautet:

$$\hat{t} = \begin{cases} 1, & y \geq \tau, \\ 0, & y < \tau. \end{cases}$$

Wenn False Positives teuer sind, wählt man oft eine höhere Schwelle. Wenn False Negatives teuer sind, wählt man oft eine niedrigere Schwelle.

Beispiel

Bei einer medizinischen Voruntersuchung kann es schlimmer sein, eine kranke Person fälschlich als gesund einzustufen, als eine gesunde Person zur weiteren Untersuchung zu schicken.

Dann möchte man False Negatives reduzieren und kann eine niedrigere Schwelle τ wählen.

7.34 Mehrklassen-Klassifikation

Logistische Regression kann auf mehr als zwei Klassen erweitert werden. Für K Klassen gilt:

$$t \in \{1, 2, \dots, K\}.$$

Für jede Klasse k berechnet man einen Score:

$$a_k = \vec{w}_k^\top \vec{x}.$$

Diese Scores werden mit der Softmax-Funktion in Wahrscheinlichkeiten umgewandelt.

Definition

Die Softmax-Funktion (softmax function) ist:

$$y_k = p(t = k \mid \vec{x}) = \frac{\exp(a_k)}{\sum_{j=1}^K \exp(a_j)}.$$

Die Wahrscheinlichkeiten erfüllen:

$$y_k \geq 0$$

und

$$\sum_{k=1}^K y_k = 1.$$

Idee

Die Softmax-Funktion ist die Mehrklassen-Verallgemeinerung der Sigmoid-Funktion. Sie wandelt mehrere Scores in eine Wahrscheinlichkeitsverteilung über Klassen um.

7.35 Mehrklassen-Kreuzentropie

Für Mehrklassen-Klassifikation verwendet man oft One-Hot-Kodierung (one-hot encoding). Der Zielvektor ist:

$$\vec{t} = \begin{pmatrix} t_1 \\ \vdots \\ t_K \end{pmatrix},$$

wobei genau eine Komponente 1 ist und alle anderen 0 sind.

Für einen Datenpunkt ist die Kreuzentropie:

$$E_n = - \sum_{k=1}^K t_{nk} \log y_{nk}.$$

Für den ganzen Datensatz:

$$E = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \log y_{nk}.$$

Definition

Die Mehrklassen-Kreuzentropie (multiclass cross-entropy) ist:

$$E = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \log y_{nk}.$$

7.36 Softmax als probabilistisches Modell

Die Softmax-Ausgabe wird interpretiert als:

$$y_k = p(t = k \mid \vec{x}).$$

Die vorhergesagte Klasse ist:

$$\hat{t} = \arg \max_k y_k.$$

Da Softmax monoton in den Scores ist, gilt auch:

$$\hat{t} = \arg \max_k a_k.$$

Hinweis

Für die Entscheidung reicht der größte Score. Für Wahrscheinlichkeiten und Training braucht man aber die Softmax-normalisierten Werte.

7.37 Zusammenhang zu neuronalen Netzen

Logistische Regression kann als einfachstes neuronales Netz betrachtet werden:

$$\vec{x} \longrightarrow \vec{w}^\top \vec{x} \longrightarrow \sigma(\vec{w}^\top \vec{x}).$$

Es gibt keine versteckte Schicht (hidden layer). Das Modell ist daher ein linearer Klassifikator mit nichtlinearer Ausgabefunktion.

Idee

Ein neuronales Netz für binäre Klassifikation verwendet am Ende oft genau dieselbe Kombination:

$$\text{Logit} \longrightarrow \text{Sigmoid} \longrightarrow \text{Kreuzentropie}.$$

Logistische Regression ist daher ein wichtiger Spezialfall neuronaler Netze.

7.38 Praktisches Beispiel: Binäre Zielvariable erzeugen

In vielen Datensätzen ist der ursprüngliche Zielwert kontinuierlich. Für binäre Klassifikation kann man daraus eine binäre Zielvariable erzeugen.

Angenommen, ein Zielwert z_n ist kontinuierlich. Wir wählen eine Schwelle c und definieren:

$$t_n = \begin{cases} 1, & z_n \geq c, \\ 0, & z_n < c. \end{cases}$$

Beispiel

Wenn z_n ein Wohnungspreis ist, kann man definieren:

$$t_n = \begin{cases} 1, & \text{Preis ist hoch,} \\ 0, & \text{Preis ist niedrig.} \end{cases}$$

Dann kann logistische Regression verwendet werden, um die Wahrscheinlichkeit für einen hohen Preis zu modellieren.

Hinweis

Das Umwandeln eines kontinuierlichen Zielwerts in eine binäre Klasse kann nützlich sein, verliert aber Information.

Ein Preis von 101 und ein Preis von 1000 können nach der Schwelle beide dieselbe Klasse bekommen.

7.39 Feature-Skalierung

Wie bei linearer Regression ist Feature-Skalierung wichtig. Wenn Merkmale sehr unterschiedliche Größenordnungen haben, kann Gradientenabstieg schlecht funktionieren.

Eine Standardisierung ist:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}.$$

Dabei sind μ_j und σ_j Mittelwert und Standardabweichung des Merkmals im Trainingsdatensatz.

Prüfungshinweis

Wichtig: Skalierungsparameter werden nur aus Trainingsdaten berechnet.
Dieselben Parameter werden dann auf Validierungs- und Testdaten angewendet.

7.40 Häufige Fehler

Prüfungshinweis

Typische Fehler bei logistischer Regression:

1. Denken, logistische Regression sei ein Regressionsmodell für kontinuierliche Zielwerte.
2. Die Ausgabe $\vec{w}^\top \vec{x}$ direkt als Wahrscheinlichkeit interpretieren.
3. Vergessen, die Sigmoid-Funktion anzuwenden.
4. $p(t = 1 \mid \vec{x})$ und $p(t = 0 \mid \vec{x})$ verwechseln.
5. Kreuzentropie mit quadratischem Fehler ersetzen, ohne die probabilistische Bedeutung zu beachten.
6. Die Ableitung der Sigmoid-Funktion falsch berechnen.
7. Beim Gradienten das einfache Ergebnis $(y_n - t_n)\vec{x}_n$ nicht erkennen.
8. Die Entscheidungsgrenze mit der Sigmoid-Kurve verwechseln.
9. Accuracy verwenden, obwohl die Klassen stark unausgeglichen sind.
10. Bei MAP-Regularisierung den Prior vergessen.
11. Den Bias-Term versehentlich regularisieren, obwohl dies nicht gewünscht ist.

7.41 Mini-Aufgaben

7.41.1 Aufgabe 1: Sigmoid berechnen

Berechne

$$\sigma(0).$$

Beispiel

Mit

$$\sigma(a) = \frac{1}{1 + \exp(-a)}$$

folgt:

$$\sigma(0) = \frac{1}{1 + \exp(0)} = \frac{1}{1 + 1} = \frac{1}{2}.$$

7.41.2 Aufgabe 2: Wahrscheinlichkeit und Gegenwahrscheinlichkeit

Ein logistisches Regressionsmodell gibt aus:

$$p(t = 1 \mid \vec{x}) = 0,8.$$

Berechne

$$p(t = 0 \mid \vec{x}).$$

Beispiel

Da es nur zwei Klassen gibt:

$$p(t = 0 \mid \vec{x}) = 1 - p(t = 1 \mid \vec{x}).$$

Also:

$$p(t = 0 \mid \vec{x}) = 1 - 0,8 = 0,2.$$

7.41.3 Aufgabe 3: Binäre Kreuzentropie

Für einen Datenpunkt gilt:

$$t = 1, \quad y = 0,9.$$

Berechne den Kreuzentropie-Verlust.

Beispiel

Der Verlust ist:

$$E = -[t \log y + (1 - t) \log(1 - y)].$$

Mit $t = 1$:

$$E = -\log y.$$

Also:

$$E = -\log(0,9).$$

7.41.4 Aufgabe 4: Gradient für einen Datenpunkt

Gegeben seien

$$y_n = 0,7, \quad t_n = 1,$$

und

$$\vec{x}_n = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

Berechne den Gradienten

$$\nabla_{\vec{w}} E_n(\vec{w}).$$

Beispiel

Der Gradient ist:

$$\nabla_{\vec{w}} E_n(\vec{w}) = (y_n - t_n) \vec{x}_n.$$

Einsetzen:

$$y_n - t_n = 0,7 - 1 = -0,3.$$

Also:

$$\nabla_{\vec{w}} E_n(\vec{w}) = -0,3 \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} -0,3 \\ -0,6 \end{pmatrix}.$$

7.41.5 Aufgabe 5: Entscheidungsgrenze

Ein Modell hat

$$w_0 = -1, \quad w_1 = 2.$$

Für eine eindimensionale Eingabe mit Bias gilt:

$$a = w_0 + w_1 x.$$

Bestimme die Entscheidungsgrenze.

Beispiel

Die Entscheidungsgrenze ist:

$$a = 0.$$

Also:

$$-1 + 2x = 0.$$

Damit:

$$2x = 1$$

und

$$x = \frac{1}{2}.$$

Für $x \geq \frac{1}{2}$ sagt das Modell Klasse 1 voraus. Für $x < \frac{1}{2}$ sagt es Klasse 0 voraus.

7.42 Zusammenfassung

- Logistische Regression (logistic regression) ist ein Modell für binäre Klassifikation.
- Das Modell berechnet zuerst einen linearen Score:

$$a = \vec{w}^\top \vec{x}.$$

- Die Sigmoid-Funktion wandelt den Score in eine Wahrscheinlichkeit um:

$$y = \sigma(a) = \frac{1}{1 + \exp(-a)}.$$

- Die Modellwahrscheinlichkeit ist:

$$p(t = 1 \mid \vec{x}, \vec{w}) = \sigma(\vec{w}^\top \vec{x}).$$

- Für die Gegenklasse gilt:

$$p(t = 0 \mid \vec{x}, \vec{w}) = 1 - y.$$

- Der Logit ist linear:

$$\log\left(\frac{y}{1-y}\right) = \vec{w}^\top \vec{x}.$$

- Die Bernoulli-Likelihood ist:

$$L(\vec{w}) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n}.$$

- Die negative Log-Likelihood ist die binäre Kreuzentropie:

$$E(\vec{w}) = - \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

- Die Ableitung der Sigmoid-Funktion ist:

$$\sigma'(a) = \sigma(a)(1 - \sigma(a)).$$

- Der Gradient für einen Datenpunkt ist:

$$\nabla_{\vec{w}} E_n(\vec{w}) = (y_n - t_n) \vec{x}_n.$$

- Der Gradient für den Datensatz ist:

$$\nabla_{\vec{w}} E(\vec{w}) = X^\top (\vec{y} - \vec{t}).$$

- Die Entscheidungsgrenze ist:

$$\vec{w}^\top \vec{x} = 0.$$

- L_2 -regularisierte logistische Regression entspricht MAP mit Gauß-Prior.
- Für mehrere Klassen verwendet man die Softmax-Funktion:

$$y_k = \frac{\exp(a_k)}{\sum_{j=1}^K \exp(a_j)}.$$

7.43 Typische Prüfungsfragen

Prüfungshinweis

Mögliche Prüfungsfragen zu diesem Kapitel:

1. Warum ist lineare Regression nicht direkt für Klassifikation geeignet?
2. Definiere die Sigmoid-Funktion.
3. Zeige, dass $\sigma(0) = \frac{1}{2}$.
4. Zeige, dass $\sigma'(a) = \sigma(a)(1 - \sigma(a))$.
5. Wie modelliert logistische Regression $p(t = 1 \mid \vec{x})$?
6. Was ist der Logit?
7. Zeige, dass logistische Regression lineare Log-Odds modelliert.
8. Schreibe die Bernoulli-Likelihood für logistische Regression auf.
9. Leite die Log-Likelihood her.

10. Warum ist die Kreuzentropie die negative Log-Likelihood?
11. Leite den Gradienten für einen Datenpunkt her.
12. Leite den Gradienten in Matrixform her.
13. Warum gibt es keine geschlossene Lösung wie bei linearer Regression?
14. Wie lautet das Update des Batch-Gradientenabstiegs?
15. Was ist die Entscheidungsgrenze der logistischen Regression?
16. Warum ist logistische Regression ein linearer Klassifikator?
17. Wie kann man nichtlineare Entscheidungsgrenzen erzeugen?
18. Wie funktioniert L_2 -Regularisierung bei logistischer Regression?
19. Wie hängt L_2 -Regularisierung mit MAP zusammen?
20. Was ist der Unterschied zwischen Sigmoid und Softmax?
21. Wie lautet die Mehrklassen-Kreuzentropie?
22. Warum kann Accuracy bei unausgeglichenen Klassen irreführend sein?

8 Perzeptrons

8.1 Ziel dieses Kapitels

In den vorherigen Kapiteln haben wir lineare Regression (linear regression) und logistische Regression (logistic regression) betrachtet. Beide Modelle verwenden eine lineare Kombination der Eingaben:

$$a = \vec{w}^\top \vec{x}.$$

Bei linearer Regression wird daraus direkt ein kontinuierlicher Wert vorhergesagt. Bei logistischer Regression wird der lineare Score durch die Sigmoid-Funktion (sigmoid function) in eine Wahrscheinlichkeit umgewandelt.

In diesem Kapitel betrachten wir das Perzeptron (perceptron). Das Perzeptron ist ein frühes und sehr wichtiges Modell für binäre Klassifikation. Es ist außerdem ein historischer Vorläufer neuronaler Netze (neural networks).

Idee

Ein Perzeptron (perceptron) ist ein einfacher linearer Klassifikator.

Es berechnet zuerst einen linearen Score

$$a = \vec{w}^\top \vec{x}$$

und trifft danach mit einer Schwellenfunktion (threshold function) eine binäre Entscheidung.

Nach diesem Kapitel solltest du folgende Fragen beantworten können:

- Was ist ein Perzeptron (perceptron)?
- Wie hängt das Perzeptron mit einem künstlichen Neuron zusammen?
- Was ist eine Schwellenfunktion (threshold function)?
- Was ist eine Aktivierungsfunktion (activation function)?
- Was ist eine Entscheidungsgrenze (decision boundary)?
- Was bedeutet lineare Separierbarkeit (linear separability)?
- Wie funktioniert die Perzeptron-Lernregel (perceptron learning rule)?
- Warum konvergiert das Perzeptron nur bei linear separierbaren Daten?

- Warum kann ein einzelnes Perzeptron das XOR-Problem nicht lösen?
- Wie hängen Perzeptron, logistische Regression und neuronale Netze zusammen?

8.2 Motivation

Wir betrachten ein binäres Klassifikationsproblem. Die Zielwerte seien

$$t_n \in \{-1, +1\}.$$

Die Eingaben seien

$$\vec{x}_n \in \mathbb{R}^D.$$

Das Ziel ist, eine Funktion zu lernen, die für eine neue Eingabe $\vec{x}$ eine Klasse vorhersagt:

$$y(\vec{x}) \in \{-1, +1\}.$$

Ein sehr einfacher Ansatz ist: Wir berechnen zuerst einen linearen Score

$$a = \vec{w}^\top \vec{x}$$

und entscheiden dann anhand des Vorzeichens von a .

Wenn

$$a \geq 0,$$

sagen wir Klasse +1 voraus. Wenn

$$a < 0,$$

sagen wir Klasse -1 voraus.

Definition

Ein binärer linearer Klassifikator (binary linear classifier) entscheidet anhand des Vorzeichens eines linearen Scores:

$$y(\vec{x}) = \text{sign}(\vec{w}^\top \vec{x}).$$

8.3 Das Perzeptron-Modell

Das Perzeptron besteht aus zwei Schritten:

1. Lineare Kombination der Eingabe:

$$a = \vec{w}^\top \vec{x}.$$

2. Anwendung einer Schwellenfunktion:

$$y = h(a).$$

Für Klassen

$$y \in \{-1, +1\}$$

verwendet man oft die Vorzeichenfunktion:

$$h(a) = \text{sign}(a).$$

Damit ist:

$$y(\vec{x}, \vec{w}) = \text{sign}(\vec{w}^\top \vec{x}).$$

Definition

Das Perzeptron (perceptron) ist definiert durch

$$y(\vec{x}, \vec{w}) = \text{sign}(\vec{w}^\top \vec{x}).$$

Dabei ist $\vec{x}$ der Eingabevektor, $\vec{w}$ der Gewichtsvektor und y die vorhergesagte Klasse.

Die Vorzeichenfunktion ist:

$$\text{sign}(a) = \begin{cases} +1, & a \geq 0, \\ -1, & a < 0. \end{cases}$$

Hinweis

Manche Darstellungen verwenden Klassen

$$t \in \{0, 1\}.$$

Dann verwendet man eine Schrittfunktion, die Werte 0 und 1 ausgibt. In diesem Kapitel verwenden wir meistens

$$t \in \{-1, +1\},$$

weil die Perzeptron-Lernregel damit besonders einfach aussieht.

8.4 Bias-Term

Wie bei linearer und logistischer Regression ist ein Bias-Term wichtig. Ohne Bias hätte das Modell die Form:

$$a = \sum_{j=1}^D w_j x_j.$$

Die Entscheidungsgrenze müsste dann durch den Ursprung gehen.

Mit Bias-Term:

$$a = w_0 + \sum_{j=1}^D w_j x_j.$$

Wir können den Bias wieder in den Eingabevektor einbauen:

$$\tilde{\vec{x}} = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_D \end{pmatrix}, \quad \vec{w} = \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_D \end{pmatrix}.$$

Dann gilt:

$$a = \vec{w}^\top \tilde{\vec{x}}.$$

Idee

Der Bias-Term w_0 verschiebt die Entscheidungsgrenze.

Ohne Bias muss die Entscheidungsgrenze durch den Ursprung gehen. Mit Bias kann sie frei verschoben werden.

Im Folgenden schreiben wir oft einfach $\vec{x}$, obwohl der Bias-Eintrag 1 bereits enthalten sein kann.

8.5 Interpretation als künstliches Neuron

Das Perzeptron kann als einfaches künstliches Neuron (artificial neuron) interpretiert werden.

Ein Neuron erhält Eingaben

$$x_1, x_2, \dots, x_D.$$

Jede Eingabe wird mit einem Gewicht multipliziert:

$$w_1 x_1, w_2 x_2, \dots, w_D x_D.$$

Dann werden die gewichteten Eingaben aufsummiert:

$$a = w_0 + \sum_{j=1}^D w_j x_j.$$

Danach wird eine Aktivierungsfunktion (activation function) angewendet:

$$y = h(a).$$

Definition

Ein künstliches Neuron (artificial neuron) berechnet typischerweise

$$a = \sum_{j=1}^D w_j x_j + w_0$$

und danach

$$y = h(a).$$

Dabei ist h eine Aktivierungsfunktion (activation function).

Beim Perzeptron ist die Aktivierungsfunktion eine harte Schwellenfunktion:

$$h(a) = \text{sign}(a).$$

8.6 Entscheidungsgrenze

Das Perzeptron entscheidet nach dem Vorzeichen von

$$a = \vec{w}^\top \vec{x}.$$

Die Entscheidungsgrenze (decision boundary) liegt dort, wo das Modell genau zwischen den Klassen steht:

$$\vec{w}^\top \vec{x} = 0.$$

Definition

Die Entscheidungsgrenze eines Perzeptrons ist die Menge aller Punkte $\vec{x}$, für die gilt:

$$\vec{w}^\top \vec{x} = 0.$$

Für zwei Eingabedimensionen mit Bias gilt:

$$\vec{x} = \begin{pmatrix} 1 \\ x_1 \\ x_2 \end{pmatrix}.$$

Dann ist:

$$\vec{w}^\top \vec{x} = w_0 + w_1 x_1 + w_2 x_2.$$

Die Entscheidungsgrenze ist:

$$w_0 + w_1x_1 + w_2x_2 = 0.$$

Falls

$$w_2 \neq 0,$$

kann man nach x_2 auflösen:

$$x_2 = -\frac{w_0}{w_2} - \frac{w_1}{w_2}x_1.$$

Das ist eine Gerade.

Idee

Ein einzelnes Perzeptron ist ein linearer Klassifikator.

In zwei Dimensionen ist die Entscheidungsgrenze eine Gerade. In drei Dimensionen ist sie eine Ebene. In D Dimensionen ist sie eine Hyperebene.

8.7 Geometrische Interpretation des Gewichtsvektors

Die Entscheidungsgrenze ist:

$$\vec{w}^\top \vec{x} = 0.$$

Der Gewichtsvektor $\vec{w}$ steht senkrecht auf dieser Entscheidungsgrenze. Er zeigt in die Richtung, in der der Score

$$a = \vec{w}^\top \vec{x}$$

am schnellsten größer wird.

Wenn

$$\vec{w}^\top \vec{x} > 0,$$

liegt $\vec{x}$ auf der positiven Seite der Grenze. Wenn

$$\vec{w}^\top \vec{x} < 0,$$

liegt $\vec{x}$ auf der negativen Seite.

Hinweis

Die Länge von $\vec{w}$ beeinflusst beim reinen Perzeptron nicht die vorhergesagte Klasse, solange das Vorzeichen gleich bleibt.

Wenn man $\vec{w}$ mit einer positiven Konstanten $c > 0$ multipliziert, gilt:

$$\text{sign}(c\vec{w}^\top \vec{x}) = \text{sign}(\vec{w}^\top \vec{x}).$$

8.8 Lineare Separierbarkeit

Ein Datensatz ist linear separierbar, wenn es eine Hyperebene gibt, die die beiden Klassen perfekt trennt.

Definition

Ein Datensatz

$$\mathcal{D} = \{(\vec{x}_1, t_1), \dots, (\vec{x}_N, t_N)\}$$

mit

$$t_n \in \{-1, +1\}$$

heißt linear separierbar (linearly separable), wenn es einen Gewichtsvektor $\vec{w}$ gibt, sodass für alle n gilt:

$$t_n(\vec{w}^\top \vec{x}_n) > 0.$$

Warum ist diese Bedingung sinnvoll?

Wenn

$$t_n = +1,$$

dann brauchen wir

$$\vec{w}^\top \vec{x}_n > 0.$$

Dann ist:

$$t_n(\vec{w}^\top \vec{x}_n) = +1 \cdot (\vec{w}^\top \vec{x}_n) > 0.$$

Wenn

$$t_n = -1,$$

dann brauchen wir

$$\vec{w}^\top \vec{x}_n < 0.$$

Dann ist:

$$t_n(\vec{w}^\top \vec{x}_n) = -1 \cdot (\vec{w}^\top \vec{x}_n) > 0.$$

Idee

Die Bedingung

$$t_n(\vec{w}^\top \vec{x}_n) > 0$$

bedeutet: Der Datenpunkt $\vec{x}_n$ liegt auf der richtigen Seite der Entscheidungsgrenze.

8.9 Korrekt und falsch klassifizierte Punkte

Die Perzeptron-Vorhersage ist:

$$y_n = \text{sign}(\vec{w}^\top \vec{x}_n).$$

Ein Punkt ist korrekt klassifiziert, wenn

$$y_n = t_n.$$

Für $t_n \in \{-1, +1\}$ ist das äquivalent zu:

$$t_n(\vec{w}^\top \vec{x}_n) > 0.$$

Ein Punkt ist falsch klassifiziert, wenn

$$y_n \neq t_n.$$

Das ist äquivalent zu:

$$t_n(\vec{w}^\top \vec{x}_n) \leq 0.$$

Definition

Die Menge der falsch klassifizierten Punkte ist

$$\mathcal{M} = \left\{ n \mid t_n(\vec{w}^\top \vec{x}_n) \leq 0 \right\}.$$

8.10 Perzeptron-Kriterium

Um ein Perzeptron zu trainieren, braucht man eine Fehlerfunktion. Eine Möglichkeit ist das Perzeptron-Kriterium (perceptron criterion).

Für falsch klassifizierte Punkte ist

$$t_n(\vec{w}^\top \vec{x}_n) \leq 0.$$

Wir definieren den Fehler:

$$E_P(\vec{w}) = - \sum_{n \in \mathcal{M}} t_n \vec{w}^\top \vec{x}_n.$$

Definition

Das Perzeptron-Kriterium (perceptron criterion) ist

$$E_P(\vec{w}) = - \sum_{n \in \mathcal{M}} t_n \vec{w}^\top \vec{x}_n,$$

wobei $\mathcal{M}$ die Menge der falsch klassifizierten Punkte ist.

Für einen falsch klassifizierten Punkt gilt:

$$t_n \vec{w}^\top \vec{x}_n \leq 0.$$

Daher ist

$$-t_n \vec{w}^\top \vec{x}_n \geq 0.$$

Das Perzeptron-Kriterium ist also nichtnegativ:

$$E_P(\vec{w}) \geq 0.$$

Wenn alle Punkte korrekt klassifiziert sind, ist

$$\mathcal{M} = \emptyset$$

und damit

$$E_P(\vec{w}) = 0.$$

Idee

Das Perzeptron-Kriterium bestraft nur falsch klassifizierte Punkte.
Korrekt klassifizierte Punkte tragen nicht zum Fehler bei.

8.11 Gradient des Perzeptron-Kriteriums

Für einen falsch klassifizierten Datenpunkt ist der Beitrag zum Fehler:

$$E_n(\vec{w}) = -t_n \vec{w}^\top \vec{x}_n.$$

Der Gradient nach $\vec{w}$ ist:

$$\nabla_{\vec{w}} E_n(\vec{w}) = -t_n \vec{x}_n.$$

Für alle falsch klassifizierten Punkte:

$$\nabla_{\vec{w}} E_P(\vec{w}) = - \sum_{n \in \mathcal{M}} t_n \vec{x}_n.$$

Definition

Der Gradient des Perzeptron-Kriteriums ist

$$\nabla_{\vec{w}} E_P(\vec{w}) = - \sum_{n \in \mathcal{M}} t_n \vec{x}_n.$$

8.12 Perzeptron-Lernregel

Gradientenabstieg verwendet:

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} - \eta \nabla_{\vec{w}} E(\vec{w}^{(\tau)}).$$

Für einen einzelnen falsch klassifizierten Punkt gilt:

$$\nabla_{\vec{w}} E_n(\vec{w}) = -t_n \vec{x}_n.$$

Also:

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} - \eta(-t_n \vec{x}_n).$$

Damit:

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} + \eta t_n \vec{x}_n.$$

Definition

Die Perzeptron-Lernregel (perceptron learning rule) lautet für einen falsch klassifizierten Punkt:

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} + \eta t_n \vec{x}_n.$$

Für korrekt klassifizierte Punkte wird $\vec{w}$ nicht geändert.

Dabei ist

$$\eta > 0$$

die Lernrate (learning rate).

8.13 Intuition der Lernregel

Die Lernregel ist sehr intuitiv.

8.13.1 Fall $t_n = +1$

Wenn der wahre Zielwert

$$t_n = +1$$

ist, aber der Punkt falsch klassifiziert wird, dann ist

$$\vec{w}^\top \vec{x}_n \leq 0.$$

Das Modell gibt also zu kleinen oder negativen Score aus. Die Lernregel ist:

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} + \eta \vec{x}_n.$$

Dadurch wird der Score für diesen Punkt größer, denn:

$$(\vec{w}^{(\tau)} + \eta \vec{x}_n)^\top \vec{x}_n = (\vec{w}^{(\tau)})^\top \vec{x}_n + \eta \vec{x}_n^\top \vec{x}_n.$$

Da

$$\vec{x}_n^\top \vec{x}_n = \|\vec{x}_n\|^2 \geq 0,$$

wird der Score erhöht.

8.13.2 Fall $t_n = -1$

Wenn der wahre Zielwert

$$t_n = -1$$

ist, aber der Punkt falsch klassifiziert wird, dann ist

$$\vec{w}^\top \vec{x}_n \geq 0.$$

Das Modell gibt also einen zu großen Score aus. Die Lernregel ist:

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} - \eta \vec{x}_n.$$

Dadurch wird der Score für diesen Punkt kleiner.

Idee

Die Perzeptron-Lernregel verschiebt $\vec{w}$ so, dass der falsch klassifizierte Punkt beim nächsten Mal eher auf der richtigen Seite der Entscheidungsgrenze liegt.

8.14 Algorithmus des Perzeptrons

Der Trainingsalgorithmus läuft iterativ über die Trainingsdaten.

Definition

Perzeptron-Algorithmus:

1. Initialisiere $\vec{w}$, zum Beispiel mit $\vec{w} = \vec{0}$.

2. Wähle einen Datenpunkt $(\vec{x}_n, t_n)$.

3. Berechne den Score:

$$a_n = \vec{w}^\top \vec{x}_n.$$

4. Prüfe, ob der Punkt falsch klassifiziert ist:

$$t_n a_n \leq 0.$$

5. Falls ja, aktualisiere:

$$\vec{w} \leftarrow \vec{w} + \eta t_n \vec{x}_n.$$

6. Wiederhole, bis keine Fehler mehr auftreten oder eine maximale Anzahl an Iterationen erreicht ist.

Hinweis

Die Reihenfolge der Datenpunkte kann das Training beeinflussen.

In praktischen Varianten werden die Datenpunkte oft in zufälliger Reihenfolge durchlaufen.

8.15 Perzeptron-Konvergenz

Ein klassisches Resultat ist der Perzeptron-Konvergenzsatz.

Idee

Wenn die Daten linear separierbar sind, dann findet der Perzeptron-Algorithmus nach endlich vielen Updates eine trennende Hyperebene.

Wenn die Daten nicht linear separierbar sind, muss der Algorithmus nicht konvergieren.

Der genaue Konvergenzbeweis benötigt Annahmen über den Abstand der Daten zur trennenden Hyperebene. Die wichtige Intuition ist:

linear separierbar $\Rightarrow$ Perzeptron kann eine perfekte lineare Trennung finden.

Wenn die Daten aber nicht linear separierbar sind, gibt es keine Hyperebene, die alle Punkte korrekt trennt. Dann kann das Perzeptron immer wieder Updates durchführen, ohne jemals alle Punkte korrekt zu klassifizieren.

Prüfungshinweis

Wichtig für Prüfungen:

- Das Perzeptron konvergiert bei linear separierbaren Daten.
- Das Perzeptron konvergiert nicht garantiert bei nicht separierbaren Daten.
- Ein einzelnes Perzeptron kann nur lineare Entscheidungsgrenzen darstellen.

8.16 Beispiel für ein Update

Wir betrachten einen Datenpunkt

$$\vec{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix},$$

mit Zielwert

$$t = +1.$$

Der aktuelle Gewichtsvektor sei

$$\vec{w} = \begin{pmatrix} -1 \\ 0 \end{pmatrix}.$$

Der Score ist:

$$a = \vec{w}^\top \vec{x} = (-1) \cdot 1 + 0 \cdot 2 = -1.$$

Da

$$ta = (+1)(-1) = -1 \leq 0,$$

ist der Punkt falsch klassifiziert.

Mit Lernrate

$$\eta = 1$$

ist das Update:

$$\vec{w}_{\text{neu}} = \vec{w} + \eta t \vec{x}.$$

Einsetzen:

$$\vec{w}_{\text{neu}} = \begin{pmatrix} -1 \\ 0 \end{pmatrix} + 1 \cdot (+1) \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

Also:

$$\vec{w}_{\text{neu}} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}.$$

Beispiel

Nach dem Update ist der neue Score:

$$a_{\text{neu}} = \vec{w}_{\text{neu}}^{\top} \vec{x} = 0 \cdot 1 + 2 \cdot 2 = 4.$$

Damit ist

$$ta_{\text{neu}} = (+1)4 = 4 > 0.$$

Der Punkt ist nun korrekt klassifiziert.

8.17 Vergleich mit logistischer Regression

Perzeptron und logistische Regression sind eng verwandt. Beide berechnen zuerst einen linearen Score:

$$a = \vec{w}^{\top} \vec{x}.$$

Der Unterschied liegt in der Ausgabefunktion und im Trainingsziel.

Beim Perzeptron:

$$y = \text{sign}(a).$$

Bei logistischer Regression:

$$y = \sigma(a).$$

Definition

Perzeptron:

$$y = \text{sign}(\vec{w}^{\top} \vec{x}).$$

Logistische Regression:

$$p(t = 1 \mid \vec{x}, \vec{w}) = \sigma(\vec{w}^{\top} \vec{x}).$$

8.18 Harte und weiche Entscheidung

Das Perzeptron trifft eine harte Entscheidung:

$$y \in \{-1, +1\}.$$

Logistische Regression gibt eine weiche Wahrscheinlichkeit aus:

$$y \in (0,1).$$

Idee

Das Perzeptron sagt nur, welche Klasse vorhergesagt wird.

Logistische Regression sagt zusätzlich, wie sicher das Modell in Form einer Wahrscheinlichkeit ist.

Die Sigmoid-Funktion ist glatt und differenzierbar:

$$\sigma'(a) = \sigma(a)(1 - \sigma(a)).$$

Die Vorzeichenfunktion ist nicht glatt und an

$$a = 0$$

nicht differenzierbar.

Das macht logistische Regression für gradientenbasierte Optimierung angenehmer.

8.19 Perzeptron-Verlust und Kreuzentropie

Beim Perzeptron werden falsch klassifizierte Punkte bestraft:

$$E_P(\vec{w}) = - \sum_{n \in \mathcal{M}} t_n \vec{w}^\top \vec{x}_n.$$

Bei logistischer Regression wird die negative Log-Likelihood minimiert:

$$E_{\text{CE}}(\vec{w}) = - \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

Hinweis

Die beiden Modelle können ähnliche lineare Entscheidungsgrenzen lernen, aber die Optimierungsprobleme sind unterschiedlich.

Logistische Regression ist probabilistisch motiviert. Das klassische Perzeptron ist eher algorithmisch und geometrisch motiviert.

8.20 Mehrere Ausgabekonventionen

Es gibt zwei häufige Kodierungen für binäre Klassen.

8.20.1 Kodierung mit $\{-1, +1\}$

Beim Perzeptron ist üblich:

$$t \in \{-1, +1\}.$$

Dann ist die Vorhersage:

$$y = \text{sign}(\vec{w}^\top \vec{x}).$$

Die Korrektheitsbedingung ist:

$$t\vec{w}^\top \vec{x} > 0.$$

8.20.2 Kodierung mit $\{0,1\}$

Bei logistischer Regression ist üblich:

$$t \in \{0,1\}.$$

Dann ist:

$$p(t = 1 \mid \vec{x}) = \sigma(\vec{w}^\top \vec{x}).$$

Hinweis

Beide Kodierungen beschreiben binäre Klassifikation. Man muss aber bei Formeln konsequent bleiben.

Für Perzeptron-Updates ist $\{-1, +1\}$ oft einfacher. Für Bernoulli-Likelihood und Kreuzentropie ist $\{0,1\}$ oft einfacher.

8.21 Limitierung: XOR-Problem

Ein einzelnes Perzeptron kann nur linear separierbare Daten klassifizieren. Das klassische Gegenbeispiel ist XOR.

Wir betrachten Eingaben

$$x_1, x_2 \in \{0,1\}.$$

Die XOR-Funktion ist:

x_1	x_2	t
0	0	0
0	1	1
1	0	1
1	1	0

Die positiven Punkte sind:

$$(0,1) \quad \text{und} \quad (1,0).$$

Die negativen Punkte sind:

$$(0,0) \quad \text{und} \quad (1,1).$$

Diese Punkte können in der Ebene nicht durch eine einzige Gerade getrennt werden.

Idee

Das XOR-Problem ist nicht linear separierbar.

Ein einzelnes Perzeptron kann XOR daher nicht lösen.

8.22 Warum mehrschichtige Netze helfen

Obwohl ein einzelnes Perzeptron nur lineare Entscheidungsgrenzen darstellen kann, können mehrere Perzeptrons kombiniert werden.

Ein mehrschichtiges neuronales Netz (multilayer neural network) verwendet versteckte Einheiten (hidden units). Diese berechnen Zwischenrepräsentationen:

$$h_j = g(\vec{w}_j^\top \vec{x}),$$

wobei g eine Aktivierungsfunktion ist.

Die Ausgabe kann dann aus diesen versteckten Einheiten berechnet werden:

$$y = f\left(\sum_j v_j h_j\right).$$

Idee

Mehrschichtige neuronale Netze können nichtlineare Entscheidungsgrenzen darstellen.

Die erste Schicht transformiert die Eingabe. Die nächste Schicht kann im transformierten Raum linear trennen.

8.23 Perzeptron als Baustein neuronaler Netze

Ein künstliches neuronales Netz besteht aus vielen Einheiten, die ähnlich wie Perzeptrons aufgebaut sind:

$$a_j = \sum_i w_{ji} x_i + b_j,$$

und

$$z_j = g(a_j).$$

Dabei ist:

- a_j die Aktivierung vor der Nichtlinearität,
- z_j die Ausgabe der Einheit,

- w_{ji} das Gewicht von Eingabe i zu Einheit j ,
- b_j der Bias,
- g die Aktivierungsfunktion.

Definition

Eine neuronale Einheit (neural unit) berechnet

$$z_j = g\left(\sum_i w_{ji}x_i + b_j\right).$$

Beim klassischen Perzeptron ist

$$g(a) = \text{sign}(a).$$

In modernen neuronalen Netzen verwendet man meistens glatte oder stückweise lineare Aktivierungsfunktionen, zum Beispiel Sigmoid, Tanh oder ReLU.

8.24 Aktivierungsfunktionen

Aktivierungsfunktionen (activation functions) machen neuronale Netze nichtlinear. Ohne nicht-lineare Aktivierungsfunktionen wäre die Verkettung mehrerer linearer Schichten wieder nur eine lineare Abbildung.

8.24.1 Sigmoid

Die Sigmoid-Funktion ist:

$$\sigma(a) = \frac{1}{1 + \exp(-a)}.$$

Sie gibt Werte in

$$(0,1)$$

aus.

8.24.2 Tanh

Die Hyperbolic-Tangent-Funktion ist:

$$\tanh(a) = \frac{\exp(a) - \exp(-a)}{\exp(a) + \exp(-a)}.$$

Sie gibt Werte in

$$(-1,1)$$

aus.

8.24.3 ReLU

Die ReLU-Funktion (rectified linear unit) ist:

$$\text{ReLU}(a) = \max(0, a).$$

Also:

$$\text{ReLU}(a) = \begin{cases} a, & a > 0, \\ 0, & a \leq 0. \end{cases}$$

Idee

Nichtlineare Aktivierungsfunktionen sind entscheidend.

Ohne sie kann ein mehrschichtiges Netz keine komplexeren Funktionen darstellen als ein einzelnes lineares Modell.

8.25 Warum lineare Schichten allein nicht reichen

Angenommen, wir haben zwei lineare Schichten ohne Aktivierungsfunktion:

$$\vec{h} = W_1 \vec{x},$$

und

$$\vec{y} = W_2 \vec{h}.$$

Dann gilt:

$$\vec{y} = W_2 W_1 \vec{x}.$$

Setze

$$W = W_2 W_1.$$

Dann:

$$\vec{y} = W \vec{x}.$$

Das ist wieder nur eine lineare Abbildung.

Prüfungshinweis

Mehrere lineare Schichten ohne Nichtlinearität sind äquivalent zu einer einzigen linearen Schicht.

Deshalb braucht ein neuronales Netz nichtlineare Aktivierungsfunktionen.

8.26 Perzeptron und Feature-Transformation

Wie bei logistischer Regression kann auch ein Perzeptron mit Basisfunktionen verwendet werden. Statt direkt $\vec{x}$ zu klassifizieren, verwendet man eine transformierte Darstellung:

$$\vec{\phi}(\vec{x}).$$

Dann ist:

$$y = \text{sign} \left(\vec{w}^\top \vec{\phi}(\vec{x}) \right).$$

Die Entscheidungsgrenze im transformierten Raum ist linear:

$$\vec{w}^\top \vec{\phi}(\vec{x}) = 0.$$

Im ursprünglichen Eingaberaum kann sie nichtlinear sein.

Beispiel

Für eine zweidimensionale Eingabe

$$\vec{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

kann man quadratische Features verwenden:

$$\vec{\phi}(\vec{x}) = \begin{pmatrix} 1 \\ x_1 \\ x_2 \\ x_1^2 \\ x_2^2 \\ x_1 x_2 \end{pmatrix}.$$

Dann kann das Perzeptron nichtlineare Grenzen wie Ellipsen oder Parabeln darstellen.

8.27 Online-Lernen

Das klassische Perzeptron ist ein Online-Lernalgorithmus (online learning algorithm). Das bedeutet: Die Gewichte werden nach einzelnen Datenpunkten aktualisiert.

$$\vec{w} \leftarrow \vec{w} + \eta t_n \vec{x}_n.$$

Diese Aktualisierung wird nur durchgeführt, wenn der aktuelle Datenpunkt falsch klassifiziert ist.

Definition

Online-Lernen (online learning) bedeutet, dass das Modell schrittweise mit einzelnen Datenpunkten aktualisiert wird, statt immer den gesamten Datensatz gleichzeitig zu verwenden.

Hinweis

Online-Lernen ist nützlich, wenn Daten nacheinander eintreffen oder wenn der Datensatz zu groß ist, um ihn vollständig auf einmal zu verarbeiten.

8.28 Lernrate

Die Lernrate (learning rate) η bestimmt die Schrittweite der Updates:

$$\vec{w} \leftarrow \vec{w} + \eta t_n \vec{x}_n.$$

Wenn η groß ist, sind die Updates groß. Wenn η klein ist, sind die Updates klein.

Beim klassischen Perzeptron ist die genaue Wahl von η weniger kritisch als bei vielen anderen Gradientenverfahren, weil nur das Vorzeichen von $\vec{w}^\top \vec{x}$ zählt. Für $\eta > 0$ ändert eine konstante Skalierung der Gewichte die Entscheidungsgrenze im Wesentlichen nicht.

Hinweis

Trotzdem kann η den Trainingsverlauf beeinflussen, insbesondere bei Varianten des Perzeptrons oder bei nicht separierbaren Daten.

8.29 Pocket-Algorithmus

Wenn Daten nicht linear separierbar sind, konvergiert das klassische Perzeptron nicht garantiert. Eine praktische Erweiterung ist der Pocket-Algorithmus (pocket algorithm).

Die Idee ist: Man speichert während des Trainings den bisher besten Gewichtsvektor.

Idee

Der Pocket-Algorithmus behält den besten bisher gefundenen Gewichtsvektor in der Tasche.

Auch wenn spätere Updates schlechter werden, kann man am Ende den besten gespeicherten Gewichtsvektor verwenden.

Formal:

1. Trainiere wie beim Perzeptron.
2. Nach bestimmten Schritten berechne die Anzahl der Fehlklassifikationen.
3. Wenn der aktuelle Gewichtsvektor besser ist als der bisher beste, speichere ihn.
4. Gib am Ende den besten gespeicherten Gewichtsvektor aus.

Hinweis

Der Pocket-Algorithmus löst das Problem nicht perfekt, aber er macht das Perzeptron bei nicht separierbaren Daten praktischer.

8.30 Vergleich: Perzeptron, logistische Regression und lineare Regression

Modell	Ausgabe	Zieltyp	Training
Lineare Regression	$\vec{w}^\top \vec{x}$	$t \in \mathbb{R}$	Least Squares
Logistische Regression	$\sigma(\vec{w}^\top \vec{x})$	$t \in \{0,1\}$	Kreuzentropie
Perzeptron	$\text{sign}(\vec{w}^\top \vec{x})$	$t \in \{-1, +1\}$	Perzeptron-Regel

Idee

Alle drei Modelle verwenden einen linearen Score.

Der Unterschied liegt darin, wie der Score interpretiert wird und welche Fehlerfunktion optimiert wird.

8.31 Mehrklassen-Perzeptron

Das Perzeptron kann auch auf mehrere Klassen erweitert werden. Sei

$$t \in \{1, 2, \dots, K\}.$$

Für jede Klasse k gibt es einen Gewichtsvektor $\vec{w}_k$. Der Score für Klasse k ist:

$$a_k = \vec{w}_k^\top \vec{x}.$$

Die vorhergesagte Klasse ist:

$$\hat{t} = \arg \max_k a_k.$$

Definition

Beim Mehrklassen-Perzeptron (multiclass perceptron) gilt:

$$\hat{t} = \arg \max_k \vec{w}_k^\top \vec{x}.$$

Wenn die wahre Klasse t ist und die vorhergesagte Klasse $\hat{t}$ falsch ist, kann man aktualisieren:

$$\vec{w}_t \leftarrow \vec{w}_t + \eta \vec{x},$$

und

$$\vec{w}_{\hat{t}} \leftarrow \vec{w}_{\hat{t}} - \eta \vec{x}.$$

Idee

Beim Mehrklassen-Perzeptron wird der Gewichtsvektor der richtigen Klasse in Richtung des Datenpunkts verschoben.

Der Gewichtsvektor der fälschlich vorhergesagten Klasse wird vom Datenpunkt weggeschoben.

8.32 Perzeptron mit Margin

Beim klassischen Perzeptron reicht es, wenn

$$t_n \vec{w}^\top \vec{x}_n > 0.$$

Man kann aber eine Sicherheitsmarge (margin) verlangen. Zum Beispiel:

$$t_n \vec{w}^\top \vec{x}_n \geq \gamma,$$

wobei

$$\gamma > 0$$

eine gewünschte Margin ist.

Definition

Die Margin eines Datenpunkts bezüglich $\vec{w}$ ist

$$t_n \vec{w}^\top \vec{x}_n.$$

Eine positive Margin bedeutet korrekte Klassifikation. Eine große positive Margin bedeutet, dass der Punkt mit Sicherheitsabstand korrekt klassifiziert wird.

Margin-Ideen führen später zu Modellen wie Support Vector Machines.

Hinweis

Das klassische Perzeptron versucht nur, die Punkte korrekt zu klassifizieren. Es maximiert nicht explizit die Margin.

8.33 Zusammenhang zu Support Vector Machines

Support Vector Machines (SVMs) sind ebenfalls lineare Klassifikatoren, haben aber ein anderes Trainingsziel. Während das Perzeptron irgendeine trennende Hyperebene finden kann, sucht eine SVM eine Hyperebene mit möglichst großer Margin.

Perzeptron $\Rightarrow$ irgendeine trennende Hyperebene.

SVM $\Rightarrow$ trennende Hyperebene mit großer Margin.

Idee

Das Perzeptron ist ein wichtiger Vorläufer vieler linearer Klassifikationsmethoden. Margin-basierte Methoden wie Support Vector Machines verbessern die Idee, indem sie nicht nur korrekte Klassifikation, sondern auch robuste Trennung verlangen.

8.34 Zusammenhang zu Backpropagation

Das klassische Perzeptron verwendet eine nicht differenzierbare Schwellenfunktion:

$$\text{sign}(a).$$

Für moderne neuronale Netze braucht man aber meistens differenzierbare Aktivierungsfunktionen, damit Gradienten mit Backpropagation berechnet werden können.

Deshalb verwendet man in neuronalen Netzen Funktionen wie:

$$\sigma(a), \quad \tanh(a), \quad \text{ReLU}(a).$$

Hinweis

ReLU ist bei $a = 0$ nicht differenzierbar, aber sie ist stückweise linear und funktioniert praktisch sehr gut. In Implementierungen verwendet man an $a = 0$ einfach eine geeignete Subgradienten-Konvention.

8.35 Praktische Aspekte

8.35.1 Feature-Skalierung

Wie bei logistischer Regression kann Feature-Skalierung wichtig sein. Wenn Merkmale sehr unterschiedliche Größenordnungen haben, dominieren Merkmale mit großen Zahlenwerten die Updates.

Eine typische Standardisierung ist:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}.$$

Dabei werden μ_j und σ_j auf den Trainingsdaten berechnet.

8.35.2 Datenreihenfolge

Da das Perzeptron online aktualisiert wird, kann die Reihenfolge der Datenpunkte den Trainingsverlauf beeinflussen. Oft mischt man die Daten zufällig.

8.35.3 Nicht separierbare Daten

Bei nicht separierbaren Daten sollte man nicht erwarten, dass das klassische Perzeptron perfekt konvergiert. Dann sind logistische Regression, regularisierte Modelle oder neuronale Netze oft stabiler.

8.36 Häufige Fehler

Prüfungshinweis

Typische Fehler bei Perzeptrons:

1. Perzeptron mit logistischer Regression verwechseln.
2. Die Vorzeichenkodierung $t \in \{-1, +1\}$ mit $t \in \{0,1\}$ mischen.
3. Den Bias-Term vergessen.
4. Denken, ein einzelnes Perzeptron könne nichtlinear separierbare Probleme lösen.
5. Die Entscheidungsgrenze mit der Aktivierungsfunktion verwechseln.
6. Die Perzeptron-Lernregel mit dem Gradienten der Kreuzentropie verwechseln.
7. Vergessen, dass das Perzeptron nur bei linear separierbaren Daten garantiert konvergiert.
8. Das XOR-Problem als linear separierbar ansehen.
9. Mehrere lineare Schichten ohne Aktivierungsfunktion für ein echtes tiefes Netz halten.
10. Nichtlineare Aktivierungsfunktionen in neuronalen Netzen unterschätzen.

8.37 Mini-Aufgaben

8.37.1 Aufgabe 1: Perzeptron-Vorhersage

Gegeben sei

$$\vec{w} = \begin{pmatrix} -1 \\ 2 \end{pmatrix}, \quad \vec{x} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Berechne die Perzeptron-Vorhersage

$$y = \text{sign}(\vec{w}^\top \vec{x}).$$

Beispiel

Zuerst:

$$\vec{w}^\top \vec{x} = (-1) \cdot 1 + 2 \cdot 1 = 1.$$

Da

$$1 > 0,$$

gilt:

$$y = +1.$$

8.37.2 Aufgabe 2: Falsch klassifiziert oder korrekt?

Gegeben sei

$$t = -1$$

und

$$\vec{w}^\top \vec{x} = 2.$$

Ist der Punkt korrekt klassifiziert?

Beispiel

Ein Punkt ist korrekt klassifiziert, wenn

$$t\vec{w}^\top \vec{x} > 0.$$

Hier:

$$t\vec{w}^\top \vec{x} = (-1) \cdot 2 = -2.$$

Da

$$-2 \leq 0,$$

ist der Punkt falsch klassifiziert.

8.37.3 Aufgabe 3: Perzeptron-Update

Gegeben sei

$$\vec{w} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad \vec{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad t = -1,$$

und

$$\eta = 1.$$

Angenommen, der Punkt ist falsch klassifiziert. Berechne das Update.

Beispiel

Die Lernregel ist:

$$\vec{w}_{\text{neu}} = \vec{w} + \eta t \vec{x}.$$

Einsetzen:

$$\vec{w}_{\text{neu}} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} + 1 \cdot (-1) \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

Also:

$$\vec{w}_{\text{neu}} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} - \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \end{pmatrix}.$$

8.37.4 Aufgabe 4: Entscheidungsgrenze

Gegeben sei ein Perzeptron mit

$$a = w_0 + w_1 x_1 + w_2 x_2.$$

Die Gewichte seien

$$w_0 = -2, \quad w_1 = 1, \quad w_2 = 1.$$

Bestimme die Entscheidungsgrenze.

Beispiel

Die Entscheidungsgrenze ist:

$$a = 0.$$

Also:

$$-2 + x_1 + x_2 = 0.$$

Damit:

$$x_2 = 2 - x_1.$$

Das ist eine Gerade.

8.37.5 Aufgabe 5: XOR

Warum kann ein einzelnes Perzeptron XOR nicht lösen?

Beispiel

Ein einzelnes Perzeptron hat eine lineare Entscheidungsgrenze:

$$\vec{w}^\top \vec{x} = 0.$$

Für $D = 2$ ist das eine Gerade.

Die XOR-Datenpunkte sind aber nicht durch eine einzige Gerade trennbar. Daher kann ein einzelnes Perzeptron XOR nicht lösen.

8.38 Zusammenfassung

- Ein Perzeptron (perceptron) ist ein linearer Klassifikator.

- Es berechnet einen Score:

$$a = \vec{w}^\top \vec{x}.$$

- Die Ausgabe ist:

$$y = \text{sign}(a).$$

- Mit Klassen $t \in \{-1, +1\}$ ist ein Punkt korrekt klassifiziert, wenn:

$$t\vec{w}^\top \vec{x} > 0.$$

- Die Entscheidungsgrenze ist:

$$\vec{w}^\top \vec{x} = 0.$$

- Ein einzelnes Perzeptron kann nur linear separierbare Daten perfekt trennen.

- Das Perzeptron-Kriterium ist:

$$E_P(\vec{w}) = - \sum_{n \in \mathcal{M}} t_n \vec{w}^\top \vec{x}_n.$$

- Die Perzeptron-Lernregel ist:

$$\vec{w} \leftarrow \vec{w} + \eta t_n \vec{x}_n.$$

- Das Perzeptron konvergiert garantiert, wenn die Daten linear separierbar sind.
- Das XOR-Problem ist nicht linear separierbar und kann daher nicht von einem einzelnen Perzeptron gelöst werden.
- Mehrschichtige neuronale Netze verwenden Perzeptron-ähnliche Einheiten mit nichtlinearen Aktivierungsfunktionen.
- Nichtlineare Aktivierungsfunktionen sind notwendig, weil mehrere lineare Schichten ohne Nichtlinearität wieder nur eine lineare Abbildung ergeben.

8.39 Typische Prüfungsfragen

Prüfungshinweis

Mögliche Prüfungsfragen zu diesem Kapitel:

1. Definiere ein Perzeptron.
2. Was ist der Unterschied zwischen Perzeptron und logistischer Regression?
3. Welche Rolle spielt der Bias-Term?
4. Was ist die Entscheidungsgrenze eines Perzeptrons?
5. Warum ist ein einzelnes Perzeptron ein linearer Klassifikator?
6. Was bedeutet lineare Separierbarkeit?
7. Zeige, warum $t_n \vec{w}^\top \vec{x}_n > 0$ korrekte Klassifikation bedeutet.
8. Definiere das Perzeptron-Kriterium.
9. Leite den Gradienten des Perzeptron-Kriteriums für einen falsch klassifizierten Punkt her.
10. Leite die Perzeptron-Lernregel her.
11. Erkläre intuitiv, warum die Perzeptron-Lernregel sinnvoll ist.
12. Wann konvergiert der Perzeptron-Algorithmus?
13. Warum konvergiert er bei nicht separierbaren Daten nicht garantiert?
14. Warum kann ein einzelnes Perzeptron XOR nicht lösen?
15. Wie können Basisfunktionen einem Perzeptron nichtlineare Grenzen ermöglichen?
16. Warum brauchen mehrschichtige neuronale Netze nichtlineare Aktivierungsfunktionen?
17. Was ist der Unterschied zwischen Online-Lernen und Batch-Lernen?
18. Was ist der Pocket-Algorithmus?
19. Wie funktioniert das Mehrklassen-Perzeptron?
20. Wie hängt das Perzeptron historisch und mathematisch mit neuronalen Netzen zusammen?

9 Mehrschichtige Perzeptrons und Backpropagation

9.1 Ziel dieses Kapitels

In den vorherigen Kapiteln haben wir lineare Modelle betrachtet: lineare Regression (linear regression), logistische Regression (logistic regression) und Perzeptrons (perceptrons). Diese Modelle berechnen im Kern einen linearen Score der Form

$$a = \vec{w}^\top \vec{x}.$$

Solche Modelle sind wichtig, aber sie haben eine zentrale Einschränkung: Ohne geeignete Merkmalsabbildung können sie nur lineare Entscheidungsgrenzen oder lineare Zusammenhänge darstellen.

Mehrschichtige Perzeptrons (multilayer perceptrons), kurz MLPs, erweitern diese Idee. Sie bestehen aus mehreren Schichten (layers) von künstlichen Neuronen (artificial neurons). Durch nichtlineare Aktivierungsfunktionen (activation functions) können sie komplexe nichtlineare Funktionen lernen.

Idee

Ein mehrschichtiges Perzeptron (multilayer perceptron) ist ein neuronales Netz (neural network), das Eingaben durch mehrere Schichten transformiert.

Jede Schicht berechnet eine affine Transformation und danach eine Nichtlinearität:

$$\vec{z}^{(l)} = g^{(l)} \left(W^{(l)} \vec{z}^{(l-1)} + \vec{b}^{(l)} \right).$$

Nach diesem Kapitel solltest du folgende Fragen beantworten können:

- Was ist ein mehrschichtiges Perzeptron (multilayer perceptron)?
- Was ist der Unterschied zwischen Eingabeschicht, versteckter Schicht und Ausgabeschicht?
- Warum braucht man nichtlineare Aktivierungsfunktionen?
- Was ist ein Forward Pass?
- Was ist eine Verlustfunktion (loss function)?
- Was ist Backpropagation?
- Wie berechnet man Gradienten mit der Kettenregel?

- Was bedeuten lokale Gradienten und Fehlerterme?
- Wie sehen die Gradienten für Gewichte und Biases aus?
- Wie trainiert man ein neuronales Netz mit Gradientenabstieg?
- Warum können Gradienten verschwinden oder explodieren?
- Welche Rolle spielen Sigmoid, Tanh und ReLU?

9.2 Vom Perzeptron zum neuronalen Netz

Ein einzelnes Perzeptron berechnet

$$a = \vec{w}^\top \vec{x} + b$$

und danach eine Aktivierungsfunktion:

$$z = g(a).$$

Beim klassischen Perzeptron ist g eine Schwellenfunktion. In modernen neuronalen Netzen verwendet man meistens differenzierbare Aktivierungsfunktionen wie Sigmoid, Tanh oder ReLU.

Definition

Ein künstliches Neuron (artificial neuron) berechnet

$$a = \vec{w}^\top \vec{x} + b,$$

und danach

$$z = g(a).$$

Dabei ist a die Voraktivierung (pre-activation), z die Aktivierung (activation), $\vec{w}$ der Gewichtsvektor, b der Bias und g die Aktivierungsfunktion.

Ein einzelnes Neuron ist noch stark eingeschränkt. Die Idee eines neuronalen Netzes ist, viele solcher Neuronen in Schichten zu organisieren.

9.3 Schichten eines neuronalen Netzes

Ein mehrschichtiges Perzeptron besteht aus mehreren Schichten:

- Eingabeschicht (input layer),
- eine oder mehrere versteckte Schichten (hidden layers),
- Ausgabeschicht (output layer).

Die Eingabeschicht enthält die Eingabedaten. Die versteckten Schichten berechnen interne Repräsentationen. Die Ausgabeschicht erzeugt die Vorhersage.

Definition

Eine versteckte Schicht (hidden layer) ist eine Schicht zwischen Eingabe und Ausgabe. Sie heißt versteckt, weil ihre Zielwerte nicht direkt im Datensatz beobachtet werden. Sie wird nur indirekt durch das Trainingsziel gelernt.

Für ein Netz mit L Schichten schreiben wir:

$$\vec{z}^{(0)} = \vec{x}.$$

Die Eingabe $\vec{x}$ ist also die Aktivierung der Schicht 0. Für Schicht l gilt dann:

$$\vec{a}^{(l)} = W^{(l)}\vec{z}^{(l-1)} + \vec{b}^{(l)},$$

und

$$\vec{z}^{(l)} = g^{(l)}(\vec{a}^{(l)}).$$

Dabei ist:

$$l \in \{1, \dots, L\}.$$

Die Ausgabe des Netzes ist:

$$\vec{y} = \vec{z}^{(L)}.$$

9.4 Dimensionen der Parameter

Angenommen, Schicht $l - 1$ hat D_{l-1} Einheiten und Schicht l hat D_l Einheiten. Dann gilt:

$$\vec{z}^{(l-1)} \in \mathbb{R}^{D_{l-1}},$$

$$W^{(l)} \in \mathbb{R}^{D_l \times D_{l-1}},$$

$$\vec{b}^{(l)} \in \mathbb{R}^{D_l},$$

$$\vec{a}^{(l)} \in \mathbb{R}^{D_l},$$

und

$$\vec{z}^{(l)} \in \mathbb{R}^{D_l}.$$

Hinweis

Die Gewichtsmatrix $W^{(l)}$ hat eine Zeile pro Neuron der aktuellen Schicht und eine Spalte pro Ausgabe der vorherigen Schicht.

Das Produkt

$$W^{(l)} \vec{z}^{(l-1)}$$

hat daher die Dimension $\mathbb{R}^{D_l}$.

9.5 Forward Pass

Der Forward Pass ist die Berechnung der Ausgabe aus der Eingabe.

Für ein Netz mit zwei Schichten, also einer versteckten Schicht und einer Ausgabeschicht, gilt:

$$\vec{z}^{(0)} = \vec{x}.$$

Die versteckte Schicht berechnet:

$$\vec{a}^{(1)} = W^{(1)} \vec{x} + \vec{b}^{(1)},$$

$$\vec{z}^{(1)} = g^{(1)}(\vec{a}^{(1)}).$$

Die Ausgabeschicht berechnet:

$$\vec{a}^{(2)} = W^{(2)} \vec{z}^{(1)} + \vec{b}^{(2)},$$

$$\vec{y} = \vec{z}^{(2)} = g^{(2)}(\vec{a}^{(2)}).$$

Definition

Der Forward Pass ist die schichtweise Berechnung

$$\vec{x} \longrightarrow \vec{z}^{(1)} \longrightarrow \vec{z}^{(2)} \longrightarrow \dots \longrightarrow \vec{y}.$$

Dabei werden die aktuellen Parameter $W^{(l)}$ und $\vec{b}^{(l)}$ verwendet.

9.6 Warum Nichtlinearitäten notwendig sind

Angenommen, ein Netz hätte zwei lineare Schichten ohne Aktivierungsfunktion:

$$\vec{z}^{(1)} = W^{(1)} \vec{x} + \vec{b}^{(1)}$$

und

$$\vec{y} = W^{(2)} \vec{z}^{(1)} + \vec{b}^{(2)}.$$

Einsetzen ergibt:

$$\vec{y} = W^{(2)} \left(W^{(1)} \vec{x} + \vec{b}^{(1)} \right) + \vec{b}^{(2)}.$$

Also:

$$\vec{y} = W^{(2)} W^{(1)} \vec{x} + W^{(2)} \vec{b}^{(1)} + \vec{b}^{(2)}.$$

Setze:

$$W = W^{(2)} W^{(1)}$$

und

$$\vec{b} = W^{(2)} \vec{b}^{(1)} + \vec{b}^{(2)}.$$

Dann:

$$\vec{y} = W \vec{x} + \vec{b}.$$

Das ist wieder nur eine affine Abbildung.

Idee

Mehrere lineare Schichten ohne Nichtlinearität sind äquivalent zu einer einzigen linearen Schicht.

Nichtlineare Aktivierungsfunktionen sind deshalb notwendig, damit tiefe Netze komplexe nichtlineare Funktionen darstellen können.

9.7 Wichtige Aktivierungsfunktionen

9.7.1 Sigmoid

Die Sigmoid-Funktion (sigmoid function) ist:

$$\sigma(a) = \frac{1}{1 + \exp(-a)}.$$

Sie bildet auf das Intervall (0,1) ab.

Die Ableitung ist:

$$\sigma'(a) = \sigma(a)(1 - \sigma(a)).$$

Hinweis

Sigmoid eignet sich als Ausgabeaktivierung für binäre Klassifikation. In versteckten Schichten kann Sigmoid problematisch sein, weil die Ableitung für große positive oder negative Werte sehr klein wird.

9.7.2 Tanh

Die Tanh-Funktion ist:

$$\tanh(a) = \frac{\exp(a) - \exp(-a)}{\exp(a) + \exp(-a)}.$$

Sie bildet auf das Intervall $(-1,1)$ ab.

Die Ableitung ist:

$$\frac{d}{da} \tanh(a) = 1 - \tanh^2(a).$$

9.7.3 ReLU

Die ReLU-Funktion (rectified linear unit) ist:

$$\text{ReLU}(a) = \max(0, a).$$

Also:

$$\text{ReLU}(a) = \begin{cases} a, & a > 0, \\ 0, & a \leq 0. \end{cases}$$

Die Ableitung ist für $a \neq 0$:

$$\text{ReLU}'(a) = \begin{cases} 1, & a > 0, \\ 0, & a < 0. \end{cases}$$

Idee

ReLU ist in modernen neuronalen Netzen sehr beliebt, weil sie einfach ist und für positive Aktivierungen keinen kleinen Sättigungsgradienten hat.

9.8 Ausgabeschichten

Die Wahl der Ausgabeschicht hängt vom Problemtyp ab.

9.8.1 Regression

Für Regression verwendet man oft eine lineare Ausgabe:

$$\vec{y} = \vec{a}^{(L)}.$$

Bei eindimensionaler Regression:

$$y \in \mathbb{R}.$$

Eine typische Verlustfunktion ist der quadratische Fehler:

$$E = \frac{1}{2}(y - t)^2.$$

9.8.2 Binäre Klassifikation

Für binäre Klassifikation verwendet man meistens eine Sigmoid-Ausgabe:

$$y = \sigma(a^{(L)}).$$

Dann gilt:

$$y = p(t = 1 \mid \vec{x}).$$

Die passende Verlustfunktion ist die binäre Kreuzentropie (binary cross-entropy):

$$E = -[t \log y + (1 - t) \log(1 - y)].$$

9.8.3 Mehrklassen-Klassifikation

Für K Klassen verwendet man Softmax:

$$y_k = \frac{\exp(a_k)}{\sum_{j=1}^K \exp(a_j)}.$$

Die passende Verlustfunktion ist die Mehrklassen-Kreuzentropie (multiclass cross-entropy):

$$E = - \sum_{k=1}^K t_k \log y_k.$$

9.9 Verlustfunktion für einen Datensatz

Für einen einzelnen Datenpunkt schreiben wir den Verlust als

$$E_n(\Theta),$$

wobei Θ alle Parameter des Netzes bezeichnet:

$$\Theta = \{W^{(1)}, \vec{b}^{(1)}, \dots, W^{(L)}, \vec{b}^{(L)}\}.$$

Für einen Datensatz mit N Datenpunkten ist der Gesamtverlust häufig:

$$E(\Theta) = \sum_{n=1}^N E_n(\Theta).$$

Oder als Durchschnitt:

$$E(\Theta) = \frac{1}{N} \sum_{n=1}^N E_n(\Theta).$$

Definition

Training eines neuronalen Netzes bedeutet, die Parameter Θ so zu wählen, dass eine Verlustfunktion minimiert wird:

$$\Theta^* = \arg \min_{\Theta} E(\Theta).$$

9.10 Das Problem: Viele Parameter

Ein neuronales Netz kann sehr viele Parameter haben. Für jede Schicht l gibt es:

$$W^{(l)}$$

und

$$\vec{b}^{(l)}.$$

Um Gradientenabstieg zu verwenden, brauchen wir alle Ableitungen:

$$\frac{\partial E}{\partial W^{(l)}}$$

und

$$\frac{\partial E}{\partial \vec{b}^{(l)}}.$$

Direkt alle Ableitungen einzeln auszurechnen wäre unübersichtlich und ineffizient. Backpropagation löst genau dieses Problem.

Idee

Backpropagation ist ein effizienter Algorithmus zur Berechnung aller Gradienten in einem neuronalen Netz.

Die zentrale mathematische Grundlage ist die Kettenregel (chain rule).

9.11 Kettenregel

Backpropagation basiert auf der Kettenregel. Für skalare Funktionen gilt:

$$z = f(y), \quad y = g(x).$$

Dann ist:

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}.$$

In neuronalen Netzen sind die Größen meistens Vektoren und Matrizen. Die Idee bleibt aber gleich: Eine Änderung in einer frühen Schicht beeinflusst alle späteren Schichten.

Definition

Die Kettenregel erlaubt, Ableitungen zusammengesetzter Funktionen zu berechnen.
Für neuronale Netze bedeutet das: Der Fehler wird von der Ausgabe zurück durch die Schichten propagiert.

9.12 Backpropagation-Idee

Der Forward Pass berechnet:

$$\vec{x} \longrightarrow \vec{z}^{(1)} \longrightarrow \dots \longrightarrow \vec{z}^{(L)} = \vec{y} \longrightarrow E.$$

Backpropagation läuft in umgekehrter Richtung:

$$E \longrightarrow \vec{z}^{(L)} \longrightarrow \vec{a}^{(L)} \longrightarrow \dots \longrightarrow \vec{a}^{(1)}.$$

Dabei werden sogenannte Fehlerterme (error terms) berechnet.

Definition

Der Fehlerterm einer Schicht l ist

$$\delta^{(l)} = \frac{\partial E}{\partial \vec{a}^{(l)}}.$$

Er beschreibt, wie stark der Verlust auf Änderungen der Voraktivierung $\vec{a}^{(l)}$ reagiert.

9.13 Ein einfaches Zwei-Schichten-Netz

Wir betrachten nun ein Netz mit einer versteckten Schicht und einer Ausgabeschicht.

Forward Pass:

$$\vec{z}^{(0)} = \vec{x}.$$

Versteckte Schicht:

$$\vec{a}^{(1)} = W^{(1)}\vec{x} + \vec{b}^{(1)},$$

$$\vec{z}^{(1)} = g^{(1)}(\vec{a}^{(1)}).$$

Ausgabeschicht:

$$\vec{a}^{(2)} = W^{(2)}\vec{z}^{(1)} + \vec{b}^{(2)},$$

$$\vec{y} = \vec{z}^{(2)} = g^{(2)}(\vec{a}^{(2)}).$$

Der Verlust ist:

$$E = E(\vec{y}, \vec{t}).$$

9.14 Gradienten der Ausgabeschicht

Wir wollen die Ableitung nach $W^{(2)}$ berechnen. Für die Ausgabeschicht gilt:

$$\vec{a}^{(2)} = W^{(2)} \vec{z}^{(1)} + \vec{b}^{(2)}.$$

Der Fehlerterm ist:

$$\vec{\delta}^{(2)} = \frac{\partial E}{\partial \vec{a}^{(2)}}.$$

Dann gilt für die Gewichtsmatrix:

$$\frac{\partial E}{\partial W^{(2)}} = \vec{\delta}^{(2)} (\vec{z}^{(1)})^\top.$$

Für den Bias gilt:

$$\frac{\partial E}{\partial \vec{b}^{(2)}} = \vec{\delta}^{(2)}.$$

Definition

Für eine Schicht l gilt allgemein:

$$\frac{\partial E}{\partial W^{(l)}} = \vec{\delta}^{(l)} (\vec{z}^{(l-1)})^\top,$$

und

$$\frac{\partial E}{\partial \vec{b}^{(l)}} = \vec{\delta}^{(l)}.$$

9.15 Fehlerterm der Ausgabeschicht

Der Fehlerterm der Ausgabeschicht hängt von der gewählten Ausgabefunktion und Verlustfunktion ab.

9.15.1 Quadratischer Fehler

Bei quadratischem Fehler:

$$E = \frac{1}{2} \|\vec{y} - \vec{t}\|^2$$

gilt:

$$\frac{\partial E}{\partial \vec{y}} = \vec{y} - \vec{t}.$$

Wenn

$$\vec{y} = g^{(L)}(\vec{a}^{(L)}),$$

dann ist:

$$\vec{\delta}^{(L)} = (\vec{y} - \vec{t}) \odot (g^{(L)})'(\vec{a}^{(L)}).$$

Dabei bedeutet $\odot$ komponentenweise Multiplikation.

9.15.2 Softmax mit Kreuzentropie

Bei Softmax-Ausgabe und Kreuzentropie vereinfacht sich der Fehlerterm zu:

$$\vec{\delta}^{(L)} = \vec{y} - \vec{t}.$$

9.15.3 Sigmoid mit binärer Kreuzentropie

Bei Sigmoid-Ausgabe und binärer Kreuzentropie gilt ebenfalls:

$$\delta^{(L)} = y - t.$$

Idee

Die Kombination aus Sigmoid und Kreuzentropie oder Softmax und Kreuzentropie führt zu einem einfachen Ausgabefehler:

$$\text{Ausgabefehler} = \text{Vorhersage} - \text{Ziel}.$$

9.16 Fehlerterm der versteckten Schicht

Nun wollen wir den Fehlerterm der versteckten Schicht berechnen:

$$\vec{\delta}^{(1)} = \frac{\partial E}{\partial \vec{a}^{(1)}}.$$

Die versteckte Schicht beeinflusst den Verlust über die Ausgabeschicht. Daher verwenden wir die Kettenregel.

Für allgemeine Schicht l gilt:

$$\vec{\delta}^{(l)} = \left((W^{(l+1)})^\top \vec{\delta}^{(l+1)} \right) \odot (g^{(l)})'(\vec{a}^{(l)}).$$

Für die versteckte Schicht im Zwei-Schichten-Netz:

$$\vec{\delta}^{(1)} = \left((W^{(2)})^\top \vec{\delta}^{(2)} \right) \odot (g^{(1)})'(\vec{a}^{(1)}).$$

Definition

Die Backpropagation-Rekursion für versteckte Schichten lautet:

$$\vec{\delta}^{(l)} = \left((W^{(l+1)})^\top \vec{\delta}^{(l+1)} \right) \odot (g^{(l)})'(\vec{a}^{(l)}).$$

9.17 Backpropagation-Algorithmus

Für einen einzelnen Datenpunkt läuft Backpropagation so ab:

1. Forward Pass: Berechne für alle Schichten

$$\vec{a}^{(l)} = W^{(l)} \vec{z}^{(l-1)} + \vec{b}^{(l)}$$

und

$$\vec{z}^{(l)} = g^{(l)}(\vec{a}^{(l)}).$$

2. Verlust berechnen:

$$E = E(\vec{z}^{(L)}, \vec{t}).$$

3. Ausgabefehler berechnen:

$$\vec{\delta}^{(L)} = \frac{\partial E}{\partial \vec{a}^{(L)}}.$$

4. Rückwärts durch die versteckten Schichten:

$$\vec{\delta}^{(l)} = \left((W^{(l+1)})^\top \vec{\delta}^{(l+1)} \right) \odot (g^{(l)})'(\vec{a}^{(l)}).$$

5. Gradienten berechnen:

$$\frac{\partial E}{\partial W^{(l)}} = \vec{\delta}^{(l)} (\vec{z}^{(l-1)})^\top,$$

$$\frac{\partial E}{\partial \vec{b}^{(l)}} = \vec{\delta}^{(l)}.$$

Idee

Backpropagation speichert im Forward Pass die Aktivierungen und Voraktivierungen. Im Backward Pass werden daraus effizient alle Gradienten berechnet.

9.18 Gradientenabstieg für MLPs

Nach der Berechnung der Gradienten werden die Parameter aktualisiert. Für Batch-Gradientenabstieg gilt:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial E}{\partial W^{(l)}},$$

und

$$\vec{b}^{(l)} \leftarrow \vec{b}^{(l)} - \eta \frac{\partial E}{\partial \vec{b}^{(l)}}.$$

Dabei ist $\eta > 0$ die Lernrate (learning rate).

Definition

Training mit Gradientenabstieg bedeutet:

$$\Theta \leftarrow \Theta - \eta \nabla_{\Theta} E(\Theta).$$

Dabei enthält Θ alle Gewichte und Biases des Netzes.

9.19 Mini-Batch-Training

In der Praxis verwendet man selten den ganzen Datensatz auf einmal. Stattdessen verwendet man Mini-Batches.

Ein Mini-Batch $\mathcal{B}$ ist eine Teilmenge der Trainingsdaten. Der Mini-Batch-Verlust ist:

$$E_{\mathcal{B}}(\Theta) = \frac{1}{|\mathcal{B}|} \sum_{n \in \mathcal{B}} E_n(\Theta).$$

Die Parameter werden mit dem Gradienten dieses Mini-Batch-Verlusts aktualisiert:

$$\Theta \leftarrow \Theta - \eta \nabla_{\Theta} E_{\mathcal{B}}(\Theta).$$

Idee

Mini-Batch-Training ist ein Kompromiss zwischen Batch-Gradientenabstieg und stochastischem Gradientenabstieg.

Es ist effizient, skaliert gut und nutzt moderne Hardware besser.

9.20 Matrixform für mehrere Datenpunkte

Für einen Mini-Batch kann man die Aktivierungen als Matrizen schreiben. Sei $Z^{(0)}$ eine Matrix, deren Spalten die Eingaben des Mini-Batches sind.

Für Schicht l :

$$A^{(l)} = W^{(l)} Z^{(l-1)} + \vec{b}^{(l)} \vec{1}^T.$$

Die Aktivierungen sind:

$$Z^{(l)} = g^{(l)}(A^{(l)}).$$

Die Fehlerterme werden ebenfalls als Matrizen geschrieben:

$$\Delta^{(l)} = \frac{\partial E}{\partial A^{(l)}}.$$

Dann gilt:

$$\frac{\partial E}{\partial W^{(l)}} = \Delta^{(l)} (Z^{(l-1)})^\top,$$

und

$$\frac{\partial E}{\partial \vec{b}^{(l)}} = \Delta^{(l)} \vec{1}.$$

Hinweis

Je nach Konvention liegen Datenpunkte als Zeilen oder Spalten in den Matrizen. Die Formeln müssen dann entsprechend transponiert werden. Wichtig ist nicht die Konvention, sondern konsistente Dimensionen.

9.21 Initialisierung der Gewichte

Die Wahl der Anfangswerte ist wichtig. Wenn alle Gewichte mit 0 initialisiert werden, sind Neuronen derselben Schicht symmetrisch. Dann lernen sie dieselben Features.

Idee

Gewichte neuronaler Netze sollten zufällig initialisiert werden, damit verschiedene Neuronen unterschiedliche Funktionen lernen können.

Typische Initialisierungen berücksichtigen die Anzahl der Eingänge und Ausgänge einer Schicht.

9.21.1 Xavier-Initialisierung

Xavier-Initialisierung wird häufig für Sigmoid oder Tanh verwendet. Die Varianz der Gewichte wird so gewählt, dass Aktivierungen und Gradienten nicht zu stark wachsen oder schrumpfen.

9.21.2 He-Initialisierung

He-Initialisierung wird häufig für ReLU-Netze verwendet. Sie berücksichtigt, dass ReLU ungefähr die Hälfte der Aktivierungen auf 0 setzt.

Hinweis

Die genaue Initialisierung ist ein praktisches Detail, aber die Grundidee ist wichtig: Zu große Gewichte können Aktivierungen und Gradienten explodieren lassen. Zu kleine Gewichte können Signale verschwinden lassen.

9.22 Verschwindende und explodierende Gradienten

Bei tiefen Netzen werden viele Ableitungen miteinander multipliziert. Dadurch können Gradienten sehr klein oder sehr groß werden.

Wenn viele Faktoren kleiner als 1 sind, kann der Gradient verschwinden:

$$\|\nabla E\| \approx 0.$$

Das nennt man verschwindende Gradienten (vanishing gradients).

Wenn viele Faktoren größer als 1 sind, kann der Gradient explodieren:

$$\|\nabla E\| \gg 1.$$

Das nennt man explodierende Gradienten (exploding gradients).

Idee

Verschwindende Gradienten erschweren das Lernen früher Schichten.

Explodierende Gradienten machen das Training instabil.

Sigmoid und Tanh können verschwindende Gradienten begünstigen, wenn sie in Sättigungsbereiche geraten. ReLU reduziert dieses Problem teilweise, hat aber eigene Probleme wie tote Neuronen.

9.23 Tote ReLU-Neuronen

Ein ReLU-Neuron ist tot, wenn es für alle oder fast alle Eingaben

$$a \leq 0$$

hat. Dann ist:

$$\text{ReLU}(a) = 0$$

und

$$\text{ReLU}'(a) = 0.$$

Der Gradient für dieses Neuron ist dann 0, und es kann kaum noch lernen.

Hinweis

Tote ReLUs können durch zu große Lernraten oder ungünstige Initialisierung entstehen.

Varianten wie Leaky ReLU reduzieren dieses Problem.

9.24 Regularisierung neuronaler Netze

Neuronale Netze können sehr flexibel sein und daher überanpassen. Typische Regularisierungsmethoden sind:

- L_2 -Regularisierung,
- Early Stopping,
- Dropout,
- Datenaugmentation (data augmentation),

- kleinere Netzwerkarchitekturen.

9.24.1 L_2 -Regularisierung

Man ergänzt den Verlust um:

$$\frac{\lambda}{2} \sum_l \|W^{(l)}\|_F^2.$$

Dann ist:

$$E_{\text{reg}}(\Theta) = E(\Theta) + \frac{\lambda}{2} \sum_l \|W^{(l)}\|_F^2.$$

9.24.2 Early Stopping

Beim Early Stopping wird das Training beendet, wenn der Validierungsfehler nicht mehr besser wird.

Idee

Early Stopping wirkt als Regularisierung, weil das Netz nicht beliebig lange an die Trainingsdaten angepasst wird.

9.24.3 Dropout

Bei Dropout werden während des Trainings zufällig Aktivierungen auf 0 gesetzt. Dadurch soll das Netz robuster werden und sich weniger auf einzelne Neuronen verlassen.

9.25 Universal Approximation

Ein wichtiger theoretischer Satz sagt: Ein neuronales Netz mit mindestens einer versteckten Schicht und geeigneter Aktivierungsfunktion kann unter bestimmten Bedingungen jede stetige Funktion auf einem kompakten Bereich beliebig gut approximieren.

Idee

Der Universal-Approximation-Satz sagt nicht, dass ein Netz automatisch gut lernt. Er sagt nur, dass es prinzipiell genug Ausdruckskraft haben kann. Training, Datenmenge, Optimierung und Regularisierung bleiben trotzdem entscheidend.

9.26 Vergleich mit linearen Modellen

Modell	Form	Nichtlinearität
Lineare Regression	$\vec{w}^\top \vec{x}$	keine
Logistische Regression	$\sigma(\vec{w}^\top \vec{x})$	nur Ausgabe
MLP	$g^{(L)}(W^{(L)} \dots g^{(1)}(W^{(1)} \vec{x} + \vec{b}^{(1)}))$	mehrere Schichten

Idee

Lineare und logistische Regression lernen eine direkte Abbildung aus den gegebenen Features.

Ein MLP lernt zusätzlich interne Features in den versteckten Schichten.

9.27 Häufige Fehler

Prüfungshinweis

Typische Fehler bei MLPs und Backpropagation:

1. Denken, mehrere lineare Schichten ohne Aktivierungsfunktion seien mächtiger als eine lineare Schicht.
2. Die Dimensionen von $W^{(l)}$, $\vec{z}^{(l)}$ und $\vec{b}^{(l)}$ verwechseln.
3. Voraktivierung $\vec{a}^{(l)}$ und Aktivierung $\vec{z}^{(l)}$ verwechseln.
4. Die Ableitung der Aktivierungsfunktion in Backpropagation vergessen.
5. Den Fehlerterm $\vec{\delta}^{(l)}$ mit dem Gradienten nach den Gewichten verwechseln.
6. Bei $\frac{\partial E}{\partial W^{(l)}}$ das äußere Produkt vergessen.
7. Softmax mit Sigmoid verwechseln.
8. Kreuzentropie und quadratischen Fehler unpassend kombinieren.
9. Gewichte mit 0 initialisieren und dadurch Symmetrieprobleme erzeugen.
10. Training Error und Validation Error verwechseln.

9.28 Mini-Aufgaben

9.28.1 Aufgabe 1: Dimensionen einer Schicht

Eine Schicht erhält Eingaben

$$\vec{z}^{(l-1)} \in \mathbb{R}^5$$

und hat

$$3$$

Neuronen. Welche Dimensionen haben $W^{(l)}$, $\vec{b}^{(l)}$, $\vec{a}^{(l)}$ und $\vec{z}^{(l)}$?

Beispiel

Da die Schicht 3 Ausgaben und 5 Eingaben hat:

$$W^{(l)} \in \mathbb{R}^{3 \times 5}.$$

Der Bias hat eine Komponente pro Neuron:

$$\vec{b}^{(l)} \in \mathbb{R}^3.$$

Daher:

$$\vec{a}^{(l)} \in \mathbb{R}^3, \quad \vec{z}^{(l)} \in \mathbb{R}^3.$$

9.28.2 Aufgabe 2: ReLU berechnen

Berechne

$$\text{ReLU}(-2), \quad \text{ReLU}(3).$$

Beispiel

Mit

$$\text{ReLU}(a) = \max(0, a)$$

folgt:

$$\text{ReLU}(-2) = 0,$$

und

$$\text{ReLU}(3) = 3.$$

9.28.3 Aufgabe 3: Ausgabefehler bei Softmax

Für einen Datenpunkt sei

$$\vec{y} = \begin{pmatrix} 0,1 \\ 0,7 \\ 0,2 \end{pmatrix}, \quad \vec{t} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}.$$

Berechne den Fehlerterm der Ausgabeschicht bei Softmax mit Kreuzentropie.

Beispiel

Bei Softmax mit Kreuzentropie gilt:

$$\vec{\delta}^{(L)} = \vec{y} - \vec{t}.$$

Also:

$$\vec{\delta}^{(L)} = \begin{pmatrix} 0,1 \\ 0,7 \\ 0,2 \end{pmatrix} - \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0,1 \\ -0,3 \\ 0,2 \end{pmatrix}.$$

9.28.4 Aufgabe 4: Gewichtgradient einer Schicht

Gegeben seien

$$\vec{\delta}^{(l)} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad \vec{z}^{(l-1)} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}.$$

Berechne

$$\frac{\partial E}{\partial W^{(l)}}.$$

Beispiel

Es gilt:

$$\frac{\partial E}{\partial W^{(l)}} = \vec{\delta}^{(l)} (\vec{z}^{(l-1)})^\top.$$

Also:

$$\frac{\partial E}{\partial W^{(l)}} = \begin{pmatrix} 2 \\ -1 \end{pmatrix} \begin{pmatrix} 3 & 4 \end{pmatrix} = \begin{pmatrix} 6 & 8 \\ -3 & -4 \end{pmatrix}.$$

9.28.5 Aufgabe 5: Warum Nichtlinearität?

Warum ist ein Netz aus mehreren linearen Schichten ohne Aktivierungsfunktionen nicht mächtiger als eine einzelne lineare Schicht?

Beispiel

Zwei lineare Schichten ergeben:

$$\vec{h} = W^{(1)} \vec{x},$$

$$\vec{y} = W^{(2)} \vec{h}.$$

Einsetzen liefert:

$$\vec{y} = W^{(2)} W^{(1)} \vec{x}.$$

Setze

$$W = W^{(2)}W^{(1)}.$$

Dann:

$$\vec{y} = W\vec{x}.$$

Das ist wieder eine einzige lineare Schicht.

9.29 Zusammenfassung

- Ein mehrschichtiges Perzeptron (multilayer perceptron) besteht aus mehreren Schichten künstlicher Neuronen.

- Eine Schicht berechnet:

$$\vec{a}^{(l)} = W^{(l)}\vec{z}^{(l-1)} + \vec{b}^{(l)}.$$

- Danach folgt eine Aktivierung:

$$\vec{z}^{(l)} = g^{(l)}(\vec{a}^{(l)}).$$

- Nichtlineare Aktivierungsfunktionen sind notwendig, weil mehrere lineare Schichten sonst wieder nur eine lineare Abbildung ergeben.
- Der Forward Pass berechnet die Ausgabe des Netzes aus der Eingabe.
- Backpropagation berechnet Gradienten effizient mit der Kettenregel.
- Der Fehlerterm einer Schicht ist:

$$\vec{\delta}^{(l)} = \frac{\partial E}{\partial \vec{a}^{(l)}}.$$

- Für versteckte Schichten gilt:

$$\vec{\delta}^{(l)} = \left((W^{(l+1)})^\top \vec{\delta}^{(l+1)} \right) \odot (g^{(l)})'(\vec{a}^{(l)}).$$

- Die Gradienten sind:

$$\frac{\partial E}{\partial W^{(l)}} = \vec{\delta}^{(l)} (\vec{z}^{(l-1)})^\top,$$

$$\frac{\partial E}{\partial \vec{b}^{(l)}} = \vec{\delta}^{(l)}.$$

- Bei Softmax mit Kreuzentropie gilt:

$$\vec{\delta}^{(L)} = \vec{y} - \vec{t}.$$

- Bei Sigmoid mit binärer Kreuzentropie gilt:

$$\delta^{(L)} = y - t.$$

- MLPs werden meist mit Mini-Batch-Gradientenabstieg trainiert.
- Initialisierung, Aktivierungsfunktionen und Regularisierung sind entscheidend für stabiles Training.

9.30 Typische Prüfungsfragen

Prüfungshinweis

Mögliche Prüfungsfragen zu diesem Kapitel:

1. Was ist ein mehrschichtiges Perzeptron?
2. Was ist der Unterschied zwischen Eingabeschicht, versteckter Schicht und Ausgangsschicht?
3. Warum braucht man nichtlineare Aktivierungsfunktionen?
4. Zeige, dass mehrere lineare Schichten ohne Aktivierung äquivalent zu einer linearen Schicht sind.
5. Was ist ein Forward Pass?
6. Was ist Backpropagation?
7. Welche Rolle spielt die Kettenregel?
8. Definiere den Fehlerterm $\delta^{(l)}$.
9. Leite die Backpropagation-Rekursion für versteckte Schichten her.
10. Wie berechnet man $\frac{\partial E}{\partial W^{(l)}}$?
11. Wie berechnet man $\frac{\partial E}{\partial b^{(l)}}$?
12. Warum vereinfacht sich der Ausgabefehler bei Softmax mit Kreuzentropie zu $\vec{y} - \vec{t}$?
13. Was ist der Unterschied zwischen Sigmoid, Tanh und ReLU?
14. Was sind verschwindende Gradienten?
15. Was sind explodierende Gradienten?
16. Warum sollte man Gewichte nicht alle mit 0 initialisieren?
17. Was ist Mini-Batch-Training?
18. Was ist Early Stopping?
19. Was ist Dropout?
20. Was sagt der Universal-Approximation-Satz aus und was sagt er nicht aus?

10 Generalisierung und Validierung

10.1 Ziel dieses Kapitels

In den bisherigen Kapiteln haben wir verschiedene Modelle und Trainingsmethoden betrachtet: lineare Regression (linear regression), logistische Regression (logistic regression), Perzeptrons (perceptrons) und mehrschichtige Perzeptrons (multilayer perceptrons). Dabei ging es oft darum, eine Fehlerfunktion auf den Trainingsdaten zu minimieren.

Das eigentliche Ziel im maschinellen Lernen ist aber nicht, auf den Trainingsdaten möglichst gut zu sein. Das eigentliche Ziel ist Generalisierung (generalization).

Idee

Generalisierung (generalization) bedeutet, dass ein Modell auf neuen, bisher nicht gesehenen Daten gut funktioniert.

Ein Modell soll also nicht nur die Trainingsdaten erklären, sondern die zugrunde liegende Struktur lernen.

Nach diesem Kapitel solltest du folgende Fragen beantworten können:

- Was ist Generalisierung (generalization)?
- Was ist der Unterschied zwischen Trainingsfehler (training error), Validierungsfehler (validation error) und Testfehler (test error)?
- Was ist Überanpassung (overfitting)?
- Was ist Unteranpassung (underfitting)?
- Was ist Modellkomplexität (model complexity)?
- Was ist der Bias-Varianz-Trade-off (bias-variance trade-off)?
- Warum braucht man Validierungsdaten (validation data)?
- Was ist Kreuzvalidierung (cross-validation)?
- Was sind Hyperparameter (hyperparameters)?
- Wie funktioniert Modellselektion (model selection)?
- Was ist Regularisierung (regularization)?
- Was ist Early Stopping?
- Warum darf man Testdaten nicht zum Modellauswählen verwenden?

10.2 Trainingsfehler und Generalisierung

Beim Training minimieren wir meistens eine Fehlerfunktion auf den Trainingsdaten:

$$E_{\text{train}}(\theta) = \frac{1}{N_{\text{train}}} \sum_{n=1}^{N_{\text{train}}} E_n(\theta).$$

Dabei bezeichnet θ allgemein alle Parameter des Modells. Bei linearer Regression kann $\theta = \vec{w}$ sein. Bei neuronalen Netzen enthält θ alle Gewichte und Biases.

Definition

Der Trainingsfehler (training error) ist der Fehler eines Modells auf den Daten, die zum Lernen der Parameter verwendet wurden.

Ein kleiner Trainingsfehler ist notwendig, aber nicht ausreichend. Ein Modell kann auf den Trainingsdaten sehr gut sein und trotzdem auf neuen Daten schlecht funktionieren.

Idee

Das Ziel ist nicht:

minimiere nur den Trainingsfehler.

Das Ziel ist:

minimiere den erwarteten Fehler auf neuen Daten.

10.3 Daten erzeugende Verteilung

In vielen theoretischen Betrachtungen nehmen wir an, dass Daten aus einer unbekannten Verteilung stammen. Ein Datenpunkt besteht aus Eingabe und Zielwert:

$$(\vec{x}, t).$$

Diese Paare werden aus einer gemeinsamen Verteilung gezogen:

$$(\vec{x}, t) \sim p(\vec{x}, t).$$

Die Trainingsdaten sind dann Stichproben:

$$\mathcal{D}_{\text{train}} = \{(\vec{x}_1, t_1), \dots, (\vec{x}_N, t_N)\}.$$

Meistens nimmt man an:

$$(\vec{x}_1, t_1), \dots, (\vec{x}_N, t_N) \stackrel{\text{i.i.d.}}{\sim} p(\vec{x}, t).$$

Hinweis

Die wahre Verteilung $p(\vec{x}, t)$ kennen wir normalerweise nicht. Wir sehen nur endlich viele Beispiele aus dieser Verteilung. Genau deshalb ist Generalisierung schwierig.

10.4 Erwarteter Fehler

Der ideale Fehler eines Modells wäre der erwartete Fehler über die wahre Datenverteilung. Für eine Verlustfunktion $L(y(\vec{x}), t)$ ist der erwartete Fehler:

$$E_{\text{true}}(\theta) = \mathbb{E}_{(\vec{x}, t) \sim p(\vec{x}, t)} [L(y(\vec{x}, \theta), t)].$$

Als Integral:

$$E_{\text{true}}(\theta) = \int L(y(\vec{x}, \theta), t) p(\vec{x}, t) d\vec{x} dt.$$

Definition

Der wahre Generalisierungsfehler (true generalization error) ist der erwartete Fehler eines Modells auf neuen Daten aus der gleichen Datenverteilung:

$$E_{\text{true}}(\theta) = \mathbb{E} [L(y(\vec{x}, \theta), t)].$$

Da $p(\vec{x}, t)$ unbekannt ist, können wir $E_{\text{true}}(\theta)$ nicht exakt berechnen. Wir müssen ihn mit Daten schätzen.

10.5 Trainingsfehler als Schätzer

Der Trainingsfehler ist ein empirischer Durchschnitt:

$$E_{\text{train}}(\theta) = \frac{1}{N} \sum_{n=1}^N L(y(\vec{x}_n, \theta), t_n).$$

Wenn θ fest wäre und nicht aus denselben Daten gelernt würde, wäre dieser Durchschnitt ein natürlicher Schätzer des erwarteten Fehlers.

Das Problem ist aber: θ wird genau so gewählt, dass $E_{\text{train}}(\theta)$ klein wird.

Idee

Der Trainingsfehler ist normalerweise optimistisch verzerrt.

Er unterschätzt häufig den Fehler auf neuen Daten, weil das Modell direkt auf diese Trainingsdaten angepasst wurde.

Deshalb brauchen wir getrennte Daten zur Bewertung.

10.6 Trainings-, Validierungs- und Testdaten

Typischerweise teilt man die verfügbaren Daten in drei Mengen:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}.$$

Definition

Trainingsdaten (training data) werden verwendet, um die Modellparameter zu lernen.

Validierungsdaten (validation data) werden verwendet, um Hyperparameter zu wählen und Modelle zu vergleichen.

Testdaten (test data) werden erst ganz am Ende verwendet, um die endgültige Modellleistung zu schätzen.

10.6.1 Trainingsfehler

$$E_{\text{train}} = \frac{1}{N_{\text{train}}} \sum_{n \in \mathcal{D}_{\text{train}}} L(y(\vec{x}_n, \theta), t_n).$$

10.6.2 Validierungsfehler

$$E_{\text{val}} = \frac{1}{N_{\text{val}}} \sum_{n \in \mathcal{D}_{\text{val}}} L(y(\vec{x}_n, \theta), t_n).$$

10.6.3 Testfehler

$$E_{\text{test}} = \frac{1}{N_{\text{test}}} \sum_{n \in \mathcal{D}_{\text{test}}} L(y(\vec{x}_n, \theta), t_n).$$

Prüfungshinweis

Die Testdaten dürfen nicht zur Modellselektion verwendet werden.

Wenn man wiederholt Modelle auf dem Testdatensatz bewertet und danach Entscheidungen trifft, wird der Testdatensatz indirekt zum Validierungsdatensatz. Dann ist der Testfehler keine ehrliche Schätzung der Generalisierung mehr.

10.7 Überanpassung

Überanpassung (overfitting) tritt auf, wenn ein Modell die Trainingsdaten sehr gut beschreibt, aber auf neuen Daten schlecht ist.

Typisches Verhalten:

E_{train} ist klein,

aber

E_{test} ist groß.

Definition

Überanpassung (overfitting) bedeutet, dass ein Modell zu stark an die Trainingsdaten angepasst ist und dabei auch Rauschen oder zufällige Besonderheiten der Trainingsdaten lernt.

Beispiel

Ein hochgradiges Polynom kann durch viele Trainingspunkte fast exakt verlaufen. Zwischen den Trainingspunkten kann es aber stark oszillieren. Dann ist der Trainingsfehler klein, aber der Fehler auf neuen Punkten groß.

Idee

Überanpassung bedeutet:

Modell lernt nicht nur Signal, sondern auch Rauschen.

10.8 Unteranpassung

Unteranpassung (underfitting) ist das Gegenteil. Das Modell ist zu einfach, um die Struktur der Daten zu erfassen.

Typisches Verhalten:

E_{train} ist groß

und

E_{test} ist ebenfalls groß.

Definition

Unteranpassung (underfitting) bedeutet, dass ein Modell zu wenig flexibel ist, um die relevante Struktur in den Daten zu lernen.

Beispiel

Wenn die wahre Beziehung gekrümmt ist, aber wir nur eine Gerade verwenden, kann das Modell die Daten nicht gut beschreiben. Dann liegt Unteranpassung vor.

10.9 Modellkomplexität

Modellkomplexität (model complexity) beschreibt, wie flexibel ein Modell ist.

Beispiele für Komplexität:

- Grad eines Polynoms,
- Anzahl der Basisfunktionen,

- Anzahl der Parameter,
- Tiefe eines neuronalen Netzes,
- Anzahl der Neuronen,
- Stärke der Regularisierung.

Idee

Mit steigender Modellkomplexität sinkt typischerweise der Trainingsfehler.
Der Testfehler sinkt aber nur bis zu einem gewissen Punkt. Danach kann er wieder steigen, weil das Modell überanpasst.

Typisches Verhalten:

zu einfach $\Rightarrow$ Unteranpassung.

passend komplex $\Rightarrow$ gute Generalisierung.

zu komplex $\Rightarrow$ Überanpassung.

10.10 Beispiel: Polynomiale Regression

Wir betrachten polynomiale Regression mit Basisfunktionen

$$\phi_j(x) = x^j.$$

Das Modell ist:

$$y(x, \vec{w}) = \sum_{j=0}^M w_j x^j.$$

Der Polynomgrad M steuert die Modellkomplexität.

10.10.1 Kleines M

Wenn $M = 1$, ist das Modell eine Gerade:

$$y(x, \vec{w}) = w_0 + w_1 x.$$

Das kann für gekrümmte Daten zu einfach sein.

10.10.2 Mittleres M

Ein mittlerer Polynomgrad kann die Struktur gut erfassen, ohne zu stark zu oszillieren.

10.10.3 Großes M

Ein sehr großer Polynomgrad kann viele Trainingspunkte exakt treffen, aber zwischen den Punkten stark schwanken.

Idee

Der Polynomgrad M ist ein Hyperparameter.
Er wird nicht direkt durch Gradientenabstieg gelernt, sondern muss durch Modellselektion gewählt werden.

10.11 Hyperparameter

Parameter und Hyperparameter sind unterschiedliche Dinge.

Definition

Parameter (parameters) werden während des Trainings aus den Trainingsdaten gelernt.
Hyperparameter (hyperparameters) werden vor oder außerhalb des eigentlichen Trainings festgelegt.

Beispiele für Parameter:

$$\vec{w}$$

bei linearer Regression,

$$W^{(l)}, \vec{b}^{(l)}$$

bei neuronalen Netzen.

Beispiele für Hyperparameter:

- Polynomgrad M ,
- Regularisierungsstärke λ ,
- Lernrate η ,
- Anzahl der versteckten Schichten,
- Anzahl der Neuronen pro Schicht,
- Batch-Größe,
- Anzahl der Trainings-Epochen,
- Wahl der Aktivierungsfunktion.

Hinweis

Hyperparameter werden typischerweise mit Validierungsdaten gewählt. Wenn man sie mit Testdaten wählt, ist der Testfehler nicht mehr unabhängig.

10.12 Validierung

Validierung (validation) bedeutet, Modelle oder Hyperparameter auf Daten zu bewerten, die nicht für das Fitten der Parameter verwendet wurden.

Vorgehen:

1. Teile die Daten in Trainings-, Validierungs- und Testdaten.
2. Trainiere verschiedene Modelle auf den Trainingsdaten.
3. Berechne den Validierungsfehler.
4. Wähle das Modell mit dem besten Validierungsfehler.
5. Bewerte das gewählte Modell einmal auf den Testdaten.

Definition

Modellselektion (model selection) ist die Auswahl eines Modells oder von Hyperparametern anhand von Validierungsleistung.

Idee

Trainingsdaten beantworten:

Welche Parameter soll das Modell haben?

Validierungsdaten beantworten:

Welches Modell oder welche Hyperparameter sollen gewählt werden?

Testdaten beantworten:

Wie gut ist das endgültige Modell auf neuen Daten?

10.13 Typische Datenaufteilung

Eine einfache Aufteilung ist zum Beispiel:

60% Training, 20% Validierung, 20% Test.

Oder:

80% Training, 10% Validierung, 10% Test.

Die genaue Aufteilung hängt von der Datenmenge ab.

Hinweis

Bei sehr großen Datensätzen reicht oft ein kleiner Testanteil. Bei kleinen Datensätzen möchte man möglichst viele Daten fürs Training nutzen. Dann kann Kreuzvalidierung hilfreich sein.

10.14 Kreuzvalidierung

Bei kleinen Datensätzen ist eine einzelne Validierungsaufteilung oft instabil. Das Ergebnis kann stark davon abhängen, welche Datenpunkte zufällig in Training oder Validierung landen. Eine Lösung ist Kreuzvalidierung (cross-validation).

Definition

Bei K -facher Kreuzvalidierung (K -fold cross-validation) wird der Datensatz in K Teile, sogenannte Folds, aufgeteilt.

Das Modell wird K -mal trainiert. Jedes Mal wird ein anderer Fold zur Validierung verwendet und die übrigen $K - 1$ Folds zum Training.

Der Kreuzvalidierungsfehler ist der Durchschnitt der Validierungsfehler:

$$E_{CV} = \frac{1}{K} \sum_{k=1}^K E_{\text{val}}^{(k)}.$$

Dabei ist $E_{\text{val}}^{(k)}$ der Validierungsfehler im k -ten Durchlauf.

Idee

Kreuzvalidierung nutzt die Daten effizienter als eine einzelne Validierungsaufteilung. Jeder Datenpunkt wird einmal zur Validierung und $K - 1$ -mal zum Training verwendet.

10.15 Leave-One-Out-Kreuzvalidierung

Ein Spezialfall ist Leave-One-Out-Kreuzvalidierung (leave-one-out cross-validation).

Dabei ist

$$K = N.$$

Das bedeutet: Jeder Fold enthält genau einen Datenpunkt.

Für jeden Durchlauf wird auf $N - 1$ Datenpunkten trainiert und auf dem übrig gebliebenen Datenpunkt validiert.

Definition

Leave-One-Out-Kreuzvalidierung ist K -fold cross-validation mit

$$K = N.$$

Hinweis

Leave-One-Out nutzt die Daten sehr effizient, kann aber rechnerisch teuer sein, weil das Modell N -mal trainiert werden muss.

10.16 Stratifizierte Aufteilung

Bei Klassifikationsproblemen mit unausgebalancierten Klassen ist es wichtig, dass Training, Validierung und Test ähnliche Klassenanteile haben.

Definition

Stratifizierte Aufteilung (stratified split) bedeutet, dass die Klassenverteilung in Training, Validierung und Test ungefähr gleich bleibt.

Beispiel: Wenn im gesamten Datensatz

90%

der Datenpunkte Klasse 0 und

10%

Klasse 1 sind, sollten Training, Validierung und Test ungefähr dieselben Anteile haben.

Prüfungshinweis

Bei unausgebalancierten Klassen sollte man nicht rein zufällig ohne Kontrolle splitten. Sonst kann es passieren, dass eine seltene Klasse im Validierungs- oder Testdatensatz kaum vertreten ist.

10.17 Data Leakage

Data Leakage ist ein häufiger und gefährlicher Fehler. Es bedeutet, dass Information aus Validierungs- oder Testdaten indirekt ins Training gelangt.

Definition

Data Leakage bedeutet, dass ein Modell während des Trainings oder der Modellselektion Information verwendet, die eigentlich erst in Validierungs- oder Testdaten verfügbar sein sollte.

Beispiele:

- Normalisierung mit Mittelwert und Standardabweichung des gesamten Datensatzes statt nur der Trainingsdaten.
- Feature-Auswahl auf allen Daten vor dem Split.
- Hyperparameterwahl anhand des Testfehlers.

- Duplikate desselben Falls in Training und Test.
- Zeitreihendaten zufällig mischen, obwohl zukünftige Daten nicht zur Vergangenheit verfügbar sein sollten.

Idee

Data Leakage führt zu zu optimistischen Ergebnissen.
Das Modell wirkt besser, als es auf wirklich neuen Daten wäre.

10.18 Vorverarbeitung und Splits

Viele Vorverarbeitungsschritte müssen nur auf den Trainingsdaten gefittet werden.

Beispiel Standardisierung:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}.$$

Dabei müssen μ_j und σ_j nur aus den Trainingsdaten berechnet werden:

$$\mu_j = \frac{1}{N_{\text{train}}} \sum_{n \in \mathcal{D}_{\text{train}}} x_{nj},$$

$$\sigma_j^2 = \frac{1}{N_{\text{train}}} \sum_{n \in \mathcal{D}_{\text{train}}} (x_{nj} - \mu_j)^2.$$

Dann werden dieselben Werte auf Validierungs- und Testdaten angewendet.

Prüfungshinweis

Regel:

Fit preprocessing on training data only.

Dann:

Apply same transformation to validation and test data.

10.19 Bias-Varianz-Trade-off

Der Bias-Varianz-Trade-off wurde bereits bei Schätzern eingeführt. Bei Generalisierung ist er besonders wichtig.

Informell:

Bias = Fehler durch zu einfache Modellannahmen.

Varianz = Empfindlichkeit gegenüber Trainingsdaten.

Idee

Hoher Bias führt häufig zu Unteranpassung.
Hohe Varianz führt häufig zu Überanpassung.

10.19.1 Hoher Bias

Ein Modell mit hohem Bias ist zu starr. Es kann die wahre Struktur nicht gut approximieren.
Typisch:

$$E_{\text{train}} \text{ hoch,} \quad E_{\text{test}} \text{ hoch.}$$

10.19.2 Hohe Varianz

Ein Modell mit hoher Varianz reagiert stark auf die konkrete Trainingsmenge. Es passt sich zufälligen Details an.
Typisch:

$$E_{\text{train}} \text{ niedrig,} \quad E_{\text{test}} \text{ hoch.}$$

10.20 Bias-Varianz-Zerlegung für Regression

Für quadratischen Fehler kann man den erwarteten Vorhersagefehler in Bias, Varianz und Rauschen zerlegen.

Sei die wahre Beziehung:

$$t = f(\vec{x}) + \epsilon,$$

mit

$$\mathbb{E}[\epsilon] = 0$$

und

$$\text{Var}(\epsilon) = \sigma^2.$$

Ein Lernalgorithmus trainiert auf einem zufälligen Datensatz $\mathcal{D}$ ein Modell

$$y(\vec{x}; \mathcal{D}).$$

Der erwartete quadratische Fehler an einem festen Punkt $\vec{x}$ kann geschrieben werden als:

$$\mathbb{E}_{\mathcal{D}, t} [(y(\vec{x}; \mathcal{D}) - t)^2] = \underbrace{(\mathbb{E}_{\mathcal{D}}[y(\vec{x}; \mathcal{D})] - f(\vec{x}))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}} [(y(\vec{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[y(\vec{x}; \mathcal{D})])^2]}_{\text{Varianz}} + \underbrace{\sigma^2}_{\text{Rauschen}}.$$

Definition

Für quadratischen Fehler zerfällt der erwartete Fehler in:

$$\text{erwarteter Fehler} = \text{Bias}^2 + \text{Varianz} + \text{Rauschen}.$$

Hinweis

Das Rauschen σ^2 ist irreduzibel. Auch das beste Modell kann zufälliges Messrauschen nicht vorhersagen.

10.21 Regularisierung

Regularisierung (regularization) ist eine zentrale Methode gegen Overfitting. Man bestraft zu komplexe Modelle oder große Parameterwerte.

Ein typischer regularisierter Verlust ist:

$$E_{\text{reg}}(\theta) = E_{\text{data}}(\theta) + \lambda E_{\text{regterm}}(\theta).$$

Dabei steuert λ die Stärke der Regularisierung.

Definition

Regularisierung (regularization) fügt dem Trainingsziel einen Zusatzterm hinzu, der unerwünschte Modellkomplexität bestraft.

10.21.1 L_2 -Regularisierung

Bei L_2 -Regularisierung verwendet man:

$$E_{\text{regterm}}(\vec{w}) = \frac{1}{2} \vec{w}^\top \vec{w}.$$

Der Verlust ist:

$$E_{\text{reg}}(\vec{w}) = E_{\text{data}}(\vec{w}) + \frac{\lambda}{2} \vec{w}^\top \vec{w}.$$

Idee

L_2 -Regularisierung bestraft große Gewichte.
Sie führt oft zu glatteren, stabileren Modellen.

10.21.2 L_1 -Regularisierung

Bei L_1 -Regularisierung verwendet man:

$$E_{\text{regterm}}(\vec{w}) = \sum_j |w_j|.$$

Der Verlust ist:

$$E_{\text{reg}}(\vec{w}) = E_{\text{data}}(\vec{w}) + \lambda \sum_j |w_j|.$$

Idee

L_1 -Regularisierung fördert sparse Lösungen.
Das bedeutet: Viele Gewichte werden exakt oder ungefähr 0.

10.22 Regularisierung als MAP

Regularisierung kann probabilistisch als Maximum-a-posteriori-Schätzung (maximum a-posteriori estimation) interpretiert werden.

Bei MAP maximieren wir:

$$p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta)p(\theta).$$

Äquivalent minimieren wir den negativen Log-Posterior:

$$-\log p(\theta \mid \mathcal{D}) = -\log p(\mathcal{D} \mid \theta) - \log p(\theta) + \text{const.}$$

Der Datenfehler ist:

$$E_{\text{data}}(\theta) = -\log p(\mathcal{D} \mid \theta).$$

Der Regularisierungsterm kommt aus dem Prior:

$$E_{\text{regterm}}(\theta) = -\log p(\theta).$$

Idee

Regularisierung entspricht einem Prior auf Parameter.

$$\text{Regularisierung} \iff \text{Prior.}$$

Ein Gauß-Prior führt zu L_2 -Regularisierung. Ein Laplace-Prior führt zu L_1 -Regularisierung.

10.23 Early Stopping

Early Stopping ist eine Regularisierungsmethode für iterative Trainingsverfahren.

Beim Training sinkt der Trainingsfehler meistens weiter. Der Validierungsfehler sinkt zunächst ebenfalls, kann aber später wieder steigen.

$$E_{\text{train}} \downarrow$$

während

$$E_{\text{val}}$$

nach einer gewissen Zeit wieder steigen kann.

Definition

Early Stopping bedeutet, das Training zu stoppen, wenn der Validierungsfehler nicht mehr besser wird.

Idee

Early Stopping verhindert, dass das Modell zu lange an die Trainingsdaten angepasst wird. Es wirkt daher als Regularisierung.

Typisches Vorgehen:

1. Trainiere iterativ.
2. Berechne nach jeder Epoche den Validierungsfehler.
3. Speichere das Modell mit dem besten Validierungsfehler.
4. Stoppe, wenn sich der Validierungsfehler über mehrere Epochen nicht verbessert.

10.24 Lernkurven

Lernkurven (learning curves) zeigen Trainings- und Validierungsfehler als Funktion der Trainingsmenge oder der Trainingszeit.

10.24.1 Fehler über Trainingszeit

Man zeichnet:

$$E_{\text{train}}(\tau)$$

und

$$E_{\text{val}}(\tau)$$

gegen die Epoche τ .

Typisches Overfitting-Muster:

$$E_{\text{train}}(\tau) \text{ sinkt weiter,}$$

aber

$$E_{\text{val}}(\tau) \text{ steigt wieder.}$$

10.24.2 Fehler über Datenmenge

Man trainiert mit verschiedenen Trainingsgrößen N und zeichnet:

$$E_{\text{train}}(N)$$

und

$$E_{\text{val}}(N).$$

Idee

Lernkurven helfen zu diagnostizieren, ob ein Modell eher unteranpasst oder überanpasst.

10.25 Diagnose: Underfitting oder Overfitting

10.25.1 Fall 1: Beide Fehler hoch

Wenn

$$E_{\text{train}} \text{ hoch}$$

und

$$E_{\text{val}} \text{ hoch}$$

sind, liegt wahrscheinlich Unteranpassung vor.

Mögliche Maßnahmen:

- komplexeres Modell wählen,
- mehr Features verwenden,
- schwächere Regularisierung verwenden,
- länger trainieren,
- bessere Optimierung verwenden.

10.25.2 Fall 2: Trainingsfehler niedrig, Validierungsfehler hoch

Wenn

$$E_{\text{train}} \text{ niedrig}$$

aber

$$E_{\text{val}} \text{ hoch}$$

ist, liegt wahrscheinlich Überanpassung vor.

Mögliche Maßnahmen:

- einfacheres Modell wählen,

- stärkere Regularisierung verwenden,
- mehr Trainingsdaten sammeln,
- Datenaugmentation verwenden,
- Early Stopping verwenden.

10.25.3 Fall 3: Beide Fehler niedrig

Wenn beide Fehler niedrig sind, generalisiert das Modell wahrscheinlich gut. Trotzdem sollte am Ende ein unabhängiger Testdatensatz verwendet werden.

10.26 Datenaugmentation

Datenaugmentation (data augmentation) erzeugt zusätzliche Trainingsbeispiele durch transformationsbasierte Varianten vorhandener Daten.

Bei Bildern zum Beispiel:

- kleine Rotationen,
- Spiegelungen,
- leichte Verschiebungen,
- zufällige Ausschnitte,
- Farbänderungen.

Definition

Datenaugmentation (data augmentation) erweitert die Trainingsdaten durch sinnvolle Transformationen, die das Label erhalten.

Idee

Datenaugmentation kann Overfitting reduzieren, weil das Modell mehr Variation sieht.

Wichtig: Die Transformationen müssen label-erhaltend sein. Wenn eine Transformation das Label verändert, erzeugt man falsche Trainingsdaten.

10.27 Ensembles

Ein Ensemble kombiniert mehrere Modelle. Die Idee ist, dass verschiedene Modelle unterschiedliche Fehler machen. Durch Mittelung oder Mehrheitsentscheidung kann die Gesamtleistung besser werden.

Für Regression kann man Vorhersagen mitteln:

$$y_{\text{ens}}(\vec{x}) = \frac{1}{M} \sum_{m=1}^M y_m(\vec{x}).$$

Für Klassifikation kann man Wahrscheinlichkeiten mitteln:

$$p_{\text{ens}}(c \mid \vec{x}) = \frac{1}{M} \sum_{m=1}^M p_m(c \mid \vec{x}).$$

Definition

Ein Ensemble ist eine Kombination mehrerer Modelle zur Verbesserung von Stabilität und Generalisierung.

Hinweis

Ensembles reduzieren oft Varianz, sind aber rechnerisch teurer und schwerer zu interpretieren.

10.28 Modellselektion

Modellselektion bedeutet, aus mehreren Kandidaten ein Modell auszuwählen.

Beispiele:

- Polynomgrad M wählen,
- Regularisierung λ wählen,
- Lernrate η wählen,
- Netzwerkarchitektur wählen,
- Aktivierungsfunktion wählen.

Für jeden Kandidaten trainiert man auf den Trainingsdaten und bewertet auf den Validierungsdaten.

$$\lambda^* = \arg \min_{\lambda} E_{\text{val}}(\lambda).$$

Definition

Hyperparameteroptimierung (hyperparameter optimization) ist die Suche nach Hyperparametern, die den Validierungsfehler minimieren.

10.29 Grid Search und Random Search

10.29.1 Grid Search

Bei Grid Search definiert man eine Menge möglicher Werte. Zum Beispiel:

$$\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}.$$

Dann trainiert man für jeden Wert ein Modell und wählt den besten Validierungsfehler.

10.29.2 Random Search

Bei Random Search werden Hyperparameter zufällig aus vorgegebenen Bereichen gezogen. Das kann effizienter sein, wenn nur wenige Hyperparameter wirklich wichtig sind.

Hinweis

Für Hyperparameter wie λ sucht man oft auf logarithmischer Skala, weil Unterschiede über Größenordnungen wichtig sind.

10.30 Nested Cross-Validation

Wenn sowohl Modellselektion als auch Leistungsbewertung mit Kreuzvalidierung gemacht werden sollen, verwendet man verschachtelte Kreuzvalidierung (nested cross-validation).

Es gibt:

- eine äußere Kreuzvalidierung zur Bewertung,
- eine innere Kreuzvalidierung zur Hyperparameterwahl.

Definition

Nested Cross-Validation trennt Modellselektion und Leistungsbewertung sauber durch zwei verschachtelte Validierungsschleifen.

Idee

Nested Cross-Validation verhindert, dass die Leistungsbewertung durch Hyperparameterwahl zu optimistisch wird.

10.31 Verteilungsshift

Alle bisherigen Aussagen setzen voraus, dass Trainings- und Testdaten aus derselben Verteilung stammen:

$$p_{\text{train}}(\vec{x}, t) = p_{\text{test}}(\vec{x}, t).$$

In der Praxis kann das verletzt sein.

Definition

Verteilungsshift (distribution shift) bedeutet, dass sich die Datenverteilung zwischen Training und Anwendung unterscheidet.

Beispiele:

- Ein medizinisches Modell wird in einem anderen Krankenhaus verwendet.
- Ein Spamfilter wird nach mehreren Jahren verwendet, obwohl sich Spam verändert hat.
- Ein Bildmodell wird auf Bildern mit anderer Kameraqualität getestet.

- Trainingsdaten stammen aus Simulation, Testdaten aus der realen Welt.

Hinweis

Ein niedriger Testfehler ist nur aussagekräftig, wenn die Testdaten die spätere Anwendung gut repräsentieren.

10.32 Metriken für Regression

Bei Regression sind typische Metriken:

10.32.1 Mean Squared Error

$$\text{MSE} = \frac{1}{N} \sum_{n=1}^N (y_n - t_n)^2.$$

10.32.2 Root Mean Squared Error

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{n=1}^N (y_n - t_n)^2}.$$

10.32.3 Mean Absolute Error

$$\text{MAE} = \frac{1}{N} \sum_{n=1}^N |y_n - t_n|.$$

Idee

MSE bestraft große Fehler stärker als MAE, weil Fehler quadriert werden.
MAE ist robuster gegenüber Ausreißern.

10.33 Metriken für Klassifikation

Bei Klassifikation sind typische Metriken:

10.33.1 Accuracy

$$\text{accuracy} = \frac{1}{N} \sum_{n=1}^N \mathbb{1}(\hat{t}_n = t_n).$$

10.33.2 Precision

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}.$$

10.33.3 Recall

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

10.33.4 F1-Score

$$F_1 = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

Prüfungshinweis

Bei unausgeglichenen Klassen ist Accuracy oft nicht ausreichend. Dann sind Precision, Recall, F1-Score oder ROC-AUC oft aussagekräftiger.

10.34 Unsicherheit der Testschätzung

Der Testfehler ist selbst nur ein Schätzer des wahren Generalisierungsfehlers. Wenn der Testdatensatz klein ist, kann diese Schätzung stark schwanken.

Für Accuracy kann man grob modellieren:

$$\hat{p} = \frac{k}{N},$$

wobei k die Anzahl korrekter Vorhersagen ist. Die Standardabweichung dieses Anteils ist ungefähr:

$$\sqrt{\frac{\hat{p}(1 - \hat{p})}{N}}.$$

Idee

Ein Testfehler auf wenigen Testpunkten ist unsicher.
Mehr Testdaten liefern eine stabilere Schätzung der echten Generalisierung.

10.35 Häufige Fehler

Prüfungshinweis

Typische Fehler bei Generalisierung und Validierung:

1. Trainingsfehler mit Generalisierungsfehler verwechseln.
2. Testdaten zur Hyperparameterwahl verwenden.
3. Vorverarbeitung auf dem gesamten Datensatz fitten.
4. Overfitting nur an großem Trainingsfehler erkennen wollen.
5. Underfitting und Overfitting verwechseln.
6. Bei unausgeglichenen Klassen nur Accuracy betrachten.
7. Kreuzvalidierung falsch durchführen, sodass Datenleakage entsteht.
8. Bei Zeitreihen zufällig splitten, obwohl die zeitliche Reihenfolge wichtig ist.

9. Den Validierungsfehler mehrfach optimieren und danach so tun, als wäre er ein unabhängiger Testfehler.
10. Ein Modell auf einem Testdatensatz bewerten, der die spätere Anwendung nicht repräsentiert.

10.36 Mini-Aufgaben

10.36.1 Aufgabe 1: Trainings- und Testfehler interpretieren

Ein Modell hat:

$$E_{\text{train}} = 0,01$$

und

$$E_{\text{test}} = 0,50.$$

Was ist wahrscheinlich passiert?

Beispiel

Der Trainingsfehler ist sehr klein, aber der Testfehler ist groß. Das spricht für Überanpassung (overfitting).

Das Modell hat die Trainingsdaten sehr gut gelernt, generalisiert aber schlecht auf neue Daten.

10.36.2 Aufgabe 2: Underfitting erkennen

Ein Modell hat:

$$E_{\text{train}} = 0,45$$

und

$$E_{\text{val}} = 0,48.$$

Was ist wahrscheinlich passiert?

Beispiel

Beide Fehler sind hoch. Das spricht für Unteranpassung (underfitting).

Das Modell ist vermutlich zu einfach oder wurde nicht ausreichend trainiert.

10.36.3 Aufgabe 3: Kreuzvalidierung

Ein 5-fold cross-validation Verfahren liefert Validierungsfehler:

0,20, 0,25, 0,22, 0,23, 0,30.

Berechne den Kreuzvalidierungsfehler.

Beispiel

Der Kreuzvalidierungsfehler ist der Durchschnitt:

$$E_{CV} = \frac{1}{5}(0,20 + 0,25 + 0,22 + 0,23 + 0,30).$$

Die Summe ist:

$$0,20 + 0,25 + 0,22 + 0,23 + 0,30 = 1,20.$$

Also:

$$E_{CV} = \frac{1,20}{5} = 0,24.$$

10.36.4 Aufgabe 4: Accuracy

Ein Klassifikator macht auf 100 Testpunkten 87 korrekte Vorhersagen. Berechne die Accuracy.

Beispiel

Die Accuracy ist:

$$\text{accuracy} = \frac{87}{100} = 0,87.$$

Also:

$$\text{accuracy} = 87\%.$$

10.36.5 Aufgabe 5: Precision und Recall

Gegeben seien:

$$TP = 30, \quad FP = 10, \quad FN = 20.$$

Berechne Precision und Recall.

Beispiel

Precision:

$$\text{precision} = \frac{TP}{TP + FP} = \frac{30}{30 + 10} = \frac{30}{40} = 0,75.$$

Recall:

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{30}{30 + 20} = \frac{30}{50} = 0,60.$$

10.37 Zusammenfassung

- Generalisierung (generalization) bedeutet gute Leistung auf neuen Daten.
- Der Trainingsfehler misst Leistung auf Trainingsdaten.
- Der Validierungsfehler dient zur Modell- und Hyperparameterwahl.
- Der Testfehler dient zur finalen Leistungsabschätzung.
- Überanpassung (overfitting) bedeutet niedriger Trainingsfehler, aber hoher Fehler auf neuen Daten.
- Unteranpassung (underfitting) bedeutet, dass das Modell zu einfach ist und selbst die Trainingsdaten schlecht beschreibt.
- Modellkomplexität beeinflusst den Bias-Varianz-Trade-off.
- Hyperparameter werden nicht direkt durch Training gelernt.
- Kreuzvalidierung nutzt Daten effizienter für Modellselektion.
- Testdaten dürfen nicht zur Hyperparameterwahl verwendet werden.
- Data Leakage führt zu zu optimistischen Ergebnissen.
- Regularisierung reduziert Overfitting durch Bestrafung zu komplexer Modelle.
- Early Stopping ist eine Regularisierungsmethode für iterative Trainingsverfahren.
- Lernkurven helfen, Underfitting und Overfitting zu diagnostizieren.
- Die passende Metrik hängt vom Problem ab.

10.38 Typische Prüfungsfragen

Prüfungshinweis

Mögliche Prüfungsfragen zu diesem Kapitel:

1. Was bedeutet Generalisierung?
2. Warum reicht ein kleiner Trainingsfehler nicht aus?
3. Was ist der Unterschied zwischen Trainings-, Validierungs- und Testdaten?
4. Warum darf man Testdaten nicht zur Hyperparameterwahl verwenden?
5. Was ist Overfitting?

6. Was ist Underfitting?
7. Wie erkennt man Overfitting an Trainings- und Validierungsfehler?
8. Was ist Modellkomplexität?
9. Was ist ein Hyperparameter?
10. Was ist Modellselektion?
11. Erkläre K -fold cross-validation.
12. Was ist Leave-One-Out-Kreuzvalidierung?
13. Was ist Data Leakage?
14. Warum muss Standardisierung nur auf Trainingsdaten gefittet werden?
15. Erkläre den Bias-Varianz-Trade-off.
16. Was ist der irreduzible Fehler?
17. Wie hilft Regularisierung gegen Overfitting?
18. Wie hängt Regularisierung mit MAP zusammen?
19. Was ist Early Stopping?
20. Was zeigen Lernkurven?
21. Wann ist Accuracy eine schlechte Metrik?
22. Was ist der Unterschied zwischen Precision und Recall?
23. Was ist Verteilungssshift?
24. Warum ist ein Testfehler auf wenigen Daten unsicher?

11 PCA und Dimensionsreduktion

11.1 Ziel dieses Kapitels

In vielen Anwendungen des maschinellen Lernens sind Daten hochdimensional. Ein Bild mit 28×28 Pixeln kann zum Beispiel als Vektor in

$$\mathbb{R}^{784}$$

dargestellt werden. Moderne Datensätze können sogar Tausende oder Millionen Merkmale enthalten.

Hohe Dimensionen sind aber nicht immer hilfreich. Oft liegen die Daten näherungsweise auf einer niedrigdimensionalen Struktur. Dimensionsreduktion (dimensionality reduction) versucht, eine niedrigdimensionale Darstellung der Daten zu finden, die möglichst viel relevante Information erhält.

Idee

Dimensionsreduktion (dimensionality reduction) bedeutet, Daten aus einem hochdimensionalen Raum in einen niedrigdimensionalen Raum abzubilden.

Dabei soll möglichst viel wichtige Struktur der Daten erhalten bleiben.

In diesem Kapitel betrachten wir vor allem Hauptkomponentenanalyse (principal component analysis), kurz PCA. PCA ist eine der wichtigsten linearen Methoden zur Dimensionsreduktion.

Nach diesem Kapitel solltest du folgende Fragen beantworten können:

- Warum braucht man Dimensionsreduktion (dimensionality reduction)?
- Was ist der Fluch der Dimensionalität (curse of dimensionality)?
- Was ist PCA (principal component analysis)?
- Was bedeutet Zentrierung der Daten?
- Was ist eine Kovarianzmatrix (covariance matrix)?
- Was sind Eigenwerte und Eigenvektoren?
- Wie findet PCA die Richtungen maximaler Varianz?
- Wie projiziert man Daten auf Hauptkomponenten?
- Wie rekonstruiert man Daten aus einer reduzierten Darstellung?
- Was ist Rekonstruktionsfehler (reconstruction error)?

- Wie wählt man die Anzahl der Hauptkomponenten?
- Was ist erklärte Varianz (explained variance)?
- Wie hängt PCA mit Singular Value Decomposition zusammen?

11.2 Motivation für Dimensionsreduktion

Ein Datenpunkt sei ein Vektor

$$\vec{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_D \end{pmatrix} \in \mathbb{R}^D.$$

Wenn D groß ist, sprechen wir von hochdimensionalen Daten.

Beispiele:

- Bilder mit vielen Pixeln,
- Genexpressionsdaten mit vielen Genen,
- medizinische Zeitreihen mit vielen Messwerten,
- Textdaten mit vielen Wortmerkmalen,
- Sensordaten mit vielen Kanälen.

Dimensionsreduktion versucht, eine Abbildung zu finden:

$$\vec{x} \in \mathbb{R}^D \longrightarrow \vec{z} \in \mathbb{R}^M,$$

wobei

$$M < D.$$

Der Vektor $\vec{z}$ ist die niedrigdimensionale Darstellung (low-dimensional representation).

Definition

Dimensionsreduktion (dimensionality reduction) sucht eine Darstellung

$$\vec{z} \in \mathbb{R}^M$$

für Datenpunkte

$$\vec{x} \in \mathbb{R}^D,$$

wobei

$$M < D.$$

11.3 Warum hohe Dimensionen problematisch sind

Hohe Dimensionen können mehrere Probleme verursachen.

11.3.1 Rechenaufwand

Viele Algorithmen werden mit steigender Dimension teurer. Matrixoperationen, Distanzberechnungen und Optimierung können deutlich langsamer werden.

11.3.2 Speicherbedarf

Ein Datensatz mit N Datenpunkten und D Merkmalen kann als Matrix geschrieben werden:

$$X \in \mathbb{R}^{N \times D}.$$

Wenn N und D groß sind, braucht diese Matrix viel Speicher.

11.3.3 Rauschen und irrelevante Merkmale

Nicht alle Merkmale sind gleich wichtig. Manche Merkmale enthalten vor allem Rauschen (noise) oder redundante Information.

11.3.4 Visualisierung

Menschen können Daten in zwei oder drei Dimensionen gut visualisieren. Daten in

$$\mathbb{R}^{100}$$

oder

$$\mathbb{R}^{1000}$$

sind dagegen schwer direkt darzustellen.

Idee

Dimensionsreduktion hilft bei:

- Kompression,
- Visualisierung,
- Rauschreduktion,
- schnellerer Berechnung,
- besserer Generalisierung,

- Erkennen wichtiger Datenrichtungen.

11.4 Der Fluch der Dimensionalität

Der Fluch der Dimensionalität (curse of dimensionality) beschreibt Probleme, die auftreten, wenn die Dimension sehr groß wird.

Eine wichtige Intuition: In hohen Dimensionen wird der Raum extrem groß. Bei gleicher Anzahl von Datenpunkten wird der Raum sehr dünn besetzt.

Beispiel

Betrachte das Intervall $[0,1]$ in einer Dimension. Wenn wir 10 Punkte gleichmäßig verteilen, ist der Abstand ungefähr klein.

In zwei Dimensionen hat das Einheitsquadrat $[0,1]^2$. Um in jeder Richtung 10 Positionen zu haben, brauchen wir

$$10^2 = 100$$

Punkte.

In D Dimensionen brauchen wir

$$10^D$$

Punkte.

Für $D = 10$ wären das bereits

$$10^{10}$$

Punkte.

Idee

Der Fluch der Dimensionalität (curse of dimensionality) bedeutet: Mit steigender Dimension wächst der Raum so schnell, dass endliche Datensätze sehr dünn werden.

Dadurch werden Distanzen, Dichteschätzungen und Nachbarschaftsbegriffe schwieriger.

11.5 Lineare Dimensionsreduktion

PCA ist eine lineare Methode. Das bedeutet: Die niedrigdimensionale Darstellung entsteht durch Projektion auf lineare Richtungen.

Wir suchen M Richtungen

$$\vec{u}_1, \dots, \vec{u}_M \in \mathbb{R}^D,$$

und projizieren einen Datenpunkt $\vec{x}$ auf diese Richtungen.

Für eine Richtung $\vec{u}_j$ ist die Projektion:

$$z_j = \vec{u}_j^\top \vec{x}.$$

Die reduzierte Darstellung ist:

$$\vec{z} = \begin{pmatrix} z_1 \\ \vdots \\ z_M \end{pmatrix}.$$

Wenn wir die Richtungen als Spalten einer Matrix sammeln,

$$U_M = \begin{pmatrix} \vec{u}_1 & \vec{u}_2 & \dots & \vec{u}_M \end{pmatrix} \in \mathbb{R}^{D \times M},$$

dann ist:

$$\vec{z} = U_M^\top \vec{x}.$$

Definition

Eine lineare Dimensionsreduktion projiziert Daten mit einer Matrix U_M auf einen niedrigdimensionalen Raum:

$$\vec{z} = U_M^\top \vec{x},$$

wobei

$$U_M \in \mathbb{R}^{D \times M}$$

und

$$M < D.$$

11.6 Zentrierung der Daten

PCA wird auf zentrierten Daten durchgeführt. Das bedeutet: Man zieht von jedem Datenpunkt den empirischen Mittelwert ab.

Gegeben seien Datenpunkte

$$\vec{x}_1, \dots, \vec{x}_N \in \mathbb{R}^D.$$

Der empirische Mittelwert ist:

$$\vec{\mu} = \frac{1}{N} \sum_{n=1}^N \vec{x}_n.$$

Die zentrierten Daten sind:

$$\tilde{\vec{x}}_n = \vec{x}_n - \vec{\mu}.$$

Definition

Zentrierung (centering) bedeutet, den empirischen Mittelwert von allen Datenpunkten abzuziehen:

$$\tilde{\vec{x}}_n = \vec{x}_n - \vec{\mu}.$$

Nach der Zentrierung gilt:

$$\frac{1}{N} \sum_{n=1}^N \tilde{\vec{x}}_n = \vec{0}.$$

Hinweis

PCA sucht Richtungen der Varianz um den Mittelwert. Deshalb müssen die Daten vorher zentriert werden.

Wenn man nicht zentriert, kann die erste Hauptkomponente stark durch die Lage des Mittelwerts beeinflusst werden.

11.7 Datenmatrix

Wir sammeln die zentrierten Daten in einer Matrix:

$$\tilde{X} = \begin{pmatrix} \tilde{\vec{x}}_1^\top \\ \tilde{\vec{x}}_2^\top \\ \vdots \\ \tilde{\vec{x}}_N^\top \end{pmatrix} \in \mathbb{R}^{N \times D}.$$

Jede Zeile ist ein Datenpunkt. Jede Spalte ist ein Merkmal.

Hinweis

Je nach Literatur liegen Datenpunkte manchmal als Spalten statt als Zeilen in der Datenmatrix. Dann ändern sich manche Matrixformeln durch Transposition.

In diesem Kapitel verwenden wir:

$$\tilde{X} \in \mathbb{R}^{N \times D}.$$

11.8 Kovarianzmatrix

Die Kovarianzmatrix beschreibt, wie Merkmale gemeinsam variieren. Für zentrierte Daten ist die empirische Kovarianzmatrix:

$$S = \frac{1}{N} \tilde{X}^\top \tilde{X}.$$

Damit gilt:

$$S \in \mathbb{R}^{D \times D}.$$

Definition

Die empirische Kovarianzmatrix (empirical covariance matrix) zentrierter Daten ist:

$$S = \frac{1}{N} \sum_{n=1}^N \tilde{x}_n \tilde{x}_n^\top.$$

In Matrixform:

$$S = \frac{1}{N} \tilde{X}^\top \tilde{X}.$$

Der Eintrag S_{ij} ist die empirische Kovarianz zwischen Merkmal i und Merkmal j .

$$S_{ij} = \frac{1}{N} \sum_{n=1}^N \tilde{x}_{ni} \tilde{x}_{nj}.$$

Idee

Die Diagonale der Kovarianzmatrix enthält Varianzen.

Die Nicht-Diagonaleinträge enthalten Kovarianzen zwischen verschiedenen Merkmalen.

11.9 Symmetrie und positive Semidefinitheit

Die Kovarianzmatrix S ist symmetrisch:

$$S^\top = S.$$

Außerdem ist sie positiv semidefinit. Das bedeutet:

$$\vec{v}^\top S \vec{v} \geq 0$$

für alle

$$\vec{v} \in \mathbb{R}^D.$$

Warum?

$$\vec{v}^\top S \vec{v} = \vec{v}^\top \left(\frac{1}{N} \tilde{X}^\top \tilde{X} \right) \vec{v}.$$

Also:

$$\vec{v}^\top S \vec{v} = \frac{1}{N} \vec{v}^\top \tilde{X}^\top \tilde{X} \vec{v}.$$

Das ist:

$$\vec{v}^\top S \vec{v} = \frac{1}{N} (\tilde{X} \vec{v})^\top (\tilde{X} \vec{v}).$$

Damit:

$$\vec{v}^\top S \vec{v} = \frac{1}{N} \|\tilde{X} \vec{v}\|^2 \geq 0.$$

Definition

Eine Matrix S heißt positiv semidefinit (positive semidefinite), wenn gilt:

$$\vec{v}^\top S \vec{v} \geq 0$$

für alle Vektoren $\vec{v}$.

11.10 Varianz in einer Richtung

PCA sucht Richtungen, in denen die Daten möglichst stark variieren. Sei $\vec{u}$ eine Richtung mit Norm 1:

$$\|\vec{u}\| = 1.$$

Die Projektion eines zentrierten Datenpunkts auf $\vec{u}$ ist:

$$z_n = \vec{u}^\top \tilde{\vec{x}}_n.$$

Die empirische Varianz der projizierten Daten ist:

$$\frac{1}{N} \sum_{n=1}^N z_n^2.$$

Einsetzen von z_n :

$$\frac{1}{N} \sum_{n=1}^N (\vec{u}^\top \tilde{\vec{x}}_n)^2.$$

Da

$$(\vec{u}^\top \tilde{\vec{x}}_n)^2 = \vec{u}^\top \tilde{\vec{x}}_n \tilde{\vec{x}}_n^\top \vec{u},$$

gilt:

$$\frac{1}{N} \sum_{n=1}^N (\vec{u}^\top \tilde{\vec{x}}_n)^2 = \vec{u}^\top \left(\frac{1}{N} \sum_{n=1}^N \tilde{\vec{x}}_n \tilde{\vec{x}}_n^\top \right) \vec{u}.$$

Der Ausdruck in der Klammer ist die Kovarianzmatrix S . Also:

$$\frac{1}{N} \sum_{n=1}^N z_n^2 = \vec{u}^\top S \vec{u}.$$

Definition

Die Varianz der Daten in Richtung $\vec{u}$ ist:

$$\text{Var}_{\vec{u}} = \vec{u}^\top S \vec{u},$$

wobei

$$\|\vec{u}\| = 1.$$

11.11 Erste Hauptkomponente

Die erste Hauptkomponente (first principal component) ist die Richtung mit maximaler Varianz.

Wir suchen:

$$\vec{u}_1 = \arg \max_{\vec{u}} \vec{u}^\top S \vec{u}$$

unter der Nebenbedingung

$$\vec{u}^\top \vec{u} = 1.$$

Definition

Die erste Hauptkomponente ist die Richtung

$$\vec{u}_1$$

mit maximaler projizierter Varianz:

$$\vec{u}_1 = \arg \max_{\|\vec{u}\|=1} \vec{u}^\top S \vec{u}.$$

11.12 Herleitung mit Lagrange-Multiplikatoren

Wir maximieren:

$$\vec{u}^\top S \vec{u}$$

unter der Nebenbedingung:

$$\vec{u}^\top \vec{u} = 1.$$

Dafür verwenden wir einen Lagrange-Multiplikator λ . Die Lagrange-Funktion ist:

$$\mathcal{L}(\vec{u}, \lambda) = \vec{u}^\top S \vec{u} - \lambda(\vec{u}^\top \vec{u} - 1).$$

Ableiten nach $\vec{u}$:

$$\nabla_{\vec{u}} \mathcal{L} = 2S\vec{u} - 2\lambda\vec{u}.$$

Setze gleich $\vec{0}$:

$$2S\vec{u} - 2\lambda\vec{u} = \vec{0}.$$

Teilen durch 2:

$$S\vec{u} = \lambda\vec{u}.$$

Das ist ein Eigenwertproblem.

Definition

Die Hauptkomponenten von PCA sind Eigenvektoren der Kovarianzmatrix:

$$S\vec{u} = \lambda\vec{u}.$$

Der Wert der Varianz in Richtung $\vec{u}$ ist:

$$\vec{u}^\top S\vec{u}.$$

Wenn $\vec{u}$ ein Eigenvektor mit Eigenwert λ ist, gilt:

$$S\vec{u} = \lambda\vec{u}.$$

Dann:

$$\vec{u}^\top S\vec{u} = \vec{u}^\top \lambda\vec{u} = \lambda\vec{u}^\top \vec{u}.$$

Da

$$\vec{u}^\top \vec{u} = 1,$$

ist:

$$\vec{u}^\top S\vec{u} = \lambda.$$

Idee

Der Eigenwert λ einer Hauptkomponente ist die Varianz der Daten in dieser Richtung.

Die erste Hauptkomponente ist daher der Eigenvektor zum größten Eigenwert.

11.13 Weitere Hauptkomponenten

Die zweite Hauptkomponente soll ebenfalls möglichst viel Varianz erklären, aber orthogonal zur ersten Hauptkomponente sein.

Allgemein: Die m -te Hauptkomponente ist die Richtung maximaler Varianz unter der Nebenbedingung, dass sie orthogonal zu den vorherigen Hauptkomponenten ist.

$$\vec{u}_m^\top \vec{u}_j = 0 \quad \text{für } j < m.$$

Definition

Die Hauptkomponenten sind orthonormale Eigenvektoren der Kovarianzmatrix. Sie erfüllen:

$$\vec{u}_i^\top \vec{u}_j = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

Da S symmetrisch ist, können die Eigenvektoren orthonormal gewählt werden. Wir ordnen die Eigenwerte absteigend:

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_D \geq 0.$$

Die zugehörigen Eigenvektoren sind:

$$\vec{u}_1, \vec{u}_2, \dots, \vec{u}_D.$$

Idee

PCA nimmt die Eigenvektoren mit den größten Eigenwerten. Diese Richtungen erklären die größte Varianz der Daten.

11.14 Projektion auf Hauptkomponenten

Wenn wir nur die ersten M Hauptkomponenten behalten, definieren wir:

$$U_M = \begin{pmatrix} \vec{u}_1 & \vec{u}_2 & \dots & \vec{u}_M \end{pmatrix} \in \mathbb{R}^{D \times M}.$$

Ein zentrierter Datenpunkt $\tilde{\vec{x}}$ wird projiziert durch:

$$\vec{z} = U_M^\top \tilde{\vec{x}}.$$

Dabei ist:

$$\vec{z} \in \mathbb{R}^M.$$

Definition

Die PCA-Projektion auf M Hauptkomponenten ist:

$$\vec{z} = U_M^\top (\vec{x} - \vec{\mu}).$$

Für alle Datenpunkte in Matrixform:

$$Z = \tilde{X}U_M.$$

Hier gilt:

$$Z \in \mathbb{R}^{N \times M}.$$

11.15 Rekonstruktion aus der reduzierten Darstellung

Aus der reduzierten Darstellung $\vec{z}$ kann man den Datenpunkt näherungsweise rekonstruieren.

Für zentrierte Daten:

$$\hat{\vec{x}} = U_M \vec{z}.$$

Für ursprüngliche nichtzentrierte Daten addieren wir den Mittelwert zurück:

$$\hat{\vec{x}} = \vec{\mu} + U_M \vec{z}.$$

Da

$$\vec{z} = U_M^\top (\vec{x} - \vec{\mu}),$$

gilt:

$$\hat{\vec{x}} = \vec{\mu} + U_M U_M^\top (\vec{x} - \vec{\mu}).$$

Definition

Die PCA-Rekonstruktion aus M Komponenten ist:

$$\hat{\vec{x}} = \vec{\mu} + U_M U_M^\top (\vec{x} - \vec{\mu}).$$

Wenn

$$M = D,$$

und alle Hauptkomponenten verwendet werden, dann ist die Rekonstruktion exakt:

$$\hat{\vec{x}} = \vec{x}.$$

Wenn

$$M < D,$$

geht Information verloren.

11.16 Rekonstruktionsfehler

Der Rekonstruktionsfehler misst, wie stark ein Datenpunkt durch die niedrigdimensionale Darstellung verändert wird:

$$\|\vec{x} - \hat{\vec{x}}\|^2.$$

Für alle Datenpunkte:

$$E_{\text{rec}} = \frac{1}{N} \sum_{n=1}^N \|\vec{x}_n - \hat{\vec{x}}_n\|^2.$$

Definition

Der Rekonstruktionsfehler (reconstruction error) ist:

$$E_{\text{rec}} = \frac{1}{N} \sum_{n=1}^N \|\vec{x}_n - \hat{\vec{x}}_n\|^2.$$

Ein wichtiges Resultat: PCA minimiert den mittleren quadratischen Rekonstruktionsfehler unter allen M -dimensionalen linearen Unterräumen.

Idee

PCA kann auf zwei äquivalente Arten verstanden werden:

- Finde Richtungen maximaler Varianz.
- Finde einen niedrigdimensionalen linearen Unterraum mit minimalem quadratischem Rekonstruktionsfehler.

11.17 Erklärte Varianz

Die Eigenwerte der Kovarianzmatrix geben an, wie viel Varianz jede Hauptkomponente erklärt.

Die Gesamtvarianz der Daten ist:

$$\sum_{j=1}^D \lambda_j.$$

Die durch die ersten M Hauptkomponenten erklärte Varianz ist:

$$\sum_{j=1}^M \lambda_j.$$

Der erklärte Varianzanteil (explained variance ratio) ist:

$$r_M = \frac{\sum_{j=1}^M \lambda_j}{\sum_{j=1}^D \lambda_j}.$$

Definition

Der erklärte Varianzanteil der ersten M Hauptkomponenten ist:

$$r_M = \frac{\sum_{j=1}^M \lambda_j}{\sum_{j=1}^D \lambda_j}.$$

Beispiel

Wenn

$$r_M = 0,95,$$

dann erklären die ersten M Hauptkomponenten

$$95\%$$

der Gesamtvarianz.

11.18 Wahl der Anzahl der Komponenten

Die Anzahl M der Hauptkomponenten ist ein Hyperparameter.

Typische Kriterien:

- Wähle M , sodass ein bestimmter Varianzanteil erklärt wird.
- Verwende einen Scree Plot.
- Wähle M mit Validierungsdaten für eine nachgelagerte Aufgabe.
- Wähle M so, dass Visualisierung möglich ist, zum Beispiel $M = 2$ oder $M = 3$.

11.18.1 Varianzschwelle

Man kann M so wählen, dass:

$$r_M \geq 0,95.$$

Das bedeutet: Mindestens 95% der Varianz sollen erhalten bleiben.

11.18.2 Scree Plot

Ein Scree Plot zeigt die Eigenwerte

$$\lambda_1, \lambda_2, \dots, \lambda_D$$

in absteigender Reihenfolge. Oft sucht man nach einem Knickpunkt. Nach diesem Punkt bringen zusätzliche Komponenten nur noch wenig zusätzliche Varianz.

Hinweis

Ein hoher erklärter Varianzanteil bedeutet nicht automatisch gute Leistung für jede Aufgabe. Manchmal liegen für Klassifikation wichtige Informationen in Richtungen mit kleiner Varianz.

11.19 PCA-Algorithmus

Der PCA-Algorithmus kann so zusammengefasst werden:

1. Datenmatrix X aufstellen.
2. Empirischen Mittelwert berechnen:

$$\vec{\mu} = \frac{1}{N} \sum_{n=1}^N \vec{x}_n.$$

3. Daten zentrieren:

$$\tilde{\vec{x}}_n = \vec{x}_n - \vec{\mu}.$$

4. Kovarianzmatrix berechnen:

$$S = \frac{1}{N} \tilde{X}^\top \tilde{X}.$$

5. Eigenwertproblem lösen:

$$S \vec{u}_j = \lambda_j \vec{u}_j.$$

6. Eigenwerte absteigend sortieren.

7. Die ersten M Eigenvektoren wählen:

$$U_M = (\vec{u}_1, \dots, \vec{u}_M).$$

8. Daten projizieren:

$$Z = \tilde{X} U_M.$$

Definition

PCA-Projektion für einen neuen Datenpunkt:

$$\vec{z} = U_M^\top (\vec{x} - \vec{\mu}).$$

Wichtig: Der Mittelwert $\vec{\mu}$ und die Matrix U_M werden auf den Trainingsdaten gelernt.

11.20 PCA und Data Leakage

Wenn PCA als Vorverarbeitung für ein überwachtes Lernproblem verwendet wird, muss PCA nur auf den Trainingsdaten gefittet werden.

Falsch wäre:

PCA auf allen Daten fitten $\rightarrow$ Train/Test Split.

Richtig ist:

Train/Test Split $\rightarrow$ PCA auf Training fitten $\rightarrow$ Training und Test transformieren.

Prüfungshinweis

PCA muss bei einem ML-Workflow wie andere Vorverarbeitung behandelt werden. Fit auf Trainingsdaten. Transformation mit denselben PCA-Komponenten auf Validierungs- und Testdaten.

11.21 Standardisierung vor PCA

PCA ist empfindlich gegenüber Skalierung. Wenn ein Merkmal sehr große numerische Werte hat, kann es die Hauptkomponenten dominieren.

Beispiel: Ein Merkmal wird in Metern gemessen, ein anderes in Millimetern. Dann ist die Varianz des Millimeter-Merkmals numerisch viel größer, obwohl es nicht unbedingt wichtiger ist.

Eine Standardisierung ist:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}.$$

Hinweis

Ob man vor PCA standardisieren sollte, hängt vom Problem ab.

Wenn die Skalen der Merkmale bedeutungsvoll sind, kann Zentrierung reichen. Wenn Merkmale unterschiedliche Einheiten oder Größenordnungen haben, ist Standardisierung oft sinnvoll.

11.22 PCA und Dekorrelation

Wenn wir alle Hauptkomponenten verwenden, sind die projizierten Koordinaten unkorreliert.

Sei

$$U = (\vec{u}_1, \dots, \vec{u}_D)$$

die Matrix aller Eigenvektoren. Dann gilt:

$$U^\top S U = \Lambda,$$

wobei

$$\Lambda = \text{diag}(\lambda_1, \dots, \lambda_D).$$

Das bedeutet: Die Kovarianzmatrix im PCA-Koordinatensystem ist diagonal.

Idee

PCA rotiert das Koordinatensystem so, dass die Kovarianzmatrix diagonal wird.
Die neuen Koordinaten sind unkorreliert.

11.23 Whitening

Whitening geht einen Schritt weiter. Nach PCA kann man jede Komponente durch die Wurzel ihres Eigenwerts teilen.

Wenn

$$\vec{z} = U^\top (\vec{x} - \vec{\mu}),$$

dann ist eine whitened Darstellung:

$$\vec{z}_{\text{white}} = \Lambda^{-1/2} \vec{z}.$$

Dabei ist:

$$\Lambda^{-1/2} = \text{diag} \left(\frac{1}{\sqrt{\lambda_1}}, \dots, \frac{1}{\sqrt{\lambda_D}} \right).$$

Definition

Whitening transformiert Daten so, dass sie Mittelwert 0 und Kovarianzmatrix I haben.

Hinweis

Whitening kann nützlich sein, verstärkt aber Komponenten mit kleinen Eigenwerten. Diese können viel Rauschen enthalten.

11.24 PCA über Singular Value Decomposition

In der Praxis berechnet man PCA häufig über Singular Value Decomposition (SVD).

Für die zentrierte Datenmatrix

$$\tilde{X} \in \mathbb{R}^{N \times D}$$

lautet die SVD:

$$\tilde{X} = A \Sigma V^\top.$$

Dabei sind:

$$A \in \mathbb{R}^{N \times N},$$

$$\Sigma \in \mathbb{R}^{N \times D},$$

$$V \in \mathbb{R}^{D \times D}.$$

Die Kovarianzmatrix ist:

$$S = \frac{1}{N} \tilde{X}^\top \tilde{X}.$$

Einsetzen der SVD:

$$\tilde{X}^\top \tilde{X} = (A \Sigma V^\top)^\top (A \Sigma V^\top).$$

Also:

$$\tilde{X}^\top \tilde{X} = V \Sigma^\top A^\top A \Sigma V^\top.$$

Da A orthogonal ist:

$$A^\top A = I.$$

Somit:

$$\tilde{X}^\top \tilde{X} = V \Sigma^\top \Sigma V^\top.$$

Also:

$$S = V \left(\frac{1}{N} \Sigma^\top \Sigma \right) V^\top.$$

Idee

Die rechten Singulärvektoren der zentrierten Datenmatrix sind die PCA-Richtungen. Die Eigenwerte der Kovarianzmatrix hängen mit den quadrierten Singulärwerten zusammen.

Wenn σ_j der j -te Singulärwert ist, dann gilt:

$$\lambda_j = \frac{\sigma_j^2}{N}.$$

Je nach Konvention wird statt N auch $N - 1$ verwendet.

11.25 PCA für Visualisierung

Eine häufige Anwendung von PCA ist Visualisierung. Dazu setzt man:

$$M = 2$$

oder

$$M = 3.$$

Dann kann man Datenpunkte in zwei oder drei Dimensionen darstellen:

$$\vec{z}_n = U_M^\top(\vec{x}_n - \vec{\mu}).$$

Hinweis

PCA-Visualisierung zeigt nur lineare Projektionen.

Wenn die interessante Struktur nichtlinear ist, kann PCA sie möglicherweise nicht gut sichtbar machen.

Für nichtlineare Visualisierung verwendet man oft Methoden wie *t*-SNE oder UMAP. Diese sind aber schwieriger zu interpretieren und haben eigene Hyperparameter.

11.26 PCA für Rauschreduktion

Wenn Rauschen vor allem in Richtungen kleiner Varianz liegt, kann PCA zur Rauschreduktion verwendet werden. Man behält die Hauptkomponenten mit großen Eigenwerten und verwirft die mit kleinen Eigenwerten.

$$\vec{x} \longrightarrow \vec{z} = U_M^\top(\vec{x} - \vec{\mu}) \longrightarrow \hat{\vec{x}} = \vec{\mu} + U_M \vec{z}.$$

Idee

PCA-Rauschreduktion funktioniert gut, wenn Signal in Richtungen großer Varianz liegt und Rauschen in Richtungen kleiner Varianz.

Diese Annahme ist nicht immer erfüllt.

11.27 PCA und Kompression

PCA kann auch als lineare Kompression verstanden werden. Ein Datenpunkt in $\mathbb{R}^D$ wird durch M Koeffizienten beschrieben:

$$\vec{z} \in \mathbb{R}^M,$$

wobei

$$M < D.$$

Zusätzlich braucht man den Mittelwert $\vec{\mu}$ und die Komponenten U_M , um zu rekonstruieren.

$$\hat{\vec{x}} = \vec{\mu} + U_M \vec{z}.$$

Beispiel

Ein Bildvektor mit

$$D = 784$$

Pixelwerten könnte mit

$$M = 50$$

PCA-Koeffizienten approximiert werden.

Das ist eine starke Reduktion der Dimension.

11.28 PCA und Klassifikation

PCA ist unüberwacht (unsupervised). Das bedeutet: PCA verwendet die Labels nicht.

PCA sucht Richtungen großer Varianz in den Eingaben $\vec{x}$, nicht Richtungen, die besonders gut Klassen trennen.

Hinweis

PCA kann für Klassifikation hilfreich sein, muss es aber nicht.

Eine Richtung mit kleiner Varianz kann für die Klassentrennung sehr wichtig sein. PCA könnte diese Richtung verwerfen.

Wenn das Ziel Klassifikation ist, sollte die Anzahl der PCA-Komponenten oft mit Validierungsdaten gewählt werden.

11.29 Probabilistische Interpretation

PCA kann auch probabilistisch interpretiert werden. In probabilistischer PCA (probabilistic PCA) nimmt man an, dass Daten aus einem niedrigdimensionalen latenten Vektor entstehen:

$$\vec{z} \in \mathbb{R}^M.$$

Ein lineares generatives Modell ist:

$$\vec{x} = W\vec{z} + \vec{\mu} + \vec{\epsilon}.$$

Dabei ist:

$$\vec{\epsilon} \sim \mathcal{N}(\vec{0}, \sigma^2 I).$$

Idee

Probabilistische PCA betrachtet PCA als generatives Modell mit latenten Variablen.

Die beobachteten Daten entstehen aus niedrigdimensionalen latenten Faktoren plus Gauß-schem Rauschen.

Die Details der probabilistischen PCA gehen über die klassische PCA hinaus, sind aber wichtig für den Zusammenhang zu latenten Variablenmodellen und Gaussian Mixture Models.

11.30 Lineare und nichtlineare Dimensionsreduktion

PCA ist linear. Das heißt: Die reduzierte Darstellung entsteht durch lineare Projektion.

Viele Daten liegen aber auf nichtlinearen Mannigfaltigkeiten (manifolds). Dann kann lineare PCA unzureichend sein.

Nichtlineare Methoden sind zum Beispiel:

- Kernel PCA,
- Isomap,
- Locally Linear Embedding,
- *t*-SNE,
- UMAP,
- Autoencoder.

Hinweis

Nichtlineare Methoden können komplexere Strukturen erfassen, sind aber oft schwerer zu interpretieren.

PCA bleibt wichtig, weil sie einfach, schnell, stabil und mathematisch gut verstanden ist.

11.31 PCA im Vergleich zu Autoencodern

Ein Autoencoder ist ein neuronales Netz, das lernt, Eingaben zu rekonstruieren.

Encoder:

$$\vec{z} = f_{\text{enc}}(\vec{x}).$$

Decoder:

$$\hat{\vec{x}} = f_{\text{dec}}(\vec{z}).$$

Das Training minimiert typischerweise:

$$\sum_{n=1}^N \|\vec{x}_n - \hat{\vec{x}}_n\|^2.$$

Wenn Encoder und Decoder linear sind und der Fehler quadratisch ist, hängt der Autoencoder eng mit PCA zusammen.

Idee

PCA kann als lineares Rekonstruktionsmodell verstanden werden.

Autoencoder verallgemeinern diese Idee mit nichtlinearen Encodern und Decodern.

11.32 Praktische Aspekte

11.32.1 Vorverarbeitung

Vor PCA sollte man meistens:

- fehlende Werte behandeln,
- Daten zentrieren,
- je nach Problem standardisieren,
- Ausreißer prüfen.

11.32.2 Ausreißer

PCA kann empfindlich gegenüber Ausreißern sein. Ein einzelner extremer Datenpunkt kann die Kovarianzmatrix und damit die Hauptkomponenten stark beeinflussen.

11.32.3 Interpretierbarkeit

Die Hauptkomponenten sind Linearkombinationen der ursprünglichen Merkmale:

$$z_j = \vec{u}_j^\top (\vec{x} - \vec{\mu}).$$

Die Einträge von $\vec{u}_j$ heißen Loadings. Sie zeigen, welche ursprünglichen Merkmale stark zur Komponente beitragen.

Definition

Loadings sind die Einträge eines Hauptkomponentenvektors $\vec{u}_j$. Sie beschreiben, wie stark jedes ursprüngliche Merkmal in die jeweilige Hauptkomponente eingeht.

11.33 Häufige Fehler

Prüfungshinweis

Typische Fehler bei PCA und Dimensionsreduktion:

1. Daten vor PCA nicht zentrieren.
2. PCA auf allen Daten fitten und dadurch Data Leakage erzeugen.
3. PCA-Komponenten mit ursprünglichen Merkmalen verwechseln.
4. Denken, PCA verwende Labels.
5. Hohe erklärte Varianz mit guter Klassifikationsleistung gleichsetzen.
6. Standardisierung vergessen, obwohl Merkmale unterschiedliche Einheiten haben.
7. Eigenwerte und Eigenvektoren verwechseln.

8. Die Dimensionen von U_M , $\vec{z}$ und $\vec{x}$ verwechseln.
9. PCA als nichtlineare Methode interpretieren.
10. Rekonstruktionsfehler und Klassifikationsfehler verwechseln.
11. Scree Plot als exakte mathematische Regel statt als Diagnosewerkzeug verstehen.

11.34 Mini-Aufgaben

11.34.1 Aufgabe 1: Zentrierung

Gegeben seien eindimensionale Daten:

$$x_1 = 2, \quad x_2 = 4, \quad x_3 = 6.$$

Berechne den Mittelwert und die zentrierten Daten.

Beispiel

Der Mittelwert ist:

$$\mu = \frac{2 + 4 + 6}{3} = 4.$$

Die zentrierten Daten sind:

$$\tilde{x}_1 = 2 - 4 = -2,$$

$$\tilde{x}_2 = 4 - 4 = 0,$$

$$\tilde{x}_3 = 6 - 4 = 2.$$

11.34.2 Aufgabe 2: Erklärte Varianz

Gegeben seien Eigenwerte:

$$\lambda_1 = 5, \quad \lambda_2 = 3, \quad \lambda_3 = 2.$$

Wie viel Varianz erklären die ersten zwei Hauptkomponenten?

Beispiel

Die Gesamtvarianz ist:

$$\lambda_1 + \lambda_2 + \lambda_3 = 5 + 3 + 2 = 10.$$

Die ersten zwei Komponenten erklären:

$$\lambda_1 + \lambda_2 = 5 + 3 = 8.$$

Der erklärte Varianzanteil ist:

$$r_2 = \frac{8}{10} = 0,8.$$

Also erklären die ersten zwei Hauptkomponenten

$$80\%$$

der Varianz.

11.34.3 Aufgabe 3: Projektion

Gegeben sei

$$\vec{x} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}, \quad \vec{\mu} = \begin{pmatrix} 1 \\ 1 \end{pmatrix},$$

und die Hauptkomponente

$$\vec{u}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Berechne die eindimensionale PCA-Koordinate z_1 .

Beispiel

Zuerst zentrieren:

$$\tilde{\vec{x}} = \vec{x} - \vec{\mu} = \begin{pmatrix} 3 \\ 4 \end{pmatrix} - \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}.$$

Dann projizieren:

$$z_1 = \vec{u}_1^\top \tilde{\vec{x}} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \end{pmatrix}.$$

Also:

$$z_1 = \frac{1}{\sqrt{2}}(2 + 3) = \frac{5}{\sqrt{2}}.$$

11.34.4 Aufgabe 4: Rekonstruktion

Verwende die Werte aus Aufgabe 3 und rekonstruiere $\hat{\vec{x}}$ aus einer Komponente.

Beispiel

Die Rekonstruktion ist:

$$\hat{\vec{x}} = \vec{\mu} + \vec{u}_1 z_1.$$

Einsetzen:

$$\vec{u}_1 z_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \frac{5}{\sqrt{2}} = \frac{5}{2} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Also:

$$\vec{u}_1 z_1 = \begin{pmatrix} 2,5 \\ 2,5 \end{pmatrix}.$$

Damit:

$$\hat{\vec{x}} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} + \begin{pmatrix} 2,5 \\ 2,5 \end{pmatrix} = \begin{pmatrix} 3,5 \\ 3,5 \end{pmatrix}.$$

11.34.5 Aufgabe 5: Dimensionen

Gegeben sei eine Datenmatrix mit

$$N = 100$$

Datenpunkten und

$$D = 20$$

Merkmale. PCA soll auf

$$M = 3$$

Dimensionen reduzieren. Welche Dimensionen haben $\tilde{X}$, S , U_M und Z ?

Beispiel

Die zentrierte Datenmatrix ist:

$$\tilde{X} \in \mathbb{R}^{100 \times 20}.$$

Die Kovarianzmatrix ist:

$$S \in \mathbb{R}^{20 \times 20}.$$

Die ersten $M = 3$ Hauptkomponenten bilden:

$$U_M \in \mathbb{R}^{20 \times 3}.$$

Die projizierten Daten sind:

$$Z = \tilde{X}U_M \in \mathbb{R}^{100 \times 3}.$$

11.35 Zusammenfassung

- Dimensionsreduktion (dimensionality reduction) bildet Daten aus $\mathbb{R}^D$ nach $\mathbb{R}^M$ ab, wobei $M < D$.
- PCA (principal component analysis) ist eine lineare Methode zur Dimensionsreduktion.
- PCA sucht Richtungen maximaler Varianz.
- Die Daten müssen vor PCA zentriert werden:

$$\tilde{\vec{x}}_n = \vec{x}_n - \vec{\mu}.$$

- Die empirische Kovarianzmatrix ist:

$$S = \frac{1}{N} \tilde{X}^\top \tilde{X}.$$

- Die Varianz in Richtung $\vec{u}$ ist:

$$\vec{u}^\top S \vec{u}.$$

- Die Hauptkomponenten sind Eigenvektoren von S :

$$S \vec{u}_j = \lambda_j \vec{u}_j.$$

- Die Eigenwerte λ_j geben die erklärte Varianz der Komponenten an.
- Die PCA-Projektion ist:

$$\vec{z} = U_M^\top (\vec{x} - \vec{\mu}).$$

- Die PCA-Rekonstruktion ist:

$$\hat{\vec{x}} = \vec{\mu} + U_M U_M^\top (\vec{x} - \vec{\mu}).$$

- PCA minimiert den quadratischen Rekonstruktionsfehler unter allen linearen M -dimensionalen Unterräumen.
- Der erklärte Varianzanteil ist:

$$r_M = \frac{\sum_{j=1}^M \lambda_j}{\sum_{j=1}^D \lambda_j}.$$

- PCA ist unüberwacht und verwendet keine Labels.
- PCA muss bei ML-Workflows nur auf Trainingsdaten gefittet werden, um Data Leakage zu vermeiden.

11.36 Typische Prüfungsfragen

Prüfungshinweis

Mögliche Prüfungsfragen zu diesem Kapitel:

1. Was ist Dimensionsreduktion?
2. Warum kann hohe Dimensionalität problematisch sein?
3. Was ist der Fluch der Dimensionalität?
4. Was ist PCA?
5. Warum müssen Daten vor PCA zentriert werden?
6. Definiere die empirische Kovarianzmatrix.
7. Zeige, dass die Kovarianzmatrix positiv semidefinit ist.
8. Was bedeutet Varianz in einer Richtung?
9. Leite das Eigenwertproblem der PCA mit Lagrange-Multiplikatoren her.
10. Warum ist die erste Hauptkomponente der Eigenvektor zum größten Eigenwert?
11. Warum sind Hauptkomponenten orthogonal?
12. Wie projiziert man einen Datenpunkt auf die ersten M Hauptkomponenten?
13. Wie rekonstruiert man einen Datenpunkt aus PCA-Koordinaten?
14. Was ist Rekonstruktionsfehler?
15. Erkläre die Äquivalenz zwischen maximaler Varianz und minimalem Rekonstruktionsfehler.
16. Was ist erklärte Varianz?
17. Wie wählt man die Anzahl der Komponenten?
18. Warum kann PCA für Klassifikation ungeeignet sein?
19. Wie hängt PCA mit SVD zusammen?
20. Was ist Whitening?
21. Warum kann PCA Data Leakage verursachen?
22. Was ist der Unterschied zwischen PCA und Autoencodern?

12 Clustering

12.1 Ziel dieses Kapitels

In den bisherigen Kapiteln haben wir hauptsächlich überwachte Lernverfahren (supervised learning) betrachtet. Dort gibt es Trainingsdaten mit Eingaben und Zielwerten:

$$\mathcal{D} = \{(\vec{x}_1, t_1), \dots, (\vec{x}_N, t_N)\}.$$

Beim Clustering ist die Situation anders. Wir haben nur Eingaben:

$$\mathcal{D} = \{\vec{x}_1, \dots, \vec{x}_N\}.$$

Es gibt keine beobachteten Zielwerte t_n . Das Ziel ist, Struktur in den Daten zu finden.

Idee

Clustering ist ein unüberwachtes Lernverfahren (unsupervised learning). Das Ziel ist, Datenpunkte in Gruppen einzuteilen, sodass Punkte innerhalb einer Gruppe ähnlich sind und Punkte aus verschiedenen Gruppen möglichst unterschiedlich sind.

Nach diesem Kapitel solltest du folgende Fragen beantworten können:

- Was ist Clustering?
- Was ist der Unterschied zwischen Klassifikation und Clustering?
- Was bedeutet Ähnlichkeit oder Distanz zwischen Datenpunkten?
- Was ist k -means?
- Wie funktioniert der k -means-Algorithmus?
- Welche Zielfunktion minimiert k -means?
- Warum besteht k -means aus einem Zuordnungsschritt und einem Aktualisierungsschritt?
- Warum konvergiert k -means?
- Warum hängt k -means von der Initialisierung ab?
- Wie wählt man die Anzahl der Cluster K ?
- Was sind Elbow-Methode und Silhouette-Koeffizient?
- Was sind hierarchisches Clustering und DBSCAN?
- Welche Grenzen hat Clustering?

12.2 Was ist Clustering?

Clustering bezeichnet Methoden, die Datenpunkte automatisch in Gruppen einteilen. Diese Gruppen nennt man Cluster.

Gegeben seien Datenpunkte

$$\vec{x}_1, \dots, \vec{x}_N \in \mathbb{R}^D.$$

Ein Clustering ordnet jedem Datenpunkt eine Clusterzugehörigkeit zu. Wenn es K Cluster gibt, dann ist die Zuordnung:

$$z_n \in \{1, \dots, K\}.$$

Dabei bedeutet

$$z_n = k,$$

dass der Datenpunkt $\vec{x}_n$ zum Cluster k gehört.

Definition

Ein Clustering ist eine Zuordnung

$$z_n \in \{1, \dots, K\}$$

für jeden Datenpunkt $\vec{x}_n$.

Der Wert z_n gibt an, zu welchem Cluster der Datenpunkt gehört.

Beispiel

Beispiele für Clustering:

- Kundinnen und Kunden in Gruppen ähnlichen Kaufverhaltens einteilen.
- Patientinnen und Patienten mit ähnlichen Messwerten gruppieren.
- Bilder nach visueller Ähnlichkeit gruppieren.
- Dokumente nach Themen gruppieren.
- Messdaten in typische Zustände eines Systems aufteilen.

12.3 Clustering versus Klassifikation

Clustering und Klassifikation wirken ähnlich, sind aber grundlegend verschieden.

Bei Klassifikation (classification) gibt es Labels:

$$t_n \in \{1, \dots, K\}.$$

Das Modell lernt aus Beispielen mit bekannten Klassen.

Bei Clustering gibt es keine Labels. Das Modell muss Gruppen selbst finden.

Definition

Klassifikation ist überwachtes Lernen (supervised learning):

$$(\vec{x}_n, t_n)$$

sind gegeben.

Clustering ist unüberwachtes Lernen (unsupervised learning):

$$\vec{x}_n$$

sind gegeben, aber keine Labels t_n .

Idee

Klassifikation beantwortet:

Welche bekannte Klasse hat dieser Datenpunkt?

Clustering beantwortet:

Welche Gruppenstruktur scheint in den Daten vorhanden zu sein?

12.4 Ähnlichkeit und Distanz

Clustering braucht eine Vorstellung davon, wann Datenpunkte ähnlich sind. Oft wird Ähnlichkeit über eine Distanz definiert.

Für zwei Datenpunkte

$$\vec{x}_n, \vec{x}_m \in \mathbb{R}^D$$

ist die euklidische Distanz:

$$d(\vec{x}_n, \vec{x}_m) = \|\vec{x}_n - \vec{x}_m\|.$$

Ausgeschrieben:

$$\|\vec{x}_n - \vec{x}_m\| = \sqrt{\sum_{j=1}^D (x_{nj} - x_{mj})^2}.$$

Häufig verwendet man auch die quadrierte euklidische Distanz:

$$\|\vec{x}_n - \vec{x}_m\|^2 = \sum_{j=1}^D (x_{nj} - x_{mj})^2.$$

Definition

Die euklidische Distanz zwischen zwei Datenpunkten ist:

$$d(\vec{x}_n, \vec{x}_m) = \sqrt{\sum_{j=1}^D (x_{nj} - x_{mj})^2}.$$

Hinweis

Welche Distanz sinnvoll ist, hängt vom Problem ab.

Für numerische Vektoren ist die euklidische Distanz häufig. Für Texte, Sequenzen oder kategoriale Daten können andere Ähnlichkeitsmaße sinnvoller sein.

12.5 Feature-Skalierung

Clustering ist oft stark abhängig von der Skalierung der Merkmale. Wenn ein Merkmal sehr große Zahlenwerte hat, dominiert es die Distanz.

Beispiel

Angenommen, ein Datenpunkt enthält:

$$x_1 = \text{Alter in Jahren}$$

und

$$x_2 = \text{Einkommen in Euro.}$$

Dann kann das Einkommen numerisch viel größere Werte haben als das Alter. Ohne Skalierung wird die euklidische Distanz fast nur durch das Einkommen bestimmt.

Eine Standardisierung ist:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}.$$

Dabei sind μ_j und σ_j Mittelwert und Standardabweichung des Merkmals j .

Prüfungshinweis

Vor distanzbasiertem Clustering sollte man prüfen, ob die Merkmale sinnvoll skaliert sind. Bei unterschiedlichen Einheiten oder Größenordnungen ist Standardisierung oft notwendig.

12.6 k -means: Grundidee

Der bekannteste Clustering-Algorithmus ist k -means. Dabei wird die Anzahl der Cluster K vorgegeben.

Die Idee ist: Jeder Cluster wird durch einen Mittelpunkt beschrieben. Diese Mittelpunkte heißen Clusterzentren (cluster centers) oder Zentroiden (centroids).

Wir schreiben die Zentren als

$$\vec{\mu}_1, \dots, \vec{\mu}_K \in \mathbb{R}^D.$$

Jeder Datenpunkt wird dem nächstgelegenen Zentrum zugeordnet.

Definition

Beim k -means-Clustering gibt es K Clusterzentren

$$\vec{\mu}_1, \dots, \vec{\mu}_K.$$

Jeder Datenpunkt $\vec{x}_n$ wird dem nächstgelegenen Zentrum zugeordnet.

Idee

k -means versucht, Cluster so zu finden, dass Datenpunkte möglichst nahe am Zentrum ihres Clusters liegen.

12.7 Cluster-Zuordnungen

Für jeden Datenpunkt $\vec{x}_n$ gibt es eine Zuordnung

$$z_n \in \{1, \dots, K\}.$$

Wenn

$$z_n = k,$$

dann gehört $\vec{x}_n$ zu Cluster k .

Die Zuordnung bei gegebenen Zentren ist:

$$z_n = \arg \min_{k \in \{1, \dots, K\}} \|\vec{x}_n - \vec{\mu}_k\|^2.$$

Definition

Der Zuordnungsschritt bei k -means ist:

$$z_n = \arg \min_k \|\vec{x}_n - \vec{\mu}_k\|^2.$$

Man kann statt der quadrierten Distanz auch die normale euklidische Distanz verwenden. Da die Wurzelfunktion monoton wachsend ist, ist das nächstgelegene Zentrum dasselbe:

$$\arg \min_k \|\vec{x}_n - \vec{\mu}_k\| = \arg \min_k \|\vec{x}_n - \vec{\mu}_k\|^2.$$

12.8 Zielfunktion von k -means

k -means minimiert die Summe der quadrierten Abstände jedes Datenpunkts zu seinem Clusterzentrum.

$$J = \sum_{n=1}^N \|\vec{x}_n - \vec{\mu}_{z_n}\|^2.$$

Diese Größe heißt innerhalb-Cluster-Quadratsumme (within-cluster sum of squares).

Definition

Die k -means-Zielfunktion ist:

$$J(\{z_n\}, \{\vec{\mu}_k\}) = \sum_{n=1}^N \|\vec{x}_n - \vec{\mu}_{z_n}\|^2.$$

Dabei ist z_n die Clusterzuordnung von $\vec{x}_n$.

Äquivalent kann man mit Indikatorvariablen schreiben:

$$r_{nk} = \begin{cases} 1, & z_n = k, \\ 0, & z_n \neq k. \end{cases}$$

Dann ist:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|\vec{x}_n - \vec{\mu}_k\|^2.$$

Hinweis

Für jeden Datenpunkt gilt:

$$\sum_{k=1}^K r_{nk} = 1.$$

Jeder Punkt gehört also genau zu einem Cluster.

12.9 Der k -means-Algorithmus

Die Zielfunktion hängt von zwei Arten von Variablen ab:

- Zuordnungen z_n oder r_{nk} ,
- Clusterzentren $\vec{\mu}_k$.

Wenn die Zentren fest sind, sind die optimalen Zuordnungen leicht zu bestimmen. Wenn die Zuordnungen fest sind, sind die optimalen Zentren leicht zu bestimmen.

Der k -means-Algorithmus wechselt daher zwischen zwei Schritten:

1. Zuordnungsschritt (assignment step),
2. Aktualisierungsschritt (update step).

Definition

Der k -means-Algorithmus:

1. Wähle initiale Zentren $\vec{\mu}_1, \dots, \vec{\mu}_K$.
2. Ordne jeden Datenpunkt dem nächsten Zentrum zu:

$$z_n = \arg \min_k \|\vec{x}_n - \vec{\mu}_k\|^2.$$

3. Aktualisiere jedes Zentrum als Mittelwert der zugeordneten Punkte:

$$\vec{\mu}_k = \frac{1}{N_k} \sum_{n: z_n=k} \vec{x}_n.$$

4. Wiederhole Schritt 2 und 3, bis sich die Zuordnungen oder Zentren nicht mehr ändern.

Dabei ist

$$N_k = \sum_{n=1}^N r_{nk}$$

die Anzahl der Punkte in Cluster k .

12.10 Herleitung des Aktualisierungsschritts

Wir nehmen an, dass die Zuordnungen r_{nk} fest sind. Dann minimieren wir

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|\vec{x}_n - \vec{\mu}_k\|^2$$

nach den Zentren $\vec{\mu}_k$.

Für ein bestimmtes Cluster k betrachten wir nur die Terme, die $\vec{\mu}_k$ enthalten:

$$J_k = \sum_{n=1}^N r_{nk} \|\vec{x}_n - \vec{\mu}_k\|^2.$$

Wir leiten nach $\vec{\mu}_k$ ab:

$$\frac{\partial J_k}{\partial \vec{\mu}_k} = \sum_{n=1}^N r_{nk} 2(\vec{\mu}_k - \vec{x}_n).$$

Setze gleich $\vec{0}$:

$$\sum_{n=1}^N r_{nk} 2(\vec{\mu}_k - \vec{x}_n) = \vec{0}.$$

Teilen durch 2:

$$\sum_{n=1}^N r_{nk} (\vec{\mu}_k - \vec{x}_n) = \vec{0}.$$

Ausmultiplizieren:

$$\left(\sum_{n=1}^N r_{nk} \right) \vec{\mu}_k - \sum_{n=1}^N r_{nk} \vec{x}_n = \vec{0}.$$

Also:

$$\left(\sum_{n=1}^N r_{nk} \right) \vec{\mu}_k = \sum_{n=1}^N r_{nk} \vec{x}_n.$$

Damit:

$$\vec{\mu}_k = \frac{\sum_{n=1}^N r_{nk} \vec{x}_n}{\sum_{n=1}^N r_{nk}}.$$

Da

$$N_k = \sum_{n=1}^N r_{nk},$$

ist:

$$\vec{\mu}_k = \frac{1}{N_k} \sum_{n: z_n=k} \vec{x}_n.$$

Idee

Das optimale Zentrum eines Clusters ist der Mittelwert aller Datenpunkte in diesem Cluster.

12.11 Warum k -means konvergiert

In jedem Zuordnungsschritt wird jeder Punkt dem nächstgelegenen Zentrum zugeordnet. Dadurch kann die Zielfunktion J nicht steigen.

In jedem Aktualisierungsschritt wird jedes Zentrum als Mittelwert seiner Punkte gesetzt. Das minimiert J für feste Zuordnungen. Auch dadurch kann J nicht steigen.

Also gilt nach jeder Iteration:

$$J^{(\tau+1)} \leq J^{(\tau)}.$$

Da

$$J \geq 0,$$

kann J nicht unbegrenzt fallen.

Idee

k -means konvergiert, weil die Zielfunktion in jedem Schritt nicht steigt und nach unten durch 0 beschränkt ist.

Hinweis

Konvergenz bedeutet hier: Der Algorithmus erreicht ein lokales Minimum oder einen festen Punkt.

Es bedeutet nicht, dass immer das globale Minimum gefunden wird.

12.12 Lokale Minima und Initialisierung

Die k -means-Zielfunktion ist nicht konvex in allen Variablen gemeinsam. Der Algorithmus kann daher in verschiedenen lokalen Minima landen.

Das Ergebnis hängt stark von der Initialisierung der Zentren ab.

Beispiel

Wenn zwei initiale Zentren zufällig sehr nahe beieinander liegen, können sie anfangs denselben Bereich der Daten erklären. Ein anderer Bereich bekommt dann vielleicht kein gutes Zentrum. Das kann zu einem schlechteren lokalen Minimum führen.

Idee

k -means sollte oft mehrfach mit verschiedenen Initialisierungen gestartet werden.

Dann wählt man die Lösung mit dem kleinsten Wert der Zielfunktion J .

12.13 k -means++

Eine bessere Initialisierungsmethode ist k -means++.

Die Idee: Zentren sollen nicht nur zufällig gewählt werden, sondern möglichst gut über den Datensatz verteilt sein.

Vereinfacht:

1. Wähle das erste Zentrum zufällig aus den Datenpunkten.
2. Für jeden Datenpunkt berechne die Distanz zum nächsten bereits gewählten Zentrum.
3. Wähle das nächste Zentrum mit Wahrscheinlichkeit proportional zur quadrierten Distanz.
4. Wiederhole, bis K Zentren gewählt sind.

Idee

`k-means++` wählt initiale Zentren tendenziell weit voneinander entfernt.
Dadurch startet `k-means` oft in einer besseren Ausgangslage.

12.14 Leere Cluster

Während des `k-means`-Algorithmus kann es passieren, dass einem Cluster kein Datenpunkt zugeordnet wird. Dann ist

$$N_k = 0.$$

Die Aktualisierung

$$\vec{\mu}_k = \frac{1}{N_k} \sum_{n:z_n=k} \vec{x}_n$$

ist dann nicht definiert.

Hinweis

Leere Cluster müssen speziell behandelt werden.

Mögliche Strategien:

- Zentrum zufällig neu initialisieren.
- Zentrum auf einen weit entfernten Datenpunkt setzen.
- Cluster entfernen, falls weniger als K Cluster erlaubt sind.

12.15 Wahl der Clusterzahl K

Bei `k-means` muss K vorgegeben werden. Das ist oft schwierig.

Wenn K zu klein ist, werden unterschiedliche Gruppen zusammengelegt. Wenn K zu groß ist, werden natürliche Gruppen künstlich aufgeteilt.

Idee

Die Wahl von K ist ein Modellselektionsproblem.

Es gibt selten eine eindeutig richtige Clusterzahl. Die sinnvolle Wahl hängt von Daten, Ziel und Interpretation ab.

12.16 Elbow-Methode

Die Elbow-Methode betrachtet die `k-means`-Zielfunktion als Funktion von K .

Für jedes K berechnet man:

$$J_K = \sum_{n=1}^N \|\vec{x}_n - \vec{\mu}_{z_n}\|^2.$$

Wenn K größer wird, kann J_K nur sinken oder gleich bleiben. Mehr Cluster können die Daten immer mindestens genauso gut erklären.

Man zeichnet J_K gegen K . Oft sucht man nach einem Knickpunkt, dem sogenannten Ellenbogen.

Definition

Die Elbow-Methode wählt K anhand eines Knickpunkts in der Kurve

$$K \mapsto J_K.$$

Hinweis

Die Elbow-Methode ist heuristisch.

Manchmal gibt es keinen klaren Knick. Dann ist die Wahl von K nicht eindeutig.

12.17 Silhouette-Koeffizient

Der Silhouette-Koeffizient misst, wie gut ein Datenpunkt zu seinem eigenen Cluster passt im Vergleich zu anderen Clustern.

Für einen Datenpunkt $\vec{x}_n$:

- a_n : durchschnittliche Distanz zu Punkten im eigenen Cluster,
- b_n : kleinste durchschnittliche Distanz zu Punkten in einem anderen Cluster.

Der Silhouette-Wert ist:

$$s_n = \frac{b_n - a_n}{\max(a_n, b_n)}.$$

Es gilt:

$$-1 \leq s_n \leq 1.$$

Definition

Der Silhouette-Koeffizient eines Datenpunkts ist:

$$s_n = \frac{b_n - a_n}{\max(a_n, b_n)}.$$

Dabei misst a_n die Nähe zum eigenen Cluster und b_n die Nähe zum nächsten anderen Cluster.

Interpretation:

- $s_n \approx 1$: Punkt passt gut zum eigenen Cluster.
- $s_n \approx 0$: Punkt liegt zwischen Clustern.
- $s_n < 0$: Punkt passt besser zu einem anderen Cluster.

Der mittlere Silhouette-Wert ist:

$$s = \frac{1}{N} \sum_{n=1}^N s_n.$$

Hinweis

Der Silhouette-Koeffizient kann helfen, K zu wählen. Aber auch er ist nur ein internes Qualitätsmaß und ersetzt keine fachliche Interpretation.

12.18 Grenzen von k -means

k -means ist einfach und effizient, hat aber wichtige Einschränkungen.

12.18.1 Sphärische Cluster

k -means funktioniert besonders gut, wenn Cluster ungefähr kugelförmig sind und ähnliche Größe haben.

Das liegt an der euklidischen Distanz zu Zentren.

12.18.2 Empfindlichkeit gegenüber Ausreißern

Da Mittelwerte verwendet werden, können Ausreißer die Zentren stark verschieben.

12.18.3 Feste Clusterzahl

Die Anzahl K muss vorher festgelegt werden.

12.18.4 Harte Zuordnungen

Jeder Punkt gehört genau zu einem Cluster. Unsicherheit wird nicht modelliert.

12.18.5 Skalierung

Unterschiedliche Merkmalsskalen können das Ergebnis stark verzerren.

Prüfungshinweis

Typische Grenzen von k -means:

- K muss vorgegeben werden.
- Ergebnis hängt von Initialisierung ab.
- Cluster sollten ungefähr konvex und kugelförmig sein.
- Ausreißer können starke Effekte haben.
- Feature-Skalierung ist wichtig.
- Es gibt keine probabilistische Unsicherheit der Zuordnung.

12.19 Harte und weiche Clusterzuordnung

Bei k -means ist die Zuordnung hart:

$$r_{nk} \in \{0,1\}.$$

Ein Datenpunkt gehört genau zu einem Cluster.

Bei weichem Clustering (soft clustering) kann ein Punkt teilweise mehreren Clustern zugeordnet sein. Dann sind die Verantwortlichkeiten (responsibilities):

$$r_{nk} \in [0,1],$$

mit

$$\sum_{k=1}^K r_{nk} = 1.$$

Definition

Hartes Clustering (hard clustering) ordnet jeden Punkt genau einem Cluster zu.

Weiches Clustering (soft clustering) ordnet jedem Punkt Wahrscheinlichkeiten oder Verantwortlichkeiten für mehrere Cluster zu.

Idee

Gaussian Mixture Models verwenden weiche Clusterzuordnungen.

k -means kann als harte und vereinfachte Variante einer Mischungs idee verstanden werden.

12.20 Hierarchisches Clustering

Hierarchisches Clustering erzeugt nicht nur eine einzelne Clusterlösung, sondern eine Hierarchie von Clustern.

Es gibt zwei Haupttypen:

- agglomeratives Clustering (agglomerative clustering),
- divisives Clustering (divisive clustering).

Beim agglomerativen Clustering startet jeder Datenpunkt als eigener Cluster. Dann werden schrittweise die ähnlichsten Cluster zusammengeführt.

Beim divisiven Clustering startet man mit einem großen Cluster und teilt ihn schrittweise auf.

Definition

Hierarchisches Clustering erzeugt eine verschachtelte Struktur von Clustern.

Diese Struktur kann als Dendrogramm dargestellt werden.

12.21 Agglomeratives Clustering

Agglomeratives Clustering läuft so ab:

1. Jeder Datenpunkt ist zunächst ein eigener Cluster.
2. Berechne Distanzen zwischen Clustern.
3. Führe die zwei ähnlichsten Cluster zusammen.
4. Aktualisiere die Distanzen.
5. Wiederhole, bis nur noch ein Cluster übrig ist.

Idee

Agglomeratives Clustering baut eine Clusterhierarchie von unten nach oben auf.

N einzelne Cluster $\longrightarrow$ 1 großer Cluster.

12.22 Linkage-Kriterien

Beim hierarchischen Clustering muss definiert werden, wie die Distanz zwischen zwei Clustern gemessen wird. Das nennt man Linkage-Kriterium.

Seien A und B zwei Cluster.

12.22.1 Single Linkage

$$d_{\text{single}}(A, B) = \min_{\vec{x} \in A, \vec{x}' \in B} d(\vec{x}, \vec{x}').$$

Single Linkage verwendet den kleinsten Abstand zwischen Punkten der beiden Cluster.

12.22.2 Complete Linkage

$$d_{\text{complete}}(A, B) = \max_{\vec{x} \in A, \vec{x}' \in B} d(\vec{x}, \vec{x}').$$

Complete Linkage verwendet den größten Abstand.

12.22.3 Average Linkage

$$d_{\text{average}}(A, B) = \frac{1}{|A||B|} \sum_{\vec{x} \in A} \sum_{\vec{x}' \in B} d(\vec{x}, \vec{x}').$$

Average Linkage verwendet den durchschnittlichen Abstand.

12.22.4 Ward-Linkage

Ward-Linkage führt diejenigen Cluster zusammen, bei denen die Zunahme der innerhalb-Cluster-Quadratsumme am kleinsten ist.

Definition

Ein Linkage-Kriterium definiert die Distanz zwischen Clustern.
Es bestimmt, welche Cluster beim hierarchischen Clustering zusammengeführt werden.

Hinweis

Unterschiedliche Linkage-Kriterien können sehr unterschiedliche Clusterstrukturen ergeben.

12.23 Dendrogramm

Ein Dendrogramm ist eine Baumdarstellung der Clusterhierarchie. Die Blätter sind einzelne Datenpunkte. Innere Knoten entsprechen Zusammenführungen von Clustern.

Die Höhe einer Zusammenführung zeigt typischerweise die Distanz, bei der die Cluster zusammengeführt wurden.

Idee

Ein Dendrogramm erlaubt, die Anzahl der Cluster nachträglich zu wählen.
Man schneidet den Baum auf einer bestimmten Höhe.

Wenn man das Dendrogramm hoch schneidet, erhält man wenige große Cluster. Wenn man es niedrig schneidet, erhält man viele kleine Cluster.

12.24 DBSCAN

DBSCAN steht für Density-Based Spatial Clustering of Applications with Noise. Es ist ein dichtebasiertes Clustering-Verfahren.

Die Idee: Cluster sind Bereiche hoher Punktdichte, getrennt durch Bereiche niedriger Punktdichte.

DBSCAN benötigt zwei wichtige Hyperparameter:

$$\varepsilon$$

und

MinPts.

Definition

DBSCAN definiert Cluster als zusammenhängende Regionen hoher Dichte.
Punkte in dünn besetzten Regionen können als Rauschen oder Ausreißer markiert werden.

12.25 Kernpunkte, Randpunkte und Rauschen

Für einen Punkt $\vec{x}_n$ betrachtet man seine ε -Nachbarschaft:

$$\mathcal{N}_\varepsilon(\vec{x}_n) = \{\vec{x}_m : d(\vec{x}_n, \vec{x}_m) \leq \varepsilon\}.$$

Ein Kernpunkt (core point) ist ein Punkt, dessen ε -Nachbarschaft mindestens MinPts Punkte enthält.

$$|\mathcal{N}_\varepsilon(\vec{x}_n)| \geq \text{MinPts}.$$

Ein Randpunkt (border point) ist kein Kernpunkt, liegt aber in der Nachbarschaft eines Kernpunkts.

Ein Rauschpunkt (noise point) ist weder Kernpunkt noch Randpunkt.

Definition

DBSCAN unterscheidet:

- Kernpunkte (core points),
- Randpunkte (border points),
- Rauschpunkte (noise points).

12.26 Eigenschaften von DBSCAN

DBSCAN hat andere Eigenschaften als k -means.

- Die Anzahl der Cluster muss nicht vorgegeben werden.
- Cluster können nicht-kugelförmig sein.
- Ausreißer können als Rauschen erkannt werden.
- Die Wahl von ε und MinPts ist wichtig.
- DBSCAN hat Probleme, wenn Cluster sehr unterschiedliche Dichten haben.

Idee

DBSCAN ist besonders nützlich, wenn Cluster unregelmäßige Formen haben und Ausreißer erkannt werden sollen.

12.27 Vergleich wichtiger Clustering-Methoden

Methode	Clusterform	Braucht K ?	Ausreißer
k -means	kugelförmig	ja	empfindlich
Hierarchisch	abhängig von Linkage	nicht direkt	abhängig von Methode
DBSCAN	beliebiger	nein	kann Rauschen erkennen

Hinweis

Es gibt nicht den einen besten Clustering-Algorithmus.
Die passende Methode hängt von Datenstruktur, Ziel und gewünschter Interpretation ab.

12.28 Clustering und probabilistische Modelle

k -means ordnet Punkte hart zu Clustern zu. Eine probabilistische Alternative sind Mischmodelle (mixture models).

Ein Gaussian Mixture Model nimmt an, dass Daten aus einer Mischung von Normalverteilungen entstehen:

$$p(\vec{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\vec{x} \mid \vec{\mu}_k, \Sigma_k).$$

Dabei ist:

- π_k das Mischgewicht von Komponente k ,
- $\vec{\mu}_k$ der Mittelwert,
- Σ_k die Kovarianzmatrix.

Idee

Gaussian Mixture Models verallgemeinern k -means:

- weiche Zuordnungen statt harte Zuordnungen,
- ellipsenförmige Cluster statt nur kugelförmige Cluster,
- probabilistische Interpretation.

Gaussian Mixture Models werden typischerweise mit dem EM-Algorithmus trainiert. Das wird in den nächsten Kapiteln behandelt.

12.29 Evaluation von Clustering

Clustering ist schwer zu evaluieren, weil keine Labels vorhanden sind. Man unterscheidet interne und externe Maße.

12.29.1 Interne Maße

Interne Maße verwenden nur die Daten und das Clustering. Beispiele:

- k -means-Zielfunktion J ,
- Silhouette-Koeffizient,
- innerhalb-Cluster-Distanzen,
- zwischen-Cluster-Distanzen.

12.29.2 Externe Maße

Externe Maße verwenden bekannte Labels, falls sie für Evaluation verfügbar sind. Dann kann man Clustering mit echten Klassen vergleichen.

Beispiele:

- Adjusted Rand Index,
- Normalized Mutual Information,
- Purity.

Hinweis

Wenn echte Labels vorhanden sind, ist die Aufgabe nicht mehr rein unüberwacht. Man kann Labels aber verwenden, um ein Clustering nachträglich zu bewerten.

12.30 Interpretation von Clustern

Ein Clustering-Algorithmus findet immer irgendeine Struktur. Das bedeutet nicht automatisch, dass diese Struktur fachlich sinnvoll ist.

Idee

Cluster müssen interpretiert werden.

Ein mathematisch gutes Clustering ist nicht automatisch ein fachlich sinnvolles Clustering.

Wichtige Fragen:

- Sind die Cluster stabil gegenüber kleinen Änderungen der Daten?
- Haben die Cluster eine fachliche Bedeutung?
- Entstehen die Cluster nur durch Skalierung oder Vorverarbeitung?
- Sind Ausreißer verantwortlich für die Clusterstruktur?
- Ist die gewählte Distanz sinnvoll?

12.31 Clustering-Workflow

Ein typischer Clustering-Workflow ist:

1. Daten verstehen.
2. Fehlende Werte behandeln.
3. Merkmale sinnvoll auswählen.
4. Merkmale skalieren oder standardisieren.
5. Distanzmaß wählen.

6. Clustering-Methode wählen.
7. Hyperparameter wählen.
8. Clustering durchführen.
9. Cluster visualisieren.
10. Cluster fachlich interpretieren.
11. Stabilität prüfen.

Prüfungshinweis

Bei Clustering ist Vorverarbeitung besonders wichtig.

Da es keine Labels gibt, kann ein Algorithmus nicht wissen, welche Merkmale fachlich relevant sind.

12.32 Mini-Aufgaben

12.32.1 Aufgabe 1: Nächstes Zentrum

Gegeben seien zwei Clusterzentren:

$$\vec{\mu}_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \vec{\mu}_2 = \begin{pmatrix} 4 \\ 0 \end{pmatrix}.$$

Ein Datenpunkt sei:

$$\vec{x} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Welchem Zentrum wird $\vec{x}$ bei k -means zugeordnet?

Beispiel

Distanz zu $\vec{\mu}_1$:

$$\|\vec{x} - \vec{\mu}_1\|^2 = \left\| \begin{pmatrix} 1 \\ 0 \end{pmatrix} - \begin{pmatrix} 0 \\ 0 \end{pmatrix} \right\|^2 = 1^2 + 0^2 = 1.$$

Distanz zu $\vec{\mu}_2$:

$$\|\vec{x} - \vec{\mu}_2\|^2 = \left\| \begin{pmatrix} 1 \\ 0 \end{pmatrix} - \begin{pmatrix} 4 \\ 0 \end{pmatrix} \right\|^2 = (-3)^2 + 0^2 = 9.$$

Da

$$1 < 9,$$

wird $\vec{x}$ Cluster 1 zugeordnet.

12.32.2 Aufgabe 2: Zentrum aktualisieren

Ein Cluster enthält die Punkte:

$$\vec{x}_1 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad \vec{x}_2 = \begin{pmatrix} 3 \\ 4 \end{pmatrix}.$$

Berechne das neue k -means-Zentrum.

Beispiel

Das Zentrum ist der Mittelwert:

$$\vec{\mu} = \frac{1}{2} (\vec{x}_1 + \vec{x}_2).$$

Also:

$$\vec{\mu} = \frac{1}{2} \left[\begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 3 \\ 4 \end{pmatrix} \right].$$

Damit:

$$\vec{\mu} = \frac{1}{2} \begin{pmatrix} 4 \\ 6 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}.$$

12.32.3 Aufgabe 3: k -means-Zielfunktion

Gegeben seien zwei Punkte und ihr Clusterzentrum:

$$\vec{x}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \vec{x}_2 = \begin{pmatrix} 3 \\ 0 \end{pmatrix},$$
$$\vec{\mu} = \begin{pmatrix} 2 \\ 0 \end{pmatrix}.$$

Berechne die innerhalb-Cluster-Quadratsumme für diesen Cluster.

Beispiel

Für den ersten Punkt:

$$\|\vec{x}_1 - \vec{\mu}\|^2 = \left\| \begin{pmatrix} 1 \\ 0 \end{pmatrix} - \begin{pmatrix} 2 \\ 0 \end{pmatrix} \right\|^2 = (-1)^2 + 0^2 = 1.$$

Für den zweiten Punkt:

$$\|\vec{x}_2 - \vec{\mu}\|^2 = \left\| \begin{pmatrix} 3 \\ 0 \end{pmatrix} - \begin{pmatrix} 2 \\ 0 \end{pmatrix} \right\|^2 = 1^2 + 0^2 = 1.$$

Also:

$$J = 1 + 1 = 2.$$

12.32.4 Aufgabe 4: Silhouette-Wert

Für einen Datenpunkt sei:

$$a_n = 2, \quad b_n = 5.$$

Berechne den Silhouette-Wert.

Beispiel

Der Silhouette-Wert ist:

$$s_n = \frac{b_n - a_n}{\max(a_n, b_n)}.$$

Einsetzen:

$$s_n = \frac{5 - 2}{\max(2, 5)} = \frac{3}{5} = 0,6.$$

Der Wert ist positiv und relativ hoch. Der Punkt passt also eher gut zum eigenen Cluster.

12.32.5 Aufgabe 5: Hartes oder weiches Clustering

Bei einem Verfahren gilt für einen Datenpunkt:

$$r_{n1} = 0,2, \quad r_{n2} = 0,7, \quad r_{n3} = 0,1.$$

Ist das hartes oder weiches Clustering?

Beispiel

Da die Zugehörigkeiten Werte zwischen 0 und 1 annehmen und sich zu

$$0,2 + 0,7 + 0,1 = 1$$

summieren, handelt es sich um weiches Clustering.

Bei hartem Clustering wäre genau ein Wert 1 und alle anderen 0.

12.33 Häufige Fehler

Prüfungshinweis

Typische Fehler bei Clustering:

1. Clustering mit Klassifikation verwechseln.
2. Labels erwarten, obwohl Clustering unüberwacht ist.

3. Feature-Skalierung ignorieren.
4. K bei k -means ohne Prüfung festlegen.
5. Das globale Optimum von k -means erwarten.
6. Nur eine Initialisierung verwenden.
7. Nicht erkennen, dass k -means kugelförmige Cluster bevorzugt.
8. Ausreißer ignorieren.
9. Interne Metriken wie Silhouette als absolute Wahrheit interpretieren.
10. Cluster automatisch als fachlich bedeutsam ansehen.
11. PCA-Visualisierung als vollständige Clusterstruktur missverstehen.
12. Dichtebasierte und zentroidbasierte Methoden verwechseln.

12.34 Zusammenfassung

- Clustering ist unüberwachtes Lernen.
- Ziel ist, Datenpunkte in Gruppen ähnlicher Punkte einzuteilen.
- Bei k -means wird die Anzahl der Cluster K vorgegeben.
- Jeder Cluster wird durch ein Zentrum $\vec{\mu}_k$ beschrieben.
- Der Zuordnungsschritt ist:

$$z_n = \arg \min_k \|\vec{x}_n - \vec{\mu}_k\|^2.$$

- Die k -means-Zielfunktion ist:

$$J = \sum_{n=1}^N \|\vec{x}_n - \vec{\mu}_{z_n}\|^2.$$

- Das optimale Zentrum eines Clusters ist der Mittelwert seiner Punkte:

$$\vec{\mu}_k = \frac{1}{N_k} \sum_{n: z_n=k} \vec{x}_n.$$

- k -means konvergiert, aber nur zu einem lokalen Minimum.
- Initialisierung ist wichtig.
- k -means++ verbessert die Initialisierung.
- Die Wahl von K kann mit Elbow-Methode oder Silhouette-Koeffizient unterstützt werden.
- Hierarchisches Clustering erzeugt eine Clusterhierarchie.

- DBSCAN findet dichtebasierte Cluster und kann Ausreißer markieren.
- Clustering-Ergebnisse müssen fachlich interpretiert werden.
- Gaussian Mixture Models verallgemeinern k -means probabilistisch.

12.35 Typische Prüfungsfragen

Prüfungshinweis

Mögliche Prüfungsfragen zu diesem Kapitel:

1. Was ist Clustering?
2. Was ist der Unterschied zwischen Clustering und Klassifikation?
3. Warum ist Feature-Skalierung bei Clustering wichtig?
4. Definiere die k -means-Zielfunktion.
5. Erkläre den Zuordnungsschritt von k -means.
6. Leite den Aktualisierungsschritt für die Zentren her.
7. Warum ist das Zentrum eines Clusters der Mittelwert seiner Punkte?
8. Warum konvergiert k -means?
9. Warum findet k -means nicht notwendigerweise das globale Optimum?
10. Warum hängt k -means von der Initialisierung ab?
11. Was ist k -means++?
12. Was passiert bei leeren Clustern?
13. Wie kann man K wählen?
14. Was ist die Elbow-Methode?
15. Was misst der Silhouette-Koeffizient?
16. Welche Grenzen hat k -means?
17. Was ist der Unterschied zwischen hartem und weichem Clustering?
18. Was ist hierarchisches Clustering?
19. Was ist ein Dendrogramm?
20. Was sind Single Linkage, Complete Linkage und Average Linkage?
21. Was ist DBSCAN?

22. Was ist ein Kernpunkt bei DBSCAN?
23. Wie unterscheiden sich k -means und DBSCAN?
24. Warum ist Clustering schwer zu evaluieren?
25. Wie hängen k -means und Gaussian Mixture Models zusammen?

13 Gaussian Mixture Models

13.1 Ziel dieses Kapitels

Im vorherigen Kapitel haben wir Clustering betrachtet. Besonders wichtig war k -means. k -means ordnet jeden Datenpunkt genau einem Cluster zu. Diese Zuordnung ist hart:

$$r_{nk} \in \{0,1\}.$$

Ein Datenpunkt gehört also vollständig zu genau einem Cluster.

Gaussian Mixture Models, kurz GMMs, verallgemeinern diese Idee. Sie modellieren Daten als Mischung mehrerer Gauß-Verteilungen. Ein Datenpunkt kann dabei mit unterschiedlichen Wahrscheinlichkeiten zu mehreren Komponenten gehören.

Idee

Ein Gaussian Mixture Model (GMM) ist ein probabilistisches Modell, das eine Datenverteilung als gewichtete Summe mehrerer Gauß-Verteilungen beschreibt:

$$p(\vec{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\vec{x} \mid \vec{\mu}_k, \Sigma_k).$$

Dabei beschreibt jede Komponente k einen Cluster.

Nach diesem Kapitel solltest du folgende Fragen beantworten können:

- Was ist ein Gaussian Mixture Model?
- Was ist der Unterschied zwischen k -means und GMMs?
- Was sind Mischgewichte (mixing coefficients)?
- Was sind latente Variablen (latent variables)?
- Wie schreibt man die Likelihood eines GMMs?
- Warum ist die direkte Maximum-Likelihood-Schätzung schwierig?
- Was sind Verantwortlichkeiten (responsibilities)?
- Wie berechnet man $p(z = k \mid \vec{x})$?
- Wie funktioniert der Expectation-Maximization-Algorithmus?
- Was passiert im E-Schritt?

- Was passiert im M-Schritt?
- Wie werden $\vec{\mu}_k$, Σ_k und π_k aktualisiert?
- Wie hängt k -means mit GMMs zusammen?
- Welche Probleme können beim Fitten von GMMs auftreten?

13.2 Motivation

Viele reale Datensätze bestehen nicht aus einer einzigen einfachen Verteilung. Stattdessen können verschiedene Gruppen oder Unterpopulationen vorhanden sein.

Beispiele:

- Körpergrößen verschiedener Altersgruppen,
- medizinische Messwerte verschiedener Patientengruppen,
- Bildfeatures verschiedener Objektarten,
- Sensordaten verschiedener Betriebszustände,
- Kundenverhalten verschiedener Kundensegmente.

Eine einzelne Gauß-Verteilung kann solche Daten oft nicht gut beschreiben. Sie hat nur einen Mittelwert und eine Kovarianzmatrix.

$$p(\vec{x}) = \mathcal{N}(\vec{x} \mid \vec{\mu}, \Sigma).$$

Wenn die Daten mehrere Gruppen enthalten, ist eine Mischung mehrerer Gauß-Verteilungen sinnvoller:

$$p(\vec{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\vec{x} \mid \vec{\mu}_k, \Sigma_k).$$

Idee

Ein GMM kann multimodale Daten modellieren.

Multimodal bedeutet: Die Verteilung kann mehrere Häufungsbereiche oder Gipfel haben.

13.3 Eine einzelne multivariate Gauß-Verteilung

Bevor wir Mischungen betrachten, erinnern wir an die multivariate Gauß-Verteilung.

Für

$$\vec{x} \in \mathbb{R}^D$$

ist die Dichte:

$$\mathcal{N}(\vec{x} \mid \vec{\mu}, \Sigma) = \frac{1}{(2\pi)^{D/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\vec{x} - \vec{\mu})^\top \Sigma^{-1} (\vec{x} - \vec{\mu}) \right).$$

Dabei ist:

- $\vec{\mu} \in \mathbb{R}^D$ der Mittelwert,
- $\Sigma \in \mathbb{R}^{D \times D}$ die Kovarianzmatrix,
- $|\Sigma|$ die Determinante der Kovarianzmatrix.

Definition

Die multivariate Gauß-Verteilung ist durch Mittelwert und Kovarianzmatrix bestimmt:

$$\vec{\mu} = \mathbb{E}[\vec{X}],$$

$$\Sigma = \mathbb{E} \left[(\vec{X} - \vec{\mu})(\vec{X} - \vec{\mu})^\top \right].$$

13.4 MLE für eine einzelne Gauß-Verteilung

Gegeben seien Datenpunkte

$$\vec{x}_1, \dots, \vec{x}_N \in \mathbb{R}^D.$$

Wir nehmen an:

$$\vec{x}_n \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(\vec{x} \mid \vec{\mu}, \Sigma).$$

Die Log-Likelihood ist:

$$\log p(X \mid \vec{\mu}, \Sigma) = \sum_{n=1}^N \log \mathcal{N}(\vec{x}_n \mid \vec{\mu}, \Sigma).$$

Maximum-Likelihood liefert:

$$\hat{\vec{\mu}} = \frac{1}{N} \sum_{n=1}^N \vec{x}_n,$$

und

$$\hat{\Sigma} = \frac{1}{N} \sum_{n=1}^N (\vec{x}_n - \hat{\vec{\mu}})(\vec{x}_n - \hat{\vec{\mu}})^\top.$$

Definition

Für eine einzelne Gauß-Verteilung sind die Maximum-Likelihood-Schätzer:

$$\hat{\vec{\mu}} = \frac{1}{N} \sum_{n=1}^N \vec{x}_n,$$

$$\hat{\Sigma} = \frac{1}{N} \sum_{n=1}^N (\vec{x}_n - \hat{\vec{\mu}})(\vec{x}_n - \hat{\vec{\mu}})^\top.$$

Hinweis

Wie bei der eindimensionalen Normalverteilung verwendet der ML-Schätzer der Kovarianz den Faktor $\frac{1}{N}$, nicht $\frac{1}{N-1}$.

13.5 Gaussian Mixture Model

Ein Gaussian Mixture Model besteht aus K Gauß-Komponenten. Jede Komponente hat eigene Parameter:

$$\vec{\mu}_k, \quad \Sigma_k, \quad \pi_k.$$

Die Dichte ist:

$$p(\vec{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\vec{x} \mid \vec{\mu}_k, \Sigma_k).$$

Definition

Ein Gaussian Mixture Model mit K Komponenten ist:

$$p(\vec{x} \mid \Theta) = \sum_{k=1}^K \pi_k \mathcal{N}(\vec{x} \mid \vec{\mu}_k, \Sigma_k),$$

wobei

$$\Theta = \{\pi_k, \vec{\mu}_k, \Sigma_k\}_{k=1}^K$$

alle Modellparameter bezeichnet.

Die Mischgewichte müssen gültige Wahrscheinlichkeiten sein:

$$\pi_k \geq 0$$

und

$$\sum_{k=1}^K \pi_k = 1.$$

Idee

Das Mischgewicht π_k beschreibt, wie häufig Komponente k vor der Beobachtung eines Datenpunkts ausgewählt wird.

Großes π_k bedeutet: Komponente k erklärt einen großen Anteil der Daten.

13.6 Generative Interpretation

Ein GMM kann als generatives Modell verstanden werden. Ein Datenpunkt entsteht in zwei Schritten:

1. Wähle eine Komponente z aus einer kategorialen Verteilung:

$$p(z = k) = \pi_k.$$

2. Ziehe den Datenpunkt aus der entsprechenden Gauß-Verteilung:

$$p(\vec{x} \mid z = k) = \mathcal{N}(\vec{x} \mid \vec{\mu}_k, \Sigma_k).$$

Definition

Die latente Variable z gibt an, aus welcher Komponente ein Datenpunkt erzeugt wurde. Sie ist latent, weil wir z in den beobachteten Daten normalerweise nicht sehen.

Die gemeinsame Verteilung von $\vec{x}$ und z ist:

$$p(\vec{x}, z = k) = p(z = k)p(\vec{x} \mid z = k).$$

Also:

$$p(\vec{x}, z = k) = \pi_k \mathcal{N}(\vec{x} \mid \vec{\mu}_k, \Sigma_k).$$

Wenn wir z nicht beobachten, müssen wir über alle möglichen Komponenten summieren:

$$p(\vec{x}) = \sum_{k=1}^K p(\vec{x}, z = k).$$

Damit:

$$p(\vec{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\vec{x} \mid \vec{\mu}_k, \Sigma_k).$$

Idee

Die Mischung entsteht durch Marginalisierung der latenten Clusterzugehörigkeit:

$$p(\vec{x}) = \sum_z p(\vec{x}, z).$$

13.7 One-Hot-Darstellung der latenten Variablen

Man kann die latente Variable auch als One-Hot-Vektor schreiben:

$$\vec{z}_n = (z_{n1}, \dots, z_{nK})^\top.$$

Dabei gilt:

$$z_{nk} \in \{0,1\},$$

und

$$\sum_{k=1}^K z_{nk} = 1.$$

Wenn

$$z_{nk} = 1,$$

dann wurde Datenpunkt n von Komponente k erzeugt.

Die Prior-Verteilung über $\vec{z}_n$ ist:

$$p(\vec{z}_n) = \prod_{k=1}^K \pi_k^{z_{nk}}.$$

Die bedingte Verteilung des Datenpunkts ist:

$$p(\vec{x}_n \mid \vec{z}_n) = \prod_{k=1}^K \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k)^{z_{nk}}.$$

Hinweis

Die One-Hot-Schreibweise ist praktisch für Herleitungen des EM-Algorithmus.
Da genau ein z_{nk} gleich 1 ist, wird genau eine Komponente ausgewählt.

13.8 Likelihood eines GMMs

Für i.i.d. Daten

$$X = \{\vec{x}_1, \dots, \vec{x}_N\}$$

ist die Likelihood:

$$p(X \mid \Theta) = \prod_{n=1}^N p(\vec{x}_n \mid \Theta).$$

Da

$$p(\vec{x}_n | \Theta) = \sum_{k=1}^K \pi_k \mathcal{N}(\vec{x}_n | \vec{\mu}_k, \Sigma_k),$$

ist:

$$p(X | \Theta) = \prod_{n=1}^N \sum_{k=1}^K \pi_k \mathcal{N}(\vec{x}_n | \vec{\mu}_k, \Sigma_k).$$

Definition

Die GMM-Likelihood ist:

$$p(X | \Theta) = \prod_{n=1}^N \sum_{k=1}^K \pi_k \mathcal{N}(\vec{x}_n | \vec{\mu}_k, \Sigma_k).$$

Die Log-Likelihood ist:

$$\log p(X | \Theta) = \sum_{n=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\vec{x}_n | \vec{\mu}_k, \Sigma_k) \right).$$

Definition

Die GMM-Log-Likelihood ist:

$$\ell(\Theta) = \sum_{n=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\vec{x}_n | \vec{\mu}_k, \Sigma_k) \right).$$

13.9 Warum direkte MLE schwierig ist

Bei einer einzelnen Gauß-Verteilung konnten wir die Log-Likelihood direkt ableiten und geschlossene Lösungen erhalten.

Beim GMM steht aber ein Logarithmus über einer Summe:

$$\log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\vec{x}_n | \vec{\mu}_k, \Sigma_k) \right).$$

Dieser Ausdruck koppelt die Komponenten. Dadurch entstehen keine einfachen geschlossenen Lösungen durch direktes Ableiten.

Idee

Das Problem bei GMMs ist:

$$\log \left(\sum_k \text{Komponente}_k \right).$$

Die Summe innerhalb des Logarithmus macht direkte Maximum-Likelihood-Schätzung

schwierig.

Wenn wir die latenten Zuordnungen z_n kennen würden, wäre das Problem viel einfacher. Dann könnten wir für jede Komponente separat Mittelwert, Kovarianz und Mischgewicht schätzen.

Da z_n aber unbekannt ist, verwenden wir Expectation-Maximization.

13.10 Posterior über Komponenten

Für einen Datenpunkt $\vec{x}_n$ wollen wir wissen: Wie wahrscheinlich ist es, dass Komponente k diesen Punkt erzeugt hat?

Das ist:

$$p(z_n = k \mid \vec{x}_n).$$

Mit Bayes' Regel:

$$p(z_n = k \mid \vec{x}_n) = \frac{p(z_n = k)p(\vec{x}_n \mid z_n = k)}{p(\vec{x}_n)}.$$

Einsetzen:

$$p(z_n = k \mid \vec{x}_n) = \frac{\pi_k \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\vec{x}_n \mid \vec{\mu}_j, \Sigma_j)}.$$

Diese Größe heißt Verantwortung oder Responsibility.

Definition

Die Responsibility γ_{nk} ist die Posterior-Wahrscheinlichkeit, dass Komponente k für Datenpunkt n verantwortlich ist:

$$\gamma_{nk} = p(z_n = k \mid \vec{x}_n, \Theta) = \frac{\pi_k \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\vec{x}_n \mid \vec{\mu}_j, \Sigma_j)}.$$

Für jeden Datenpunkt gilt:

$$\sum_{k=1}^K \gamma_{nk} = 1.$$

Außerdem:

$$0 \leq \gamma_{nk} \leq 1.$$

Idee

Responsibilities sind weiche Clusterzuordnungen.

Statt

$$r_{nk} \in \{0,1\}$$

wie bei k -means haben wir

$$\gamma_{nk} \in [0,1].$$

13.11 Effektive Clustergröße

Die effektive Anzahl der Datenpunkte, die Komponente k zugeordnet sind, ist:

$$N_k = \sum_{n=1}^N \gamma_{nk}.$$

Definition

Die effektive Clustergröße einer GMM-Komponente ist:

$$N_k = \sum_{n=1}^N \gamma_{nk}.$$

Wenn alle Zuordnungen hart wären, wäre N_k einfach die Anzahl der Punkte im Cluster. Bei weichen Zuordnungen zählt jeder Punkt anteilig.

Beispiel

Wenn ein Punkt Responsibility

$$\gamma_{n1} = 0,7$$

für Komponente 1 und

$$\gamma_{n2} = 0,3$$

für Komponente 2 hat, dann zählt er zu 70% zu Komponente 1 und zu 30% zu Komponente 2.

13.12 Expectation-Maximization: Grundidee

Expectation-Maximization, kurz EM, ist ein Algorithmus für Modelle mit latenten Variablen.

Die Grundidee:

1. E-Schritt: Schätze die unbekannten latenten Zuordnungen mit den aktuellen Parametern.
2. M-Schritt: Aktualisiere die Parameter so, als wären diese geschätzten Zuordnungen bekannt.

Idee

EM wechselt zwischen:

Erwarte latente Struktur $\longrightarrow$ Maximiere Parameter.

Für GMMs bedeutet das:

E-Schritt: Responsibilities berechnen.

M-Schritt: Gauß-Parameter mit Responsibilities neu schätzen.

13.13 E-Schritt für GMMs

Im E-Schritt werden die Responsibilities mit den aktuellen Parametern berechnet. Seien die aktuellen Parameter:

$$\Theta^{\text{old}} = \{\pi_k^{\text{old}}, \vec{\mu}_k^{\text{old}}, \Sigma_k^{\text{old}}\}_{k=1}^K.$$

Dann ist:

$$\gamma_{nk} = \frac{\pi_k^{\text{old}} \mathcal{N}(\vec{x}_n | \vec{\mu}_k^{\text{old}}, \Sigma_k^{\text{old}})}{\sum_{j=1}^K \pi_j^{\text{old}} \mathcal{N}(\vec{x}_n | \vec{\mu}_j^{\text{old}}, \Sigma_j^{\text{old}})}.$$

Definition

Der E-Schritt eines GMMs berechnet für alle Datenpunkte und Komponenten:

$$\gamma_{nk} = p(z_n = k | \vec{x}_n, \Theta^{\text{old}}).$$

13.14 M-Schritt für GMMs

Im M-Schritt werden die Parameter mit den Responsibilities aktualisiert.

Zuerst:

$$N_k = \sum_{n=1}^N \gamma_{nk}.$$

Der neue Mittelwert ist der gewichtete Mittelwert:

$$\vec{\mu}_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} \vec{x}_n.$$

Die neue Kovarianzmatrix ist:

$$\Sigma_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (\vec{x}_n - \vec{\mu}_k^{\text{new}})(\vec{x}_n - \vec{\mu}_k^{\text{new}})^\top.$$

Das neue Mischgewicht ist:

$$\pi_k^{\text{new}} = \frac{N_k}{N}.$$

Definition

Der M-Schritt eines GMMs aktualisiert:

$$N_k = \sum_{n=1}^N \gamma_{nk},$$

$$\vec{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} \vec{x}_n,$$

$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (\vec{x}_n - \vec{\mu}_k)(\vec{x}_n - \vec{\mu}_k)^\top,$$

$$\pi_k = \frac{N_k}{N}.$$

13.15 Interpretation der M-Schritt-Formeln

Die Formeln sind die gleichen wie bei einer Gauß-Verteilung, aber mit Gewichten.

Der Mittelwert

$$\vec{\mu}_k$$

ist der gewichtete Durchschnitt aller Datenpunkte. Ein Punkt mit großer Responsibility γ_{nk} für Komponente k beeinflusst $\vec{\mu}_k$ stark. Ein Punkt mit kleiner Responsibility beeinflusst $\vec{\mu}_k$ wenig.

Die Kovarianz

$$\Sigma_k$$

ist die gewichtete Kovarianz um den neuen Mittelwert.

Das Mischgewicht

$$\pi_k$$

ist der Anteil der effektiven Punkte, die Komponente k erklärt.

Idee

Der M-Schritt ist wie das Fitten einer Gauß-Verteilung pro Cluster, aber jeder Datenpunkt zählt nur anteilig gemäß seiner Responsibility.

13.16 EM-Algorithmus für GMMs

Der vollständige Algorithmus lautet:

Definition

EM für Gaussian Mixture Models:

1. Wähle K .
2. Initialisiere π_k , $\vec{\mu}_k$ und Σ_k .
3. E-Schritt:

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\vec{x}_n \mid \vec{\mu}_j, \Sigma_j)}.$$

4. M-Schritt:

$$N_k = \sum_{n=1}^N \gamma_{nk},$$

$$\vec{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} \vec{x}_n,$$

$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (\vec{x}_n - \vec{\mu}_k)(\vec{x}_n - \vec{\mu}_k)^\top,$$

$$\pi_k = \frac{N_k}{N}.$$

5. Berechne die Log-Likelihood:

$$\ell(\Theta) = \sum_{n=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k) \right).$$

6. Wiederhole E- und M-Schritt, bis die Log-Likelihood kaum noch steigt.

13.17 Konvergenz von EM

EM hat eine wichtige Eigenschaft: Die Log-Likelihood sinkt nicht nach einer Iteration.

$$\ell(\Theta^{\text{new}}) \geq \ell(\Theta^{\text{old}}).$$

Idee

EM verbessert oder erhält in jeder Iteration die Log-Likelihood.

Das bedeutet: Der Algorithmus bewegt sich schrittweise zu einem lokalen Maximum der Likelihood.

Wie bei k -means bedeutet das aber nicht, dass das globale Maximum gefunden wird. EM kann in lokalen Maxima landen.

Hinweis

Konvergenz von EM bedeutet meistens: Die Änderung der Log-Likelihood wird kleiner als eine gewählte Toleranz.

Zum Beispiel:

$$|\ell^{(\tau+1)} - \ell^{(\tau)}| < \varepsilon.$$

13.18 Free Energy und EM-Idee

Eine tiefere Sicht auf EM verwendet eine untere Schranke der Log-Likelihood. Diese Schranke wird oft Free Energy genannt.

Für GMMs kann man schreiben:

$$\mathcal{F} = \sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} (\log \pi_k + \log \mathcal{N}(\vec{x}_n | \vec{\mu}_k, \Sigma_k)) - \sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} \log \gamma_{nk}.$$

Definition

Die Free Energy für ein GMM ist:

$$\mathcal{F} = \sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} (\log \pi_k + \log \mathcal{N}(\vec{x}_n | \vec{\mu}_k, \Sigma_k)) - \sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} \log \gamma_{nk}.$$

Die E- und M-Schritte können so interpretiert werden:

- E-Schritt: Wähle γ_{nk} , sodass die Schranke eng wird.
- M-Schritt: Maximiere die Schranke bezüglich der Modellparameter.

Hinweis

Für praktische Rechnungen reichen meistens die E- und M-Formeln. Die Free-Energy-Sicht erklärt aber, warum EM die Log-Likelihood nicht verringert.

13.19 Initialisierung

GMMs sind empfindlich gegenüber Initialisierung. Man muss initialisieren:

$$\vec{\mu}_k, \quad \Sigma_k, \quad \pi_k.$$

Typische Strategien:

- Mittelwerte zufällig aus Datenpunkten wählen,
- k -means verwenden und die Zentren übernehmen,
- Kovarianzmatrizen initial als empirische Kovarianz setzen,

- Mischgewichte gleich verteilen:

$$\pi_k = \frac{1}{K}.$$

Idee

Eine häufige Praxis ist: Erst k -means ausführen, dann die Zentren und Zuordnungen zur Initialisierung des GMMs verwenden.

13.20 Vergleich mit k -means

k -means und GMMs sind eng verwandt.

Eigenschaft	k -means	GMM
Zuordnung	hart	weich
Clusterform	kugelförmig	elliptisch möglich
Parameter	$\vec{\mu}_k$	$\pi_k, \vec{\mu}_k, \Sigma_k$
Optimierung	alternierend	EM
Probabilistisch	nicht direkt	ja

Idee

GMMs verallgemeinern k -means:

- Cluster haben Kovarianzmatrizen.
- Punkte können mehreren Clustern anteilig zugeordnet sein.
- Das Modell liefert Wahrscheinlichkeitsdichten.

13.21 k -means als Grenzfall eines GMMs

Man kann k -means als Grenzfall eines GMMs verstehen. Angenommen, alle Komponenten haben gleiche isotrope Kovarianz:

$$\Sigma_k = \sigma^2 I.$$

Wenn

$$\sigma^2 \rightarrow 0,$$

dann wird die Gauß-Dichte um jedes Zentrum sehr schmal. Jeder Punkt wird fast vollständig der nächsten Komponente zugeordnet.

Dann werden die Responsibilities fast hart:

$$\gamma_{nk} \approx \begin{cases} 1, & k = \arg \min_j \|\vec{x}_n - \vec{\mu}_j\|^2, \\ 0, & \text{sonst.} \end{cases}$$

Idee

k -means entspricht ungefähr einem GMM mit gleichen kugelförmigen Kovarianzen und harten Zuordnungen.

13.22 Kovarianztypen

In GMMs kann man verschiedene Formen für Σ_k verwenden.

13.22.1 Volle Kovarianz

Jede Komponente hat eine vollständige Kovarianzmatrix:

$$\Sigma_k \in \mathbb{R}^{D \times D}.$$

Das erlaubt beliebig orientierte elliptische Cluster.

13.22.2 Diagonale Kovarianz

Jede Kovarianzmatrix ist diagonal:

$$\Sigma_k = \text{diag}(\sigma_{k1}^2, \dots, \sigma_{kD}^2).$$

Dann sind die Merkmale innerhalb einer Komponente unkorreliert.

13.22.3 Sphärische Kovarianz

Jede Komponente hat:

$$\Sigma_k = \sigma_k^2 I.$$

Dann sind Cluster kugelförmig, aber jede Komponente kann eigene Varianz haben.

13.22.4 Geteilte Kovarianz

Alle Komponenten teilen dieselbe Kovarianzmatrix:

$$\Sigma_k = \Sigma \quad \text{für alle } k.$$

Hinweis

Volle Kovarianzen sind flexibel, benötigen aber viele Parameter. Diagonale oder sphärische Kovarianzen sind restriktiver, aber stabiler bei wenig Daten.

13.23 Anzahl der Parameter

Die Anzahl der Parameter eines GMMs wächst mit K und D .

Für jede Komponente:

- Mittelwert:

$$D$$

Parameter.

- Volle Kovarianzmatrix:

$$\frac{D(D+1)}{2}$$

Parameter, weil sie symmetrisch ist.

- Mischgewicht: Ein Gewicht pro Komponente, aber wegen

$$\sum_{k=1}^K \pi_k = 1$$

gibt es insgesamt nur $K - 1$ freie Mischgewichtparameter.

Idee

Ein GMM mit voller Kovarianz kann in hoher Dimension sehr viele Parameter haben. Daher braucht man genügend Daten oder Einschränkungen der Kovarianzstruktur.

13.24 Wahl der Komponentenzahl K

Die Anzahl der Komponenten K ist ein Hyperparameter. Zu kleines K führt zu einem zu groben Modell. Zu großes K kann Overfitting erzeugen.

Mögliche Methoden zur Wahl von K :

- Validierungs-Log-Likelihood,
- AIC,
- BIC,
- fachliche Interpretierbarkeit,
- Vergleich mehrerer Initialisierungen.

Definition

Die Bayesian Information Criterion, kurz BIC, ist:

$$\text{BIC} = -2\ell(\hat{\Theta}) + p \log N,$$

wobei p die Anzahl freier Parameter und N die Anzahl der Datenpunkte ist.

Ein kleinerer BIC-Wert ist besser. Der erste Term belohnt gute Likelihood. Der zweite Term bestraft viele Parameter.

Hinweis

BIC ist ein Modellselektionskriterium. Es ersetzt nicht automatisch fachliche Prüfung, kann aber bei der Wahl von K helfen.

13.25 Clustering mit GMMs

Nach dem Fitten eines GMMs kann man Datenpunkte clustern.

Die weiche Zuordnung ist:

$$\gamma_{nk} = p(z_n = k \mid \vec{x}_n).$$

Eine harte Zuordnung erhält man durch:

$$\hat{z}_n = \arg \max_k \gamma_{nk}.$$

Definition

Die harte GMM-Clusterzuordnung ist:

$$\hat{z}_n = \arg \max_k \gamma_{nk}.$$

Idee

GMMs liefern mehr Information als k -means:

- Die wahrscheinlichste Komponente.
- Die Unsicherheit der Zuordnung.
- Eine Dichte $p(\vec{x})$ für neue Datenpunkte.

13.26 Anomalieerkennung mit GMMs

Da ein GMM eine Dichte $p(\vec{x})$ modelliert, kann es auch für Anomalieerkennung verwendet werden.

Punkte mit sehr kleiner Dichte sind ungewöhnlich:

$$p(\vec{x}) \text{ klein} \quad \Rightarrow \quad \vec{x} \text{ ist potenziell anomal.}$$

Oder mit negativer Log-Likelihood:

$$-\log p(\vec{x})$$

als Anomaliescore.

Definition

Ein GMM-Anomaliescore kann definiert werden als:

$$s(\vec{x}) = -\log p(\vec{x}).$$

Große Werte bedeuten geringe Modellwahrscheinlichkeit.

13.27 Singularitäten und degenerierte Lösungen

Ein wichtiges Problem bei GMMs: Die Likelihood kann unbeschränkt werden.

Wenn eine Komponente genau auf einen einzelnen Datenpunkt gesetzt wird und ihre Kovarianz gegen 0 geht, wird die Gauß-Dichte an diesem Punkt sehr groß.

Formal:

$$\Sigma_k \rightarrow 0$$

kann zu

$$\mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k) \rightarrow \infty$$

führen, wenn

$$\vec{\mu}_k = \vec{x}_n.$$

Prüfungshinweis

Die GMM-Likelihood kann singuläre Lösungen haben.

Das ist ein wichtiger Unterschied zu vielen einfacheren Modellen.

Praktische Gegenmaßnahmen:

- Kovarianzmatrizen regularisieren:

$$\Sigma_k \leftarrow \Sigma_k + \varepsilon I.$$

- Mindestvarianzen erzwingen.
- Mehrere Initialisierungen verwenden.
- K nicht zu groß wählen.
- Diagonale oder geteilte Kovarianzen verwenden.

13.28 Numerische Stabilität

Die GMM-Dichte kann sehr kleine Werte annehmen. Bei vielen Datenpunkten können Produkte numerisch unterlaufen.

Deshalb arbeitet man mit Log-Likelihoods:

$$\ell(\Theta) = \sum_{n=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k) \right).$$

Auch der Ausdruck

$$\log \left(\sum_k \exp(a_k) \right)$$

sollte numerisch stabil berechnet werden. Dafür verwendet man die Log-Sum-Exp-Technik.

Definition

Die Log-Sum-Exp-Identität ist:

$$\log \sum_k \exp(a_k) = m + \log \sum_k \exp(a_k - m),$$

wobei

$$m = \max_k a_k.$$

Hinweis

Durch Subtraktion von m vermeidet man, dass $\exp(a_k)$ numerisch überläuft.

13.29 GMMs in hoher Dimension

In hoher Dimension können GMMs schwierig zu fiten sein.

Gründe:

- Volle Kovarianzmatrizen haben viele Parameter.
- Kovarianzmatrizen können singulär werden.
- Dichteschätzung wird durch den Fluch der Dimensionalität schwieriger.
- Viele Datenpunkte sind nötig, um Kovarianzen stabil zu schätzen.

Idee

In hoher Dimension verwendet man oft:

- PCA vor GMM,
- diagonale Kovarianzmatrizen,
- Regularisierung der Kovarianzen,
- weniger Komponenten,
- fachlich motivierte Features.

13.30 GMMs und PCA

PCA und GMMs können kombiniert werden. PCA reduziert zuerst die Dimension:

$$\vec{x} \in \mathbb{R}^D \longrightarrow \vec{z} \in \mathbb{R}^M.$$

Dann wird ein GMM im PCA-Raum gefittet:

$$p(\vec{z}) = \sum_{k=1}^K \pi_k \mathcal{N}(\vec{z} \mid \vec{\mu}_k, \Sigma_k).$$

Hinweis

Wenn PCA als Vorverarbeitung in einem Workflow verwendet wird, muss sie nur auf Trainingsdaten gefittet werden, um Data Leakage zu vermeiden.

13.31 Vor- und Nachteile von GMMs

13.31.1 Vorteile

- probabilistische Interpretation,
- weiche Clusterzuordnungen,
- elliptische Cluster möglich,
- Dichte $p(\vec{x})$ verfügbar,
- Anomalieerkennung möglich,
- gute Verbindung zu EM und latenten Variablen.

13.31.2 Nachteile

- Anzahl der Komponenten K muss gewählt werden,
- lokale Optima,
- empfindlich gegenüber Initialisierung,
- mögliche singuläre Kovarianzen,
- viele Parameter bei voller Kovarianz,
- Gauß-Annahme nicht immer passend.

Idee

GMMs sind flexibler als k -means, aber auch empfindlicher und parameterreicher.

13.32 Mini-Aufgaben

13.32.1 Aufgabe 1: Mischgewichte prüfen

Gegeben seien:

$$\pi_1 = 0,2, \quad \pi_2 = 0,5, \quad \pi_3 = 0,3.$$

Sind das gültige Mischgewichte?

Beispiel

Mischgewichte müssen nichtnegativ sein und sich zu 1 summieren.

Alle Werte sind nichtnegativ. Außerdem:

$$0,2 + 0,5 + 0,3 = 1.$$

Also sind die Mischgewichte gültig.

13.32.2 Aufgabe 2: GMM-Dichte

Ein eindimensionales GMM habe zwei Komponenten:

$$\pi_1 = 0,4, \quad \pi_2 = 0,6.$$

Die Dichten an einem Punkt x seien:

$$\mathcal{N}(x \mid \mu_1, \sigma_1^2) = 0,1,$$

$$\mathcal{N}(x \mid \mu_2, \sigma_2^2) = 0,3.$$

Berechne $p(x)$.

Beispiel

Die GMM-Dichte ist:

$$p(x) = \sum_{k=1}^2 \pi_k \mathcal{N}(x \mid \mu_k, \sigma_k^2).$$

Einsetzen:

$$p(x) = 0,4 \cdot 0,1 + 0,6 \cdot 0,3.$$

Also:

$$p(x) = 0,04 + 0,18 = 0,22.$$

13.32.3 Aufgabe 3: Responsibility berechnen

Mit den Werten aus Aufgabe 2 berechne die Responsibility von Komponente 2.

Beispiel

Die Responsibility ist:

$$\gamma_2 = \frac{\pi_2 \mathcal{N}(x \mid \mu_2, \sigma_2^2)}{\sum_{j=1}^2 \pi_j \mathcal{N}(x \mid \mu_j, \sigma_j^2)}.$$

Der Nenner wurde in Aufgabe 2 berechnet:

$$p(x) = 0,22.$$

Der Zähler ist:

$$0,6 \cdot 0,3 = 0,18.$$

Also:

$$\gamma_2 = \frac{0,18}{0,22} = \frac{18}{22} = \frac{9}{11} \approx 0,818.$$

13.32.4 Aufgabe 4: Effektive Clustergröße

Für eine Komponente seien die Responsibilities über vier Datenpunkte:

$$\gamma_{1k} = 0,8, \quad \gamma_{2k} = 0,6, \quad \gamma_{3k} = 0,1, \quad \gamma_{4k} = 0,5.$$

Berechne N_k .

Beispiel

Die effektive Clustergröße ist:

$$N_k = \sum_{n=1}^N \gamma_{nk}.$$

Also:

$$N_k = 0,8 + 0,6 + 0,1 + 0,5 = 2,0.$$

13.32.5 Aufgabe 5: Mischgewicht aktualisieren

Für ein GMM mit

$$N = 10$$

Datenpunkten sei

$$N_k = 2.$$

Berechne das neue Mischgewicht π_k .

Beispiel

Im M-Schritt gilt:

$$\pi_k = \frac{N_k}{N}.$$

Einsetzen:

$$\pi_k = \frac{2}{10} = 0,2.$$

13.32.6 Aufgabe 6: GMM oder k -means?

Ein Verfahren gibt für einen Datenpunkt folgende Zuordnungen aus:

$$\gamma_{n1} = 0,2, \quad \gamma_{n2} = 0,7, \quad \gamma_{n3} = 0,1.$$

Handelt es sich eher um k -means oder um ein GMM?

Beispiel

Die Werte sind weich und summieren sich zu 1:

$$0,2 + 0,7 + 0,1 = 1.$$

Das spricht für ein GMM oder ein anderes Soft-Clustering-Verfahren.

Bei k -means wäre die Zuordnung hart, also zum Beispiel:

$$(0,1,0).$$

13.33 Häufige Fehler

Prüfungshinweis

Typische Fehler bei Gaussian Mixture Models:

1. GMMs mit k -means gleichsetzen.
2. Harte und weiche Zuordnungen verwechseln.
3. Mischgewichte π_k nicht auf Summe 1 normieren.
4. Responsibilities nicht über k normieren.
5. Die GMM-Likelihood ohne die Summe über Komponenten schreiben.

6. Die Log-Likelihood fälschlich als Summe der Log-Komponenten schreiben.
7. Im M-Schritt $N_k = \sum_n \gamma_{nk}$ vergessen.
8. Kovarianzmatrizen ohne Gewichtung durch γ_{nk} berechnen.
9. Singuläre Kovarianzen ignorieren.
10. EM für garantiert global optimal halten.
11. Zu viele Komponenten ohne Regularisierung verwenden.
12. In hoher Dimension volle Kovarianzen ohne genügend Daten verwenden.

13.34 Zusammenfassung

- Ein Gaussian Mixture Model modelliert Daten als Mischung von Gauß-Verteilungen:

$$p(\vec{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\vec{x} \mid \vec{\mu}_k, \Sigma_k).$$

- Die Mischgewichte erfüllen:

$$\pi_k \geq 0, \quad \sum_{k=1}^K \pi_k = 1.$$

- Die latente Variable z gibt an, aus welcher Komponente ein Datenpunkt stammt.
- Die gemeinsame Verteilung ist:

$$p(\vec{x}, z = k) = \pi_k \mathcal{N}(\vec{x} \mid \vec{\mu}_k, \Sigma_k).$$

- Die GMM-Log-Likelihood ist:

$$\ell(\Theta) = \sum_{n=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k) \right).$$

- Die Responsibility ist:

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\vec{x}_n \mid \vec{\mu}_j, \Sigma_j)}.$$

- Die effektive Clustergröße ist:

$$N_k = \sum_{n=1}^N \gamma_{nk}.$$

- Im M-Schritt gilt:

$$\vec{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} \vec{x}_n,$$

$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (\vec{x}_n - \vec{\mu}_k)(\vec{x}_n - \vec{\mu}_k)^\top,$$

$$\pi_k = \frac{N_k}{N}.$$

- EM wechselt zwischen E-Schritt und M-Schritt.
- EM erhöht die Log-Likelihood monoton, findet aber nicht garantiert das globale Maximum.
- GMMs liefern weiche Zuordnungen und können elliptische Cluster modellieren.
- k -means kann als Grenzfall eines GMMs mit harten Zuordnungen verstanden werden.
- GMMs können Probleme mit lokalen Optima, Initialisierung und singulären Kovarianzen haben.

13.35 Typische Prüfungsfragen

Prüfungshinweis

Mögliche Prüfungsfragen zu diesem Kapitel:

1. Was ist ein Gaussian Mixture Model?
2. Welche Parameter hat ein GMM?
3. Welche Bedingungen müssen die Mischgewichte erfüllen?
4. Erkläre die generative Interpretation eines GMMs.
5. Was ist eine latente Variable?
6. Schreibe $p(\vec{x}, z = k)$ für ein GMM auf.
7. Leite $p(\vec{x})$ durch Marginalisierung von z her.
8. Schreibe die GMM-Likelihood und Log-Likelihood auf.
9. Warum ist direkte Maximum-Likelihood-Schätzung bei GMMs schwierig?
10. Was ist eine Responsibility?
11. Leite die Responsibility mit Bayes' Regel her.
12. Was ist N_k ?
13. Was passiert im E-Schritt?
14. Was passiert im M-Schritt?
15. Leite die Update-Formel für $\vec{\mu}_k$ her.
16. Leite die Update-Formel für Σ_k her.

17. Leite die Update-Formel für π_k her.
18. Warum steigt die Log-Likelihood bei EM monoton?
19. Warum findet EM nicht garantiert das globale Optimum?
20. Wie kann man GMMs initialisieren?
21. Was ist der Unterschied zwischen GMM und k -means?
22. In welchem Sinn ist k -means ein Grenzfall eines GMMs?
23. Welche Kovarianztypen kann man bei GMMs verwenden?
24. Warum können GMMs singuläre Lösungen haben?
25. Wie kann man K für ein GMM wählen?
26. Wie kann ein GMM für Anomalieerkennung verwendet werden?

14 Expectation-Maximization

14.1 Ziel dieses Kapitels

Im vorherigen Kapitel haben wir Gaussian Mixture Models (GMMs) betrachtet. Dort treten latente Variablen (latent variables) auf. Für jeden Datenpunkt gibt es eine unbekannte Komponentenzugehörigkeit

$$z_n.$$

Wenn diese Zugehörigkeiten bekannt wären, könnten wir die Parameter des Modells relativ einfach schätzen. Da sie aber nicht beobachtet werden, brauchen wir ein Verfahren, das mit fehlenden oder latenten Variablen umgehen kann.

Dieses Verfahren heißt Expectation-Maximization, kurz EM.

Idee

Expectation-Maximization (EM) ist ein iterativer Algorithmus zur Maximum-Likelihood-Schätzung bei Modellen mit latenten Variablen.

EM wechselt zwischen zwei Schritten:

$$\text{E-Schritt} \longrightarrow \text{M-Schritt}.$$

Im E-Schritt werden Erwartungen über latente Variablen berechnet. Im M-Schritt werden die Modellparameter aktualisiert.

Nach diesem Kapitel solltest du folgende Fragen beantworten können:

- Was sind latente Variablen (latent variables)?
- Warum ist Maximum-Likelihood mit latenten Variablen schwierig?
- Was ist der Unterschied zwischen vollständigen und unvollständigen Daten?
- Was ist die vollständige Daten-Log-Likelihood (complete-data log-likelihood)?
- Was ist die beobachtete Daten-Log-Likelihood (observed-data log-likelihood)?
- Was passiert im E-Schritt?
- Was passiert im M-Schritt?
- Was ist die Q -Funktion?
- Warum verringert EM die Log-Likelihood nicht?

- Wie hängt EM mit Jensen-Ungleichung und unteren Schranken zusammen?
- Wie wird EM auf Gaussian Mixture Models angewendet?
- Wie hängt EM mit k -means zusammen?
- Welche Probleme und Grenzen hat EM?

14.2 Motivation

In vielen Modellen gibt es Größen, die nicht direkt beobachtet werden. Diese Größen nennt man latente Variablen (latent variables).

Beispiele:

- Bei einem Gaussian Mixture Model ist unbekannt, aus welcher Komponente ein Datenpunkt stammt.
- Bei Clustering ist unbekannt, zu welchem Cluster ein Datenpunkt gehört.
- Bei Hidden Markov Models sind die verborgenen Zustände nicht direkt beobachtet.
- Bei fehlenden Daten sind manche Merkmale nicht beobachtet.
- Bei Topic Models sind die Themenzugehörigkeiten von Wörtern latent.

Definition

Eine latente Variable (latent variable) ist eine Variable, die Teil des Modells ist, aber nicht direkt beobachtet wird.

Wir unterscheiden:

X

für beobachtete Daten und

Z

für latente Variablen.

Die Parameter des Modells bezeichnen wir mit

θ .

14.3 Beobachtete und vollständige Daten

Angenommen, wir beobachten nur

X .

Die latenten Variablen

$$Z$$

sind unbekannt.

Dann nennt man X die beobachteten Daten (observed data). Das Paar

$$(X, Z)$$

nennt man vollständige Daten (complete data).

Definition

Die beobachteten Daten (observed data) sind die tatsächlich gemessenen Daten X . Die vollständigen Daten (complete data) bestehen aus beobachteten und latenten Variablen:

$$(X, Z).$$

Bei einem Gaussian Mixture Model sind die beobachteten Daten:

$$X = \{\vec{x}_1, \dots, \vec{x}_N\}.$$

Die latenten Variablen sind die unbekannten Komponentenzugehörigkeiten:

$$Z = \{z_1, \dots, z_N\}.$$

14.4 Beobachtete Daten-Likelihood

Unser Ziel ist Maximum-Likelihood-Schätzung (maximum likelihood estimation). Wir möchten also die Parameter θ so wählen, dass die beobachteten Daten möglichst wahrscheinlich sind.

Die beobachtete Daten-Likelihood ist:

$$p(X \mid \theta).$$

Da die latenten Variablen Z nicht beobachtet werden, müssen wir sie aus der gemeinsamen Verteilung herausmarginalisieren.

Im diskreten Fall:

$$p(X \mid \theta) = \sum_Z p(X, Z \mid \theta).$$

Im kontinuierlichen Fall:

$$p(X \mid \theta) = \int p(X, Z \mid \theta) dZ.$$

Definition

Die beobachtete Daten-Likelihood (observed-data likelihood) ist:

$$p(X \mid \theta) = \sum_Z p(X, Z \mid \theta).$$

Dabei wird über die latenten Variablen marginalisiert.

Die beobachtete Daten-Log-Likelihood ist:

$$\ell(\theta) = \log p(X \mid \theta).$$

Also im diskreten Fall:

$$\ell(\theta) = \log \sum_Z p(X, Z \mid \theta).$$

Idee

Das schwierige Problem bei latenten Variablen ist der Ausdruck

$$\log \sum_Z p(X, Z \mid \theta).$$

Der Logarithmus steht außerhalb der Summe über latente Variablen. Das macht direkte Maximierung oft schwierig.

14.5 Vollständige Daten-Likelihood

Wenn wir Z kennen würden, könnten wir stattdessen die vollständige Daten-Likelihood betrachten:

$$p(X, Z \mid \theta).$$

Die vollständige Daten-Log-Likelihood ist:

$$\log p(X, Z \mid \theta).$$

Definition

Die vollständige Daten-Log-Likelihood (complete-data log-likelihood) ist:

$$\log p(X, Z \mid \theta).$$

Sie wäre einfach zu verwenden, wenn die latenten Variablen Z beobachtet wären.

Bei vielen Modellen ist

$$\log p(X, Z \mid \theta)$$

viel einfacher als

$$\log p(X \mid \theta).$$

EM nutzt genau diese Tatsache.

14.6 Grundidee von EM

Da Z unbekannt ist, können wir nicht direkt mit

$$\log p(X, Z \mid \theta)$$

arbeiten. Stattdessen berechnen wir im E-Schritt eine Verteilung über Z , gegeben die aktuellen Parameter.

Dann maximieren wir im M-Schritt den Erwartungswert der vollständigen Daten-Log-Likelihood.

Idee

EM ersetzt die unbekannten latenten Variablen nicht durch einzelne feste Werte. Stattdessen verwendet EM ihre Posterior-Verteilung unter den aktuellen Parametern.

Seien die aktuellen Parameter

$$\theta^{\text{old}}.$$

Im E-Schritt berechnen wir:

$$p(Z \mid X, \theta^{\text{old}}).$$

Im M-Schritt maximieren wir:

$$Q(\theta, \theta^{\text{old}}) = \mathbb{E}_{Z \mid X, \theta^{\text{old}}} [\log p(X, Z \mid \theta)].$$

14.7 Die Q -Funktion

Die zentrale Größe bei EM ist die Q -Funktion.

Definition

Die Q -Funktion ist der Erwartungswert der vollständigen Daten-Log-Likelihood bezüglich der Posterior-Verteilung der latenten Variablen unter den alten Parametern:

$$Q(\theta, \theta^{\text{old}}) = \mathbb{E}_{Z \mid X, \theta^{\text{old}}} [\log p(X, Z \mid \theta)].$$

Im diskreten Fall ist:

$$Q(\theta, \theta^{\text{old}}) = \sum_Z p(Z \mid X, \theta^{\text{old}}) \log p(X, Z \mid \theta).$$

Im kontinuierlichen Fall:

$$Q(\theta, \theta^{\text{old}}) = \int p(Z | X, \theta^{\text{old}}) \log p(X, Z | \theta) dZ.$$

Hinweis

In $Q(\theta, \theta^{\text{old}})$ gibt es zwei Parameterrollen:

- θ^{old} bestimmt die Verteilung über die latenten Variablen.
- θ ist der Parameter, bezüglich dessen maximiert wird.

14.8 Der EM-Algorithmus

EM besteht aus zwei Schritten.

Definition

Expectation-Maximization-Algorithmus:

1. Initialisiere die Parameter θ^{old} .

2. E-Schritt: Berechne

$$p(Z | X, \theta^{\text{old}})$$

und damit

$$Q(\theta, \theta^{\text{old}}).$$

3. M-Schritt: Wähle neue Parameter

$$\theta^{\text{new}} = \arg \max_{\theta} Q(\theta, \theta^{\text{old}}).$$

4. Setze

$$\theta^{\text{old}} \leftarrow \theta^{\text{new}}.$$

5. Wiederhole, bis die Log-Likelihood kaum noch steigt.

Idee

E-Schritt bedeutet: Berechne Erwartungen über latente Variablen.

M-Schritt bedeutet: Maximiere die erwartete vollständige Daten-Log-Likelihood.

14.9 Warum EM die Log-Likelihood nicht verringert

Eine zentrale Eigenschaft von EM ist:

$$\ell(\theta^{\text{new}}) \geq \ell(\theta^{\text{old}}).$$

Die beobachtete Daten-Log-Likelihood sinkt also nicht.

Um das zu verstehen, betrachten wir eine untere Schranke (lower bound) der Log-Likelihood.

14.10 Untere Schranke mit einer Hilfsverteilung

Sei $q(Z)$ eine beliebige Verteilung über die latenten Variablen. Dann gilt:

$$\log p(X | \theta) = \log \sum_Z p(X, Z | \theta).$$

Wir multiplizieren und dividieren mit $q(Z)$:

$$\log p(X | \theta) = \log \sum_Z q(Z) \frac{p(X, Z | \theta)}{q(Z)}.$$

Da $q(Z)$ eine Verteilung ist, ist die Summe ein Erwartungswert bezüglich q :

$$\log p(X | \theta) = \log \mathbb{E}_q \left[\frac{p(X, Z | \theta)}{q(Z)} \right].$$

Da $\log$ konkav ist, folgt mit Jensen-Ungleichung:

$$\log \mathbb{E}_q \left[\frac{p(X, Z | \theta)}{q(Z)} \right] \geq \mathbb{E}_q \left[\log \frac{p(X, Z | \theta)}{q(Z)} \right].$$

Also:

$$\log p(X | \theta) \geq \sum_Z q(Z) \log \frac{p(X, Z | \theta)}{q(Z)}.$$

Definition

Die EM-Untergrenze oder Free Energy ist:

$$\mathcal{F}(q, \theta) = \sum_Z q(Z) \log \frac{p(X, Z | \theta)}{q(Z)}.$$

Sie erfüllt:

$$\mathcal{F}(q, \theta) \leq \log p(X | \theta).$$

14.11 Zerlegung der unteren Schranke

Wir können die Free Energy schreiben als:

$$\mathcal{F}(q, \theta) = \sum_Z q(Z) \log p(X, Z | \theta) - \sum_Z q(Z) \log q(Z).$$

Der erste Term ist der erwartete vollständige Daten-Log-Likelihood-Term. Der zweite Term ist die Entropie von q :

$$H(q) = - \sum_Z q(Z) \log q(Z).$$

Damit:

$$\mathcal{F}(q, \theta) = \mathbb{E}_q[\log p(X, Z \mid \theta)] + H(q).$$

Idee

Die Free Energy besteht aus:

erwartete vollständige Daten-Log-Likelihood + Entropie der Hilfsverteilung.

14.12 Zusammenhang mit KL-Divergenz

Die Differenz zwischen Log-Likelihood und Free Energy ist eine KL-Divergenz (Kullback-Leibler divergence).

Es gilt:

$$\log p(X \mid \theta) = \mathcal{F}(q, \theta) + \text{KL}(q(Z) \parallel p(Z \mid X, \theta)).$$

Da die KL-Divergenz immer nichtnegativ ist,

$$\text{KL}(q \parallel p) \geq 0,$$

folgt:

$$\mathcal{F}(q, \theta) \leq \log p(X \mid \theta).$$

Definition

Die KL-Divergenz zwischen zwei Verteilungen q und p ist:

$$\text{KL}(q \parallel p) = \sum_Z q(Z) \log \frac{q(Z)}{p(Z)}.$$

Sie ist immer nichtnegativ.

Die Schranke ist genau dann eng, wenn

$$q(Z) = p(Z \mid X, \theta).$$

Dann ist:

$$\text{KL}(q(Z) \parallel p(Z \mid X, \theta)) = 0.$$

14.13 E-Schritt als Optimierung von q

Im E-Schritt halten wir θ fest bei

$$\theta^{\text{old}}.$$

Wir wählen $q(Z)$ so, dass die Free Energy maximal wird. Das passiert, wenn die KL-Divergenz verschwindet.

Also:

$$q^{\text{new}}(Z) = p(Z \mid X, \theta^{\text{old}}).$$

Definition

Im E-Schritt wird die Hilfsverteilung auf den Posterior der latenten Variablen gesetzt:

$$q(Z) = p(Z \mid X, \theta^{\text{old}}).$$

Damit wird die untere Schranke bei den alten Parametern eng:

$$\mathcal{F}(q, \theta^{\text{old}}) = \log p(X \mid \theta^{\text{old}}).$$

14.14 M-Schritt als Optimierung von θ

Im M-Schritt halten wir $q(Z)$ fest und maximieren die Free Energy bezüglich θ .

Da der Entropieterm

$$H(q)$$

nicht von θ abhängt, maximieren wir nur:

$$\mathbb{E}_q[\log p(X, Z \mid \theta)].$$

Mit

$$q(Z) = p(Z \mid X, \theta^{\text{old}})$$

ist das genau:

$$Q(\theta, \theta^{\text{old}}) = \mathbb{E}_{Z \mid X, \theta^{\text{old}}}[\log p(X, Z \mid \theta)].$$

Definition

Im M-Schritt wird

$$Q(\theta, \theta^{\text{old}})$$

bezüglich θ maximiert:

$$\theta^{\text{new}} = \arg \max_{\theta} Q(\theta, \theta^{\text{old}}).$$

14.15 Monotonie von EM

Nun können wir die Monotonie von EM verstehen.

Nach dem E-Schritt ist die Schranke bei θ^{old} eng:

$$\mathcal{F}(q, \theta^{\text{old}}) = \ell(\theta^{\text{old}}).$$

Im M-Schritt wählen wir θ^{new} , sodass:

$$\mathcal{F}(q, \theta^{\text{new}}) \geq \mathcal{F}(q, \theta^{\text{old}}).$$

Da die Free Energy eine untere Schranke der Log-Likelihood ist:

$$\ell(\theta^{\text{new}}) \geq \mathcal{F}(q, \theta^{\text{new}}).$$

Damit:

$$\ell(\theta^{\text{new}}) \geq \mathcal{F}(q, \theta^{\text{new}}) \geq \mathcal{F}(q, \theta^{\text{old}}) = \ell(\theta^{\text{old}}).$$

Also:

$$\ell(\theta^{\text{new}}) \geq \ell(\theta^{\text{old}}).$$

Idee

EM verringert die beobachtete Daten-Log-Likelihood nicht.

Das bedeutet aber nicht, dass EM das globale Maximum findet. EM kann in lokalen Maxima oder Sattelpunkten landen.

14.16 EM für Gaussian Mixture Models

Nun wenden wir EM auf Gaussian Mixture Models an.

Das Modell ist:

$$p(\vec{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\vec{x} \mid \vec{\mu}_k, \Sigma_k).$$

Die Parameter sind:

$$\Theta = \{\pi_k, \vec{\mu}_k, \Sigma_k\}_{k=1}^K.$$

Die latente Variable z_n gibt an, aus welcher Komponente Datenpunkt $\vec{x}_n$ stammt.

14.17 Vollständige Daten-Likelihood für GMMs

Wir verwenden One-Hot-Variablen:

$$z_{nk} \in \{0, 1\},$$

mit

$$\sum_{k=1}^K z_{nk} = 1.$$

Die Verteilung der latenten Variablen ist:

$$p(\vec{z}_n | \pi) = \prod_{k=1}^K \pi_k^{z_{nk}}.$$

Die bedingte Verteilung des Datenpunkts ist:

$$p(\vec{x}_n | \vec{z}_n, \Theta) = \prod_{k=1}^K \mathcal{N}(\vec{x}_n | \vec{\mu}_k, \Sigma_k)^{z_{nk}}.$$

Die gemeinsame Verteilung ist:

$$p(\vec{x}_n, \vec{z}_n | \Theta) = \prod_{k=1}^K [\pi_k \mathcal{N}(\vec{x}_n | \vec{\mu}_k, \Sigma_k)]^{z_{nk}}.$$

Für alle Datenpunkte:

$$p(X, Z | \Theta) = \prod_{n=1}^N \prod_{k=1}^K [\pi_k \mathcal{N}(\vec{x}_n | \vec{\mu}_k, \Sigma_k)]^{z_{nk}}.$$

Definition

Die vollständige Daten-Likelihood eines GMMs ist:

$$p(X, Z | \Theta) = \prod_{n=1}^N \prod_{k=1}^K [\pi_k \mathcal{N}(\vec{x}_n | \vec{\mu}_k, \Sigma_k)]^{z_{nk}}.$$

14.18 Vollständige Daten-Log-Likelihood für GMMs

Wir nehmen den Logarithmus:

$$\log p(X, Z | \Theta) = \log \prod_{n=1}^N \prod_{k=1}^K [\pi_k \mathcal{N}(\vec{x}_n | \vec{\mu}_k, \Sigma_k)]^{z_{nk}}.$$

Der Logarithmus macht aus Produkten Summen:

$$\log p(X, Z | \Theta) = \sum_{n=1}^N \sum_{k=1}^K z_{nk} \log [\pi_k \mathcal{N}(\vec{x}_n | \vec{\mu}_k, \Sigma_k)].$$

Mit

$$\log(ab) = \log a + \log b$$

ergibt sich:

$$\log p(X, Z \mid \Theta) = \sum_{n=1}^N \sum_{k=1}^K z_{nk} [\log \pi_k + \log \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k)].$$

Definition

Die vollständige Daten-Log-Likelihood eines GMMs ist:

$$\log p(X, Z \mid \Theta) = \sum_{n=1}^N \sum_{k=1}^K z_{nk} [\log \pi_k + \log \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k)].$$

14.19 E-Schritt für GMMs

Im E-Schritt berechnen wir den Erwartungswert von z_{nk} unter dem Posterior:

$$\mathbb{E}[z_{nk}] = p(z_{nk} = 1 \mid \vec{x}_n, \Theta^{\text{old}}).$$

Diese Größe ist die Responsibility:

$$\gamma_{nk} = p(z_n = k \mid \vec{x}_n, \Theta^{\text{old}}).$$

Mit Bayes' Regel:

$$\gamma_{nk} = \frac{\pi_k^{\text{old}} \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k^{\text{old}}, \Sigma_k^{\text{old}})}{\sum_{j=1}^K \pi_j^{\text{old}} \mathcal{N}(\vec{x}_n \mid \vec{\mu}_j^{\text{old}}, \Sigma_j^{\text{old}})}.$$

Definition

Der E-Schritt für ein GMM berechnet:

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\vec{x}_n \mid \vec{\mu}_j, \Sigma_j)}.$$

Dabei werden die aktuellen Parameter verwendet.

Für jeden Datenpunkt gilt:

$$\sum_{k=1}^K \gamma_{nk} = 1.$$

14.20 Q-Funktion für GMMs

Die Q -Funktion ist der Erwartungswert der vollständigen Daten-Log-Likelihood.

Da

$$\mathbb{E}[z_{nk}] = \gamma_{nk},$$

erhalten wir:

$$Q(\Theta, \Theta^{\text{old}}) = \sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} [\log \pi_k + \log \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k)].$$

Definition

Die Q -Funktion für ein GMM ist:

$$Q(\Theta, \Theta^{\text{old}}) = \sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} [\log \pi_k + \log \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k)].$$

Im M-Schritt maximieren wir diese Funktion bezüglich

$$\pi_k, \quad \vec{\mu}_k, \quad \Sigma_k.$$

14.21 M-Schritt: effektive Clustergröße

Zuerst definieren wir die effektive Clustergröße:

$$N_k = \sum_{n=1}^N \gamma_{nk}.$$

Definition

Die effektive Anzahl der Punkte in Komponente k ist:

$$N_k = \sum_{n=1}^N \gamma_{nk}.$$

Bei harten Zuordnungen wäre N_k einfach die Anzahl der Punkte im Cluster. Bei weichen Zuordnungen zählt jeder Punkt anteilig.

14.22 M-Schritt: Update für Mittelwerte

Wir maximieren Q bezüglich $\vec{\mu}_k$. Nur der Gauß-Term hängt von $\vec{\mu}_k$ ab.

Für eine multivariate Gauß-Verteilung gilt:

$$\log \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k) = -\frac{D}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma_k| - \frac{1}{2} (\vec{x}_n - \vec{\mu}_k)^\top \Sigma_k^{-1} (\vec{x}_n - \vec{\mu}_k).$$

Die Terme, die von $\vec{\mu}_k$ abhängen, sind:

$$-\frac{1}{2} \sum_{n=1}^N \gamma_{nk} (\vec{x}_n - \vec{\mu}_k)^\top \Sigma_k^{-1} (\vec{x}_n - \vec{\mu}_k).$$

Maximieren entspricht Minimieren von:

$$\sum_{n=1}^N \gamma_{nk} (\vec{x}_n - \vec{\mu}_k)^\top \Sigma_k^{-1} (\vec{x}_n - \vec{\mu}_k).$$

Die Lösung ist der gewichtete Mittelwert:

$$\vec{\mu}_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} \vec{x}_n.$$

Definition

Das EM-Update für den Mittelwert einer GMM-Komponente ist:

$$\vec{\mu}_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} \vec{x}_n.$$

14.23 M-Schritt: Update für Kovarianzen

Das Update für die Kovarianzmatrix ist die gewichtete empirische Kovarianz:

$$\Sigma_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (\vec{x}_n - \vec{\mu}_k^{\text{new}})(\vec{x}_n - \vec{\mu}_k^{\text{new}})^\top.$$

Definition

Das EM-Update für die Kovarianzmatrix einer GMM-Komponente ist:

$$\Sigma_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (\vec{x}_n - \vec{\mu}_k^{\text{new}})(\vec{x}_n - \vec{\mu}_k^{\text{new}})^\top.$$

Idee

Die Kovarianz jeder Komponente wird aus allen Datenpunkten berechnet, aber jeder Datenpunkt wird mit seiner Responsibility γ_{nk} gewichtet.

14.24 M-Schritt: Update für Mischgewichte

Die Mischgewichte müssen die Nebenbedingungen erfüllen:

$$\pi_k \geq 0, \quad \sum_{k=1}^K \pi_k = 1.$$

Der relevante Teil der Q -Funktion ist:

$$\sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} \log \pi_k.$$

Wir verwenden einen Lagrange-Multiplikator λ für die Nebenbedingung:

$$\sum_{k=1}^K \pi_k = 1.$$

Die Lagrange-Funktion ist:

$$\mathcal{L} = \sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} \log \pi_k + \lambda \left(\sum_{k=1}^K \pi_k - 1 \right).$$

Ableiten nach π_k :

$$\frac{\partial \mathcal{L}}{\partial \pi_k} = \frac{\sum_{n=1}^N \gamma_{nk}}{\pi_k} + \lambda.$$

Setze gleich 0:

$$\frac{N_k}{\pi_k} + \lambda = 0.$$

Damit:

$$\pi_k = -\frac{N_k}{\lambda}.$$

Nun summieren wir über k :

$$\sum_{k=1}^K \pi_k = -\frac{1}{\lambda} \sum_{k=1}^K N_k.$$

Da

$$\sum_{k=1}^K \pi_k = 1$$

und

$$\sum_{k=1}^K N_k = N,$$

folgt:

$$1 = -\frac{N}{\lambda}.$$

Also:

$$\lambda = -N.$$

Damit:

$$\pi_k^{\text{new}} = \frac{N_k}{N}.$$

Definition

Das EM-Update für das Mischgewicht ist:

$$\pi_k^{\text{new}} = \frac{N_k}{N}.$$

14.25 Zusammenfassung der GMM-EM-Updates

Idee

Für ein Gaussian Mixture Model lauten die EM-Schritte:

E-Schritt:

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\vec{x}_n \mid \vec{\mu}_j, \Sigma_j)}.$$

M-Schritt:

$$N_k = \sum_{n=1}^N \gamma_{nk},$$

$$\vec{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} \vec{x}_n,$$

$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (\vec{x}_n - \vec{\mu}_k)(\vec{x}_n - \vec{\mu}_k)^\top,$$

$$\pi_k = \frac{N_k}{N}.$$

14.26 EM und k -means

EM für GMMs ist eng verwandt mit k -means.

Bei k -means gibt es harte Zuordnungen:

$$r_{nk} \in \{0,1\}.$$

Bei GMM-EM gibt es weiche Responsibilities:

$$\gamma_{nk} \in [0,1].$$

Wenn ein GMM gleiche isotrope Kovarianzen hat,

$$\Sigma_k = \sigma^2 I,$$

und

$$\sigma^2 \rightarrow 0,$$

werden die Responsibilities fast hart. Dann verhält sich EM ähnlich wie k -means.

Idee

k -means kann als Grenzfall von EM für GMMs verstanden werden:

weiche Zuordnung $\longrightarrow$ harte Zuordnung.

14.27 Generalized EM

Beim klassischen EM wird im M-Schritt Q vollständig maximiert:

$$\theta^{\text{new}} = \arg \max_{\theta} Q(\theta, \theta^{\text{old}}).$$

Manchmal ist das vollständige Maximum schwer zu finden. Dann reicht es, Q nur zu verbessern:

$$Q(\theta^{\text{new}}, \theta^{\text{old}}) \geq Q(\theta^{\text{old}}, \theta^{\text{old}}).$$

Das nennt man Generalized EM.

Definition

Generalized EM (GEM) verlangt im M-Schritt nicht das exakte Maximum von Q , sondern nur eine Verbesserung von Q .

Hinweis

Auch Generalized EM kann die Log-Likelihood monoton verbessern, solange der M-Schritt die untere Schranke erhöht.

14.28 Hard EM

Eine weitere Variante ist Hard EM. Dabei verwendet man nicht die vollständige Posterior-Verteilung der latenten Variablen. Stattdessen wählt man die wahrscheinlichste latente Zuordnung.

$$z_n^* = \arg \max_k p(z_n = k \mid \vec{x}_n, \theta).$$

Dann wird so getan, als wären diese Zuordnungen bekannt.

Definition

Hard EM ersetzt weiche Posterior-Zuordnungen durch harte Zuordnungen:

$$z_n^* = \arg \max_k p(z_n = k \mid \vec{x}_n, \theta).$$

Idee

Hard EM ist einfacher, verliert aber Unsicherheitsinformation.

k -means kann als eine Form von Hard EM für ein stark eingeschränktes GMM verstanden werden.

14.29 EM bei fehlenden Daten

EM kann auch für fehlende Daten verwendet werden. Angenommen, ein Datenpunkt besteht aus beobachteten und fehlenden Teilen:

$$\vec{x}_n = (\vec{x}_{n,\text{obs}}, \vec{x}_{n,\text{miss}}).$$

Dann können die fehlenden Werte als latente Variablen behandelt werden.

Im E-Schritt berechnet man Erwartungen über die fehlenden Werte gegeben die beobachteten Werte und aktuelle Parameter:

$$p(\vec{x}_{n,\text{miss}} \mid \vec{x}_{n,\text{obs}}, \theta^{\text{old}}).$$

Im M-Schritt aktualisiert man die Parameter basierend auf diesen Erwartungen.

Idee

EM ist nicht nur für Clustering nützlich.

EM ist ein allgemeines Prinzip für Maximum-Likelihood-Schätzung mit nicht beobachteten oder fehlenden Größen.

14.30 Initialisierung

EM benötigt Startwerte für die Parameter. Da EM nur zu lokalen Maxima konvergiert, ist Initialisierung wichtig.

Typische Strategien:

- zufällige Initialisierung,
- k -means zur Initialisierung von GMMs,
- mehrere Starts und Auswahl der besten Log-Likelihood,
- fachlich motivierte Startwerte.

Prüfungshinweis

EM findet im Allgemeinen kein globales Maximum.

Deshalb sollte man bei nichtkonvexen Modellen mehrere Initialisierungen ausprobieren.

14.31 Konvergenzkriterien

EM wird iteriert, bis ein Stoppkriterium erfüllt ist.

Typische Kriterien:

- Änderung der Log-Likelihood ist klein:

$$|\ell^{(\tau+1)} - \ell^{(\tau)}| < \varepsilon.$$

- Relative Änderung der Log-Likelihood ist klein:

$$\frac{|\ell^{(\tau+1)} - \ell^{(\tau)}|}{|\ell^{(\tau)}|} < \varepsilon.$$

- Änderung der Parameter ist klein:

$$\|\theta^{(\tau+1)} - \theta^{(\tau)}\| < \varepsilon.$$

- Maximale Anzahl an Iterationen erreicht.

Hinweis

In der Praxis verwendet man oft eine Kombination aus Log-Likelihood-Toleranz und maximaler Iterationszahl.

14.32 Numerische Stabilität

Bei EM für GMMs können sehr kleine Wahrscheinlichkeiten auftreten. Deshalb arbeitet man oft im Log-Raum.

Für die Log-Likelihood:

$$\ell(\Theta) = \sum_{n=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k) \right)$$

verwendet man die Log-Sum-Exp-Technik.

$$\log \sum_k \exp(a_k) = m + \log \sum_k \exp(a_k - m),$$

wobei

$$m = \max_k a_k.$$

Definition

Log-Sum-Exp stabilisiert Berechnungen der Form

$$\log \sum_k \exp(a_k).$$

Durch Subtraktion des Maximums werden numerische Überläufe vermieden.

14.33 Singularitäten bei GMMs

Bei GMMs kann EM in degenerierte Lösungen laufen. Eine Komponente kann sich auf einen einzelnen Datenpunkt konzentrieren und ihre Kovarianz gegen 0 schrumpfen lassen.

Dann kann die Likelihood gegen unendlich gehen.

$$\Sigma_k \rightarrow 0 \quad \Rightarrow \quad \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k) \rightarrow \infty$$

für

$$\vec{\mu}_k = \vec{x}_n.$$

Prüfungshinweis

Die GMM-Likelihood ist nicht immer wohldefiniert maximierbar, weil singuläre Lösungen auftreten können.

Gegenmaßnahmen:

- Kovarianzregularisierung:

$$\Sigma_k \leftarrow \Sigma_k + \varepsilon I.$$

- minimale Varianz erzwingen,
- zu kleine Komponenten entfernen,
- Anzahl der Komponenten K reduzieren,
- diagonale oder geteilte Kovarianzmatrizen verwenden.

14.34 Vorteile von EM

- EM nutzt latente Variablen systematisch.
- Viele M-Schritte haben geschlossene Formeln.
- Die Log-Likelihood wird monoton verbessert.
- EM ist intuitiv: fehlende Information schätzen, dann Parameter aktualisieren.
- EM ist auf viele Modelle anwendbar.

Idee

EM ist besonders nützlich, wenn die vollständige Daten-Likelihood einfach wäre, aber ein Teil der Daten latent oder fehlend ist.

14.35 Nachteile von EM

- EM findet nur lokale Optima.
- EM kann langsam konvergieren.
- Initialisierung ist wichtig.
- Der E-Schritt kann in komplexen Modellen schwierig sein.
- Der M-Schritt kann in komplexen Modellen schwierig sein.
- Bei GMMs können singuläre Lösungen auftreten.

Hinweis

EM ist kein einzelner spezieller Algorithmus nur für GMMs. EM ist ein allgemeines Optimierungsprinzip für Modelle mit latenten Variablen.

14.36 Mini-Aufgaben

14.36.1 Aufgabe 1: Latente Variable erkennen

In einem Clustering-Modell beobachtet man Datenpunkte $\vec{x}_n$, aber nicht, zu welchem Cluster sie gehören. Was ist die latente Variable?

Beispiel

Die latente Variable ist die Clusterzugehörigkeit:

$$z_n.$$

Sie gibt an, aus welchem Cluster oder welcher Komponente der Datenpunkt stammt.

14.36.2 Aufgabe 2: Responsibility berechnen

Ein GMM hat zwei Komponenten. Für einen Datenpunkt gelten:

$$\pi_1 \mathcal{N}_1(\vec{x}) = 0,2,$$

und

$$\pi_2 \mathcal{N}_2(\vec{x}) = 0,8.$$

Berechne γ_1 und γ_2 .

Beispiel

Die Summe ist:

$$0,2 + 0,8 = 1.$$

Also:

$$\gamma_1 = \frac{0,2}{1} = 0,2,$$

und

$$\gamma_2 = \frac{0,8}{1} = 0,8.$$

14.36.3 Aufgabe 3: Effektive Clustergröße

Für eine Komponente seien die Responsibilities:

$$0,1, \quad 0,4, \quad 0,8, \quad 0,7.$$

Berechne N_k .

Beispiel

Die effektive Clustergröße ist:

$$N_k = \sum_{n=1}^N \gamma_{nk}.$$

Also:

$$N_k = 0,1 + 0,4 + 0,8 + 0,7 = 2,0.$$

14.36.4 Aufgabe 4: Mischgewicht

Ein Datensatz hat

$$N = 20$$

Datenpunkte. Für Komponente k gilt:

$$N_k = 5.$$

Berechne das neue Mischgewicht.

Beispiel

Im M-Schritt gilt:

$$\pi_k = \frac{N_k}{N}.$$

Einsetzen:

$$\pi_k = \frac{5}{20} = 0,25.$$

14.36.5 Aufgabe 5: Monotonie

Warum sinkt die Log-Likelihood bei EM nicht?

Beispiel

EM konstruiert eine untere Schranke an die Log-Likelihood.

Im E-Schritt wird diese Schranke bei den aktuellen Parametern eng gemacht. Im M-Schritt wird die Schranke maximiert oder zumindest erhöht.

Dadurch gilt:

$$\ell(\theta^{\text{new}}) \geq \ell(\theta^{\text{old}}).$$

14.37 Häufige Fehler

Prüfungshinweis

Typische Fehler bei Expectation-Maximization:

1. EM nur als GMM-Algorithmus ansehen.
2. Latente Variablen mit Parametern verwechseln.
3. Beobachtete Daten-Likelihood und vollständige Daten-Likelihood verwechseln.
4. Den Logarithmus falsch durch die Summe ziehen.
5. Im E-Schritt harte Zuordnungen verwenden, obwohl Soft EM gefragt ist.
6. Die Q -Funktion mit der beobachteten Log-Likelihood verwechseln.
7. Im M-Schritt die Responsibilities nicht als Gewichte verwenden.
8. $N_k = \sum_n \gamma_{nk}$ vergessen.
9. Denken, EM finde garantiert das globale Maximum.
10. Ignorieren, dass Initialisierung wichtig ist.
11. Singuläre Kovarianzen bei GMMs nicht beachten.
12. Die Monotonie von EM mit globaler Optimalität verwechseln.

14.38 Zusammenfassung

- EM ist ein Algorithmus für Maximum-Likelihood-Schätzung mit latenten Variablen.

- Beobachtete Daten sind X .
- Latente Variablen sind Z .
- Die beobachtete Daten-Likelihood ist:

$$p(X \mid \theta) = \sum_Z p(X, Z \mid \theta).$$

- Die beobachtete Log-Likelihood enthält oft:

$$\log \sum_Z p(X, Z \mid \theta).$$

- Die vollständige Daten-Log-Likelihood ist:

$$\log p(X, Z \mid \theta).$$

- Die Q -Funktion ist:

$$Q(\theta, \theta^{\text{old}}) = \mathbb{E}_{Z \mid X, \theta^{\text{old}}} [\log p(X, Z \mid \theta)].$$

- Im E-Schritt berechnet man:

$$p(Z \mid X, \theta^{\text{old}}).$$

- Im M-Schritt berechnet man:

$$\theta^{\text{new}} = \arg \max_{\theta} Q(\theta, \theta^{\text{old}}).$$

- EM kann über eine untere Schranke an die Log-Likelihood hergeleitet werden.
- Diese Schranke hängt mit Jensen-Ungleichung und KL-Divergenz zusammen.
- EM verringert die Log-Likelihood nicht:

$$\ell(\theta^{\text{new}}) \geq \ell(\theta^{\text{old}}).$$

- Für GMMs berechnet der E-Schritt Responsibilities:

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(\vec{x}_n \mid \vec{\mu}_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\vec{x}_n \mid \vec{\mu}_j, \Sigma_j)}.$$

- Für GMMs aktualisiert der M-Schritt:

$$\vec{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} \vec{x}_n,$$

$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (\vec{x}_n - \vec{\mu}_k)(\vec{x}_n - \vec{\mu}_k)^{\top},$$

$$\pi_k = \frac{N_k}{N}.$$

- EM findet im Allgemeinen nur lokale Optima.
- Initialisierung, numerische Stabilität und Regularisierung sind praktisch wichtig.

14.39 Typische Prüfungsfragen

Prüfungshinweis

Mögliche Prüfungsfragen zu diesem Kapitel:

1. Was ist eine latente Variable?
2. Was ist der Unterschied zwischen beobachteten und vollständigen Daten?
3. Was ist die beobachtete Daten-Likelihood?
4. Was ist die vollständige Daten-Log-Likelihood?
5. Warum ist Maximum-Likelihood mit latenten Variablen oft schwierig?
6. Was ist die Q -Funktion?
7. Was passiert im E-Schritt?
8. Was passiert im M-Schritt?
9. Leite die EM-Untergrenze mit Jensen-Ungleichung her.
10. Wie hängt die EM-Untergrenze mit der KL-Divergenz zusammen?
11. Warum wird die Schranke im E-Schritt eng?
12. Warum verringert EM die Log-Likelihood nicht?
13. Warum garantiert EM kein globales Maximum?
14. Schreibe die vollständige Daten-Likelihood eines GMMs auf.
15. Leite die vollständige Daten-Log-Likelihood eines GMMs her.
16. Was sind Responsibilities?
17. Leite die Responsibilities für ein GMM mit Bayes' Regel her.
18. Schreibe die Q -Funktion für ein GMM auf.
19. Leite das Update für $\vec{\mu}_k$ her.
20. Leite das Update für Σ_k her.
21. Leite das Update für π_k mit Lagrange-Multiplikator her.

22. Wie hängt EM für GMMs mit k -means zusammen?
23. Was ist Generalized EM?
24. Was ist Hard EM?
25. Wie kann EM bei fehlenden Daten verwendet werden?
26. Welche numerischen Probleme können bei EM auftreten?
27. Warum können GMMs singuläre Lösungen haben?

15 Nichtparametrische Methoden

15.1 Ziel dieses Kapitels

In den bisherigen Kapiteln haben wir viele parametrische Modelle betrachtet. Beispiele sind lineare Regression, logistische Regression, neuronale Netze, Gaussian Mixture Models und PCA. Diese Modelle haben eine feste Anzahl von Parametern, die aus den Daten gelernt werden.

Nichtparametrische Methoden (nonparametric methods) funktionieren anders. Sie legen nicht im Voraus eine feste endliche Modellform mit fester Parameterzahl fest. Stattdessen kann ihre effektive Komplexität mit der Datenmenge wachsen.

Idee

Nichtparametrische Methoden (nonparametric methods) sind Methoden, deren effektive Modellkomplexität nicht durch eine feste endliche Parameterzahl begrenzt ist. Sie verwenden oft die Trainingsdaten selbst direkt für Vorhersagen oder Dichteschätzungen.

Nach diesem Kapitel solltest du folgende Fragen beantworten können:

- Was bedeutet nichtparametrisch (nonparametric)?
- Was ist der Unterschied zwischen parametrischen und nichtparametrischen Methoden?
- Was ist ein Histogramm als Dichteschätzer?
- Was ist Kernel Density Estimation?
- Was ist ein Kernel?
- Was ist die Bandbreite (bandwidth)?
- Was ist der Bias-Varianz-Trade-off bei Dichteschätzung?
- Wie funktioniert k -nearest neighbors?
- Wie verwendet man k -NN für Klassifikation?
- Wie verwendet man k -NN für Regression?
- Warum ist Feature-Skalierung bei nichtparametrischen Methoden wichtig?
- Was sind Vorteile und Nachteile nichtparametrischer Methoden?
- Warum leiden nichtparametrische Methoden unter dem Fluch der Dimensionalität?

15.2 Parametrische und nichtparametrische Modelle

Ein parametrisches Modell (parametric model) besitzt eine feste Anzahl von Parametern. Diese Anzahl hängt nicht von der Anzahl der Trainingsdaten ab.

Beispiele:

- Lineare Regression mit D Merkmalen:

$$y(\vec{x}, \vec{w}) = \vec{w}^\top \vec{x}.$$

- Logistische Regression:

$$p(t = 1 \mid \vec{x}, \vec{w}) = \sigma(\vec{w}^\top \vec{x}).$$

- Gauß-Verteilung:

$$\mathcal{N}(\vec{x} \mid \vec{\mu}, \Sigma).$$

Bei solchen Modellen wird eine feste Form angenommen. Die Daten bestimmen nur die Parameter.

Definition

Ein parametrisches Modell (parametric model) hat eine feste endliche Anzahl von Parametern.

Die Anzahl der Parameter bleibt gleich, auch wenn die Anzahl der Trainingsdaten wächst.

Nichtparametrische Methoden haben keine feste endliche Parameterzahl in diesem Sinn. Das bedeutet nicht, dass sie keine Parameter oder Hyperparameter haben. Sie können Hyperparameter wie Bandbreite oder Anzahl der Nachbarn besitzen. Aber ihre effektive Komplexität kann mit der Datenmenge wachsen.

Definition

Eine nichtparametrische Methode (nonparametric method) ist nicht auf eine feste endliche Parameterzahl beschränkt.

Die Modellkomplexität kann mit der Anzahl der Datenpunkte wachsen.

Hinweis

Nichtparametrisch bedeutet nicht parameterfrei.

Zum Beispiel hat Kernel Density Estimation eine Bandbreite h . k -nearest neighbors hat den Hyperparameter k . Diese Größen sind aber keine festen Modellparameter wie die Gewichte eines linearen Modells.

15.3 Warum nichtparametrische Methoden?

Parametrische Modelle sind oft effizient und gut interpretierbar. Sie können aber zu eingeschränkt sein, wenn die wahre Datenstruktur komplex ist.

Nichtparametrische Methoden sind flexibler. Sie können komplexe Strukturen abbilden, weil sie weniger starke Annahmen über die Form der Datenverteilung oder Entscheidungsfunktion machen.

Idee

Nichtparametrische Methoden sind nützlich, wenn man wenig über die Form der zugrunde liegenden Funktion oder Verteilung voraussetzen möchte.

Typische Anwendungen:

- Dichteschätzung (density estimation),
- Klassifikation (classification),
- Regression (regression),
- Glättung von Daten,
- explorative Datenanalyse,
- einfache Baseline-Modelle.

15.4 Dichteschätzung

Bei der Dichteschätzung (density estimation) wollen wir aus Daten eine Wahrscheinlichkeitsdichte schätzen.

Gegeben seien Datenpunkte

$$x_1, \dots, x_N$$

aus einer unbekannten Verteilung mit Dichte

$$p(x).$$

Wir wollen eine Schätzung

$$\hat{p}(x)$$

konstruieren.

Definition

Dichteschätzung (density estimation) bedeutet, aus Daten eine unbekannte Wahrscheinlichkeitsdichte $p(x)$ zu schätzen.

Parametrisch könnten wir zum Beispiel annehmen:

$$p(x) = \mathcal{N}(x \mid \mu, \sigma^2).$$

Dann müssten wir nur μ und σ^2 schätzen. Nichtparametrisch versuchen wir dagegen, die Dichteform flexibler aus den Daten selbst zu rekonstruieren.

15.5 Histogramm-Dichteschätzung

Das Histogramm ist eine einfache nichtparametrische Dichteschätzung.

Wir teilen den Wertebereich in Intervalle, sogenannte Bins. Für jeden Bin zählen wir, wie viele Datenpunkte hineinfallen.

Sei die Bin-Breite

$$\Delta.$$

Sei n_j die Anzahl der Datenpunkte im j -ten Bin. Dann ist die geschätzte Dichte im j -ten Bin:

$$\hat{p}(x) = \frac{n_j}{N\Delta}$$

für x im j -ten Bin.

Definition

Ein Histogramm schätzt die Dichte durch

$$\hat{p}(x) = \frac{\text{Anzahl der Punkte im Bin}}{N \cdot \text{Bin-Breite}}.$$

Warum teilen wir durch $N\Delta$?

Die Wahrscheinlichkeit für den Bin ist ungefähr:

$$\frac{n_j}{N}.$$

Für eine Dichte gilt näherungsweise:

$$p(x)\Delta \approx \frac{n_j}{N}.$$

Also:

$$p(x) \approx \frac{n_j}{N\Delta}.$$

Hinweis

Ein Histogramm ist einfach und anschaulich, aber die Schätzung hängt stark von der Bin-Breite und von der Lage der Bin-Grenzen ab.

15.6 Bin-Breite und Bias-Varianz-Trade-off

Die Bin-Breite Δ steuert die Glättung.

15.6.1 Große Bin-Breite

Wenn Δ groß ist, werden viele Datenpunkte in einem Bin zusammengefasst. Die Schätzung ist glatt, aber grob.

Δ groß $\Rightarrow$ hoher Bias, niedrige Varianz.

15.6.2 Kleine Bin-Breite

Wenn Δ klein ist, gibt es viele Bins. Dann schwankt das Histogramm stark, weil manche Bins wenige oder keine Punkte enthalten.

Δ klein $\Rightarrow$ niedriger Bias, hohe Varianz.

Idee

Die Bin-Breite eines Histogramms steuert den Bias-Varianz-Trade-off.

Zu große Bins glätten zu stark. Zu kleine Bins reagieren zu stark auf Zufallsschwankungen.

15.7 Kernel Density Estimation

Kernel Density Estimation, kurz KDE, ist eine glattere Form der nichtparametrischen Dichteschätzung.

Die Idee: Statt harte Bins zu zählen, legt man um jeden Datenpunkt eine kleine glatte Dichte. Die Gesamtschätzung ist der Durchschnitt dieser Dichten.

Für eindimensionale Daten lautet die Schätzung:

$$\hat{p}(x) = \frac{1}{N} \sum_{n=1}^N \frac{1}{h} K\left(\frac{x - x_n}{h}\right).$$

Dabei ist:

- K der Kernel,
- $h > 0$ die Bandbreite (bandwidth),
- x_n ein Trainingsdatenpunkt.

Definition

Kernel Density Estimation schätzt eine Dichte durch

$$\hat{p}(x) = \frac{1}{N} \sum_{n=1}^N \frac{1}{h} K\left(\frac{x - x_n}{h}\right).$$

Idee

KDE ersetzt jeden Datenpunkt durch eine kleine glatte Glocke.

Die geschätzte Dichte ist die Summe dieser Glocken.

15.8 Kernel

Ein Kernel $K(u)$ ist eine Funktion, die typischerweise folgende Eigenschaften hat:

$$K(u) \geq 0$$

und

$$\int K(u) du = 1.$$

Dadurch ist K selbst eine Dichte.

Ein häufiger Kernel ist der Gauß-Kernel:

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right).$$

Dann ist die KDE:

$$\hat{p}(x) = \frac{1}{N} \sum_{n=1}^N \frac{1}{h\sqrt{2\pi}} \exp\left(-\frac{(x-x_n)^2}{2h^2}\right).$$

Definition

Der Gauß-Kernel ist:

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right).$$

Andere mögliche Kernel sind:

- Rechteck-Kernel,
- Dreieck-Kernel,
- Epanechnikov-Kernel,
- Gauß-Kernel.

Hinweis

In der Praxis ist die Wahl der Bandbreite h meist wichtiger als die genaue Wahl des Kernels.

15.9 Bandbreite

Die Bandbreite (bandwidth) h steuert die Breite des Kernels.

15.9.1 Kleine Bandbreite

Bei kleiner Bandbreite ist jeder Kernel sehr schmal. Die Dichteschätzung kann stark schwanken.

$$h \text{ klein} \Rightarrow \text{hohe Varianz.}$$

15.9.2 Große Bandbreite

Bei großer Bandbreite ist jeder Kernel breit. Die Dichteschätzung wird sehr glatt.

$$h \text{ groß} \Rightarrow \text{hoher Bias.}$$

Idee

Die Bandbreite h ist der zentrale Hyperparameter bei Kernel Density Estimation. Sie kontrolliert den Bias-Varianz-Trade-off.

15.10 Mehrdimensionale KDE

Für Datenpunkte

$$\vec{x}_n \in \mathbb{R}^D$$

kann KDE verallgemeinert werden.

Mit isotroper Bandbreite h :

$$\hat{p}(\vec{x}) = \frac{1}{N} \sum_{n=1}^N \frac{1}{h^D} K\left(\frac{\vec{x} - \vec{x}_n}{h}\right).$$

Für einen multivariaten Gauß-Kernel:

$$K(\vec{u}) = \frac{1}{(2\pi)^{D/2}} \exp\left(-\frac{1}{2}\vec{u}^\top \vec{u}\right).$$

Dann:

$$\hat{p}(\vec{x}) = \frac{1}{N} \sum_{n=1}^N \frac{1}{(2\pi h^2)^{D/2}} \exp\left(-\frac{\|\vec{x} - \vec{x}_n\|^2}{2h^2}\right).$$

Definition

Mehrdimensionale KDE mit Gauß-Kernel ist:

$$\hat{p}(\vec{x}) = \frac{1}{N} \sum_{n=1}^N \mathcal{N}(\vec{x} \mid \vec{x}_n, h^2 I).$$

15.11 KDE und Gaussian Mixture Models

Mehrdimensionale KDE mit Gauß-Kernel kann als spezielles Gaussian Mixture Model interpretiert werden.

$$\hat{p}(\vec{x}) = \frac{1}{N} \sum_{n=1}^N \mathcal{N}(\vec{x} \mid \vec{x}_n, h^2 I).$$

Das ist eine Mischung aus N Gauß-Komponenten. Jede Komponente hat:

$$\vec{\mu}_n = \vec{x}_n,$$

$$\Sigma_n = h^2 I,$$

und Mischgewicht:

$$\pi_n = \frac{1}{N}.$$

Idee

KDE mit Gauß-Kernel ist wie ein GMM mit einer Komponente pro Datenpunkt.
Die Zentren sind nicht gelernt, sondern direkt die Trainingsdaten.

15.12 k -nearest neighbors

Eine weitere zentrale nichtparametrische Methode ist k -nearest neighbors, kurz k -NN.

Die Idee: Für eine neue Eingabe $\vec{x}$ suchen wir die k nächsten Trainingspunkte. Die Vorhersage wird aus deren Zielwerten gebildet.

Definition

k -nearest neighbors (k -NN) sagt für einen neuen Punkt $\vec{x}$ basierend auf den k nächsten Trainingspunkten vorher.

Gegeben sei ein Trainingsdatensatz:

$$\mathcal{D} = \{(\vec{x}_1, t_1), \dots, (\vec{x}_N, t_N)\}.$$

Für einen neuen Punkt $\vec{x}$ bestimmen wir die Menge der k nächsten Nachbarn:

$$\mathcal{N}_k(\vec{x}) = \text{Menge der } k \text{ Trainingspunkte mit kleinster Distanz zu } \vec{x}.$$

Hinweis

k -NN ist ein lazy learning Verfahren.

Das bedeutet: Beim Training wird fast nichts berechnet. Die eigentliche Arbeit passiert bei der Vorhersage.

15.13 k -NN für Klassifikation

Bei Klassifikation sind die Zielwerte Klassen:

$$t_n \in \{1, \dots, C\}.$$

Für einen neuen Punkt $\vec{x}$ suchen wir die k nächsten Nachbarn. Dann wählen wir die Klasse, die unter diesen Nachbarn am häufigsten vorkommt.

$$\hat{t} = \arg \max_c \sum_{n \in \mathcal{N}_k(\vec{x})} \mathbb{I}(t_n = c).$$

Definition

k -NN-Klassifikation verwendet Mehrheitsentscheidung:

$$\hat{t} = \arg \max_c \sum_{n \in \mathcal{N}_k(\vec{x})} \mathbb{I}(t_n = c).$$

Beispiel

Wenn $k = 5$ und die Nachbarn die Klassen

$$1, 1, 2, 1, 3$$

haben, dann kommt Klasse 1 dreimal vor. Also ist:

$$\hat{t} = 1.$$

15.14 k -NN für Regression

Bei Regression sind die Zielwerte reell:

$$t_n \in \mathbb{R}.$$

Für einen neuen Punkt $\vec{x}$ wird meist der Durchschnitt der Zielwerte der k nächsten Nachbarn verwendet:

$$\hat{y}(\vec{x}) = \frac{1}{k} \sum_{n \in \mathcal{N}_k(\vec{x})} t_n.$$

Definition

k -NN-Regression verwendet den Mittelwert der Zielwerte der k nächsten Nachbarn:

$$\hat{y}(\vec{x}) = \frac{1}{k} \sum_{n \in \mathcal{N}_k(\vec{x})} t_n.$$

Man kann auch gewichtete k -NN-Regression verwenden. Dann bekommen nähere Punkte ein größeres Gewicht:

$$\hat{y}(\vec{x}) = \frac{\sum_{n \in \mathcal{N}_k(\vec{x})} w_n t_n}{\sum_{n \in \mathcal{N}_k(\vec{x})} w_n}.$$

Ein mögliches Gewicht ist:

$$w_n = \frac{1}{\|\vec{x} - \vec{x}_n\| + \varepsilon}.$$

15.15 Wahl von k

Der Hyperparameter k steuert die Glättung.

15.15.1 Kleines k

Wenn $k = 1$, wird der Zielwert des nächsten Trainingspunkts verwendet. Das Modell kann sehr flexibel sein, aber stark auf Rauschen reagieren.

k klein $\Rightarrow$ niedriger Bias, hohe Varianz.

15.15.2 Großes k

Wenn k groß ist, werden viele Nachbarn gemittelt. Das Modell ist glatter, kann aber lokale Struktur verlieren.

k groß $\Rightarrow$ hoher Bias, niedrige Varianz.

Idee

Bei k -NN steuert k den Bias-Varianz-Trade-off.
 Kleines k : flexibel, aber rauschsensitiv.
 Großes k : stabil, aber möglicherweise zu glatt.

15.16 Entscheidungsgrenzen bei k -NN

Bei 1-NN können Entscheidungsgrenzen sehr komplex sein. Jeder Trainingspunkt beeinflusst eine Region des Raums. Diese Regionen hängen mit Voronoi-Zellen zusammen.

Definition

Eine Voronoi-Zelle eines Trainingspunkts enthält alle Punkte im Raum, die näher an diesem Trainingspunkt liegen als an allen anderen Trainingspunkten.

Bei 1-NN wird jeder Punkt im Raum der Klasse des nächstgelegenen Trainingspunkts zugeordnet. Dadurch können sehr unregelmäßige Entscheidungsgrenzen entstehen.

Wenn k größer wird, werden die Entscheidungsgrenzen glatter.

Hinweis

1-NN hat oft sehr niedrigen Trainingsfehler, kann aber stark überanpassen.

15.17 Distanzmaße

Die Wahl des Distanzmaßes ist entscheidend.

15.17.1 Euklidische Distanz

$$d(\vec{x}, \vec{x}') = \sqrt{\sum_{j=1}^D (x_j - x'_j)^2}.$$

15.17.2 Manhattan-Distanz

$$d(\vec{x}, \vec{x}') = \sum_{j=1}^D |x_j - x'_j|.$$

15.17.3 Minkowski-Distanz

$$d_p(\vec{x}, \vec{x}') = \left(\sum_{j=1}^D |x_j - x'_j|^p \right)^{1/p}.$$

Für $p = 2$ erhält man die euklidische Distanz. Für $p = 1$ erhält man die Manhattan-Distanz.

Definition

Die Minkowski-Distanz ist:

$$d_p(\vec{x}, \vec{x}') = \left(\sum_{j=1}^D |x_j - x'_j|^p \right)^{1/p}.$$

Hinweis

Das richtige Distanzmaß hängt vom Datentyp und Problem ab. Für Bilder, Texte oder kategoriale Daten sind einfache euklidische Distanzen nicht immer sinnvoll.

15.18 Feature-Skalierung bei k -NN

Da k -NN direkt auf Distanzen basiert, ist Feature-Skalierung sehr wichtig.

Wenn ein Merkmal eine viel größere Skala hat als andere, dominiert es die Distanz.

Eine Standardisierung ist:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}.$$

Dabei sind μ_j und σ_j Mittelwert und Standardabweichung des Merkmals im Trainingsdatensatz.

Prüfungshinweis

Bei k -NN und KDE sollte man Merkmale mit unterschiedlichen Einheiten oder Skalen fast immer skalieren oder standardisieren.

Die Skalierungsparameter müssen nur auf den Trainingsdaten berechnet werden, um Data Leakage zu vermeiden.

15.19 Fluch der Dimensionalität

Nichtparametrische Methoden leiden stark unter dem Fluch der Dimensionalität (curse of dimensionality).

In hohen Dimensionen werden Datenpunkte sehr dünn verteilt. Lokale Nachbarschaften enthalten dann oft kaum noch sinnvolle Information.

Bei k -NN bedeutet das: Der nächste Nachbar kann trotzdem weit entfernt sein. Dann ist die Annahme lokaler Ähnlichkeit schwächer.

Bei KDE bedeutet das: Man braucht sehr viele Datenpunkte, um eine Dichte in hoher Dimension zuverlässig zu schätzen.

Idee

Nichtparametrische Methoden brauchen in hoher Dimension sehr viele Daten.
Der Grund ist: Lokale Methoden benötigen genügend Datenpunkte in der Nähe jedes Abfragepunkts.

15.20 Speicher- und Rechenaufwand

Viele nichtparametrische Methoden speichern die Trainingsdaten.

Bei k -NN muss man für eine neue Eingabe oft Distanzen zu vielen Trainingspunkten berechnen.
Naiv kostet eine Vorhersage:

$$O(ND),$$

wobei:

- N die Anzahl der Trainingsdaten,
- D die Dimension ist.

Hinweis

k -NN ist beim Training billig, aber bei der Vorhersage teuer.
Parametrische Modelle sind oft beim Training teurer, aber bei der Vorhersage schneller.

Zur Beschleunigung kann man Datenstrukturen verwenden:

- k -d-Trees,
- Ball Trees,
- approximate nearest neighbor search.

In sehr hoher Dimension funktionieren klassische Baumstrukturen aber oft schlechter.

15.21 Parzen-Fenster

Kernel Density Estimation wird auch als Parzen-Fenster-Methode (Parzen window method) bezeichnet.

Die Grundidee: Ein Fenster wird um den Punkt x gelegt. Man zählt oder gewichtet, wie viele Trainingspunkte in diesem Fenster liegen.

Mit einem Rechteck-Kernel zählt man Punkte innerhalb eines festen Fensters. Mit einem Gauß-Kernel werden alle Punkte berücksichtigt, aber nahe Punkte stärker gewichtet.

Definition

Parzen-Fenster-Dichteschätzung ist eine nichtparametrische Dichteschätzung, bei der lokale Nachbarschaften zur Schätzung von $p(x)$ verwendet werden.

15.22 k -NN-Dichteschätzung

Neben KDE gibt es auch k -NN-Dichteschätzung. Die Idee: Für einen Punkt $\vec{x}$ sucht man das kleinste Volumen V , das k Trainingspunkte enthält.

Wenn N Datenpunkte insgesamt vorhanden sind, ist die geschätzte Wahrscheinlichkeit in diesem Volumen:

$$\frac{k}{N}.$$

Die Dichte ist ungefähr Wahrscheinlichkeit pro Volumen:

$$\hat{p}(\vec{x}) = \frac{k}{NV}.$$

Definition

Die k -NN-Dichteschätzung ist:

$$\hat{p}(\vec{x}) = \frac{k}{NV(\vec{x})},$$

wobei $V(\vec{x})$ das Volumen der kleinsten Umgebung um $\vec{x}$ ist, die k Datenpunkte enthält.

Idee

KDE verwendet feste Bandbreite und variable Punktzahl.

k -NN-Dichteschätzung verwendet feste Punktzahl und variable Bandbreite.

15.23 Nichtparametrische Regression als lokale Mittelung

k -NN-Regression kann als lokale Mittelung verstanden werden.

Für einen Punkt $\vec{x}$ wird nicht ein globaler Parametervektor gelernt. Stattdessen wird lokal in der Umgebung von $\vec{x}$ gemittelt:

$$\hat{y}(\vec{x}) = \frac{1}{k} \sum_{n \in \mathcal{N}_k(\vec{x})} t_n.$$

Allgemeiner kann man Kernel-Regression verwenden:

$$\hat{y}(\vec{x}) = \frac{\sum_{n=1}^N K\left(\frac{\|\vec{x}-\vec{x}_n\|}{h}\right) t_n}{\sum_{n=1}^N K\left(\frac{\|\vec{x}-\vec{x}_n\|}{h}\right)}.$$

Definition

Kernel-Regression sagt einen Zielwert als gewichteten Durchschnitt der Trainingszielwerte vorher:

$$\hat{y}(\vec{x}) = \frac{\sum_{n=1}^N w_n(\vec{x}) t_n}{\sum_{n=1}^N w_n(\vec{x})}.$$

Die Gewichte $w_n(\vec{x})$ sind größer für Trainingspunkte nahe bei $\vec{x}$.

15.24 Vergleich parametrisch und nichtparametrisch

Eigenschaft	parametrisch	nichtparametrisch
Parameterzahl	fest	kann mit Daten wachsen
Training	oft Optimierung	oft Speichern der Daten
Vorhersage	oft schnell	oft teuer
Flexibilität	durch Modellform begrenzt	hoch
Datenbedarf	oft geringer	oft höher
Interpretierbarkeit	modellabhängig	oft lokal

Idee

Parametrische Modelle verwenden starke Annahmen und können dadurch effizient lernen. Nichtparametrische Methoden machen weniger starke Annahmen, brauchen aber oft mehr Daten.

15.25 Vorteile nichtparametrischer Methoden

- flexibel,
- einfach zu verstehen,
- wenige Modellannahmen,
- gute Baselines,
- natürlich lokal,
- können komplexe Entscheidungsgrenzen modellieren.

Idee

Nichtparametrische Methoden sind besonders nützlich, wenn lokale Ähnlichkeit aussagekräftig ist.

15.26 Nachteile nichtparametrischer Methoden

- hoher Speicherbedarf,
- teure Vorhersage bei großen Datensätzen,
- empfindlich gegenüber Feature-Skalierung,
- empfindlich gegenüber irrelevanten Merkmalen,
- leiden stark unter hoher Dimension,
- Hyperparameter wie k oder h müssen gewählt werden,
- weniger kompakte Modellzusammenfassung.

Hinweis

Nichtparametrische Methoden sind nicht automatisch besser, nur weil sie flexibler sind. Ohne genug Daten oder sinnvolle Distanzen können sie schlechter generalisieren als einfache parametrische Modelle.

15.27 Modellwahl und Validierung

Hyperparameter nichtparametrischer Methoden sollten mit Validierungsdaten gewählt werden.
Beispiele:

$$k$$

bei k -NN,

$$h$$

bei KDE oder Kernel-Regression,

$$\Delta$$

beim Histogramm.

Man kann wählen:

$$k^* = \arg \min_k E_{\text{val}}(k),$$

oder

$$h^* = \arg \min_h E_{\text{val}}(h).$$

Prüfungshinweis

Auch bei nichtparametrischen Methoden gilt:

Testdaten dürfen nicht zur Wahl von k , h , Distanzmaß oder Vorverarbeitung verwendet werden.

15.28 Mini-Aufgaben

15.28.1 Aufgabe 1: Histogramm-Dichte

Ein Datensatz enthält

$$N = 100$$

Punkte. In einem Bin der Breite

$$\Delta = 0,5$$

liegen

$$n_j = 10$$

Punkte. Berechne die Histogramm-Dichteschätzung in diesem Bin.

Beispiel

Die Histogramm-Dichte ist:

$$\hat{p}(x) = \frac{n_j}{N\Delta}.$$

Einsetzen:

$$\hat{p}(x) = \frac{10}{100 \cdot 0,5} = \frac{10}{50} = 0,2.$$

15.28.2 Aufgabe 2: k -NN-Klassifikation

Für einen neuen Punkt seien die 5 nächsten Nachbarn mit Klassen:

$$A, A, B, A, B.$$

Welche Klasse sagt 5-NN vorher?

Beispiel

Klasse A kommt dreimal vor. Klasse B kommt zweimal vor.

Also sagt 5-NN voraus:

$$\hat{t} = A.$$

15.28.3 Aufgabe 3: k -NN-Regression

Die 3 nächsten Nachbarn haben Zielwerte:

$$2, \quad 4, \quad 10.$$

Berechne die 3-NN-Regressionsvorhersage.

Beispiel

Die Vorhersage ist der Mittelwert:

$$\hat{y} = \frac{2 + 4 + 10}{3} = \frac{16}{3}.$$

Also:

$$\hat{y} \approx 5,33.$$

15.28.4 Aufgabe 4: Bias und Varianz bei k

Was passiert typischerweise, wenn k bei k -NN sehr klein gewählt wird?

Beispiel

Ein sehr kleines k , zum Beispiel $k = 1$, macht das Modell sehr flexibel.

Typischerweise gilt:

$$\text{Bias niedrig,} \quad \text{Varianz hoch.}$$

Das Modell kann stark auf Rauschen und einzelne Trainingspunkte reagieren.

15.28.5 Aufgabe 5: KDE als GMM

Erkläre, warum KDE mit Gauß-Kernel wie ein Gaussian Mixture Model mit N Komponenten aussieht.

Beispiel

KDE mit Gauß-Kernel ist:

$$\hat{p}(\vec{x}) = \frac{1}{N} \sum_{n=1}^N \mathcal{N}(\vec{x} \mid \vec{x}_n, h^2 I).$$

Das ist eine Mischung von N Gauß-Verteilungen.
Jede Komponente hat:

$$\vec{\mu}_n = \vec{x}_n,$$

$$\Sigma_n = h^2 I,$$

und

$$\pi_n = \frac{1}{N}.$$

Daher kann man KDE als spezielles GMM mit einer Komponente pro Datenpunkt interpretieren.

15.29 Häufige Fehler

Prüfungshinweis

Typische Fehler bei nichtparametrischen Methoden:

1. Nichtparametrisch mit parameterfrei verwechseln.
2. Histogramme ohne Beachtung der Bin-Breite interpretieren.
3. Bei KDE die Bandbreite h ignorieren.
4. Bei k -NN die Feature-Skalierung vergessen.
5. Euklidische Distanz verwenden, obwohl sie für die Daten nicht sinnvoll ist.
6. $k = 1$ verwenden und Overfitting übersehen.
7. Sehr großes k verwenden und Underfitting übersehen.
8. Testdaten zur Wahl von k oder h verwenden.
9. Den Fluch der Dimensionalität unterschätzen.
10. k -NN als schnelles Vorhersagemodell bei riesigen Datensätzen erwarten.
11. Irrelevante Merkmale nicht entfernen, obwohl sie Distanzen verzerren.
12. KDE in hoher Dimension ohne genug Daten verwenden.

15.30 Zusammenfassung

- Parametrische Modelle haben eine feste endliche Parameterzahl.
- Nichtparametrische Methoden haben keine feste endliche Parameterzahl in diesem Sinn.

- Nichtparametrisch bedeutet nicht parameterfrei.
- Histogramme sind einfache nichtparametrische Dichteschätzer.
- Die Histogramm-Dichte in einem Bin ist:

$$\hat{p}(x) = \frac{n_j}{N\Delta}.$$

- Kernel Density Estimation schätzt:

$$\hat{p}(x) = \frac{1}{N} \sum_{n=1}^N \frac{1}{h} K\left(\frac{x - x_n}{h}\right).$$

- Die Bandbreite h kontrolliert Glättung, Bias und Varianz.
- Mehrdimensionale KDE mit Gauß-Kernel ist:

$$\hat{p}(\vec{x}) = \frac{1}{N} \sum_{n=1}^N \mathcal{N}(\vec{x} \mid \vec{x}_n, h^2 I).$$

- k -NN verwendet die k nächsten Trainingspunkte für Vorhersagen.
- k -NN-Klassifikation verwendet Mehrheitsentscheidung:

$$\hat{t} = \arg \max_c \sum_{n \in \mathcal{N}_k(\vec{x})} \mathbb{I}(t_n = c).$$

- k -NN-Regression verwendet lokale Mittelung:

$$\hat{y}(\vec{x}) = \frac{1}{k} \sum_{n \in \mathcal{N}_k(\vec{x})} t_n.$$

- Kleines k bedeutet niedrigen Bias und hohe Varianz.
- Großes k bedeutet höheren Bias und niedrigere Varianz.
- Nichtparametrische Methoden sind flexibel, brauchen aber oft viele Daten.
- Feature-Skalierung und Distanzwahl sind entscheidend.
- In hoher Dimension leiden nichtparametrische Methoden stark unter dem Fluch der Dimensionalität.

15.31 Typische Prüfungsfragen

Prüfungshinweis

Mögliche Prüfungsfragen zu diesem Kapitel:

1. Was bedeutet nichtparametrisch?
2. Warum heißt nichtparametrisch nicht parameterfrei?
3. Was ist der Unterschied zwischen parametrischen und nichtparametrischen Modellen?
4. Was ist Dichteschätzung?
5. Wie funktioniert ein Histogramm als Dichteschätzer?
6. Welche Rolle spielt die Bin-Breite?
7. Was ist Kernel Density Estimation?
8. Was ist ein Kernel?
9. Was ist die Bandbreite h ?
10. Wie beeinflusst h Bias und Varianz?
11. Schreibe KDE mit Gauß-Kernel auf.
12. Wie hängt KDE mit Gaussian Mixture Models zusammen?
13. Was ist k -nearest neighbors?
14. Wie funktioniert k -NN-Klassifikation?
15. Wie funktioniert k -NN-Regression?
16. Welche Rolle spielt der Hyperparameter k ?
17. Warum ist 1-NN oft varianzreich?
18. Warum kann großes k zu Underfitting führen?
19. Welche Distanzmaße können bei k -NN verwendet werden?
20. Warum ist Feature-Skalierung bei k -NN wichtig?
21. Was ist k -NN-Dichteschätzung?
22. Was ist Kernel-Regression?
23. Warum sind nichtparametrische Methoden in hoher Dimension schwierig?
24. Welche Vor- und Nachteile haben nichtparametrische Methoden?